

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

The Ford-Fulkerson Method

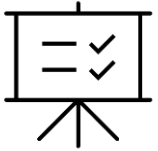
Session - 26

AIM OF THE SESSION



To familiarize students with the concept of the Ford-Fulkerson Method.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- the Ford-Fulkerson Method.
2. Describe :- Network flow and the Ford-Fulkerson maximum flow.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- the Ford-Fulkerson Method.
2. Describe :- Network flow and the Ford-Fulkerson maximum flow
3. Summarize:- Ford-Fulkerson maximum flow.

Maximum Flow

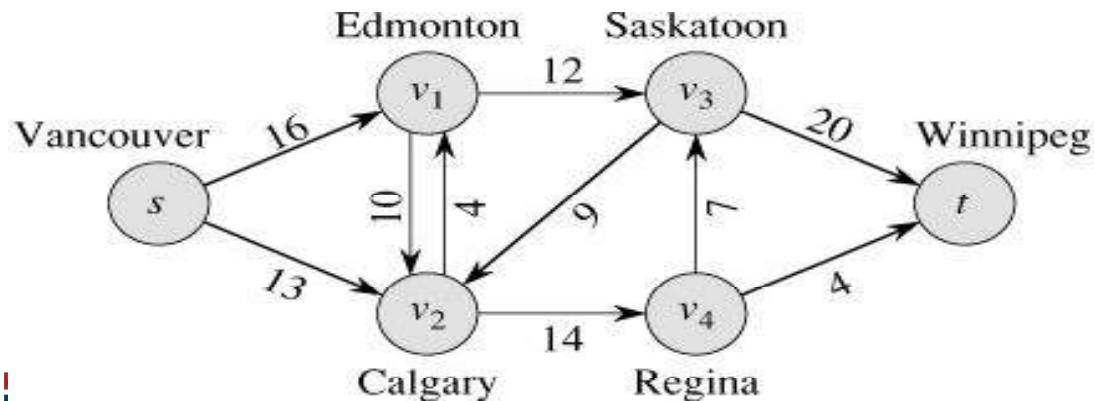
It is defined as the maximum amount of flow that the network would allow to flow from source to sink.

Multiple algorithms exist in solving the maximum flow problem.

Two major algorithms to solve these kind of problems are Ford-Fulkerson algorithm and Dinic's Algorithm.

What is Network Flowing

- In graph theory, a flow network is defined as a directed graph involving a source(S) and a sink(T) and several other nodes connected with edges.
- Each edge has an individual capacity $c(u,v) \geq 0$ which is the maximum limit of flow that edge could allow.
- If (u,v) is not in E assume $c(u,v)=0$.
- Flow in a network is an integer-valued function f defined on the edges of G satisfying $0 \leq f(u,v) \leq c(u,v)$, for every edge (u,v) in E .
- Assume that every vertex v in V is on some path from s to t .
- Following is an illustration of a network flow:



Initially,
 $c(s,v1)=16$
 $c(v1,s)=0$
 $c(v2,s)=0 \dots$

Conditions for network flow

For each edge (u,v) in E , the flow $f(u,v)$ is a real valued function that must satisfy following 3 conditions :

- **Capacity Constraint :** $\forall u,v \in V, f(u,v) \leq c(u,v)$
- **Skew Symmetry :** $\forall u,v \in V, f(u,v) = -f(v,u)$
- **Flow Conservation:** $\forall u \in V - \{s,t\} \sum_{v \in V} f(s,v) = 0$

Skew symmetry condition implies that $f(u,u)=0$.

The value of a flow

The value of a flow is given by :

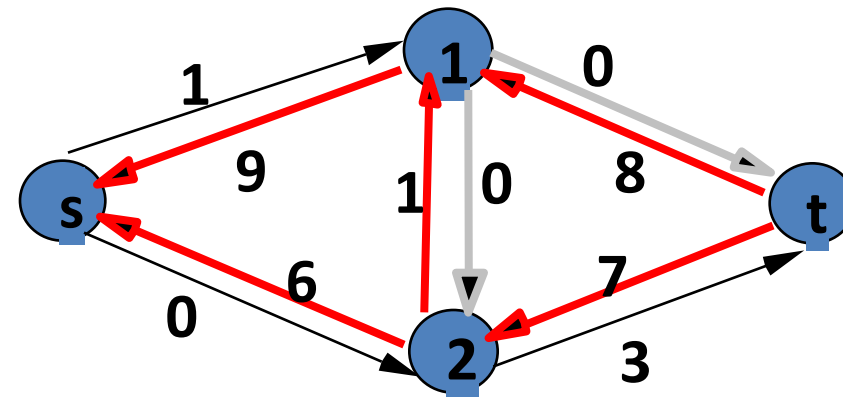
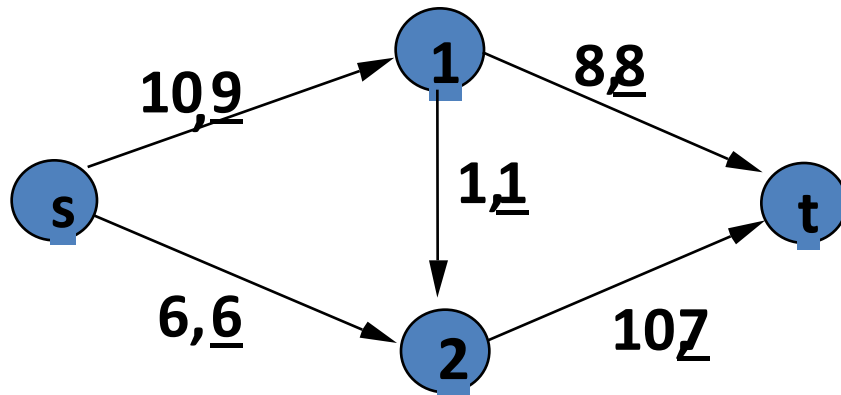
$$|f| = \sum_{v \in V} f(s, v) = \sum_{v \in V} f(v, t)$$

The flow into the node is same as flow going out from the node and thus the flow is conserved. Also the total amount of flow from source s = total amount of flow into the sink t .

The Ford-Fulkerson Method

- It was developed by L. R. Ford, Jr. and D. R. Fulkerson in 1956.
- A residual network graph indicates how much more flow is allowed in each edge in the network graph.
- The residual capacity of the network with a flow f is given by:
- The residual capacity (rc) of an edge (i,j) equals $c(i,j) - f(i,j)$ when (i,j) is a forward edge, and equals $f(i,j)$ when (i,j) is a backward edge. Moreover the residual capacity of an edge is always non-negative

$$c_f(u, v) = c(u, v) - f(u, v)$$



Augmenting Paths

- An augmenting path is a simple path from source to sink which do not include any cycles and that pass only through positive weighted edges.
- If there are no augmenting paths possible from S to T, then the flow is maximum.
- The result i.e. the maximum flow will be the total flow out of source node which is also equal to total flow in to the sink node.

Implementation

- An augmenting path in residual graph can be found using DFS or BFS.
- Updating residual graph includes following steps: (refer the diagrams for better understanding)
 - For every edge in the augmenting path, a value of minimum capacity in the path is subtracted from all the edges of that path.
 - An edge of equal amount is added to edges in reverse direction for every successive nodes in the augmenting path.

The Ford-Fulkerson's Algorithm

FORDFULKERSON(G, E, s, t)

FOREACH $e \in E$

$f(e) \leftarrow 0$

$G_f \leftarrow$ residual graph

WHILE (there exists augmenting path P)

$f \leftarrow$ augment(f, P)

update G_f

ENDWHILE

RETURN f

AUGMENT(f, P)

$b \leftarrow$ bottleneck(P)

FOREACH $e \in P$

IF ($e \in E$)

// backwards arc

$f(e) \leftarrow f(e) + b$

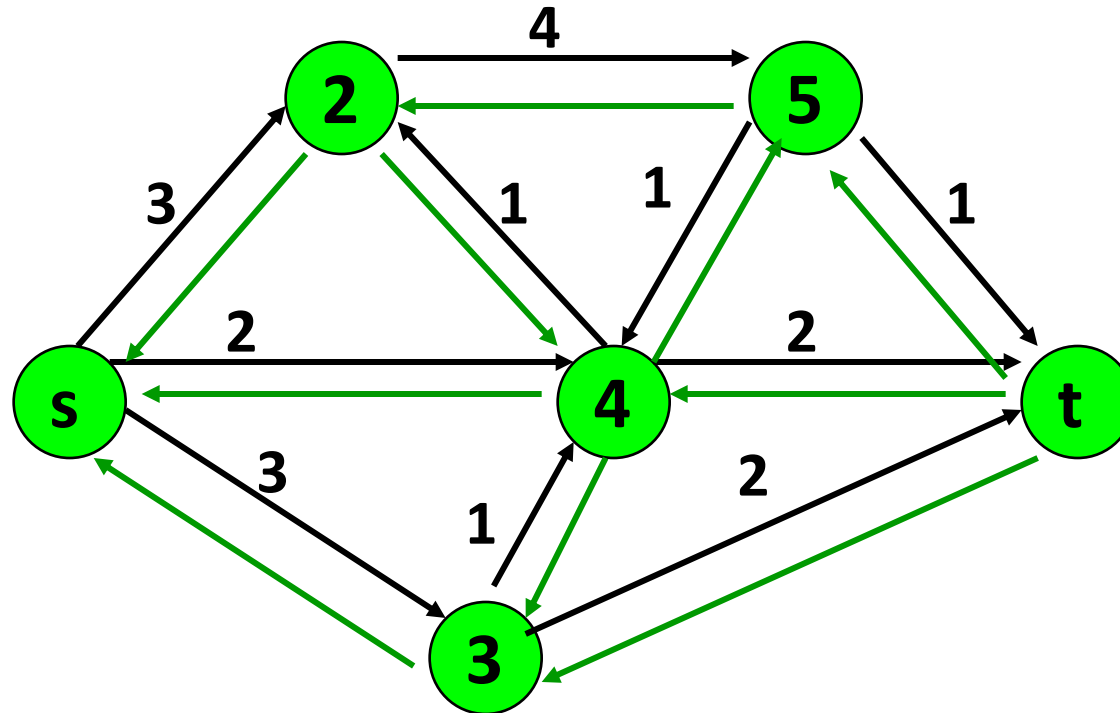
ELSE

// forward arc

$f(e^R) \leftarrow f(e) - b$

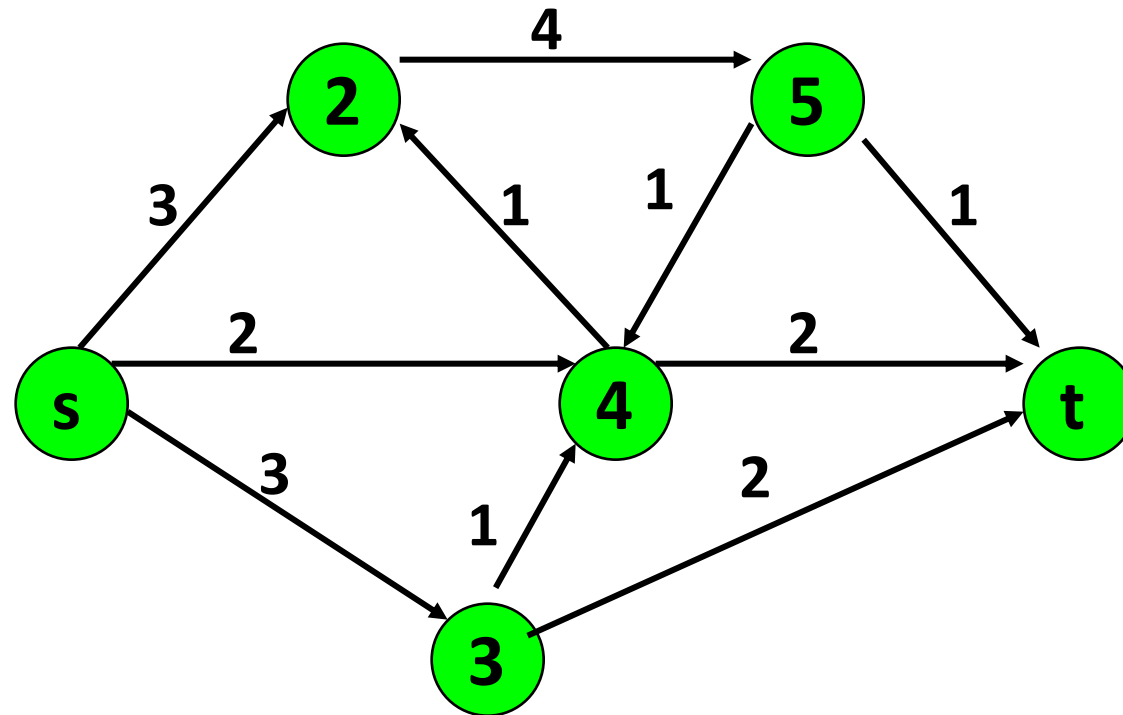
RETURN f

The Ford-Fulkerson Max Flow

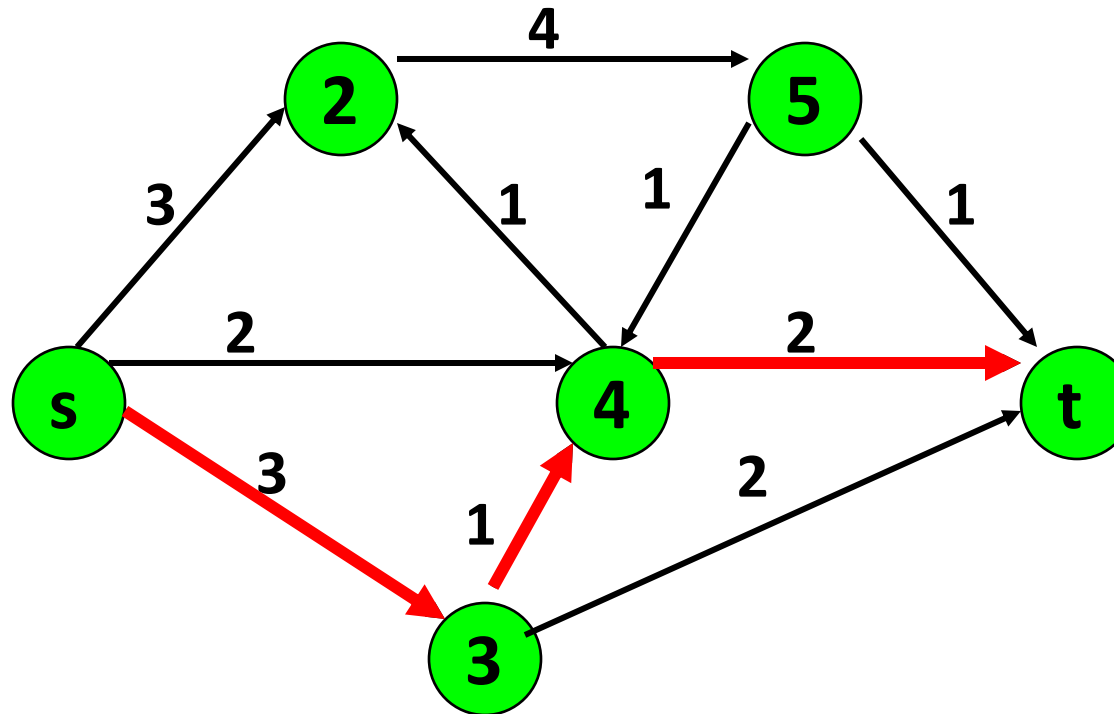


This is the original network, plus reversals of the arcs.

The Ford-Fulkerson Max Flow

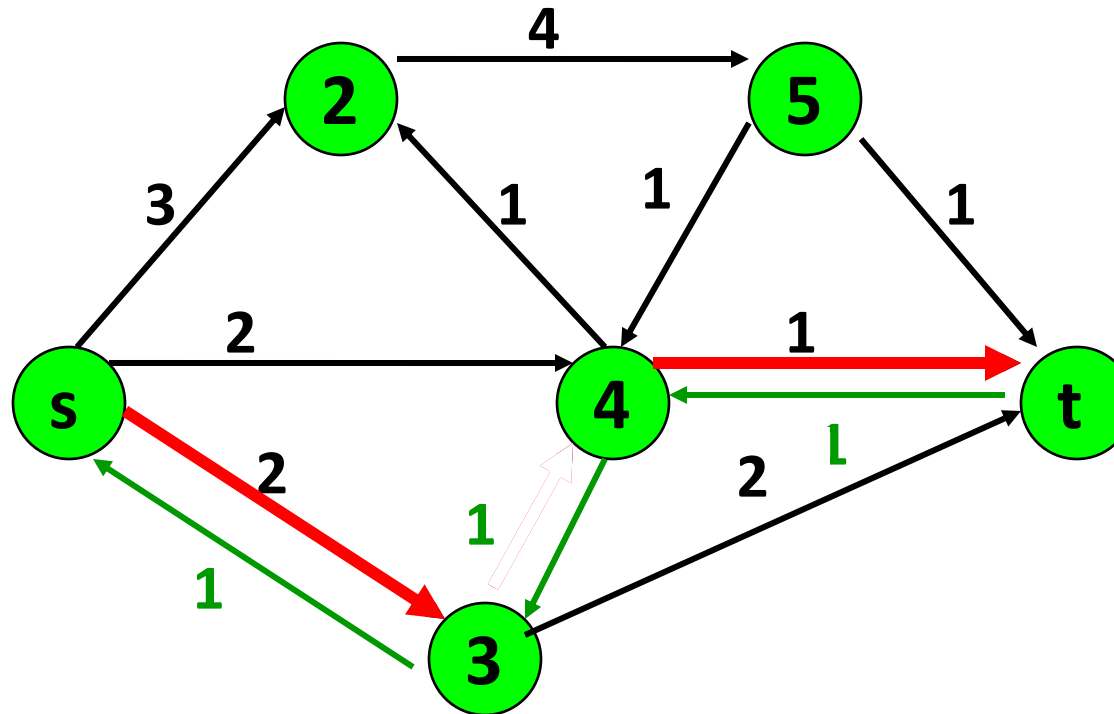


This is the original network, and the original residual network.



Find any s-t path in $G(x)$

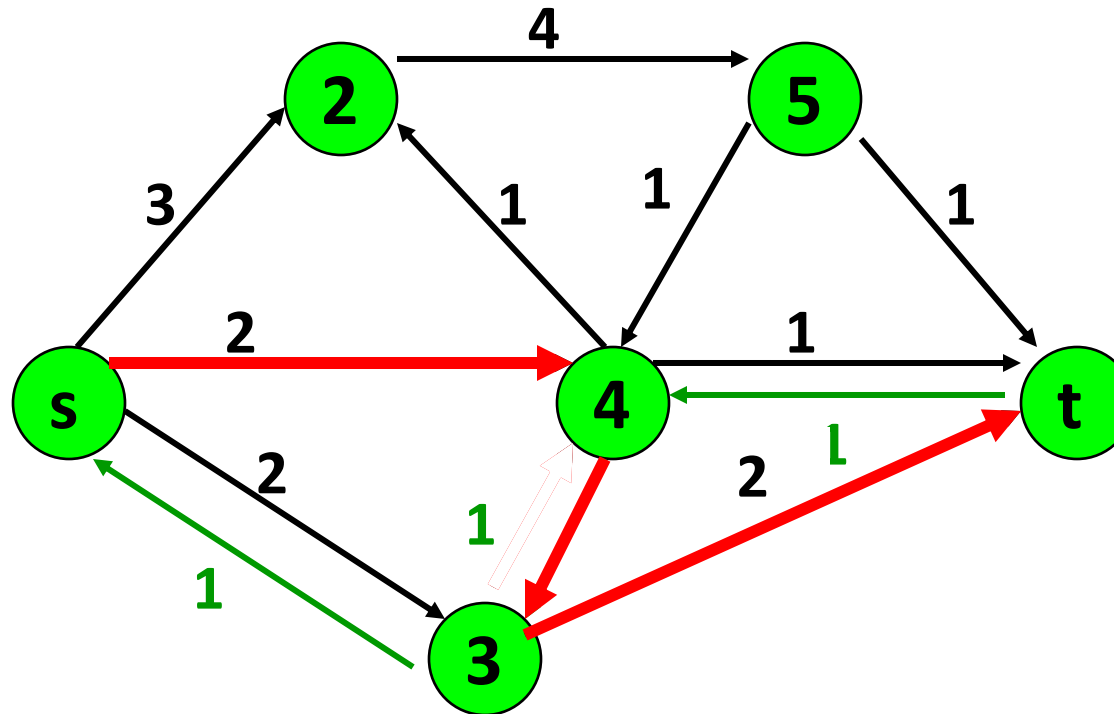
The Ford-Fulkerson Max Flow



Determine the capacity Δ of the path.

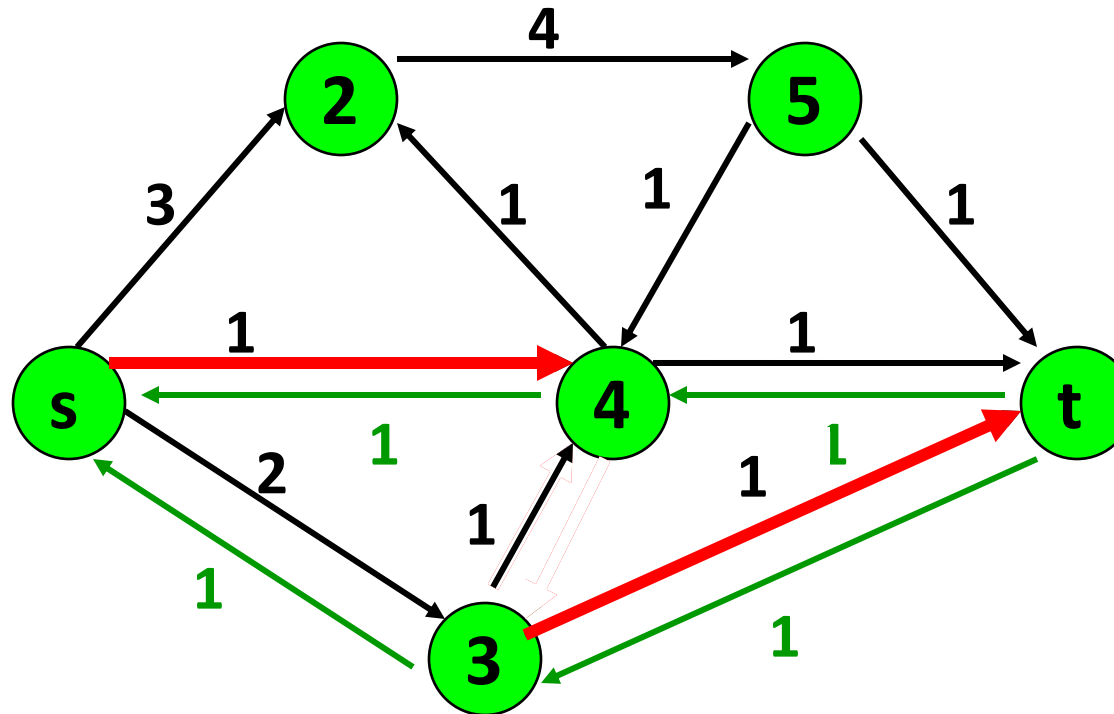
Send Δ units of flow in the path.
Update residual capacities.

The Ford-Fulkerson Max Flow



Find any s-t path

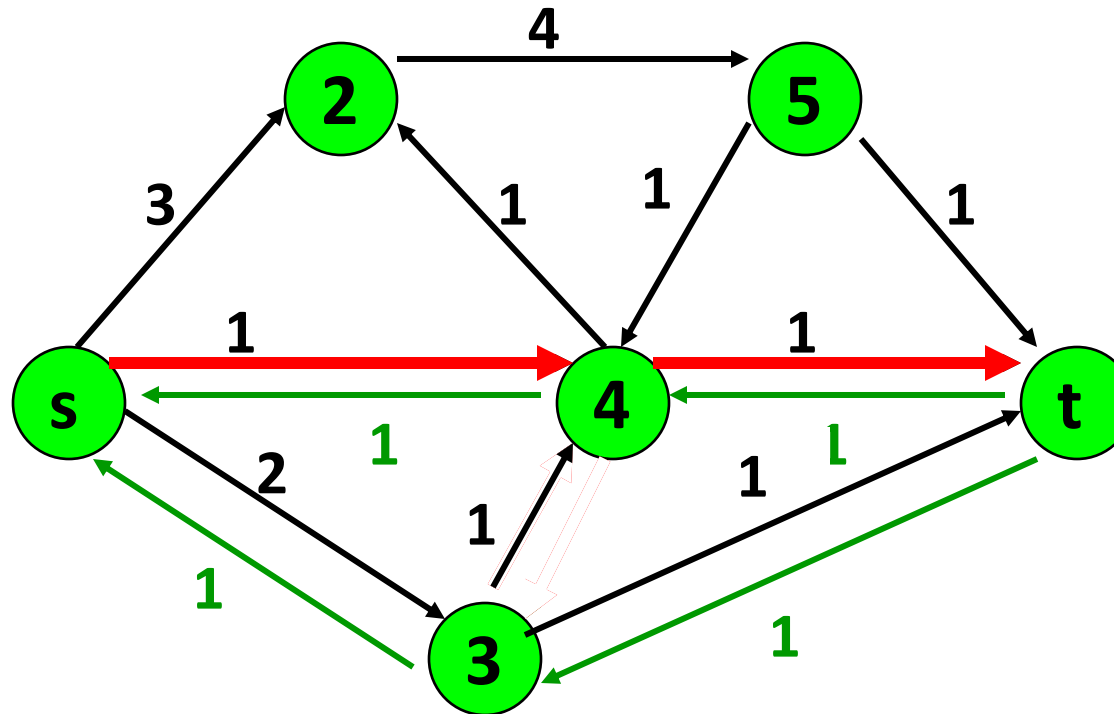
The Ford-Fulkerson Max Flow



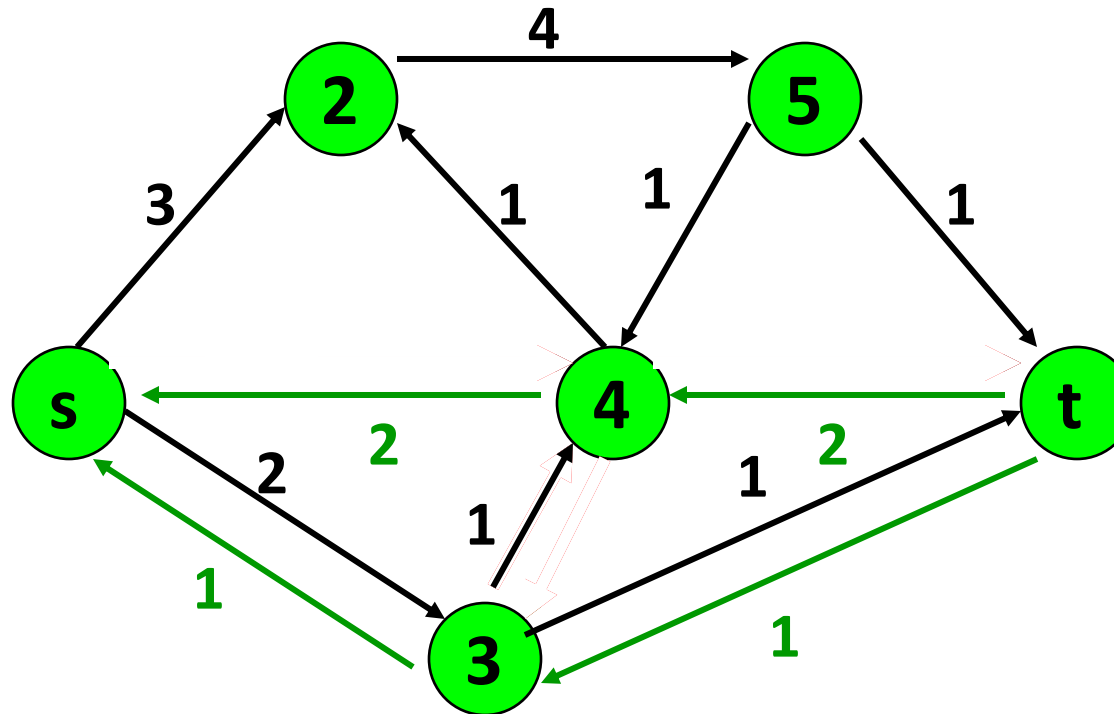
Determine the capacity Δ of the path.

Send Δ units of flow in the path.
Update residual capacities.

The Ford-Fulkerson Max Flow



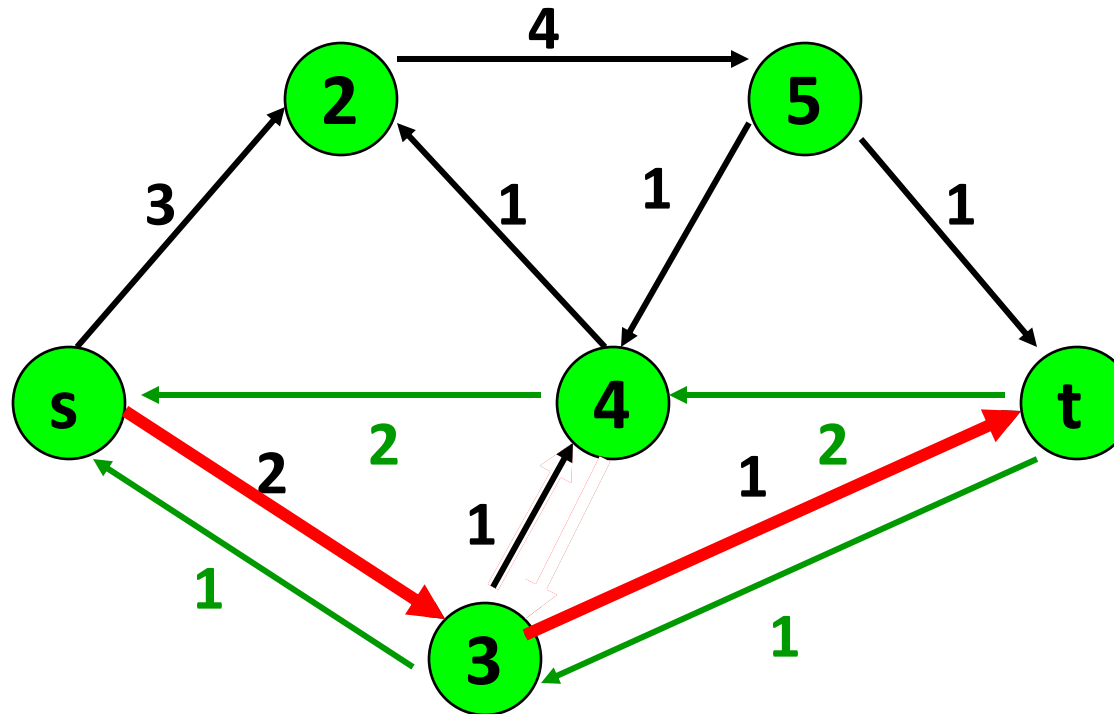
Find any s-t path



Determine the capacity Δ of the path.

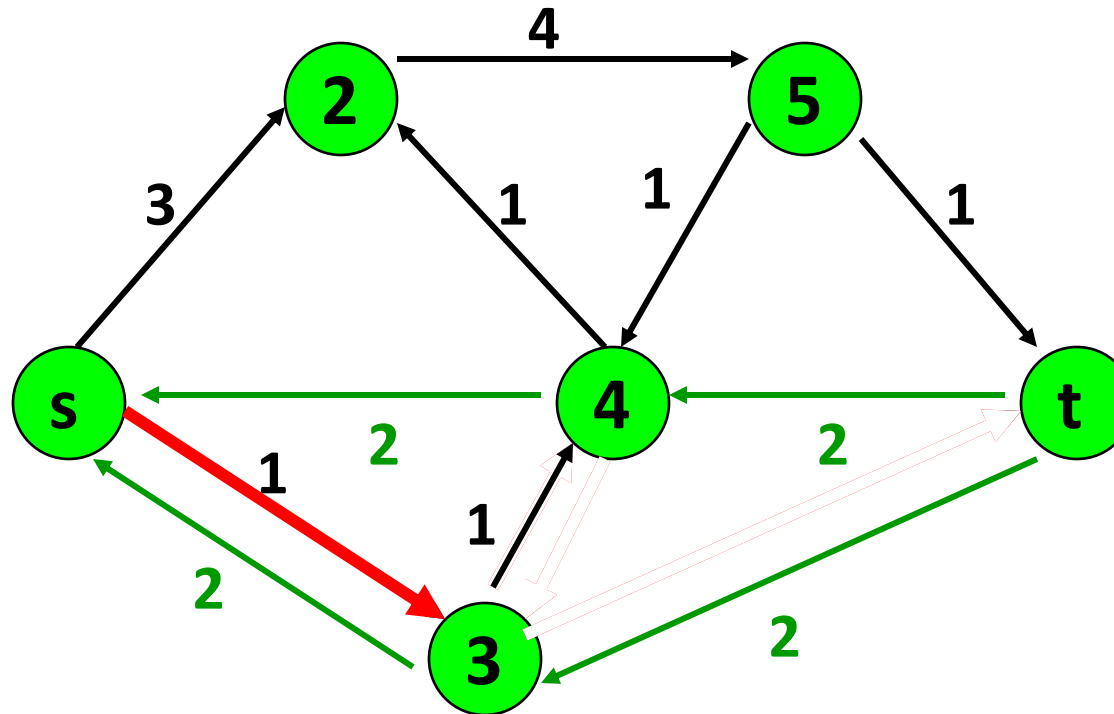
**Send Δ units of flow in the path.
Update residual capacities.**

The Ford-Fulkerson Max Flow



Find any s-t path

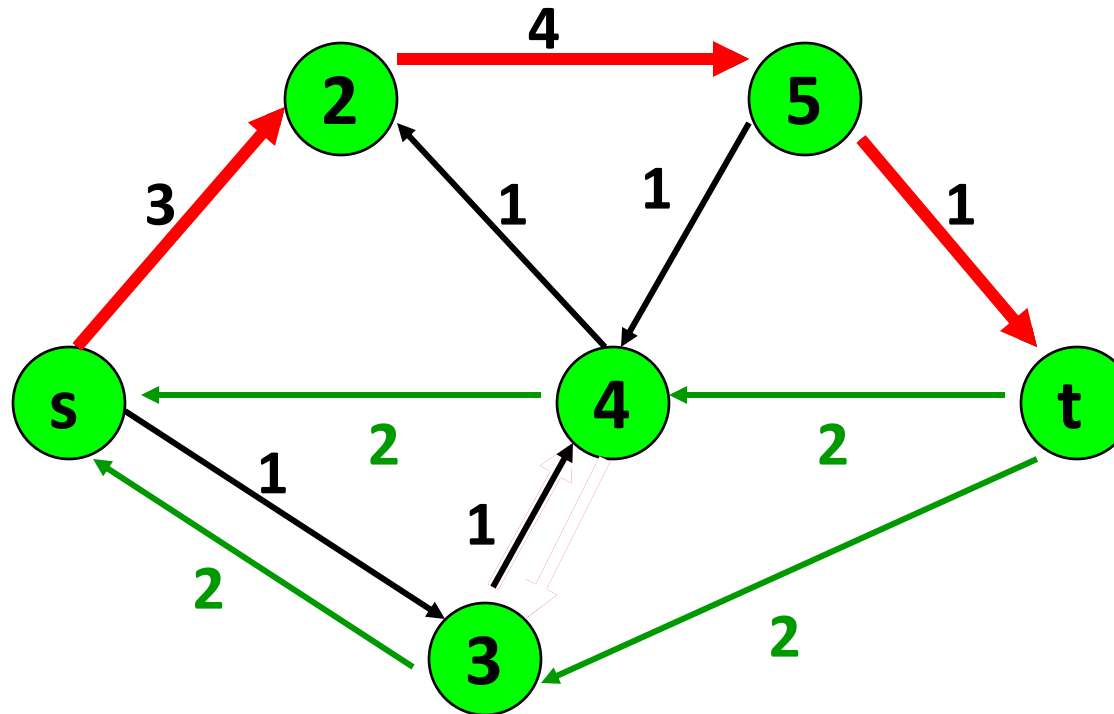
The Ford-Fulkerson Max Flow



Determine the capacity Δ of the path.

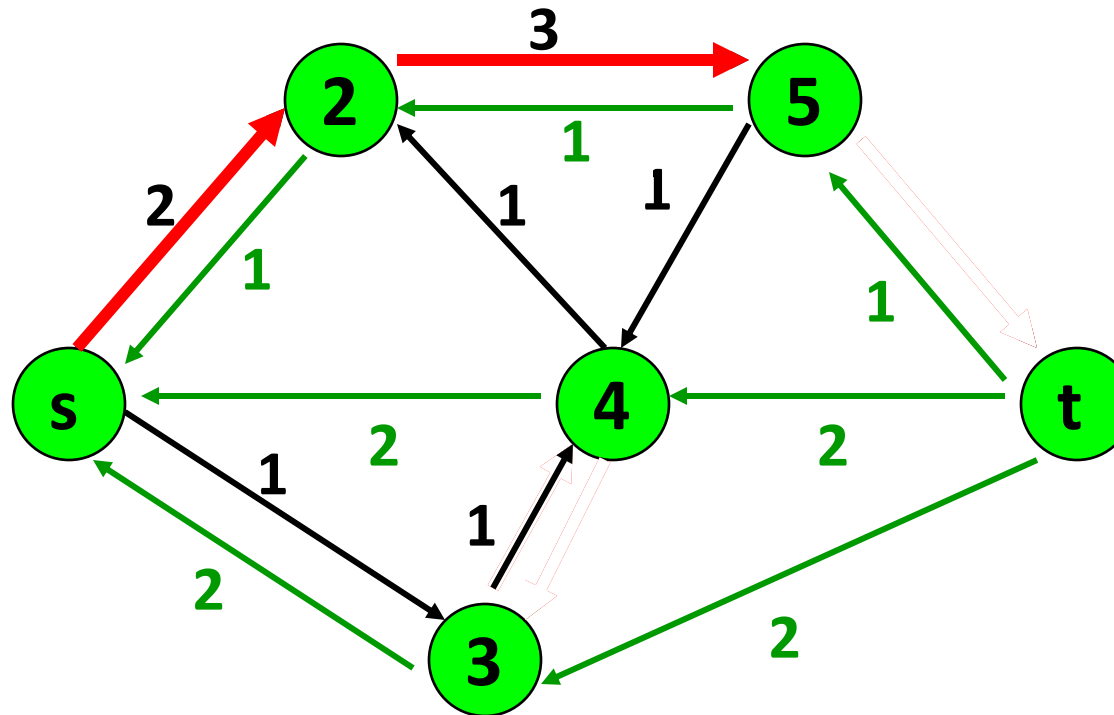
Send Δ units of flow in the path.
Update residual capacities.

The Ford-Fulkerson Max Flow



Find any s-t path

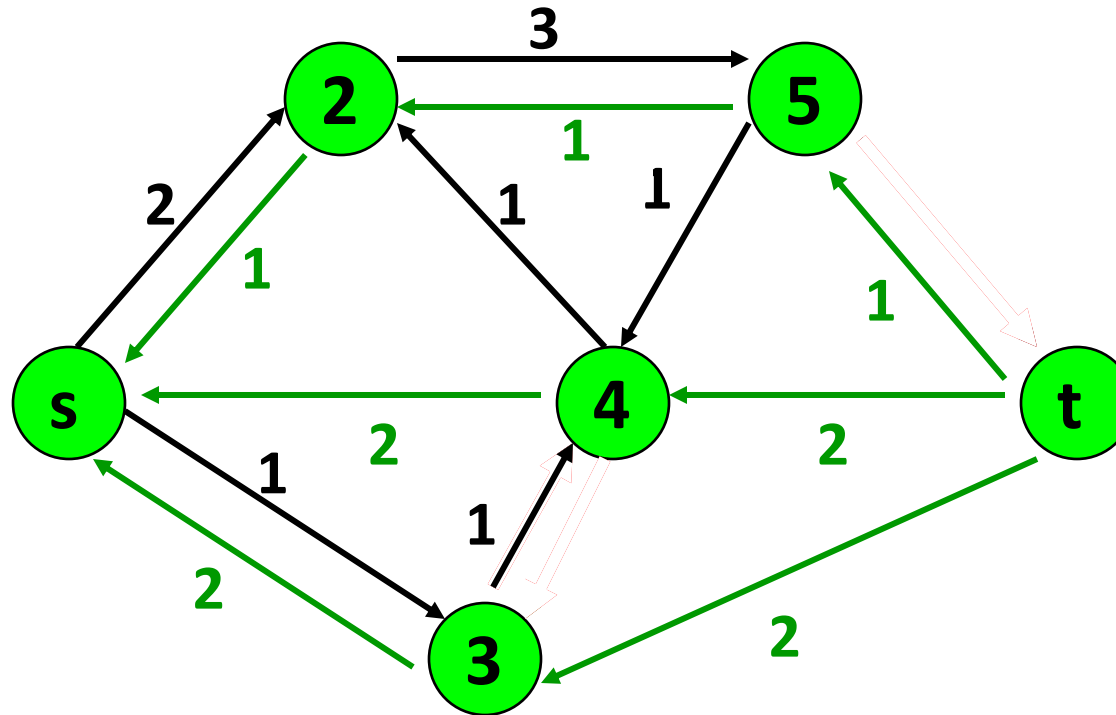
The Ford-Fulkerson Max Flow



Determine the capacity Δ of the path.

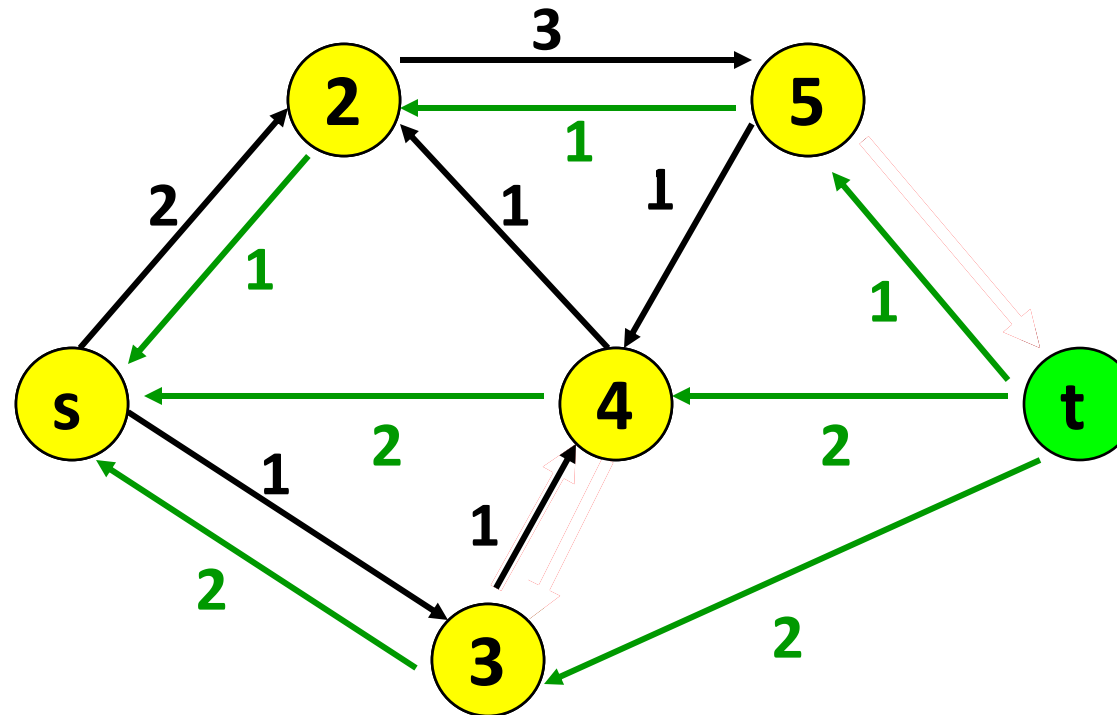
Send Δ units of flow in the path.
Update residual capacities.

The Ford-Fulkerson Max Flow



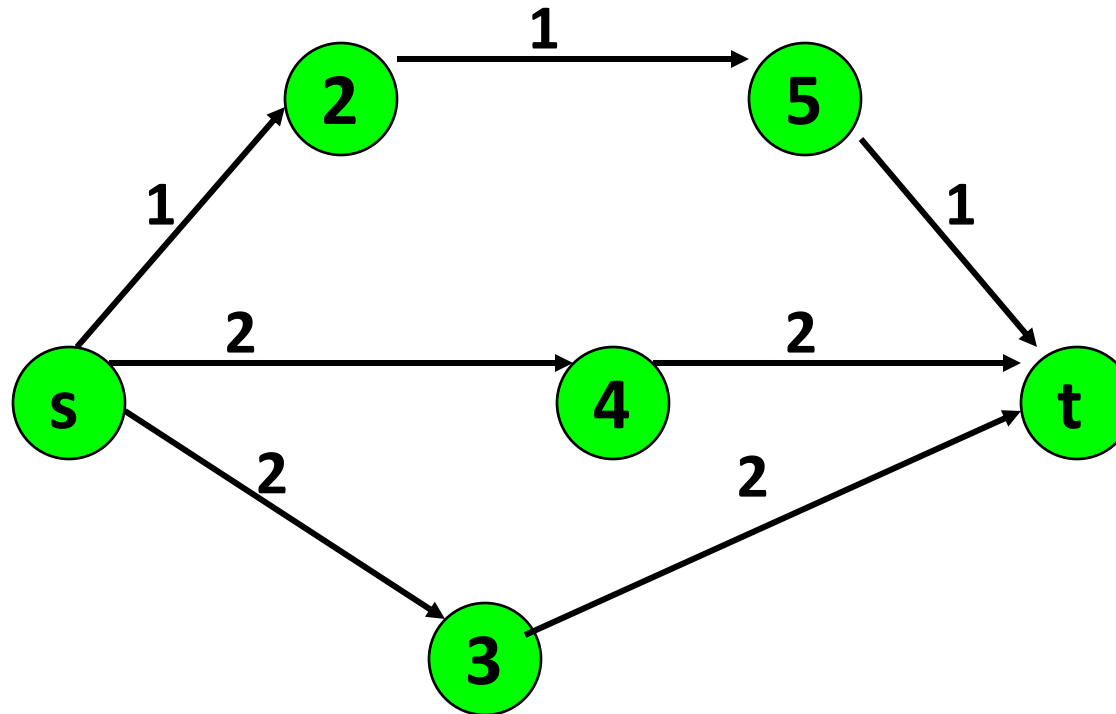
There is no s-t path in the residual network.
This flow is optimal

The Ford-Fulkerson Max Flow



These are the nodes that are reachable from node s .

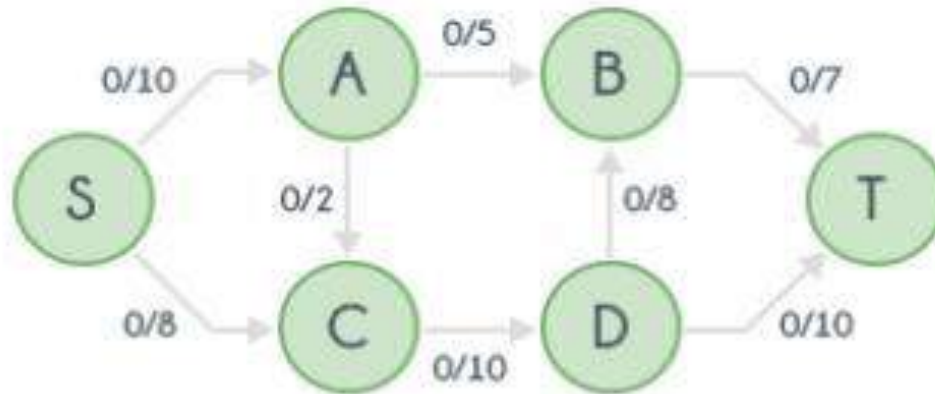
The Ford-Fulkerson Max Flow



Here is the optimal flow= $1+2+2 = 5$

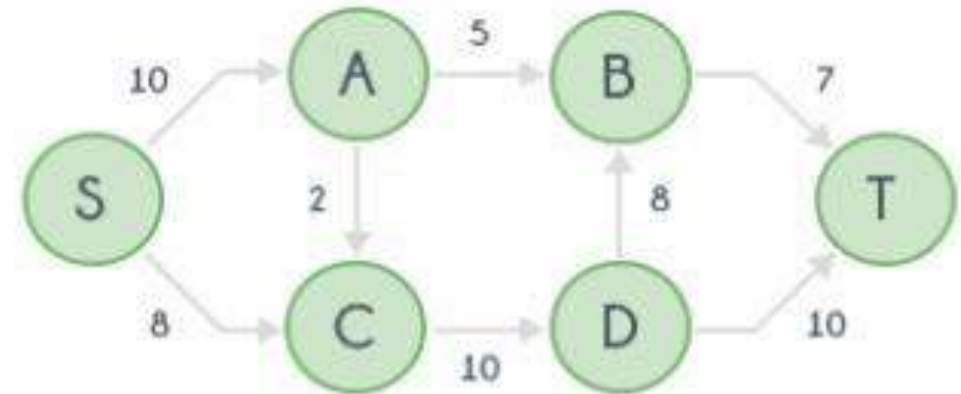
A demonstration of working of Ford-Fulkerson algorithm is shown below with the help of diagrams.

Network (G)

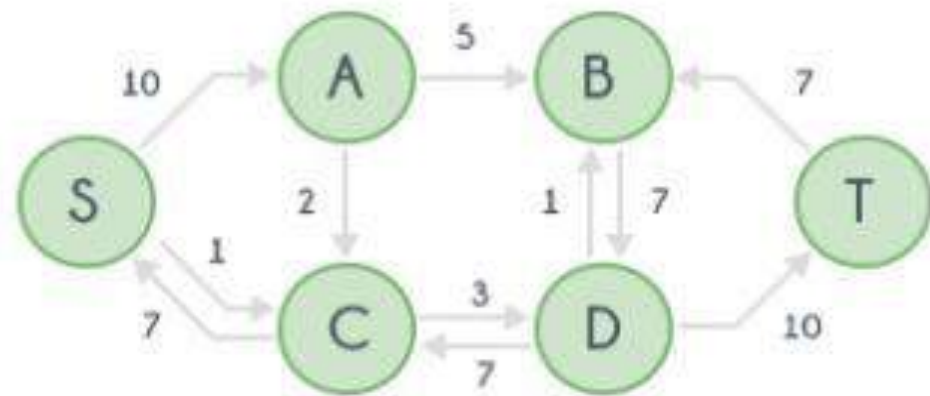
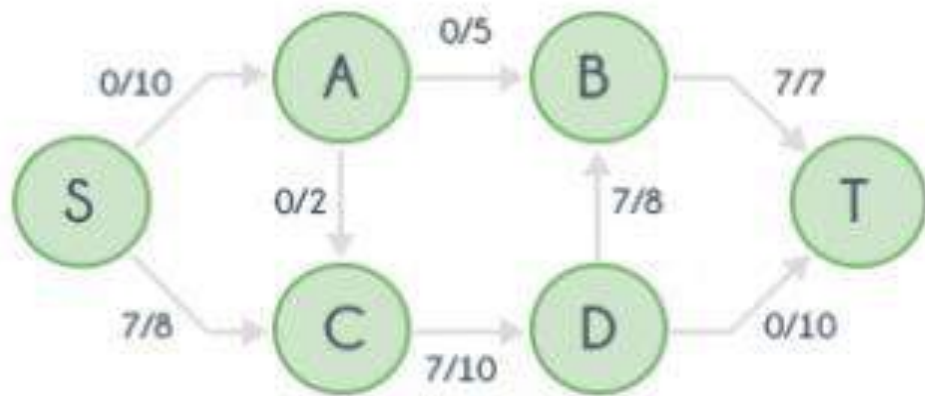


Flow = 0

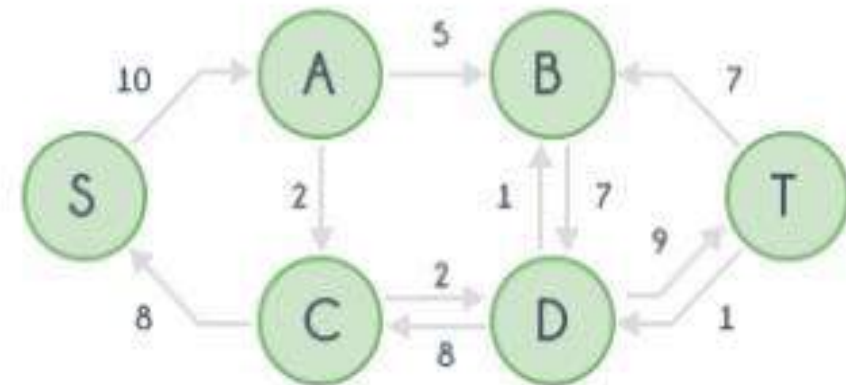
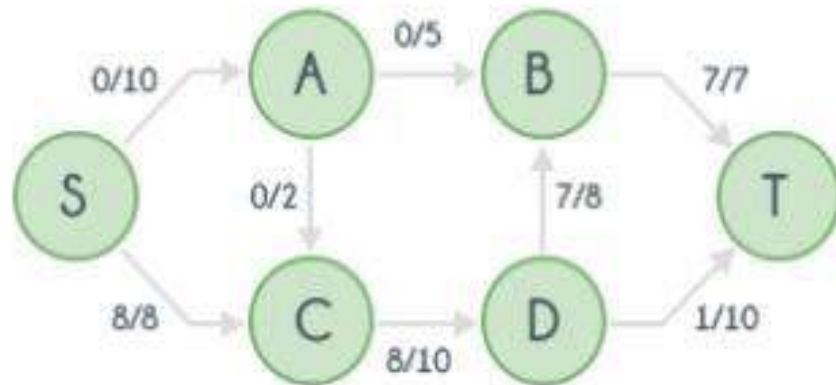
Residual Graph (G_R)



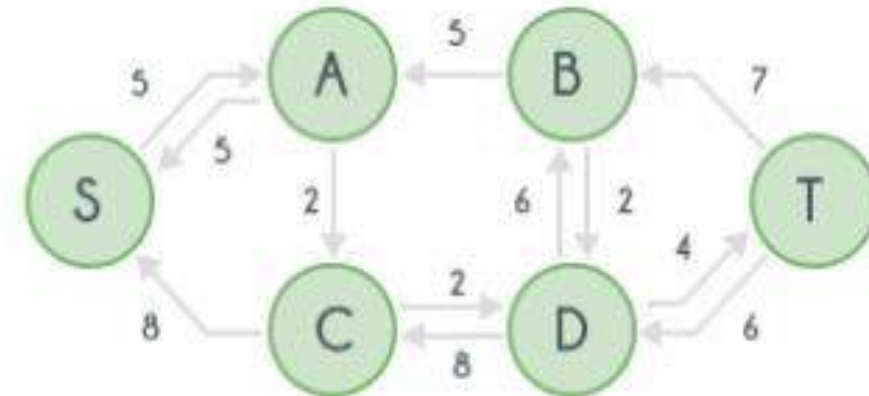
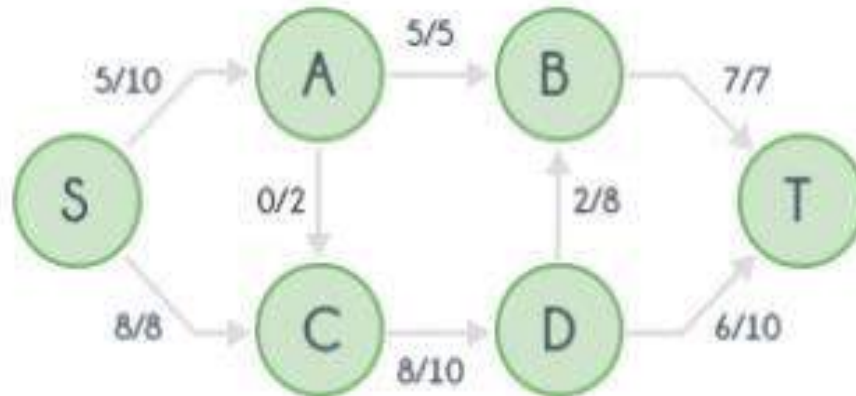
Path 1: S - C - D - B - T $\rightarrow$ Flow = Flow + 7



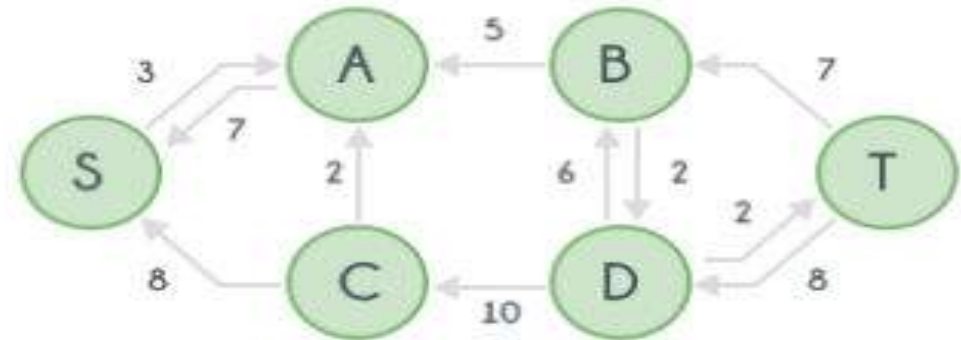
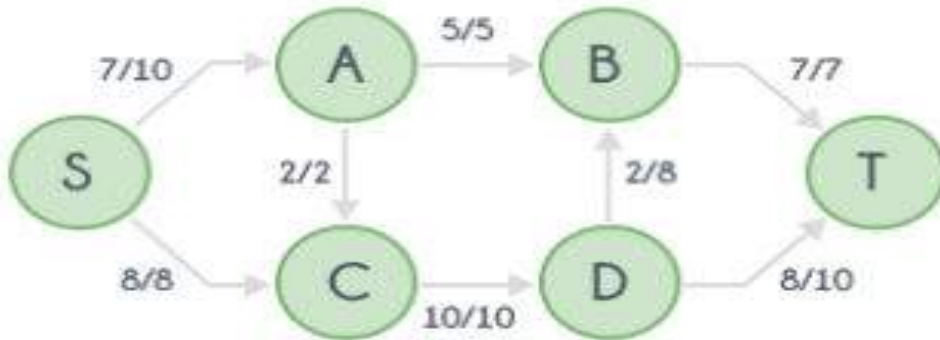
Path 2: S - C - D - T → Flow = Flow + 1



Path 3: S - A - B - T → Flow = Flow + 5



Path 4: S - A - C - D - T → Flow = Flow + 2



**No More Paths Left
Max Flow = 15**

- **Multiple algorithms exist in solving the maximum flow problem.**
- **Two major algorithms to solve this kind of problems are the Ford-Fulkerson algorithm and Dinic's Algorithm.**

SELF-ASSESSMENT QUESTIONS

- Which data structure is commonly used to find augmenting paths in the Ford-Fulkerson algorithm?

A. Stack

B. Queue

C. Binary Search Tree

D. Priority Queue

- What is the time complexity of the Ford-Fulkerson algorithm in the worst case if implemented with the breadth-first search (BFS)?

A. ($O(V + E)$)

B. ($O(V^2E)$)

C. ($O(VE^2)$)

D. ($O(V^3)$)

TERMINAL QUESTIONS

1. What is the primary objective of the Ford-Fulkerson algorithm?
2. What is the stopping criterion for the Ford-Fulkerson algorithm?
3. What does the Ford-Fulkerson algorithm use to find the augmenting path?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

- 1 Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professional.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

Edmonds-Karp Algoirthm

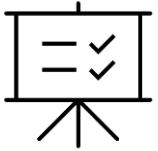
Session - 27

AIM OF THE SESSION



To familiarize students with the concept of the Network flow algorithms.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Edmonds-Karp Algorithm.
2. Describe :- Network flow using Edmonds-karp algorithm

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Edmonds-Karp Algorithm.
2. Describe :- Network flow using Edmonds-karp algorithm
3. Summarize:- Finding maximum flow in a network.

Edmonds-Karp Algorithm

- The Edmonds-Karp algorithm solves [the maximum flow problem](#) for a directed graph.
- The flow comes from a source vertex (ss) and ends up in a sink vertex (tt), and each edge in the graph allows a flow, limited by a capacity.
- The Edmonds-Karp algorithm is very similar to [the Ford-Fulkerson algorithm](#), except the Edmonds-Karp algorithm uses [Breadth First Search \(BFS\)](#) to find augmented paths to increase flow.

- The Edmonds-Karp algorithm works by using Breadth-First Search (BFS) to find a path with available capacity from the source to the sink (called an *augmented path*), and then sends as much flow as possible through that path.

How it works:

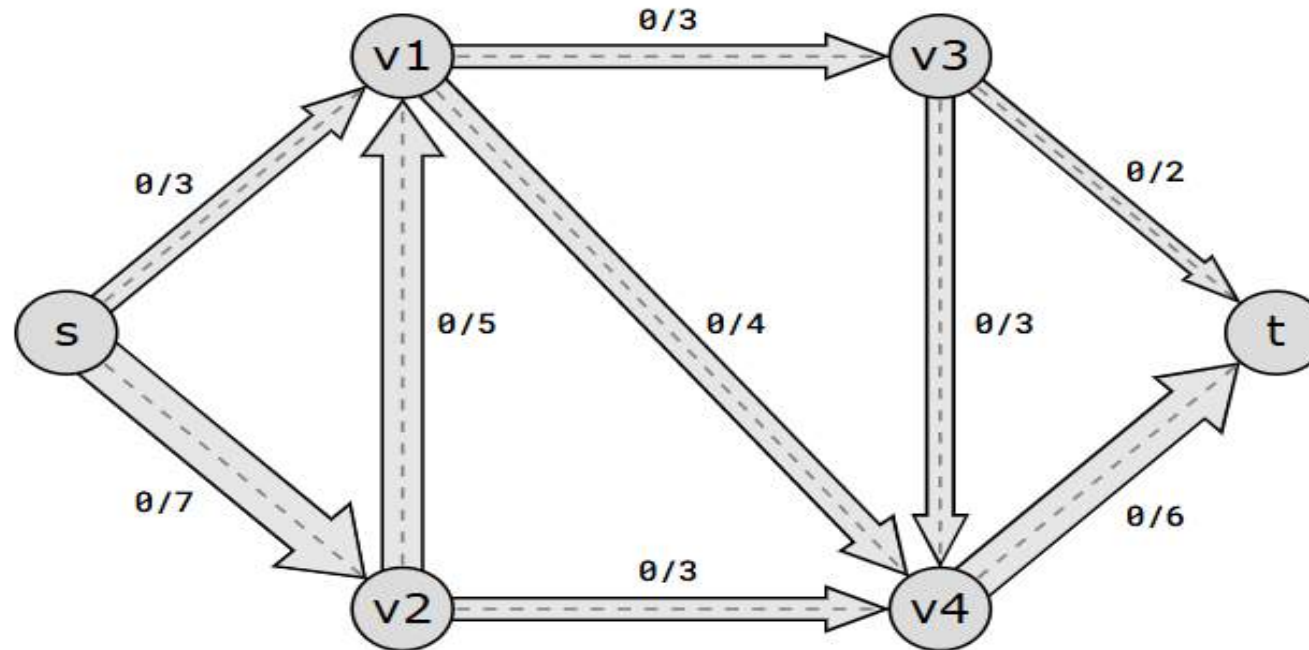
1. Start with zero flow on all edges.
2. Use BFS to find an *augmented path* where more flow can be sent.
3. Do a *bottleneck calculation* to find out how much flow can be sent through that augmented path.
4. Increase the flow found from the bottleneck calculation for each edge in the augmented path.
5. Repeat steps 2-4 until max flow is found. This happens when a new augmented path can no longer be found.

Residual Network in Edmonds-Karp

- The Edmonds-Karp algorithm works by creating and using something called a *residual network*, which is a representation of the original graph.
- In the residual network, every edge has a *residual capacity*, which is the original capacity of the edge, minus the the flow in that edge. The residual capacity can be seen as the leftover capacity in an edge with some flow.
- For example, if there is a flow of 2 in the $v_3 \rightarrow v_4$ edge, and the capacity is 3, the residual flow is 1 in that edge, because there is room for sending 1 more unit of flow through that edge.

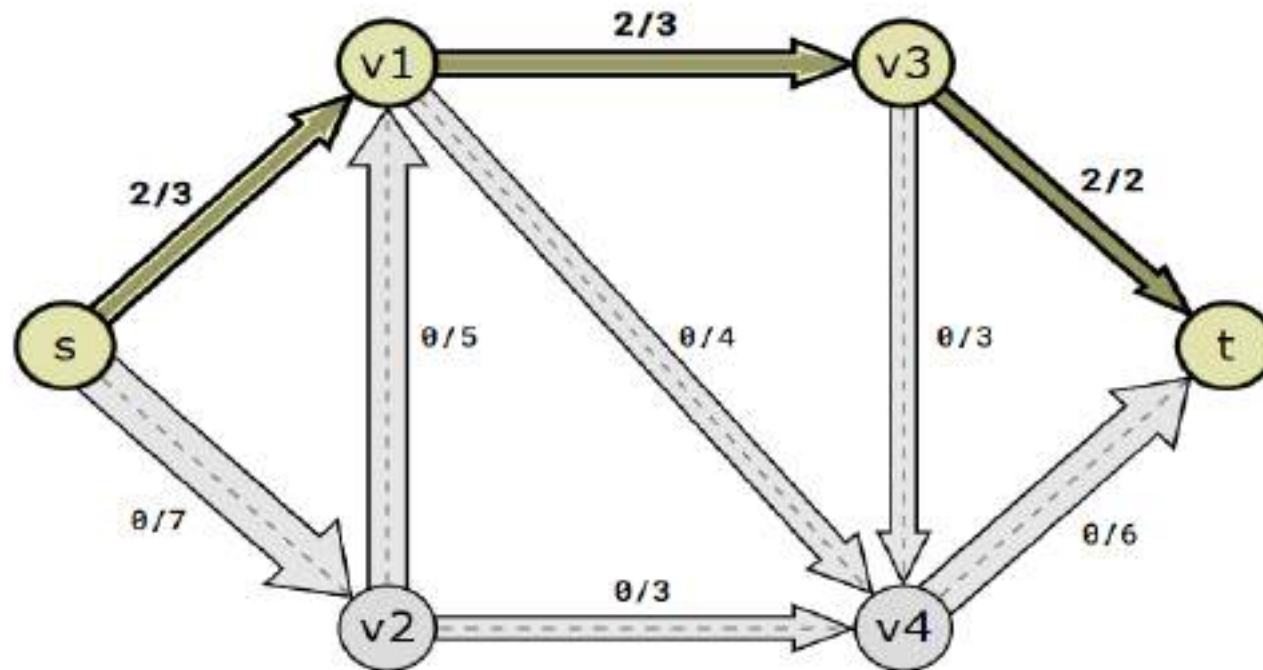
Reversed Edges in Edmonds-Karp

- The Edmonds-Karp algorithm also uses something called *reversed edges* to send flow back. This is useful to increase the total flow.
- To send flow back, in the opposite direction of the edge, a reverse edge is created for each original edge in the network. The Edmonds-Karp algorithm can then use these reverse edges to send flow in the reverse direction.
- A reversed edge has no flow or capacity, just residual capacity. The residual capacity for a reversed edge is always the same as the flow in the corresponding original edge.
- In our example, the edge $v1 \rightarrow v3$ has a flow of 2, which means there is a residual capacity of 2 on the corresponding reversed edge $v3 \rightarrow v1$.

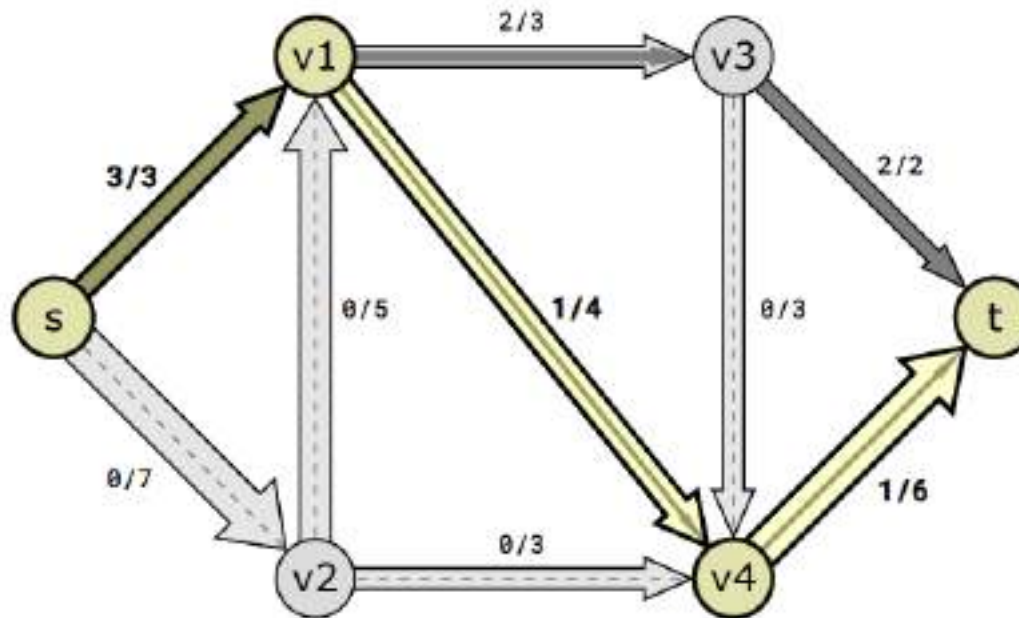


Max flow: 0

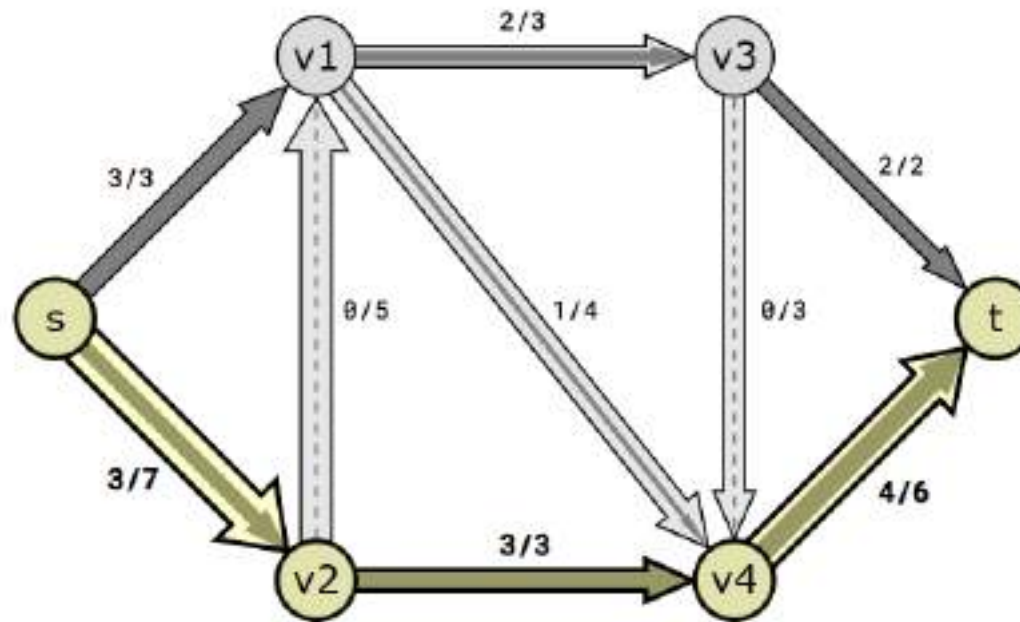
- The Edmonds-Karp algorithm starts with using Breadth-First Search to find an augmented path where flow can be increased, which is $s \rightarrow v1 \rightarrow v3 \rightarrow t$.
- After finding the augmented path, a bottleneck calculation is done to find how much flow can be sent through that path, and that flow is: 2.
- So a flow of 2 is sent over each edge in the augmented path.



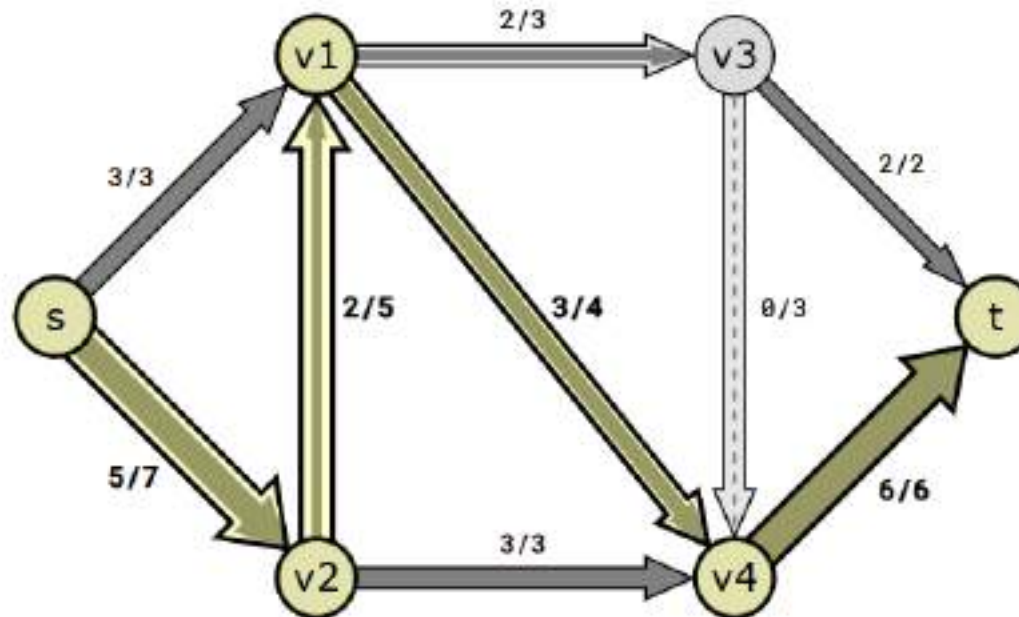
- The next augmented path is found to be $s \rightarrow v1 \rightarrow v4 \rightarrow t$.
- The flow can only be increased by 1 in this path because there is only room for one more unit of flow in the $s \rightarrow v1$ edge.



- The next augmented path is found to be $s \rightarrow v2 \rightarrow v4 \rightarrow t$.
- The flow can be increased by 3 in this path. The bottleneck (limiting edge) is $v2 \rightarrow v4$ because the capacity is 3.



- The last augmented path found is $s \rightarrow v2 \rightarrow v1 \rightarrow v4 \rightarrow t$.
- The flow can only be increased by 2 in this path because of edge $v4 \rightarrow t$ being the bottleneck in this path with only space for 2 more units of flow (capacity–flow=1).



- At this point, a new augmenting path cannot be found (it is not possible to find a path where more flow can be sent through from s to t), which means the max flow has been found, and the Edmonds-Karp algorithm is finished.
- The maximum flow is 8. As you can see in the image above, the flow (8) is the same going out of the source vertex ss , as the flow going into the sink vertex tt .

- The Edmonds-Karp algorithm computes the maximum flow in a network by using the Ford-Fulkerson method with BFS to find the shortest augmenting paths. It updates the flow and residual graph iteratively, augmenting flow along the path with the smallest residual capacity. The algorithm has a time complexity of $O(VE^2)$

SELF-ASSESSMENT QUESTIONS

What happens when no augmenting paths are found in the residual graph?

- A. The algorithm continues searching for alternative paths.
- B. The algorithm terminates, and the maximum flow is achieved.**
- C. The algorithm retries with a different source.
- D. The algorithm reduces the flow value and retries.

What does the Edmonds-Karp algorithm use to find augmenting paths?

- A. Depth-First Search (DFS)
- B. Breadth-First Search (BFS)**
- C. Dijkstra's Algorithm
- D. Prim's Algorithm

TERMINAL QUESTIONS

1. Compare the Edmonds-Karp algorithm with the basic Ford-Fulkerson method.
2. Discuss the time complexity of the Edmonds-Karp algorithm.
3. What are the real-world applications of the Edmonds-Karp algorithm?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

- 1 Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professional.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

Max Flow Min Cut Theorem

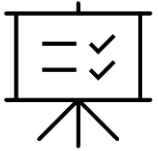
Session - 28

AIM OF THE SESSION



To familiarize students with the concept of the Max Flow Min Cut Theorem.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Max Flow Min Cut Theorem.
2. Describe :- Theorem.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Max Flow Min Cut Theorem.
2. Describe :- Theorem
3. Summarize:- The **maximum value of the flow** from a source to a sink is equal to the **minimum capacity of a cut** that separates the source and the sink..

Maximum Flow

It is defined as the maximum amount of flow that the network would allow to flow from source to sink.

Multiple algorithms exist in solving the maximum flow problem.

Two major algorithms to solve these kind of problems are Ford-Fulkerson algorithm and Dinic's Algorithm.

Max Flow Min Cut Theorem

Theorem: (Optimality conditions for max flows). The following are equivalent.

1. A flow x is maximum.
2. There is no augmenting path in $G(x)$.
3. There is an s - t cutset (S, T) whose capacity is the flow value of x .

Corollary: (Max-flow Min-Cut). The maximum flow value is the minimum value of a cut.

Proof of Theorem:

$1 \Rightarrow 2$. (not $2 \Rightarrow$ not 1)

Suppose that there is an augmenting path in $G(x)$. Then x is not maximum.

Max Flow Min Cut Theorem

Contd.

3 $\Rightarrow$ 1.

Let $v = F_x(S, T)$ be the flow from s to t . By assumption, $v = \text{CAP}(S, T)$. By weak duality, the maximum flow is at most $\text{CAP}(S, T)$. Thus the flow is maximum.

2 $\Rightarrow$ 3.

Suppose there is no augmenting path in $G(x)$.

Claim:

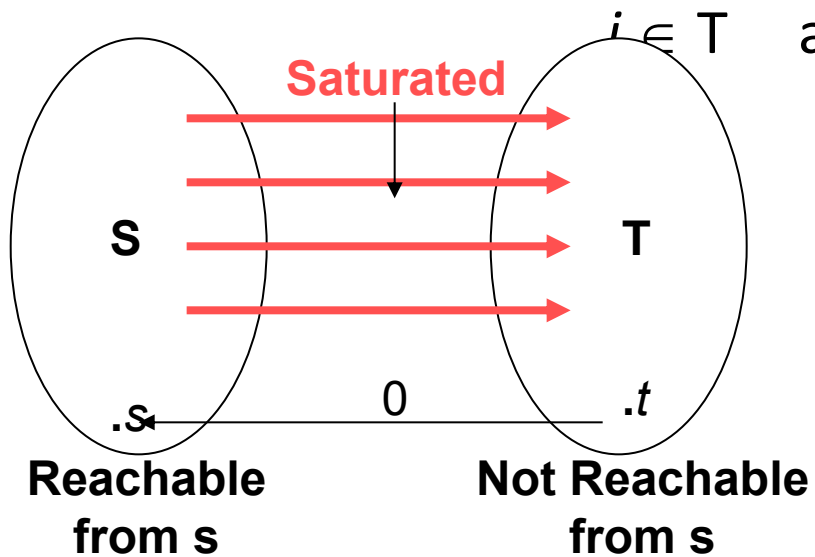
Let S be the set of nodes reachable from s in $G(x)$.

Let $T = N \setminus S$. Then there is no arc in $G(x)$ from S to T .

Max Flow Min Cut Theorem Contd.

Thus $i \in S$ and $j \in T \Rightarrow f(i, j) = c(i, j)$

$i \in T$ and $j \in S \Rightarrow f(i, j) = 0.$



There is no arc from S to T in $G(x)$

It follows that

$$F_x(S, T) = \sum_{i \in S} \sum_{j \in T} f(i, j) - \sum_{i \in S} \sum_{j \in T} f(j, i)$$

$$= \sum_{i \in S} \sum_{j \in T} c(i, j) - \sum_{i \in S} \sum_{j \in T} 0 = CAP(S, T)$$

- The maximum value of the flow in a flow network from a source s to a sink t is equal to the capacity of the minimum cut that separates the source and sink.

SELF-ASSESSMENT QUESTIONS

- What does the Max-Flow Min-Cut Theorem state?

- A. The minimum flow in a network is equal to the maximum cut capacity.
- B. The maximum flow in a network is always equal to the sum of edge capacities.
- C. **The maximum flow in a network is equal to the capacity of the minimum cut.**
- D. The maximum flow is always greater than the minimum cut.

- Which algorithm is commonly used to find the maximum flow in a network?

- A. Dijkstra's Algorithm
- B. Kruskal's Algorithm
- C. **Ford-Fulkerson Algorithm**
- D. Bellman-Ford Algorithm

TERMINAL QUESTIONS

1. What is a flow network, and how is the concept of a cut defined in the context of the Max-Flow Min-Cut Theorem? Provide an example of how a minimum cut is found in a simple network.

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Maximum flow - Ford-Fulkerson and Edmonds-Karp

Maximum flow - Ford-Fulkerson and Edmonds-Karp

It was developed by L. R. Ford, Jr. and D. R. Fulkerson in 1956.

The Edmonds-Karp algorithm is an implementation of the Ford-Fulkerson method for computing a maximal flow in a flow network.

Flow network

First let's define what a **flow network**, a **flow**, and a **maximum flow** is.

A **network** is a directed graph G with vertices V and edges E combined with a function c , which assigns each edge $e \in E$ a non-negative integer value, the **capacity** of e . Such a network is called a **flow network**, if we additionally label two vertices, one as **source** and one as **sink**.

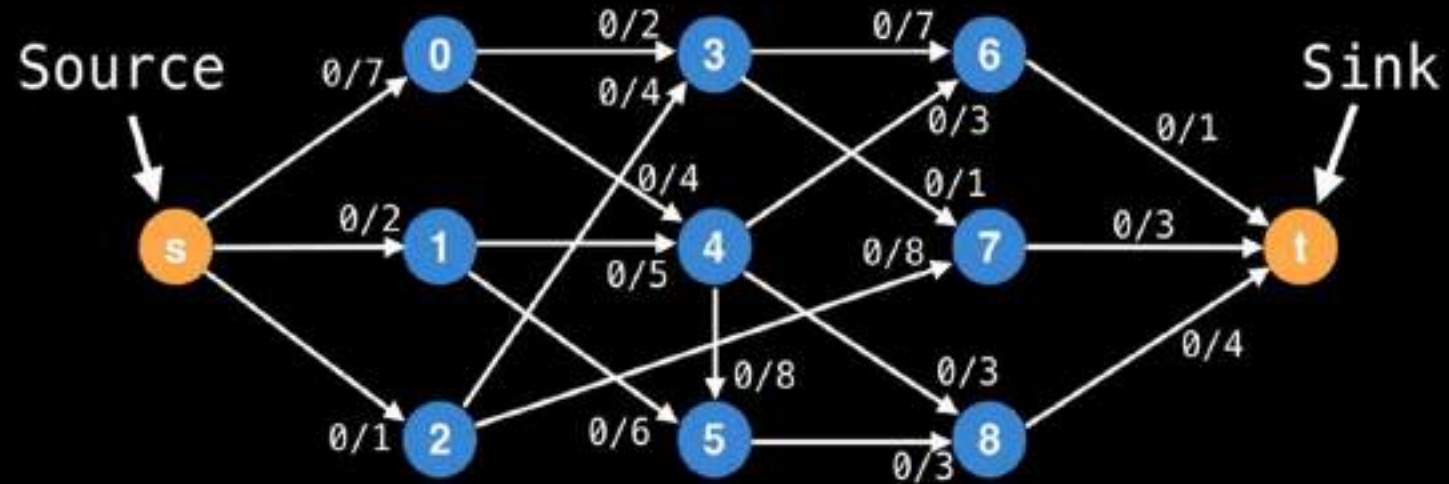
A **flow** in a flow network is function f , that again assigns each edge e a non-negative integer value, namely the flow. The function has to fulfill the following two conditions:

The flow of an edge cannot exceed the capacity.

$$f(e) \leq c(e)$$

Max flow

Q: With an infinite input source, how much “flow” can we push through the network given that each edge has a certain capacity?



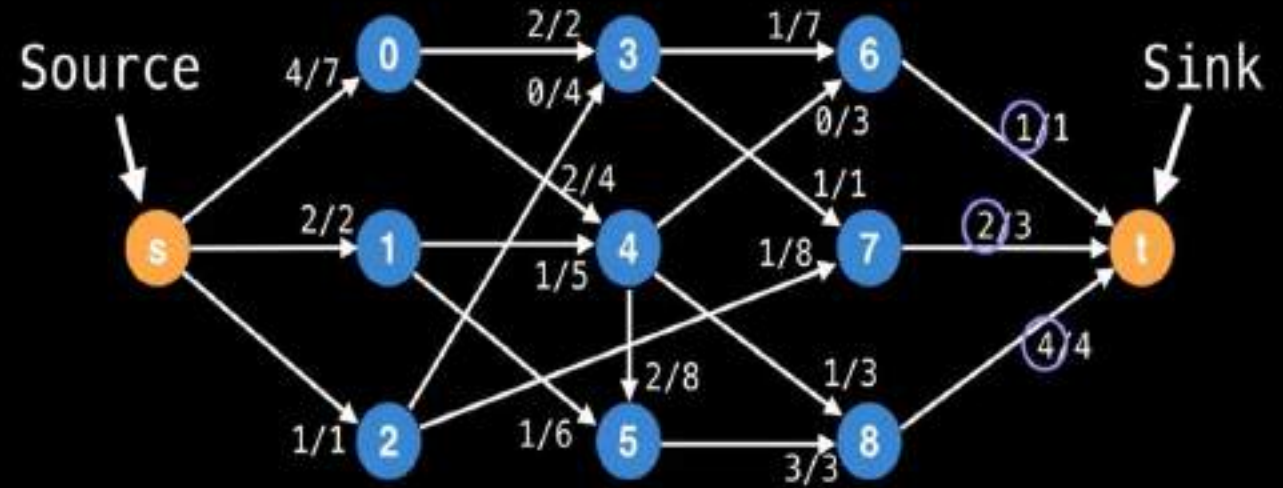
Suppose the edges are roads with cars, pipes with water or wires with electric current. Flow represents the volume of water allowed to flow through the pipes, the number of cars the roads can sustain in traffic and net electric current. Effectively, it's the “bottleneck” value for the amount of flow that can pass through the network from source to sink under all the constraints.

A **flow graph** (flow network) is a directed graph where each edge (also called an arc) has a certain **capacity** which can receive a certain amount of **flow**. The flow running through an edge must be less than or equal to the capacity.

Each edge in the flow graph has a certain flow and capacity specified by the fraction adjacent to each edge. Initially, the flow through each edge is 0 and the capacity is a non negative value.

Max flow

Q: With an infinite input source, how much "flow" can we push through the network given that each edge has a certain capacity?



In this flow graph, the max flow is 7 since $1+2+4 = 7$ is the sum of the flows entering the sink node. That's the most that we can push through the network.



And the sum of the incoming flow of a vertex u has to be equal to the sum of the outgoing flow of u except in the source and sink vertices.

$$\sum_{(v,u) \in E} f((v,u)) = \sum_{(u,v) \in E} f((u,v))$$

The source vertex s only has an outgoing flow, and the sink vertex t has only incoming flow.

It is easy to see that the following equation holds:

$$\sum_{(s,u) \in E} f((s,u)) = \sum_{(u,t) \in E} f((u,t))$$

A good analogy for a flow network is the following visualization: We represent edges as water pipes, the capacity of an edge is the maximal amount of water that can flow through the pipe per second, and the flow of an edge is the amount of water that currently flows through the pipe per second. This motivates the first flow condition. There cannot flow more water through a pipe than its capacity. The vertices act as junctions, where water comes out of some pipes, and then, these vertices distribute the water in some way to other pipes. This also motivates the second flow condition. All the incoming water has to be distributed to the other pipes in each junction. It cannot magically disappear or appear. The source s is origin of all the water, and the water can only drain in the sink t .

THE VALUE OF A FLOW.

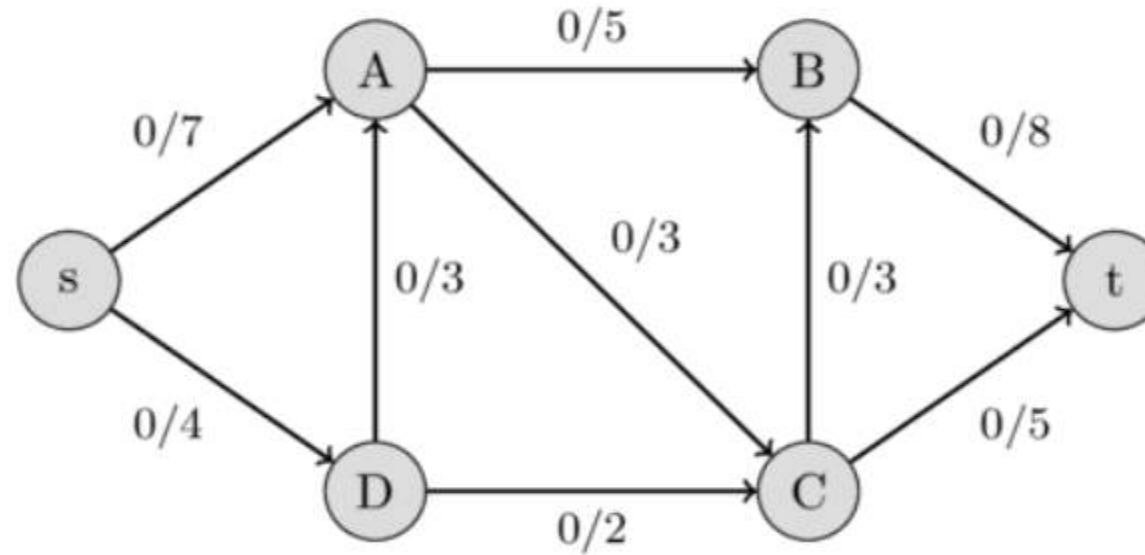
The value of a flow is given by :

$$|f| = \sum_{v \in V} f(s, v) = \sum_{v \in V} f(v, t)$$

The flow into the node is same as flow going out from the node and thus the flow is conserved. Also the total amount of flow from source s = total amount of flow into the sink t .



The following image shows a flow network. The first value of each edge represents the flow, which is initially 0, and the second value represents the capacity.



The value of the flow of a network is the sum of all the flows that get produced in the source s , or equivalently to the sum of all the flows that are consumed by the sink t . A **maximal flow** is a flow with the maximal possible value. Finding this maximal flow of a flow network is the problem that we want to solve.

In the visualization with water pipes, the problem can be formulated in the following way: how much water can we push through the pipes from the source to the sink?

Ford-Fulkerson method

Let's define one more thing. A **residual capacity** of a directed edge is the capacity minus the flow. It should be noted that if there is a flow along some directed edge (u, v) , then the reversed edge has capacity 0 and we can define the flow of it as $f((v, u)) = -f((u, v))$. This also defines the residual capacity for all the reversed edges. We can create a **residual network** from all these edges, which is just a network with the same vertices and edges, but we use the residual capacities as capacities.

The Ford-Fulkerson method works as follows. First, we set the flow of each edge to zero. Then we look for an **augmenting path** from s to t . An augmenting path is a simple path in the residual graph where residual capacity is positive for all the edges along that path. If such a path is found, then we can increase the flow along these edges. We keep on searching for augmenting paths and increasing the flow. Once an augmenting path doesn't exist anymore, the flow is maximal.

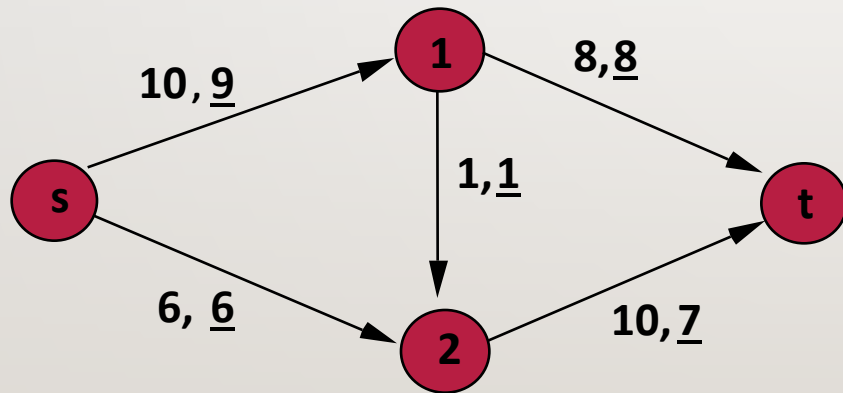
Let us specify in more detail, what increasing the flow along an augmenting path means. Let C be the smallest residual capacity of the edges in the path. Then we increase the flow in the following way: we update $f((u, v)) += C$ and $f((v, u)) -= C$ for every edge (u, v) in the path.

Here is an example to demonstrate the method. We use the same flow network as above. Initially we start with a flow of 0.

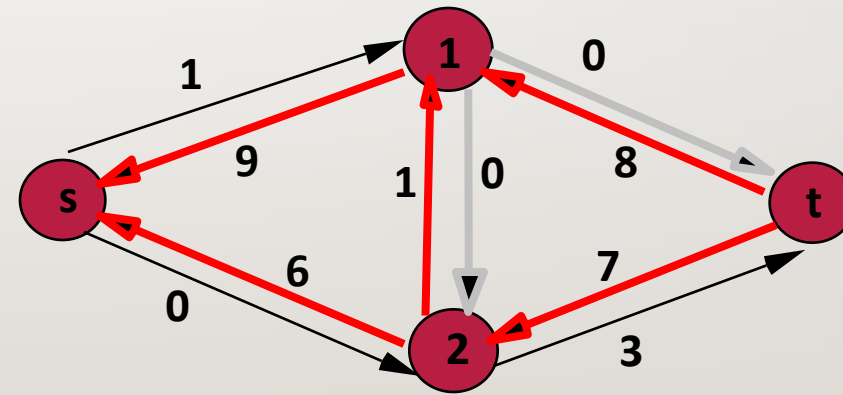
THE FORD-FULKERSON METHOD

- A residual network graph indicates how much more flow is allowed in each edge in the network graph.
- The residual capacity of the network with a flow f is given by:
 The residual capacity (rc) of an edge (i,j) equals $c(i,j) - f(i,j)$ when (i,j) is a forward edge, and equals $f(i,j)$ when (i,j) is a backward edge. Moreover the residual capacity of an edge is always non-negative

$$c_f(u, v) = c(u, v) - f(u, v)$$



Original Network



Residual Network

FORD-FULKERSON MAX FLOW

In graph theory, a flow network is defined as a directed graph involving a **source(S)** and a **sink(T)** and several other nodes connected with edges. **Each edge has an individual capacity which is the maximum limit of flow that edge could allow.**

Flow in the network should follow the following conditions:

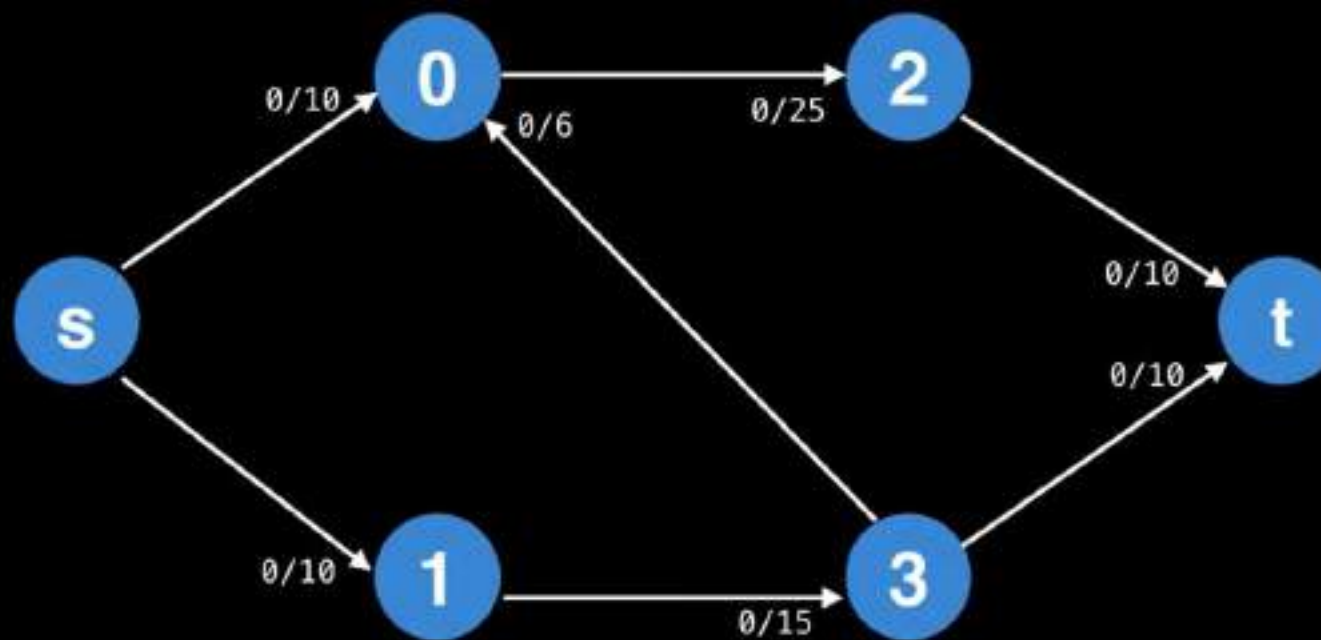
- For any non-source and non-sink node, the input flow is equal to output flow.
- For any edge(E_i) in the network, $0 \leq \text{flow}(E_i) \leq \text{Capacity}(E_i)$.
- Total flow out of the source node is equal total to flow in to the sink node.
- Net flow in the edges follows skew symmetry i.e. $F(u,v) = -F(v,u)$ where $F(u,v)$ is flow from node u to node v . This leads to a conclusion where you have to sum up all the flows between two nodes(either directions) to find net flow between the nodes initially.

FORD-FULKERSON MAX FLOW

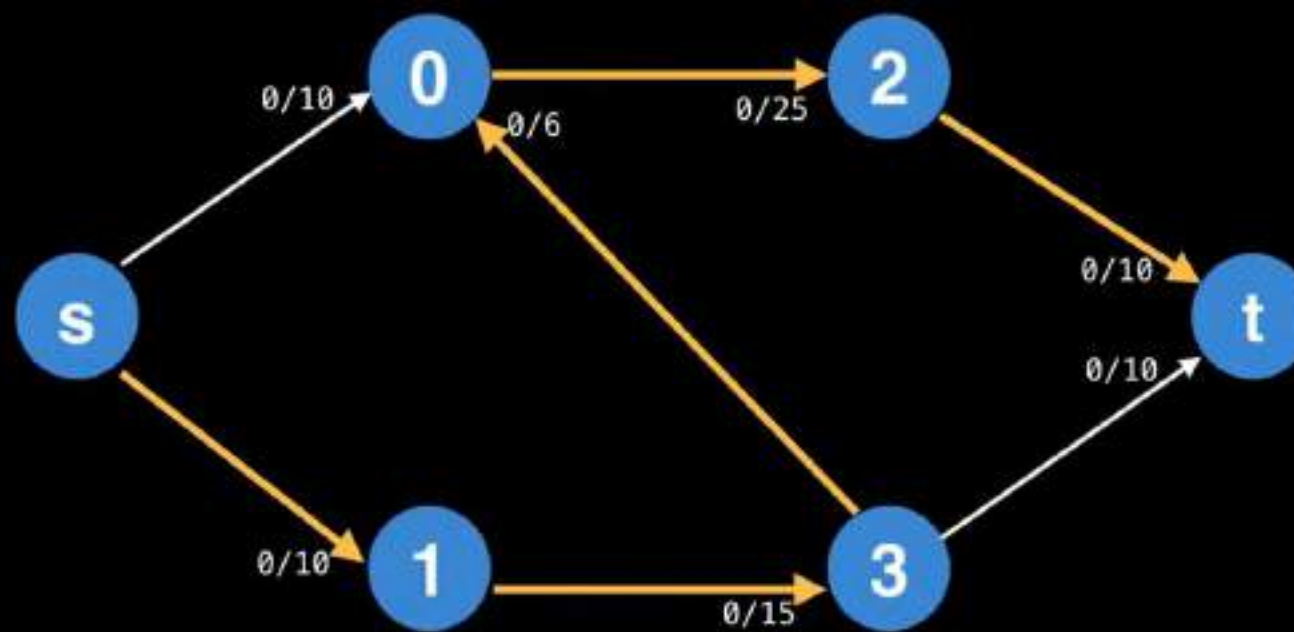
- **Maximum Flow:**

It is defined as the maximum amount of flow that the network would allow to flow from source to sink.

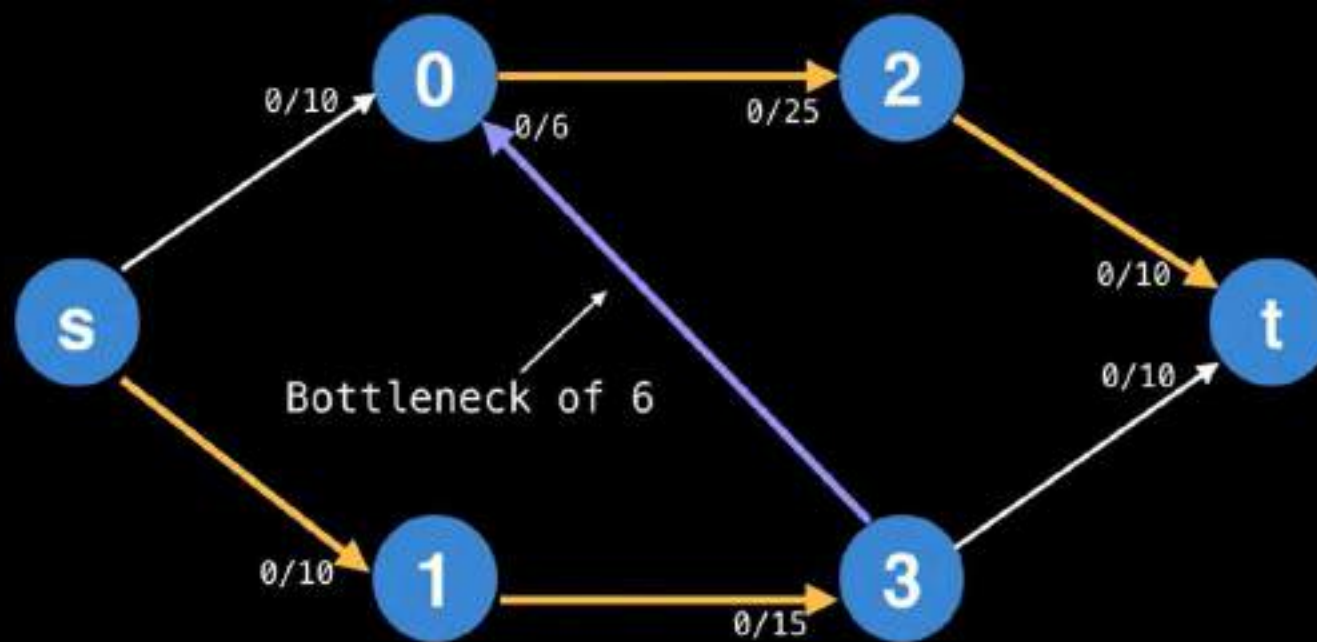
- **Ford-Fulkerson Algorithm:** Developed by L. R. Ford, Jr. and D. R. Fulkerson in 1956.
- **An augmenting path is a simple path from source to sink which do not include any cycles and that pass only through positive weighted edges.**
- **A residual network graph indicates how much more flow is allowed in each edge in the network graph.**



To find the maximum flow (and min-cut as a by product), the Ford-Fulkerson method repeatedly finds **augmenting paths** through the **residual graph** and **augments the flow** until no more augmenting paths can be found.



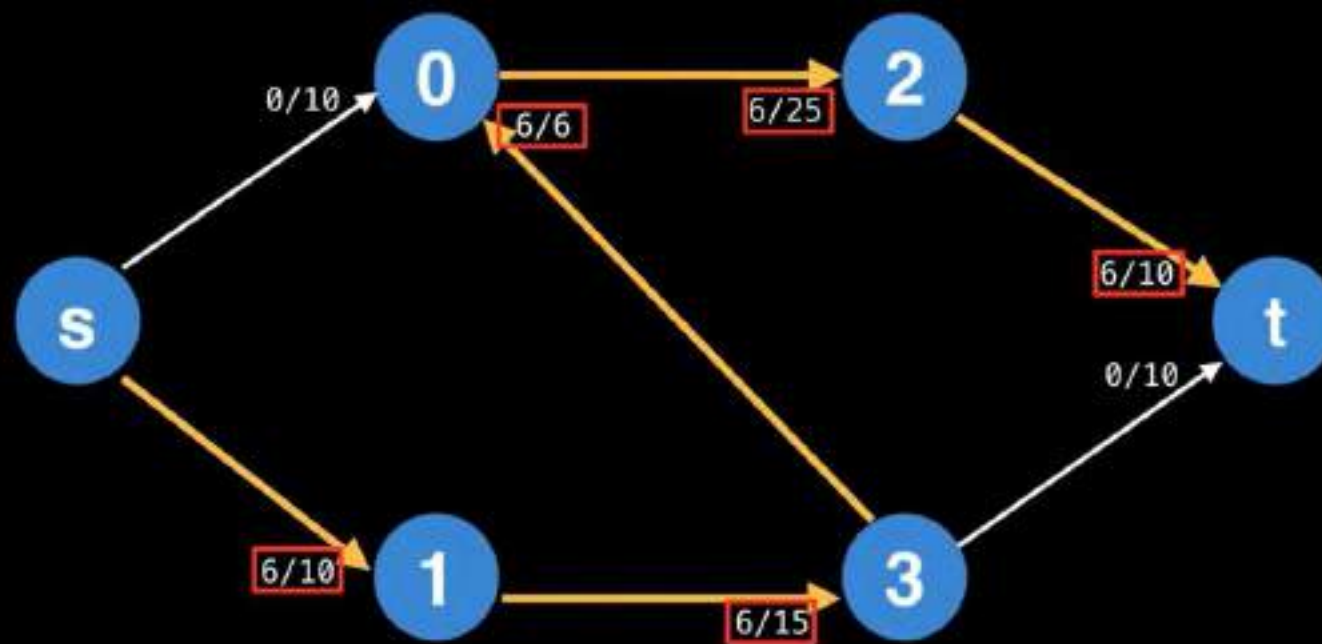
An **augmenting path** is a path of edges in the residual graph with unused capacity greater than zero from the source s to the sink t .



In the augmenting path above, the **bottleneck** is the “smallest” edge on the path. We can use the bottleneck value to **augment the flow** along the path.

The smallest remaining capacity of any edge along the augmenting path is:

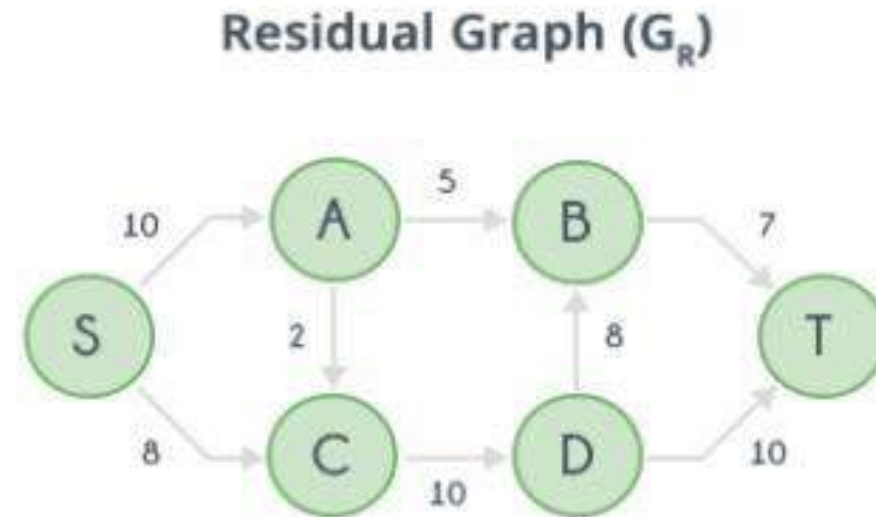
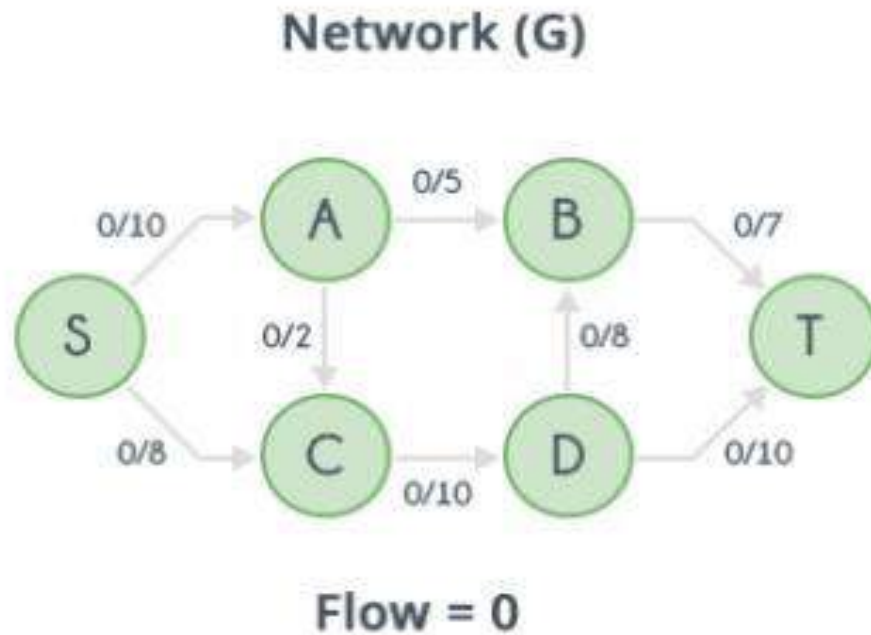
$$\min(10-0, 15-0, 6-0, 25-0, 10-0) = \min(10, 15, 6, 25, 10) = 6$$



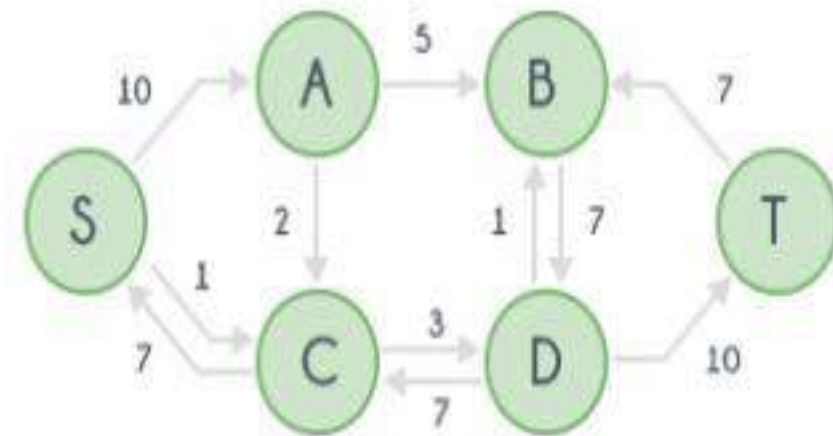
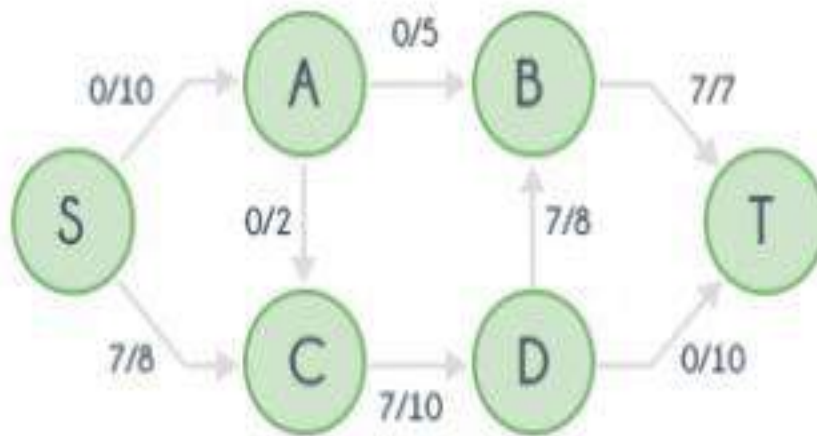
Augmenting the flow means *updating* the flow values of the edges along the augmenting path.

For forward edges, this means **increasing the flow** by the bottleneck value.

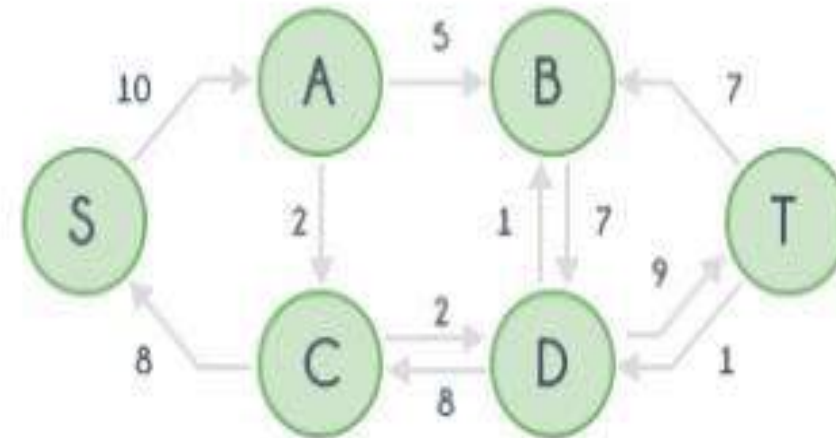
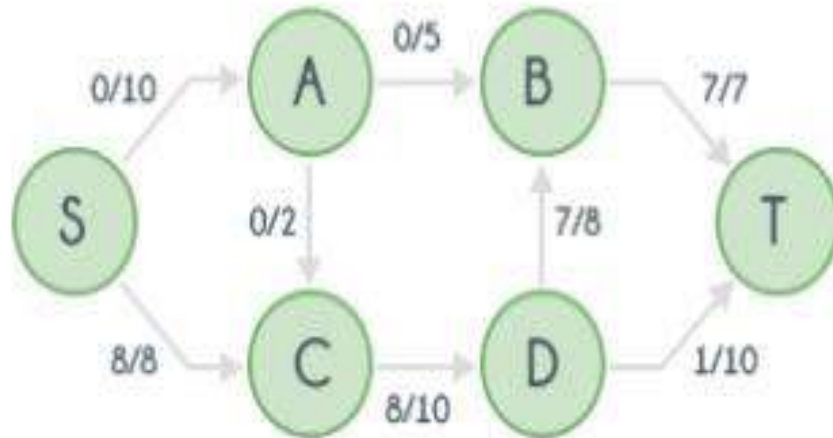
A demonstration of working of Ford-Fulkerson algorithm is shown below with the help of diagrams.



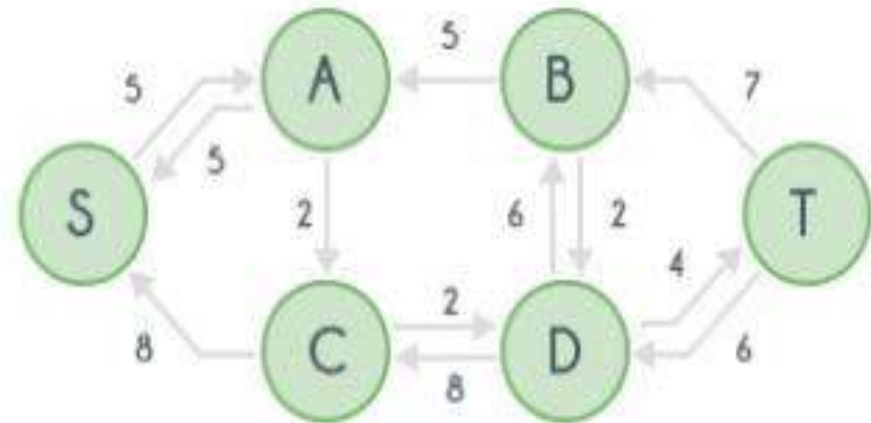
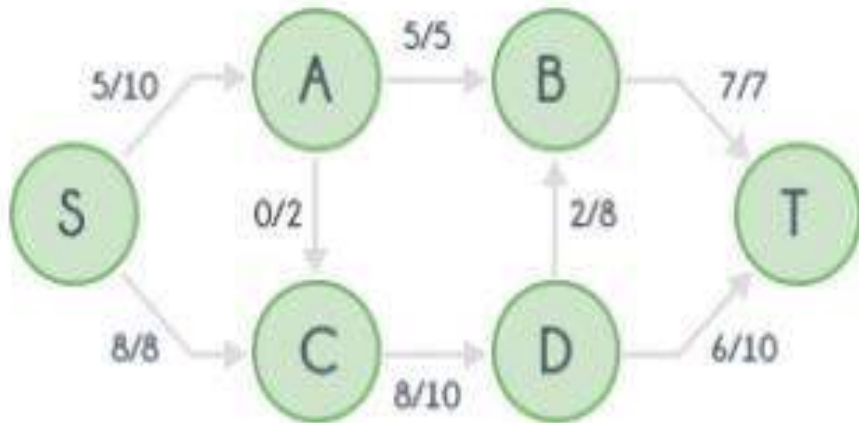
Path 1: S - C - D - B - T → Flow = Flow + 7



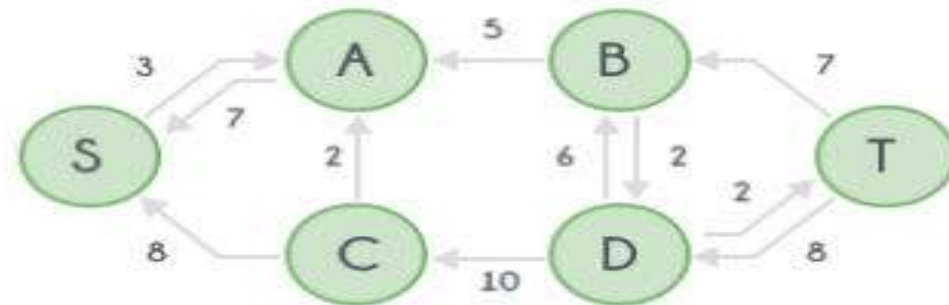
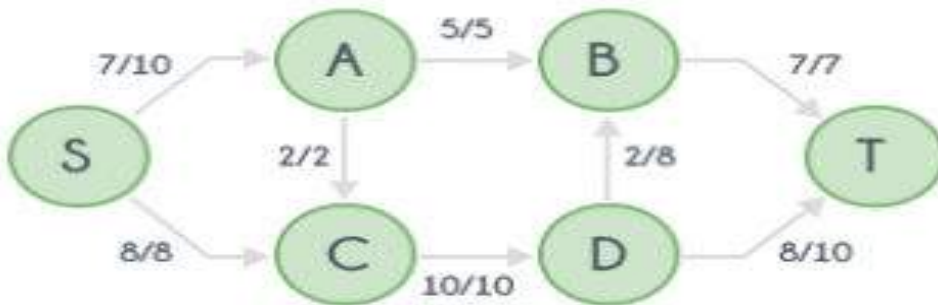
Path 2: S - C - D - T → Flow = Flow + 1



Path 3: S - A - B - T → Flow = Flow + 5



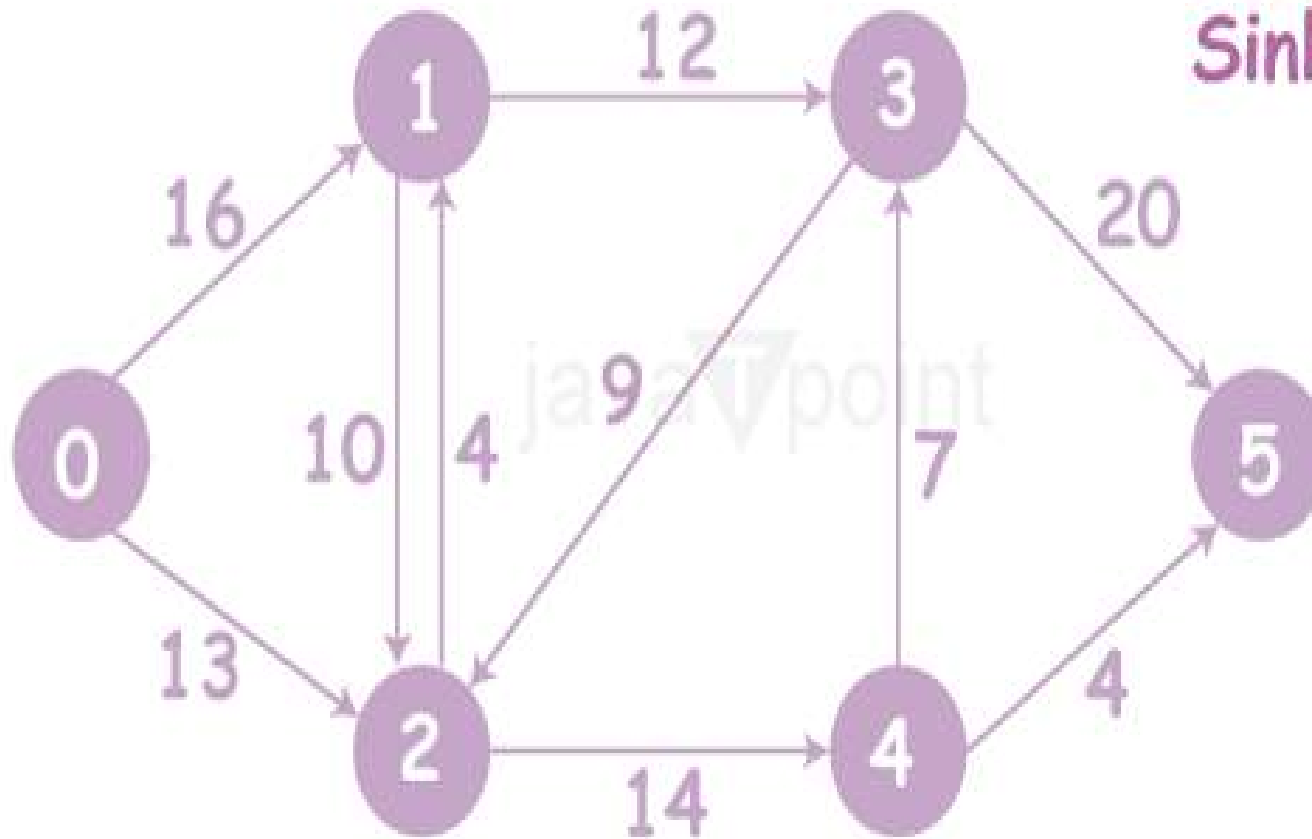
Path 4: S - A - C - D - T → Flow = Flow + 2



**No More Paths Left
Max Flow = 15**

Ford Fulkerson Algorithm

Source: 0
Sink: 5

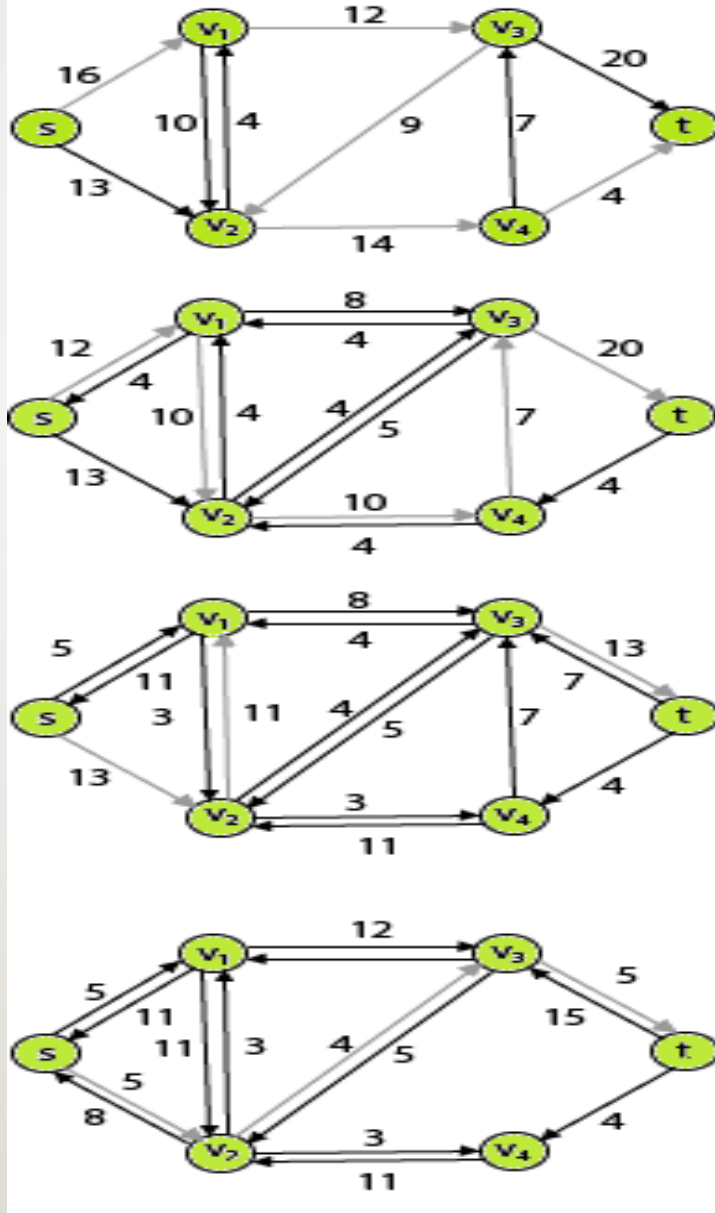


S->V1->V3->V2->V4->T
FLOW = 0+4

S->V1->V2->V4->V3->T

S->V2->V1->V3->T

S->V2->V3->T



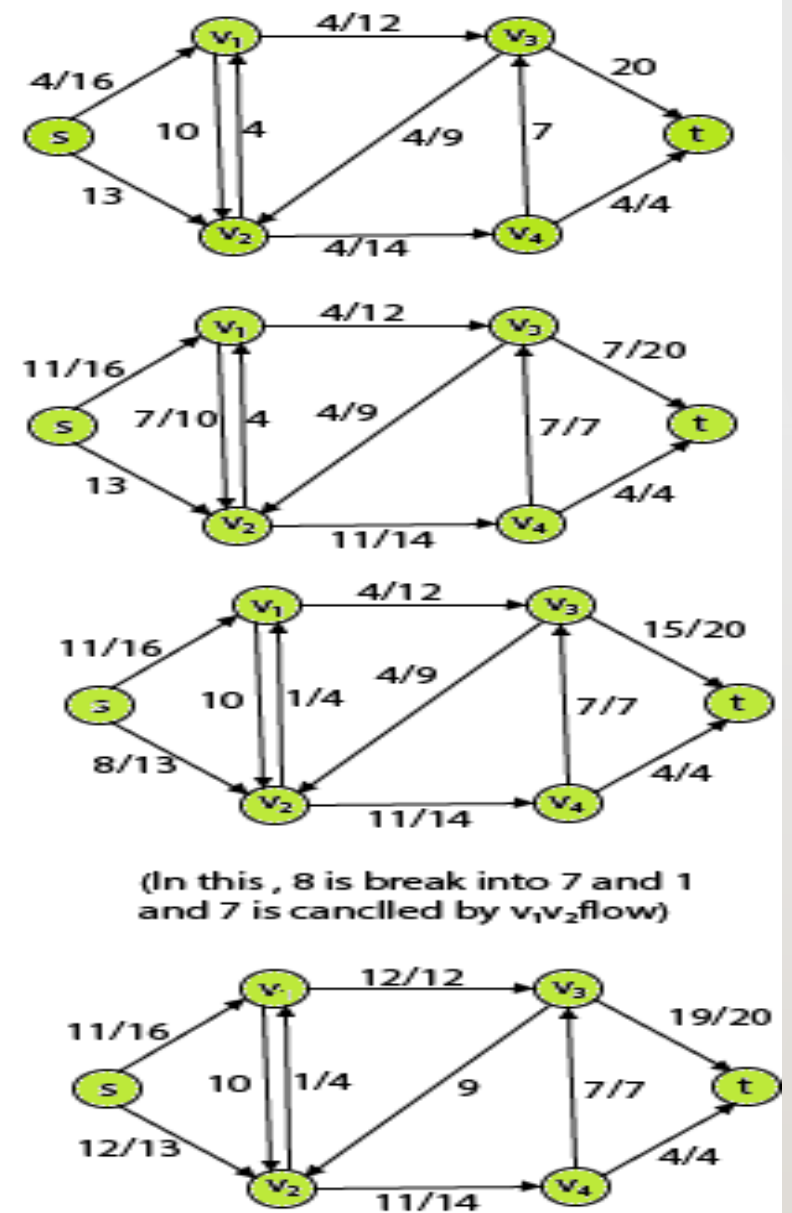
RESIDUAL NETWORK

(a)

(b)

(c)

(d)



FLOW NETWORK

Pseudocode :

function: FordFulkerson(Graph G, Node S, Node T):

 Initialise flow in all edges to 0

 while (there exists an augmenting path(P) between S and T in residual network graph):

 Augment flow between S to T along the path P

 Update residual network graph

 return

Edmonds-Karp algorithm

Edmonds-Karp algorithm is just an implementation of the Ford-Fulkerson method that uses **BFS** for finding augmenting paths. The algorithm was first published by Yefim Dinitz in 1970, and later independently published by Jack Edmonds and Richard Karp in 1972.

The complexity can be given independently of the maximal flow. The algorithm runs in $O(VE^2)$ time, even for irrational capacities. The intuition is, that every time we find an augmenting path one of the edges becomes saturated, and the distance from the edge to s will be longer if it appears later again in an augmenting path. The length of the simple paths is bounded by V .

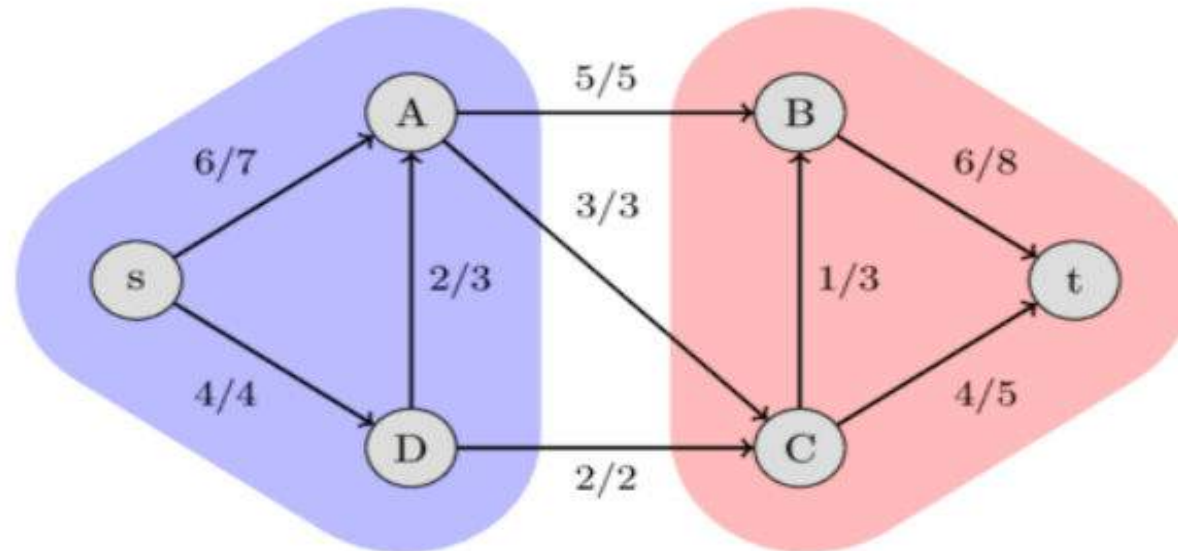
Max-flow min-cut theorem

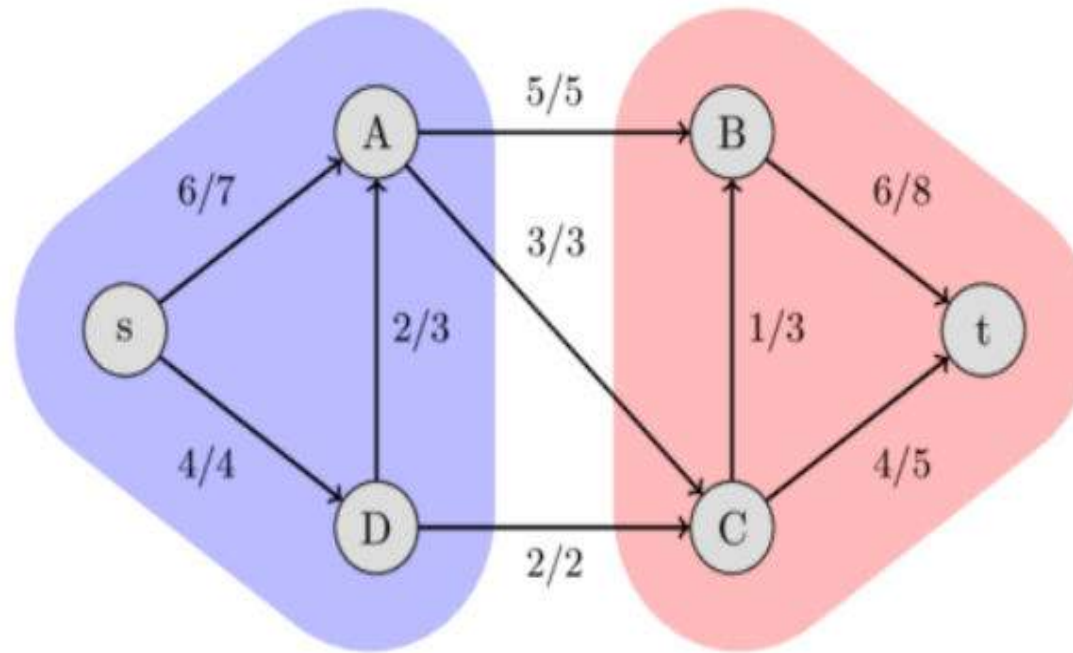
A **s - t -cut** is a partition of the vertices of a flow network into two sets, such that a set includes the source s and the other one includes the sink t . The capacity of a s - t -cut is defined as the sum of capacities of the edges from the source side to the sink side.

Obviously, we cannot send more flow from s to t than the capacity of any s - t -cut. Therefore, the maximum flow is bounded by the minimum cut capacity.

The max-flow min-cut theorem goes even further. It says that the capacity of the maximum flow has to be equal to the capacity of the minimum cut.

In the following image, you can see the minimum cut of the flow network we used earlier. It shows that the capacity of the cut $\{s, A, D\}$ and $\{B, C, t\}$ is $5 + 3 + 2 = 10$, which is equal to the maximum flow that we found. Other cuts will have a bigger capacity, like the capacity between $\{s, A\}$ and $\{B, C, D, t\}$ is $4 + 3 + 5 = 12$.





A minimum cut can be found after performing a maximum flow computation using the Ford-Fulkerson method. One possible minimum cut is the following: the set of all the vertices that can be reached from s in the residual graph (using edges with positive residual capacity), and the set of all the other vertices. This partition can be easily found using DFS starting at s .

SAMPLE QUESTIONS

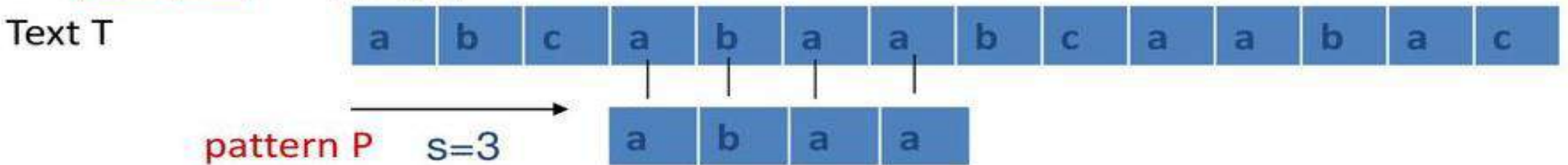
- How does the Ford-Fulkerson algorithm handle multiple sources and sinks in the network
- What is Maximum flow in Ford-Fulkerson algorithm
- What is the residual network, and how is it used in the Ford-Fulkerson algorithm
- Discuss the basic steps involved in the Ford-Fulkerson algorithm

STRING MATCHING ALGORITHMS

STRING MATCHING ALGORITHMS

- String Matching Algorithm is also called "String Searching Algorithm." This is a vital class of string algorithm is declared as "this is the method to find a place where one or several strings are found within the larger string."
- Given a text array, $T [1.....n]$, of n character and a pattern array, $P [1.....m]$, of m characters. The problems are to find an integer s , called valid shift where $0 \leq s < n-m$ and $T [s+1.....s+m] = P [1.....m]$. In other words, to find even if P in T , i.e., where P is a substring of T . The item of P and T are character drawn from some finite alphabet such as $\{0, 1\}$ or $\{A, BZ, a, b..... z\}$.

- **Given:** Two strings $T[1..n]$ and $P[1..m]$ over alphabet Σ .
- Want to find all occurrences of $P[1..m]$ “the pattern” in $T[1..n]$ “the text”.
- **Example:** $\Sigma = \{a, b, c\}$



- P occurs with shift s .
- P occurs beginning at position $s+1$.
- s is a valid shift.

The idea of the string matching problem is that we want to find all occurrences of the pattern P in the given text T

STRING MATCHING

- Text-editing programs frequently need to find all occurrences of a pattern in the text.
- Given a **text** is an array $T[1..n]$ of length n and a **pattern** $P[1..m]$ of length $m \leq n$.
- We say that pattern P occurs with shift s in text T if $0 \leq s \leq n-m$ and $T[s+1..s+m] = P[1..m]$.
- If P occurs with shift s in T , then we call s a **valid** shift; otherwise, we call s an **invalid** shift.

Shift (s): The number of position before the pattern occurrences in text.

Invalid Shift: The position after which partial matching occurs.

Valid Shift: The position after which complete matching occurs

Example: T = CARLETONUNIVERSITY
P = UNIVERSITY

Shift=0
T = CARLETONUNIVERSITY
P = UNIVERSITY

Shift=1
T = CARLETONUNIVERSITY
P = UNIVERSITY

Shift=8
T = CARLETONUNIVERSITY
P = UNIVERSITY

Outputs as pattern found in 9th (s+1) position.

STRING MATCHING

- The string-matching problem is the problem of finding all valid shifts with which a given pattern P occurs in a given text T .
- Example: $T = \text{ab} \color{red}{\text{abc}} \text{abd} \color{green}{\text{abc}} \text{a} \color{blue}{\text{abc}}$ and $P = \text{abc}$, the occurrences are:
 - first occurrence starts at $T[3]$ $[S=2]$
 - second occurrence starts at $T[9]$ $[S=8]$
 - third occurrence starts at $T[13]$ $[S=12]$ are Valid Shifts.

Applications:

- Searching keywords in a file
- Searching engines (like Google and Openfind)
- Database searching (GenBank)

There are many types of String Matching Algorithms like:-

- 1) The Naive string-matching algorithm
- 2) The Rabin-Krap algorithm
- 3) String matching with finite automata
- 4) The Knuth-Morris-Pratt algorithm

THE NAIVE STRING-MATCHING ALGORITHM

The naive algorithm finds all valid shifts using a loop that checks the condition

$P[1..m] = T[s+1..s+m]$ for each of the $n-m+1$ possible values of s .

Naïve string matching algorithm

Naïve (T, P)

- 1) $n = \text{length}(T)$
- 2) $m = \text{length}(P)$
- 3) for $s = 0$ to $n-m$
- 4) if $P[1..m] = T[s+1..s+m]$
- 5) then print "Pattern occurs with shift" s

This algorithm works by marking shift positions. After each shift position, it compares all characters of pattern with text and outputs all occurrences if there is exact match.

RUNNING TIME

- This for loop from 3 to 5 executes for $n-m + 1$ (we need at least m characters at the end) times and in iteration we are doing m comparisons.
- So the total complexity is **$O(n-m+1)$**

EXAMPLE

T :

a	b	c	a	b	d	a	a	b	c	d	e
---	---	---	---	---	---	---	---	---	---	---	---

P :

a	b	d
---	---	---

EXAMPLE (STEP – I)

T :

a	b	c	a	b	d	a	a	b	c	d	e
---	---	---	---	---	---	---	---	---	---	---	---

P :

a	b	d
---	---	---

Mismatch after 3 Comparisons

EXAMPLE (STEP – 2)

T :

a	b	c	a	b	d	a	a	b	c	d	e
---	---	---	---	---	---	---	---	---	---	---	---

P :

a	b	d
---	---	---

Mismatch after 1 Comparison

EXAMPLE (STEP – 3)

T :

a	b	c	a	b	d	a	a	b	c	d	e
---	---	---	---	---	---	---	---	---	---	---	---

P :

a	b	d
---	---	---

Mismatch after 1 Comparison

EXAMPLE (STEP – 4)

T :	a	b	c	a	b	d	a	a	b	c	d	e
P :				a	b	d						

Match found after 3 Comparisons

Thus, after 3 comparisons the substring P is found at position 4 in T.

Example:

Suppose $T = 1011101110$

$P = 111$

Find all the Valid Shift

T = Text



S=0



P = Pattern



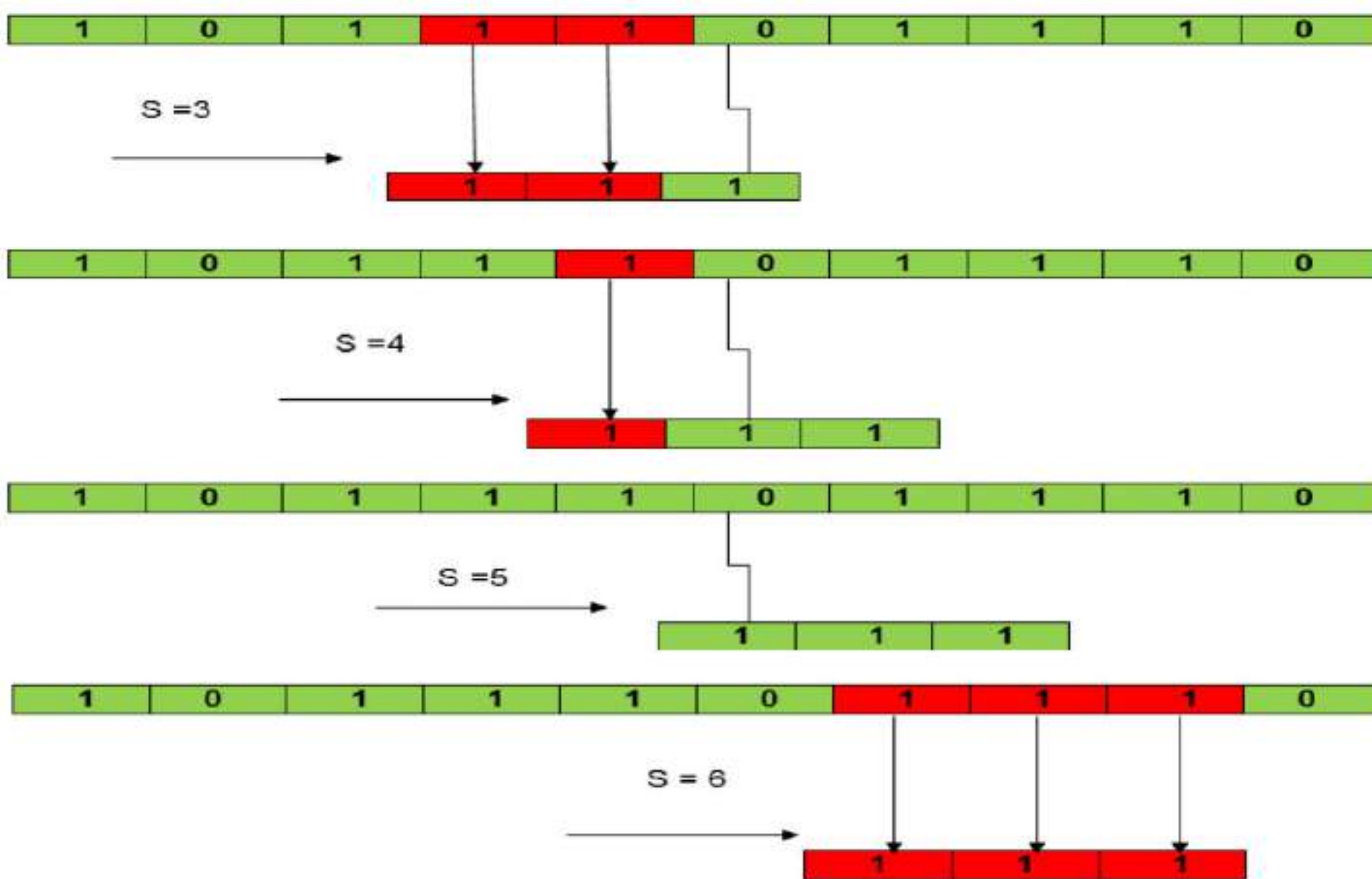
S=1



S = 2



So, S=2 is a Valid Shift



So, $S=6$ is a Valid Shift

FIND WHETHER PATTERN EXISTS IN THE TEXT OR NOT USING NAÏVE STRING MATCHING ALGORITHM

T :

a	b	c	a	a	d	c	a	b	a	d	e
---	---	---	---	---	---	---	---	---	---	---	---

P :

b	a	d
---	---	---

SAMPLE QUESTIONS

- Explain about string-matching algorithms
- List out the various string-matching algorithms.
- What is mismatch in string algorithms
- provide a real-world example where a string matching algorithm is utilized in a practical application

RABIN KARP STRING MATCHING ALGORITHM

- This algorithm performs well in practice and that also generalizes to other algorithms for related problems.
- Its worst case running time is $O(n-m+1)$ which is same as Naïve string matching algorithm but average case running time is $O(n+m)$.
- This algorithm makes use of elementary number-theoretic notions.
- Each character is converted in to decimal digit and strings of k-consecutive characters are represented as length-k decimal number.

RABIN-KARP STRING MATCHING ALGORITHM

-
- Rabin and Karp proposed a string-matching algorithm to find the pattern in a more efficient way.
 - It also checks the pattern by moving window one by one, but without checking all characters for all cases, it finds the hash value.
 - When the hash value is matched, then only it tries to check each character.
 - This procedure makes the algorithm more efficient compared to naïve string matching algorithm.

HOW RABIN-KARP WORKS

- Given a pattern P $[1..m]$, let p denote its corresponding hash value and a text T $[1..n]$, let t_s denote the hash value of the length- m substring $T[s+1..s+m]$, for $s=0, 1..n-m$.
- Certainly, s is a valid shift iff $t_s = p$ and $T[s+1..s+m] = P[1..m]$ otherwise s is a invalid shift.
- Let characters in both arrays T and P be digits in radix- Σ notation. ($\Sigma = (0, 1, \dots, 9)$)
- Choose a prime number q such that fits within a computer word to speed computations.
- Compute $(p \bmod q)$
 - The value of $p \bmod q$ is what we will be using to find all matches of the pattern P in T .

RABIN-KARP :ALGORITHM

RABIN-KARP-MATCHER(T, P, d, q)

1. $n = \text{length}[T]$

2. $m = \text{length}[P]$

3. $h = d^{m-1} \bmod q$

4. $p = 0$

5. $t_0 = 0$

6. **for** $i = 1$ **to** m **do**

7. $p = (dp + P[i]) \bmod q$

8. $t_0 = (dt_0 + T[i]) \bmod q$

RABIN-KARP :ALGORITHM(CONTINUED)

9. **for** $s = 0$ **to** $n - m$ **do**

10. **if** $p == t_s$ **then**

11. **if** $P[1 .. m] = T[s + 1 .. s + m]$ **then**

12. **Print** "Pattern occurs with shift" s

13. **if** $s < n - m$ **then**

14. $t_{s+1} = (d(t_s - T[s + 1])h) + T[s + m + 1]) \bmod q$

Example:

For string matching, working module $q = 11$, how many spurious hits does the Rabin-Karp matcher encounters in Text $T = 31415926535.....$

$T = 31415926535.....$

$P = 26$

Here $T.Length = 11$ so $Q = 11$ (can be any given prime value)

And the Hash Value is : $P \bmod Q = 26 \bmod 11 = 4$

Now find the exact match of $P \bmod Q...$

Solution:

T =

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

P =

2	6
---	---

S = 0
→

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$31 \bmod 11 = 9$ not equal to 4

S = 1
→

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$14 \bmod 11 = 3$ not equal to 4

$S = 2$

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$41 \bmod 11 = 8$ not equal to 4

$S = 3$

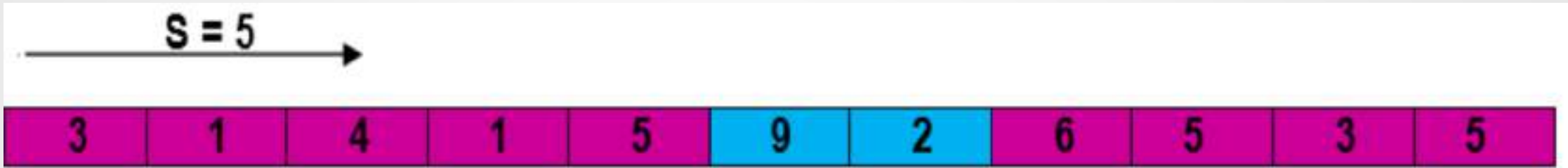
3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$15 \bmod 11 = 4$ equal to 4 SPURIOUS HIT

$S = 4$

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$59 \bmod 11 = 4$ equal to 4 SPURIOUS HIT



$92 \bmod 11 = 4$ equal to 4 SPURIOUS HIT



$26 \bmod 11 = 4$ EXACT MATCH

$S = 7$



3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$65 \bmod 11 = 10$ not equal to 4

$S = 8$



3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$53 \bmod 11 = 9$ not equal to 4



$$35 \bmod 11 = 2 \text{ not equal to } 4$$

The Pattern occurs with shift 6.

Complexity:

The running time of **RABIN-KARP-MATCHER** in the worst case scenario $O((n-m+1)m)$ but it has a good average case running time. If the expected number of strong shifts is small $O(1)$ and prime q is chosen to be quite large, then the Rabin-Karp algorithm can be expected to run in time $O(n+m)$ plus the time to require to process spurious hits.



A RABIN-KARP EXAMPLE

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$31 \bmod 11 = 9$ not equal to 4

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$14 \bmod 11 = 3$ not equal to 4

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$41 \bmod 11 = 8$ not equal to 4

RABIN-KARP EXAMPLE CONTINUED

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$15 \bmod 11 = 4$ equal to 4 -> spurious hit

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$59 \bmod 11 = 4$ equal to 4 -> spurious hit

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$92 \bmod 11 = 4$ equal to 4 -> spurious hit

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$26 \bmod 11 = 4$ equal to 4 -> an exact match!!

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$65 \bmod 11 = 10$ not equal to 4

RABIN-KARP EXAMPLE CONTINUED

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$53 \bmod 11 = 9$ not equal to 4

3	1	4	1	5	9	2	6	5	3	5
---	---	---	---	---	---	---	---	---	---	---

$35 \bmod 11 = 2$ not equal to 4

As we can see, when a match is found, further testing is done to insure that a match has indeed been found.

COMPLEXITY

- The running time of the Rabin-Karp algorithm in the worst-case scenario is $O((n-m+1)m)$ but it has a good average-case running time.
- If the expected number of valid shifts is small $O(1)$ and the prime q is chosen to be quite large, then the Rabin-Karp algorithm can be expected to run in time $O(n+m)$ plus the time required to process spurious hits.

Find whether pattern exists in the text or not using Robin-Karp string matching algorithm

T :

a	b	c	a	a	d	c	a	b	a	d	e
---	---	---	---	---	---	---	---	---	---	---	---

P :

b	a	d
---	---	---

SAMPLE QUESTIONS

- What is the Rabin-Karp algorithm for string matching? How does it utilize hashing to efficiently search for patterns in a text
- What is role of hashing
- Discuss the time complexity of the Rabin-Karp algorithm. How does it perform in terms of average and worst-case scenarios?
- Compare and contrast the Rabin-Karp algorithm with other string matching algorithms,

Knuth-Morris-Pratt Algorithm

KMP Algorithm is one of the most popular patterns matching algorithms. KMP stands for Knuth Morris Pratt. KMP algorithm was invented by **Donald Knuth** and **Vaughan Pratt** together and independently by **James H Morris** in the year 1970. In the year 1977, all the three jointly published KMP Algorithm.

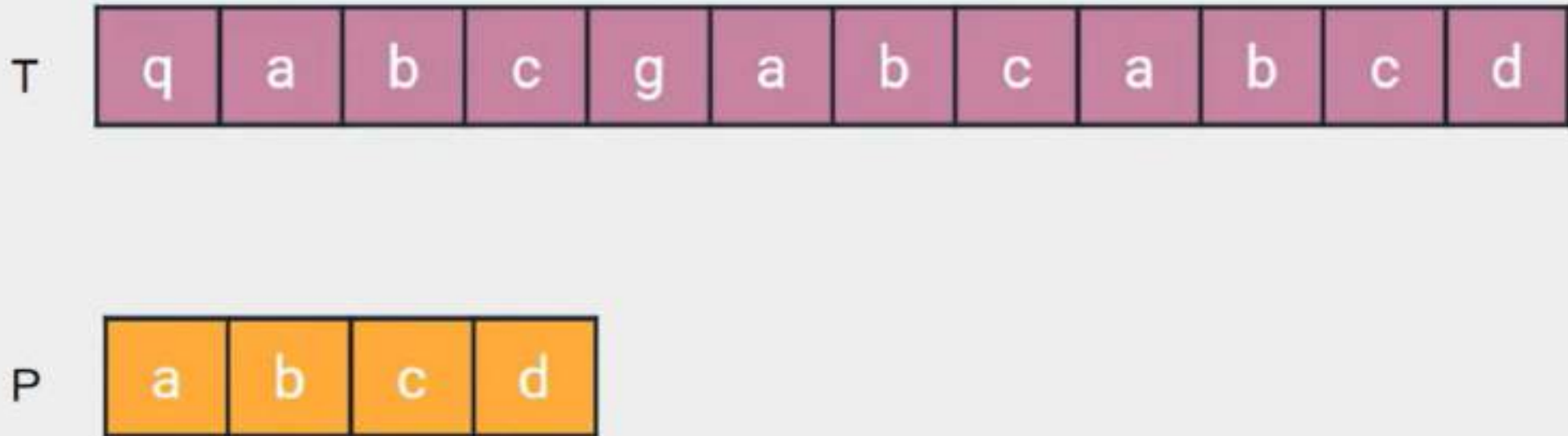
KMP algorithm was the first **linear time complexity** algorithm for string matching.

KMP algorithm is one of the string matching algorithms used to find a Pattern in a Text.

KMP algorithm is used to find a "**Pattern**" in a "**Text**". This algorithm compares character by character from left to right. But whenever a mismatch occurs, it uses a preprocessed table called "**Prefix Table**" to skip characters comparison while matching. Some times prefix table is also known as **LPS Table**. Here LPS stands for "**Longest proper Prefix which is also Suffix**".

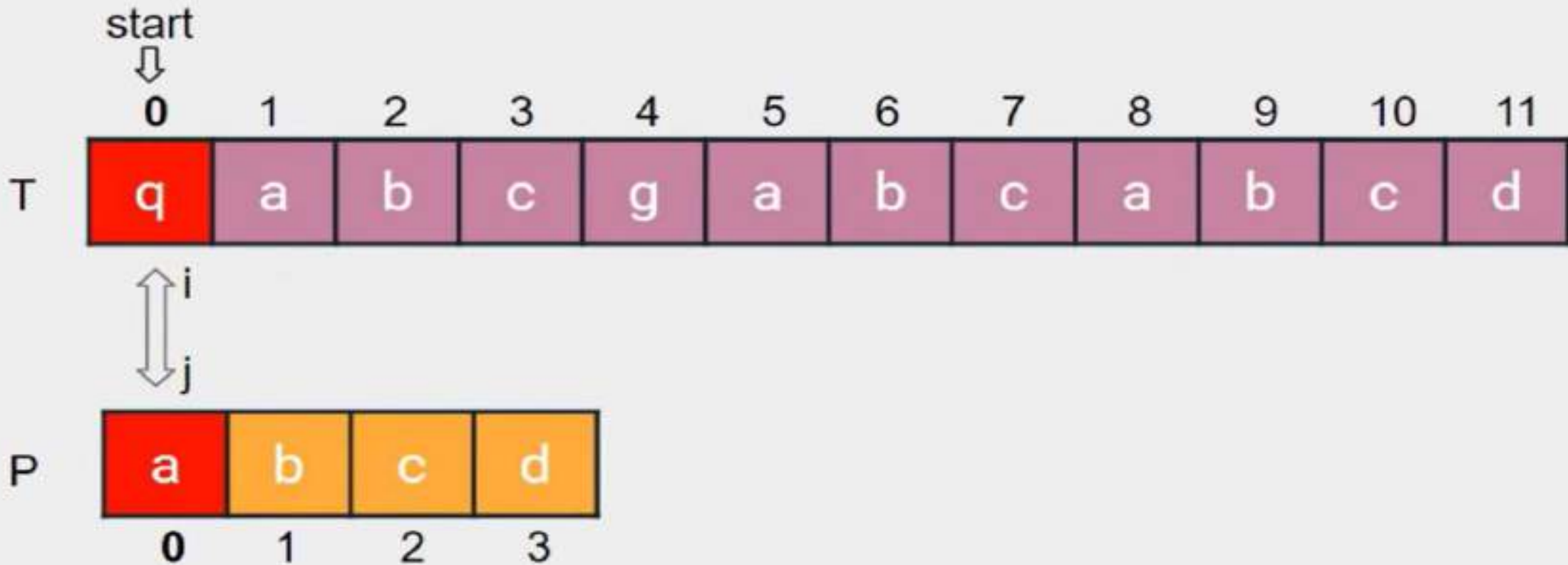
Brute Force Approach (Naive)

Check every possible fit.

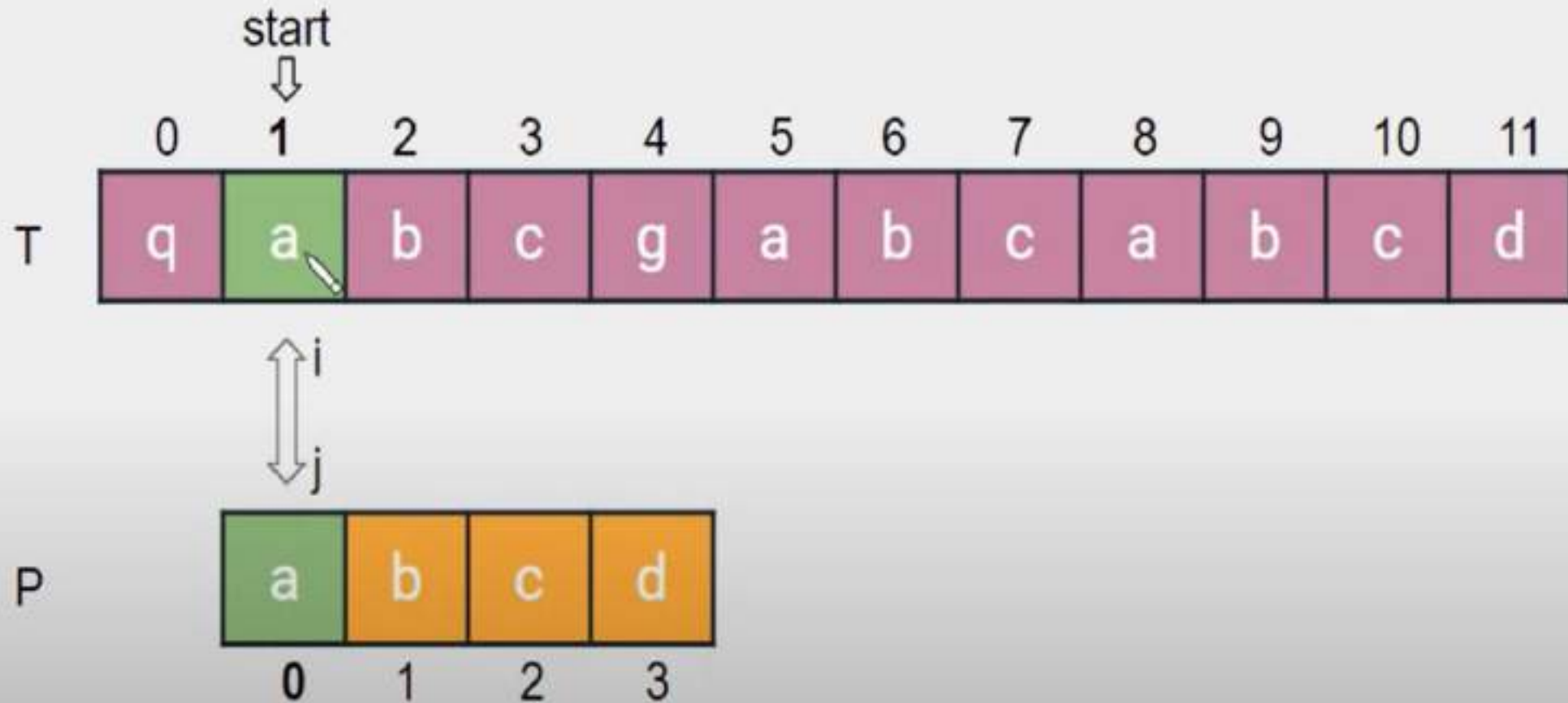


Brute Force Approach (Naive)

Check every possible fit.

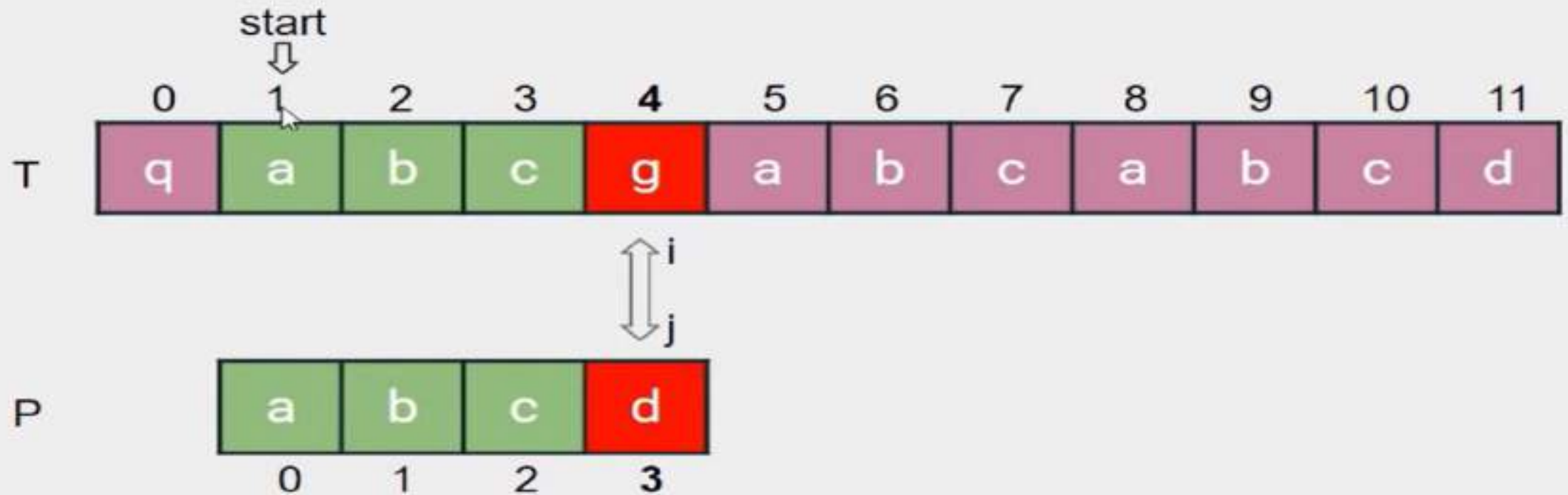


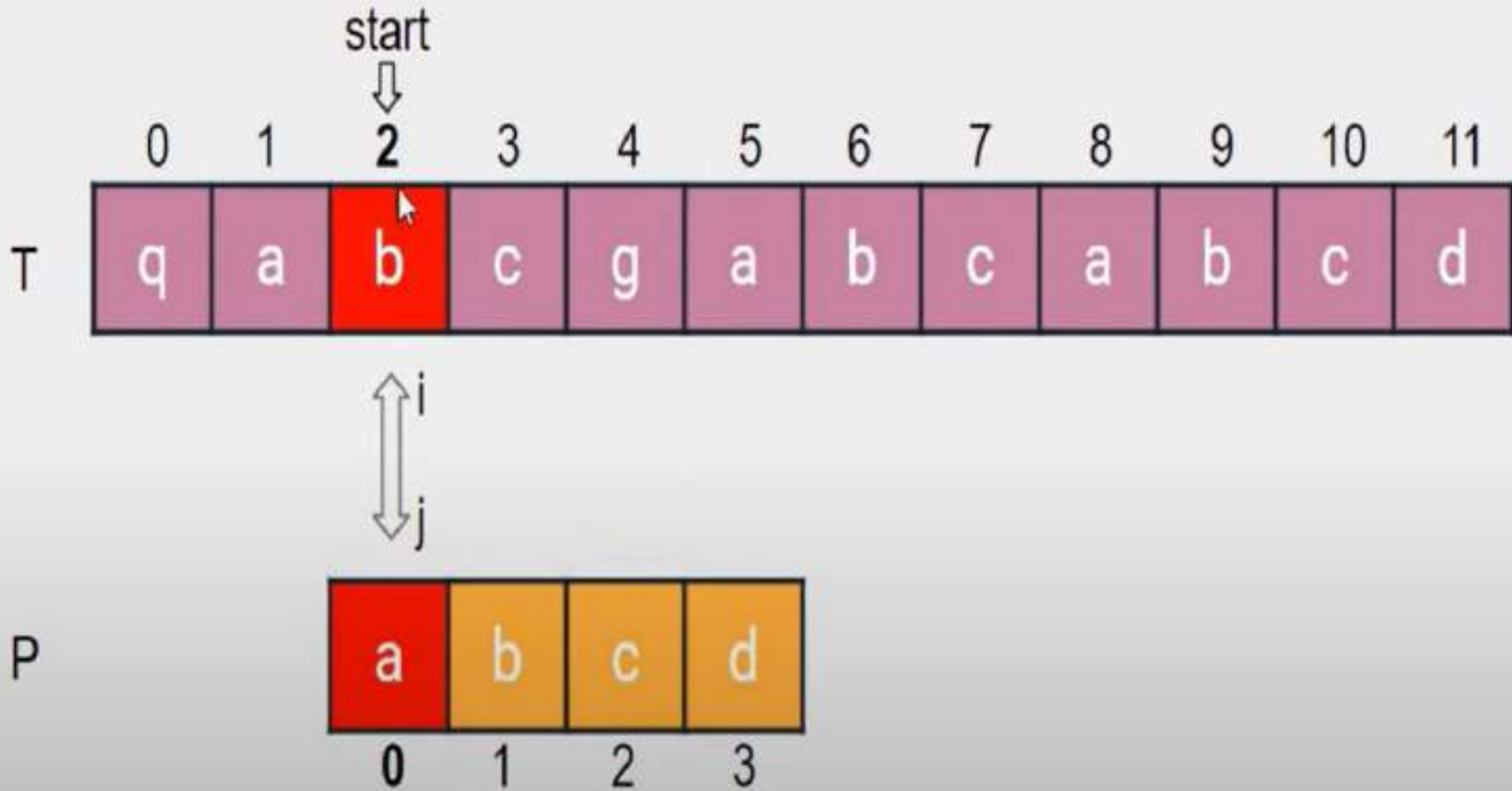
Check every possible fit.

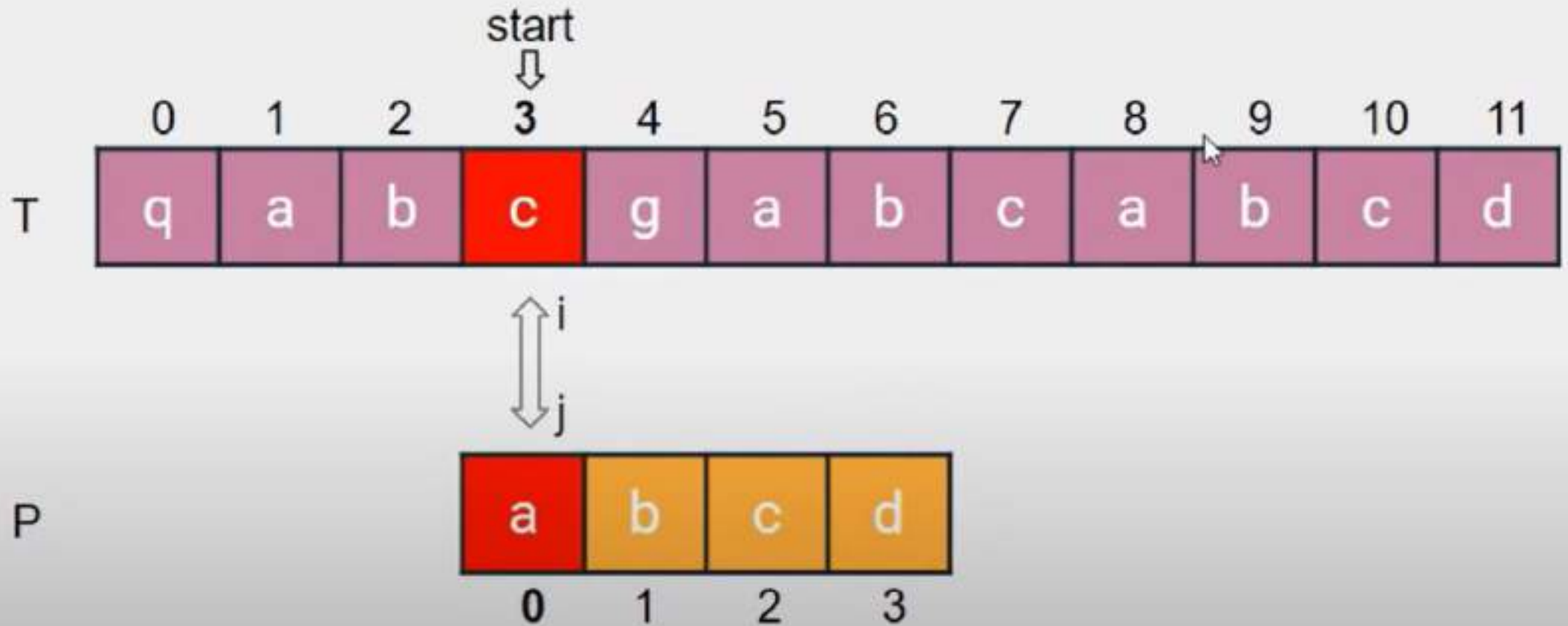


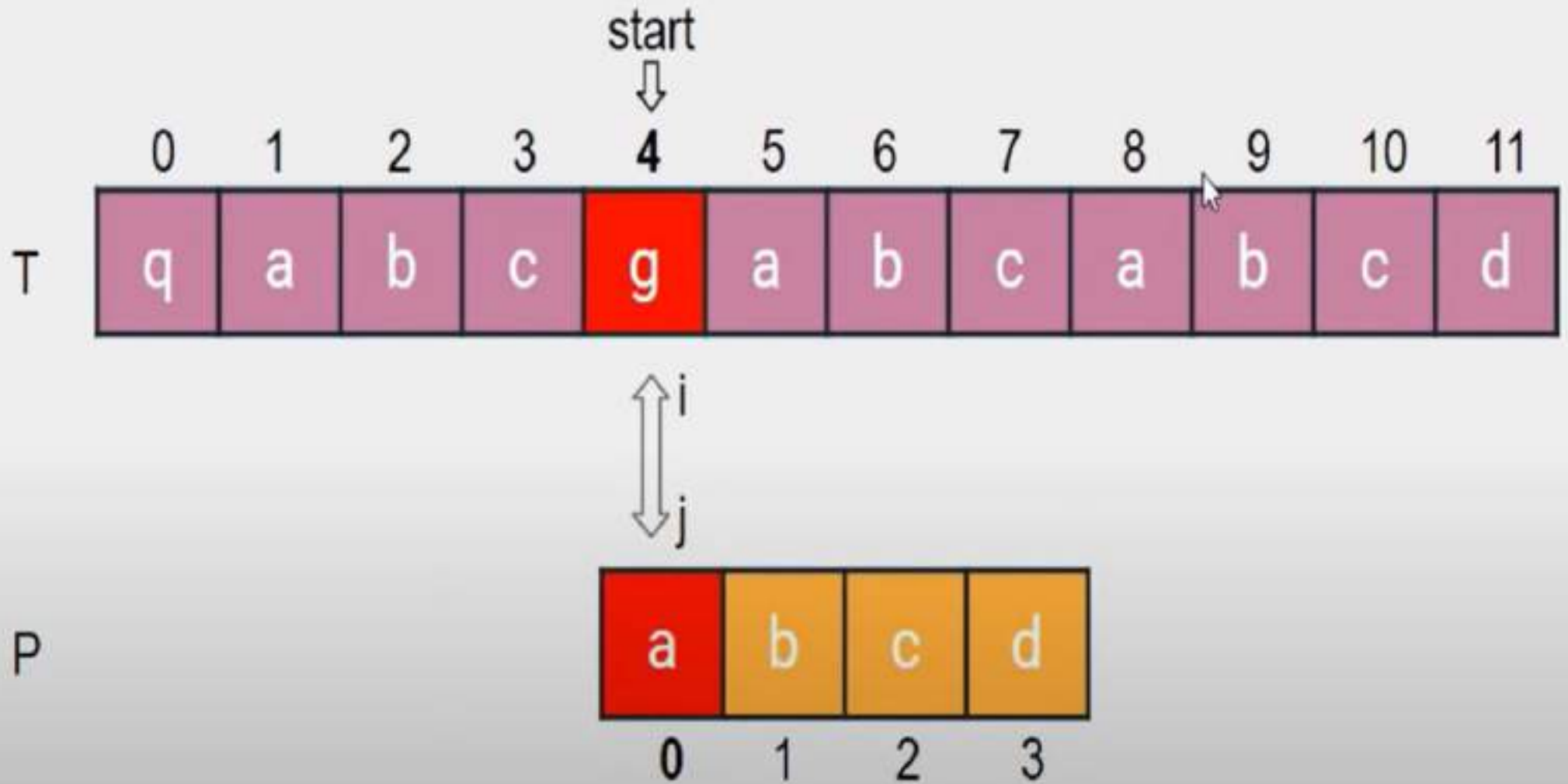
Brute Force Approach (Naive)

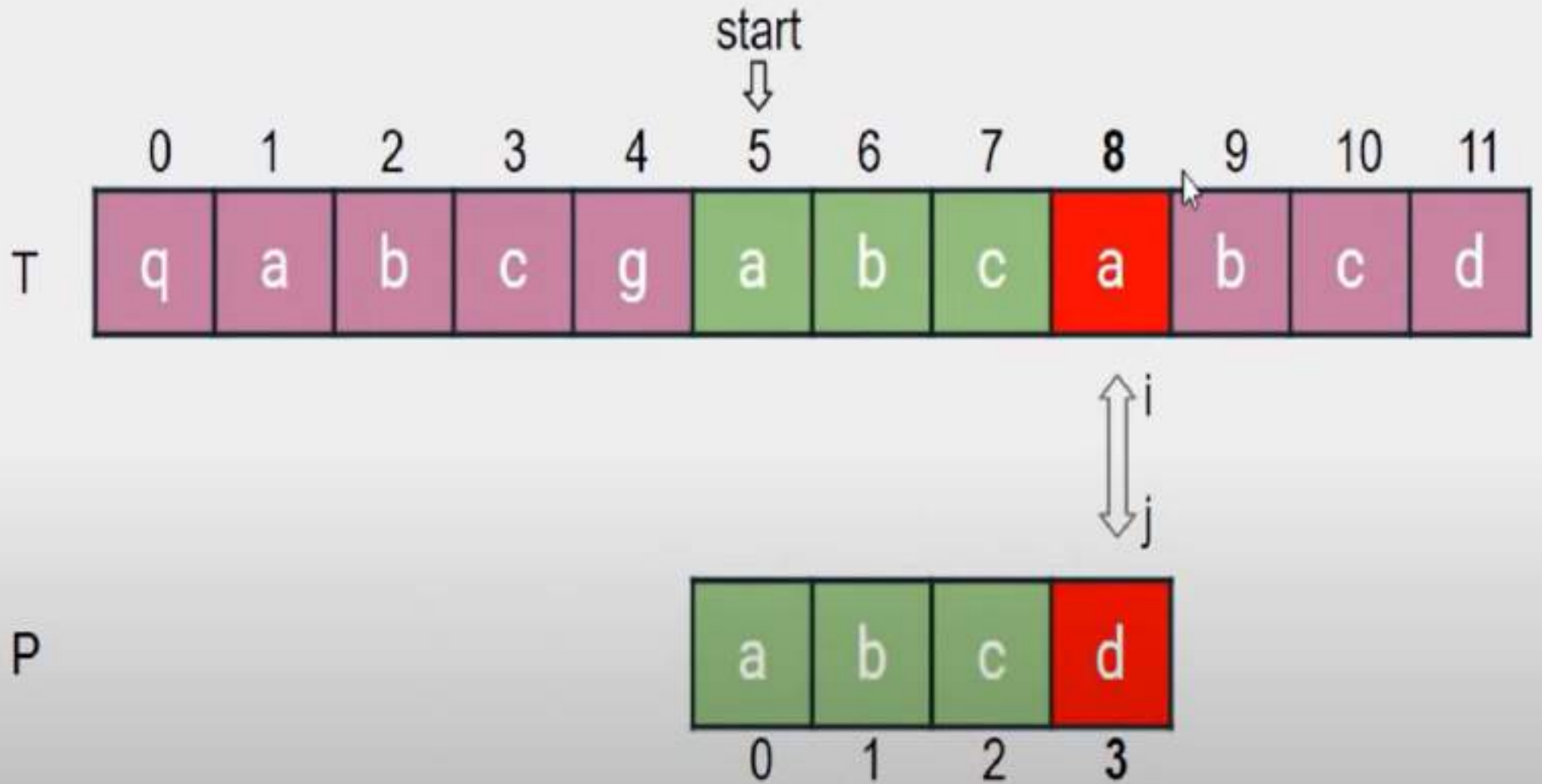
Check every possible fit.

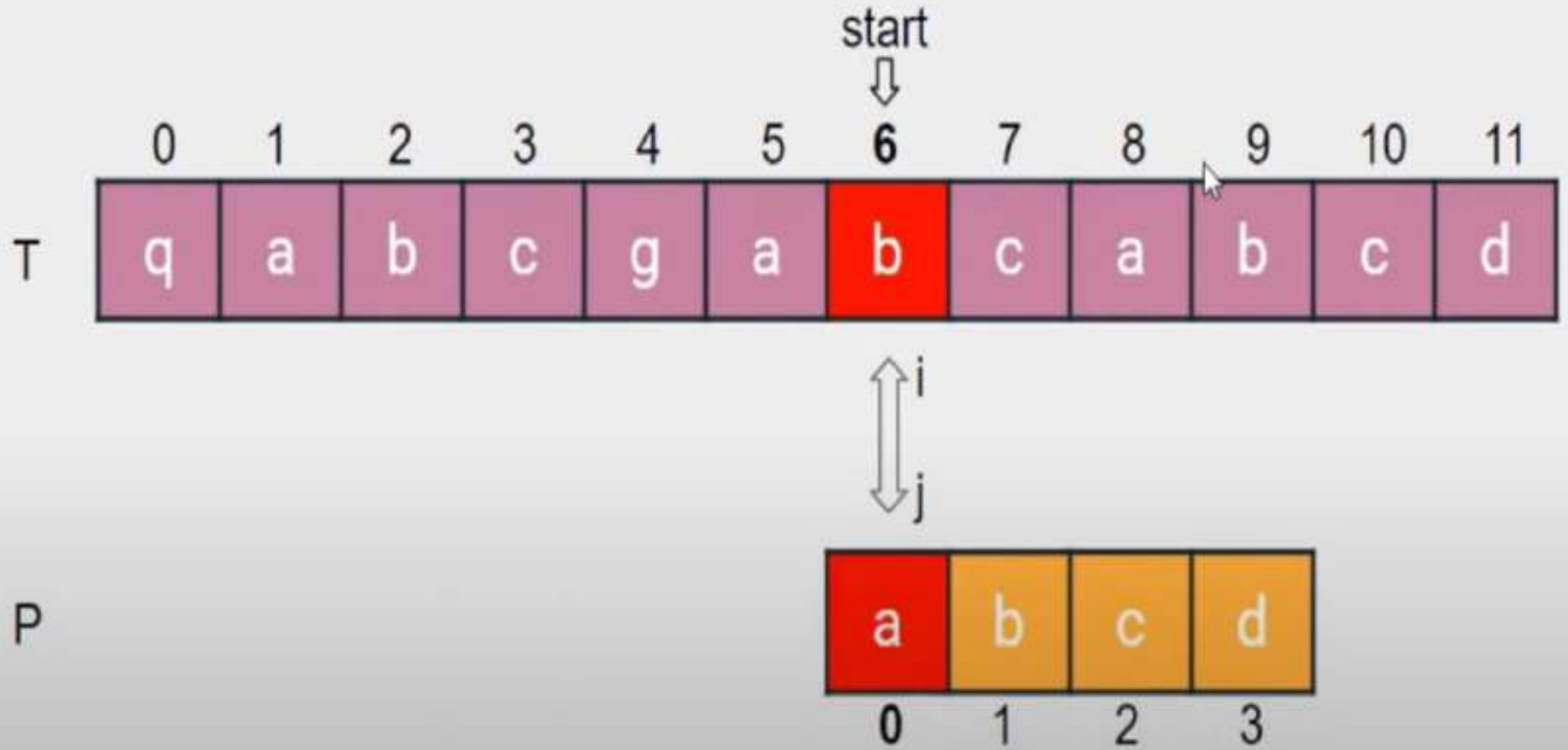


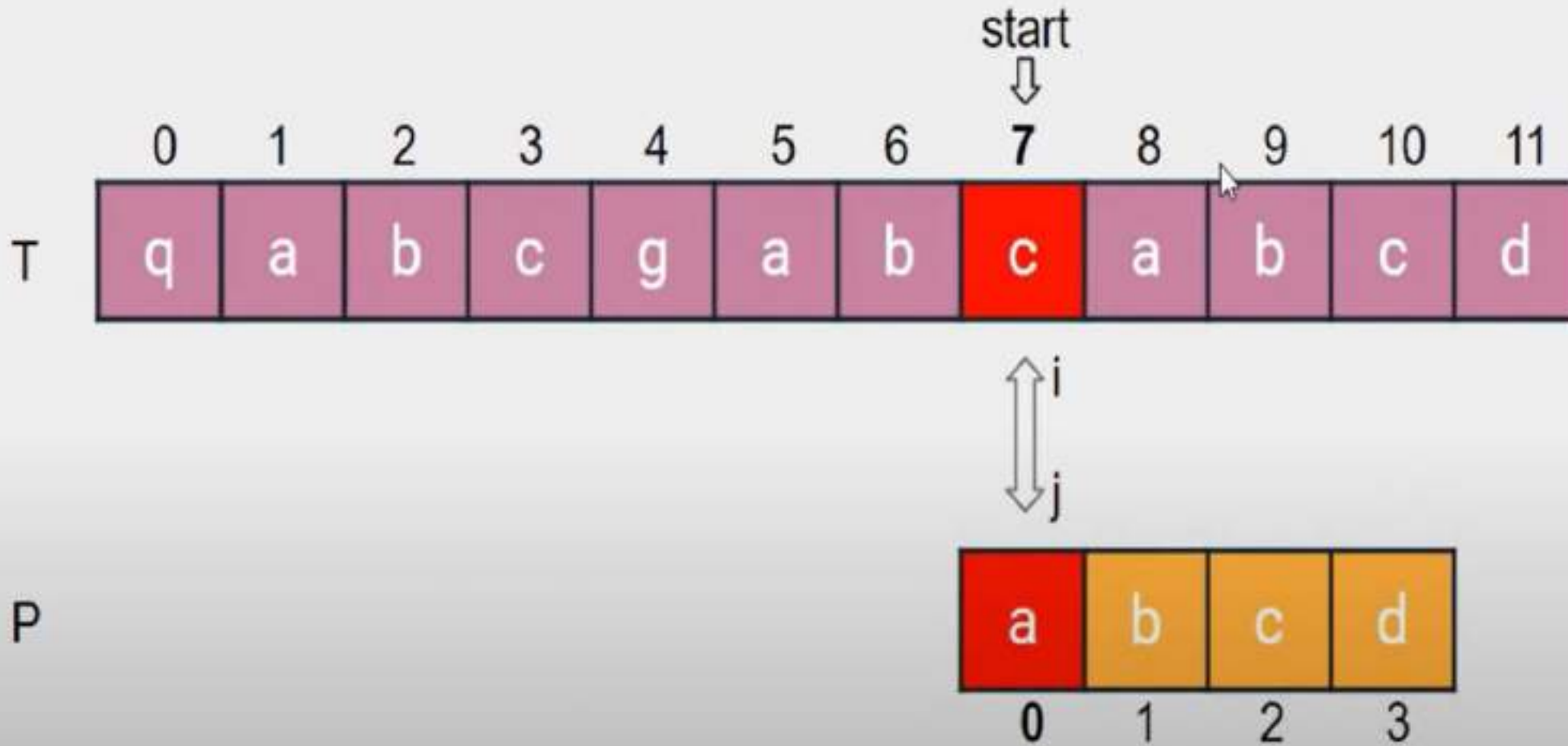




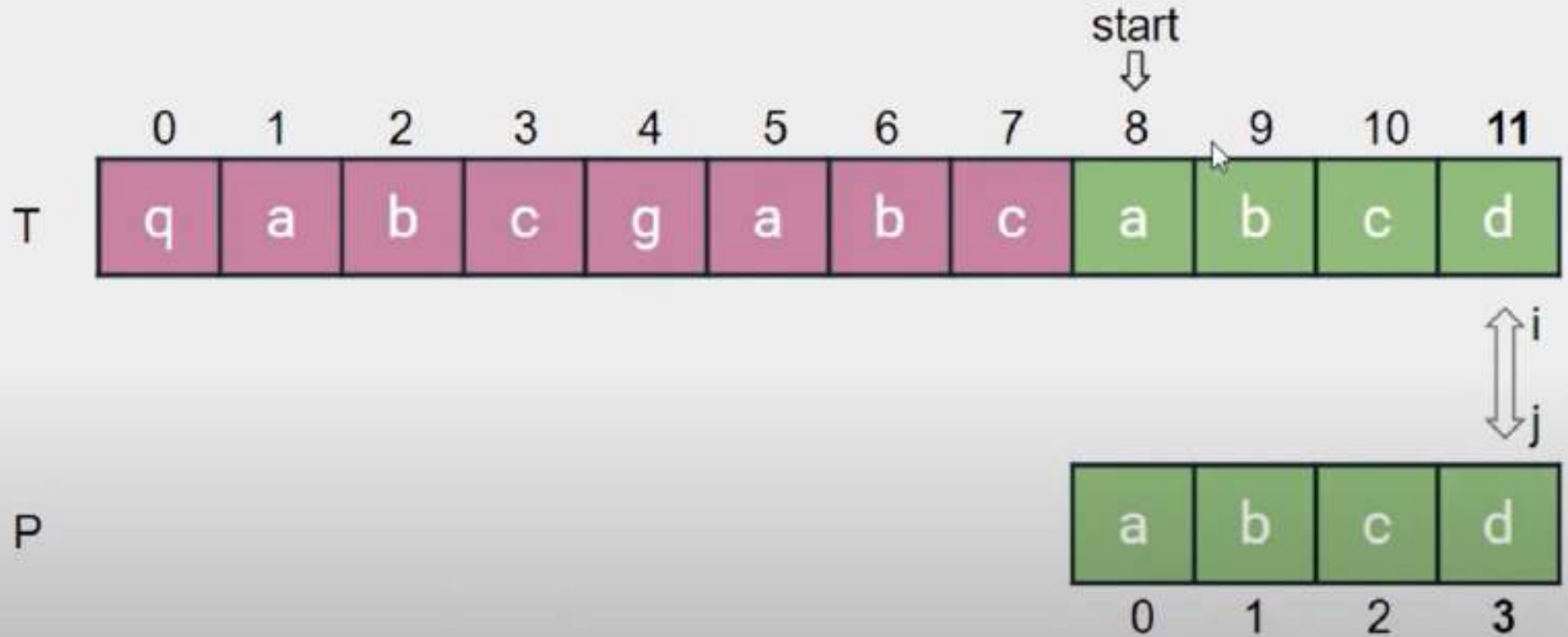




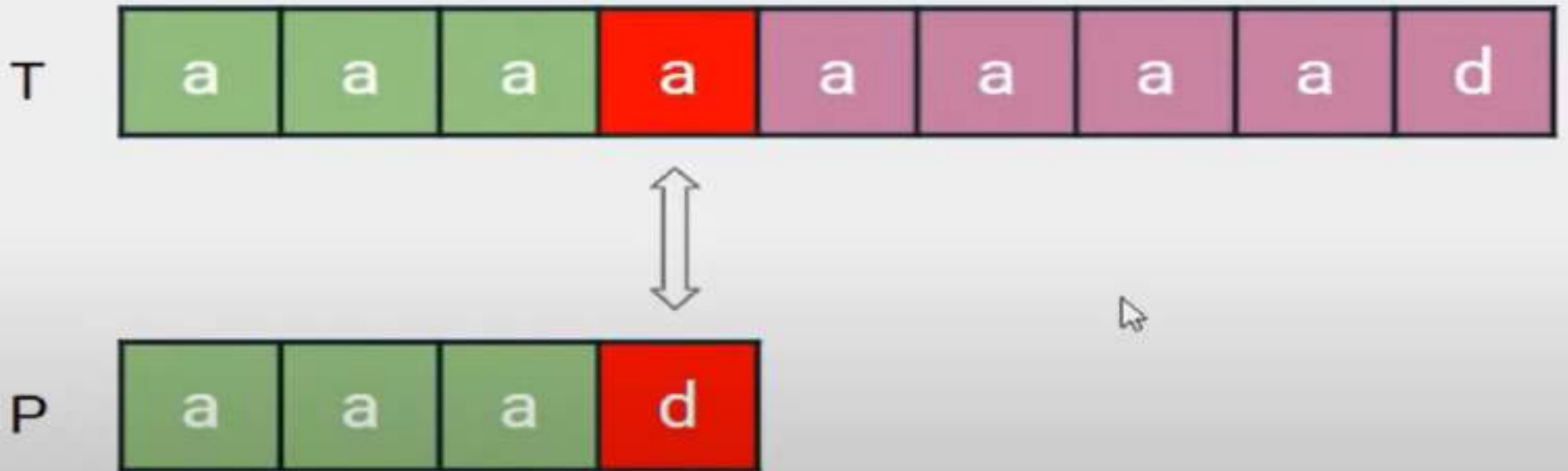




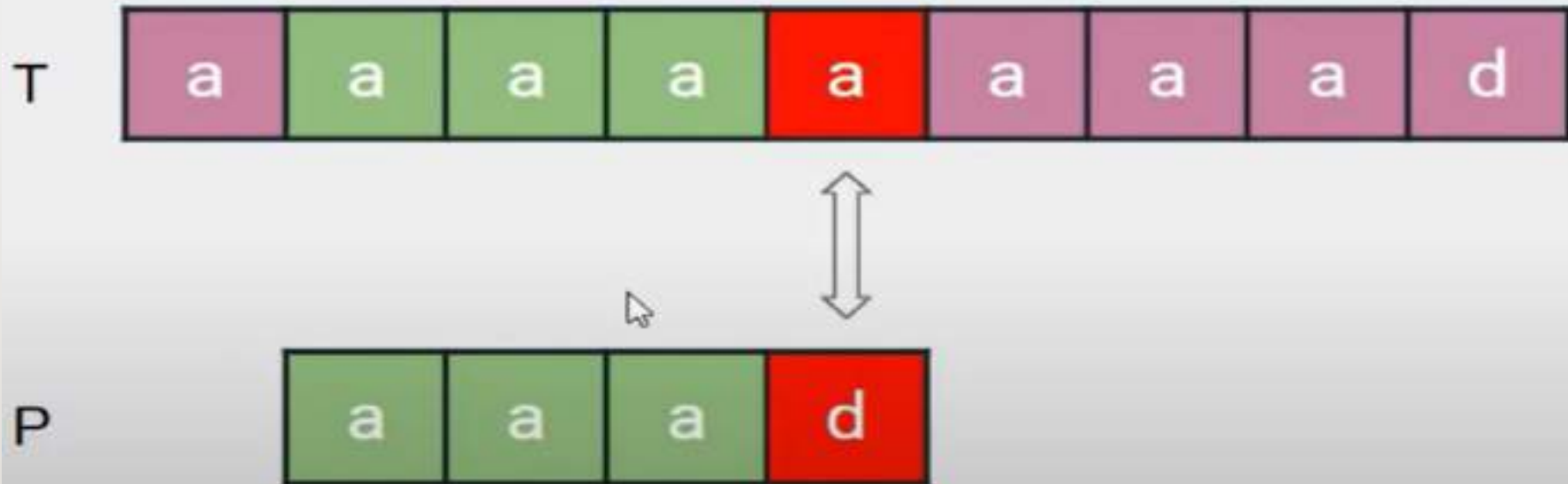
Check every possible fit.



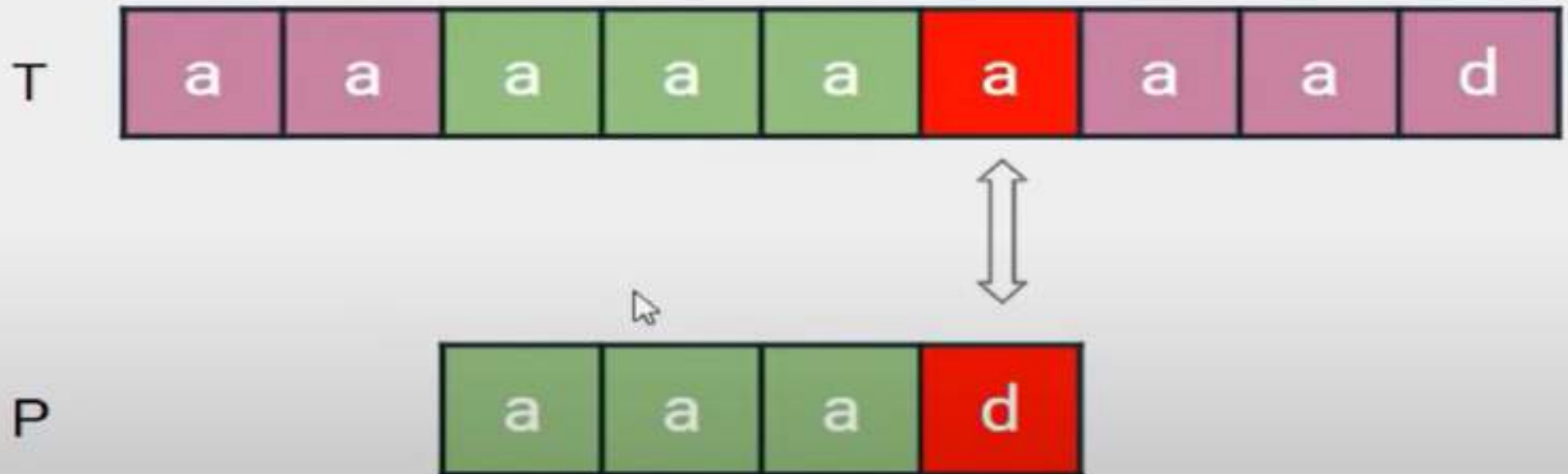
Worst Case



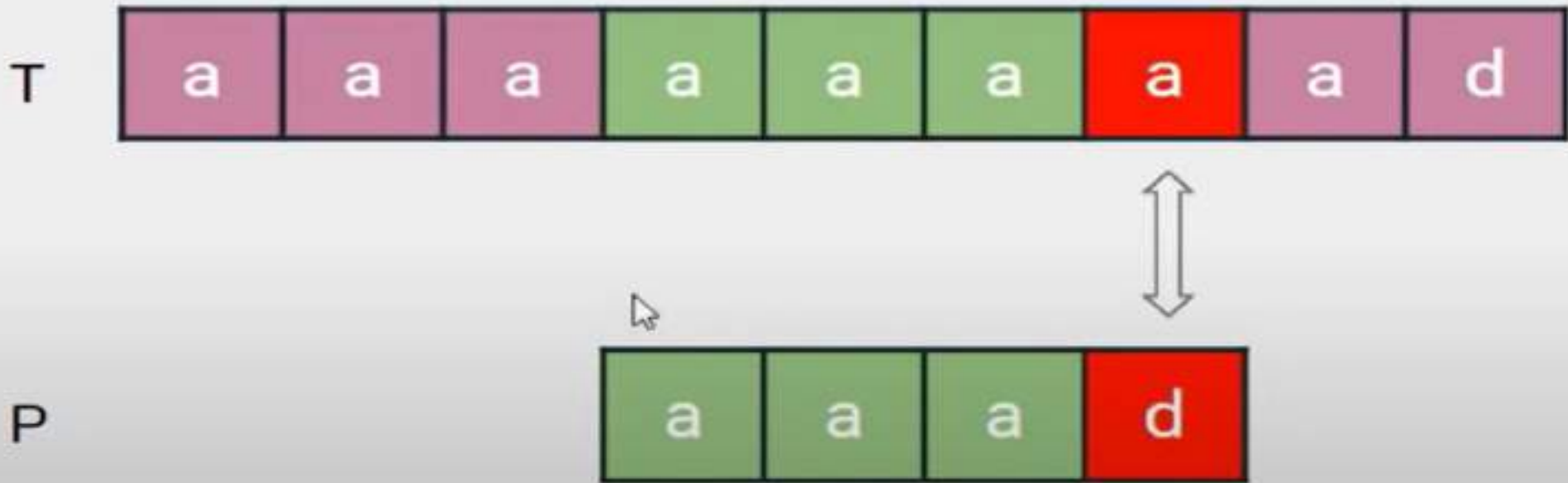
Worst Case



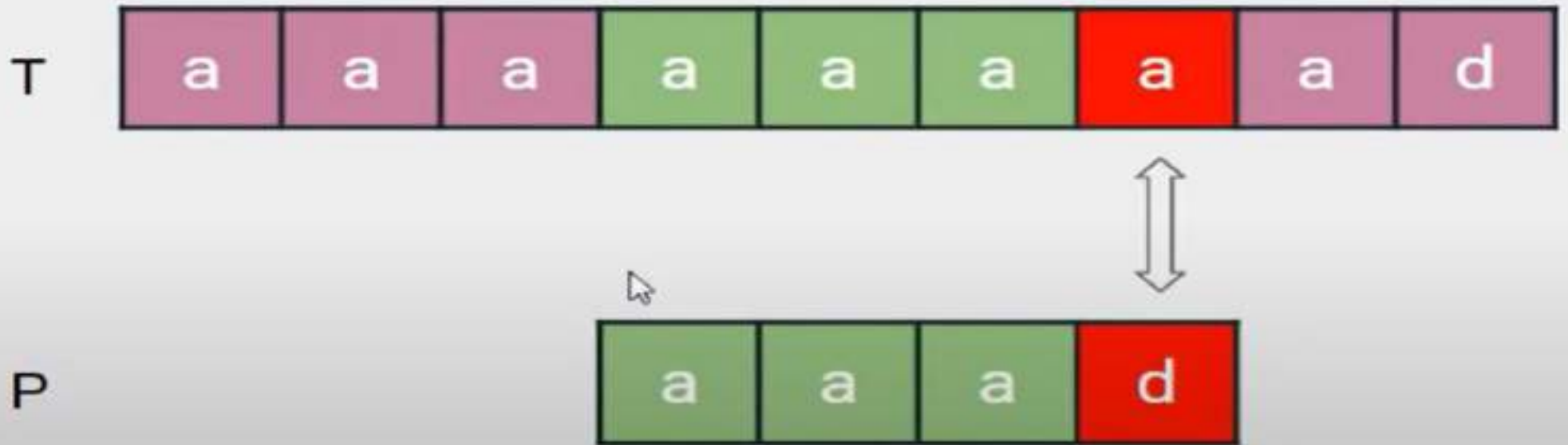
Worst Case



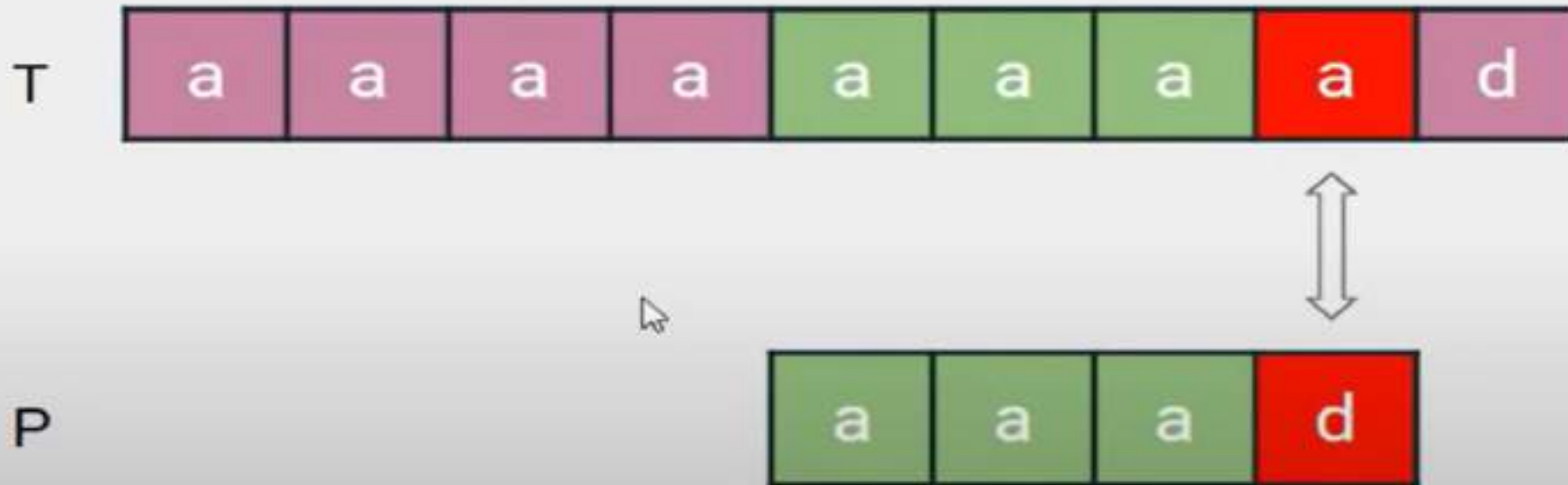
Worst Case



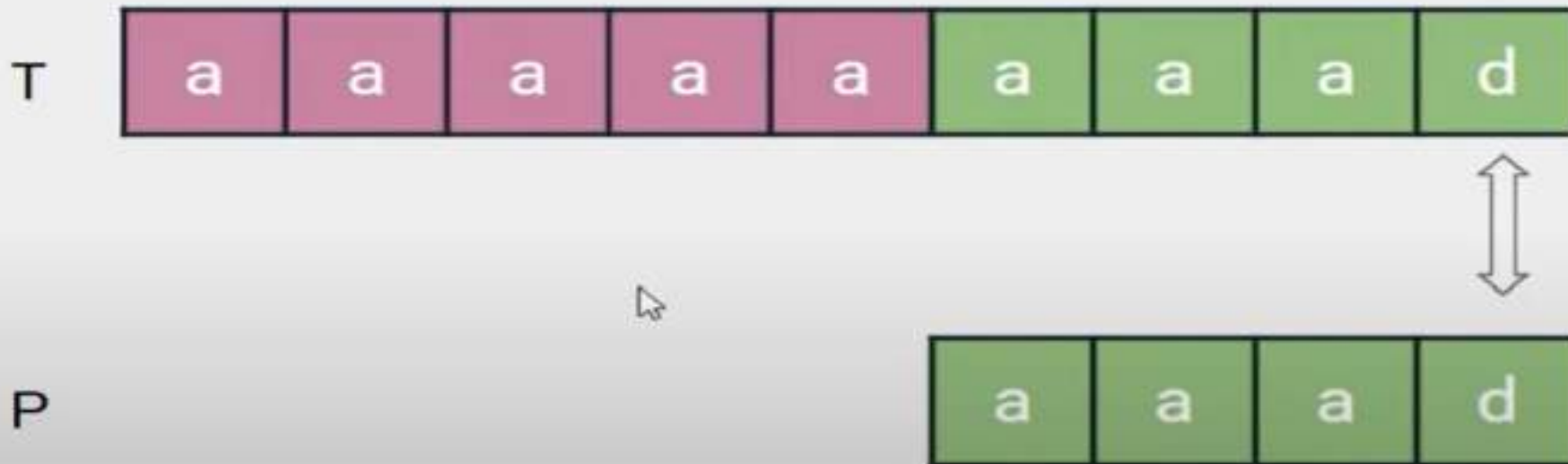
Worst Case



Worst Case

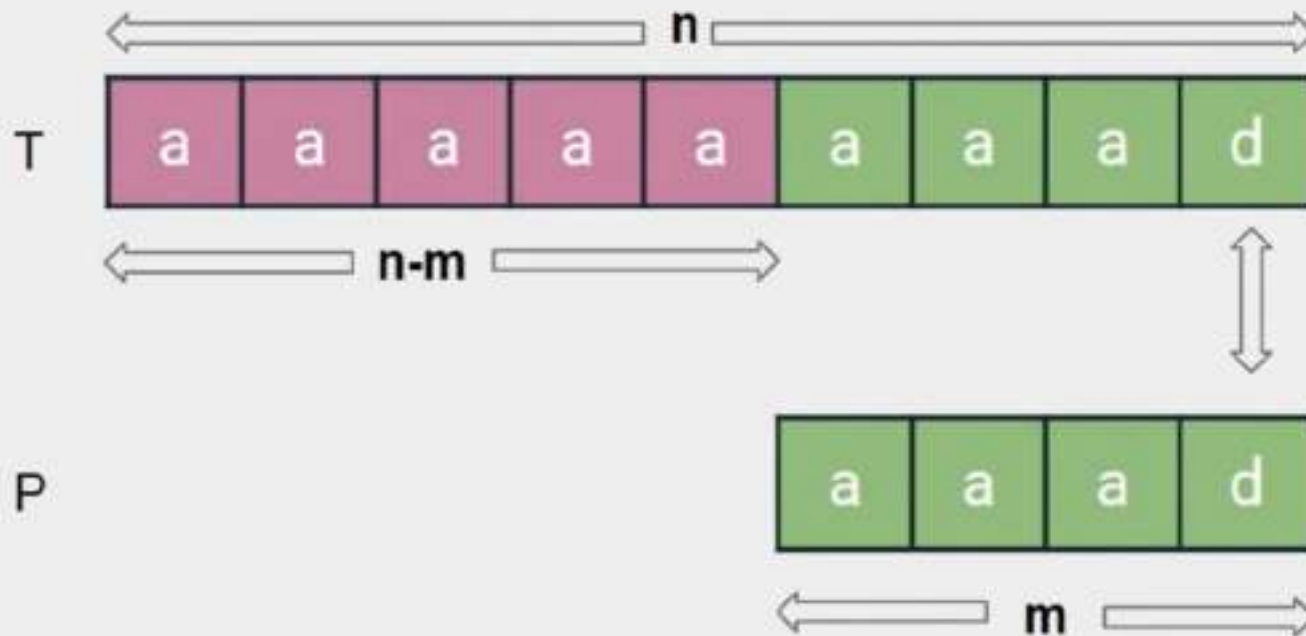


Worst Case



Brute Force Approach (Naive)

Worst Case

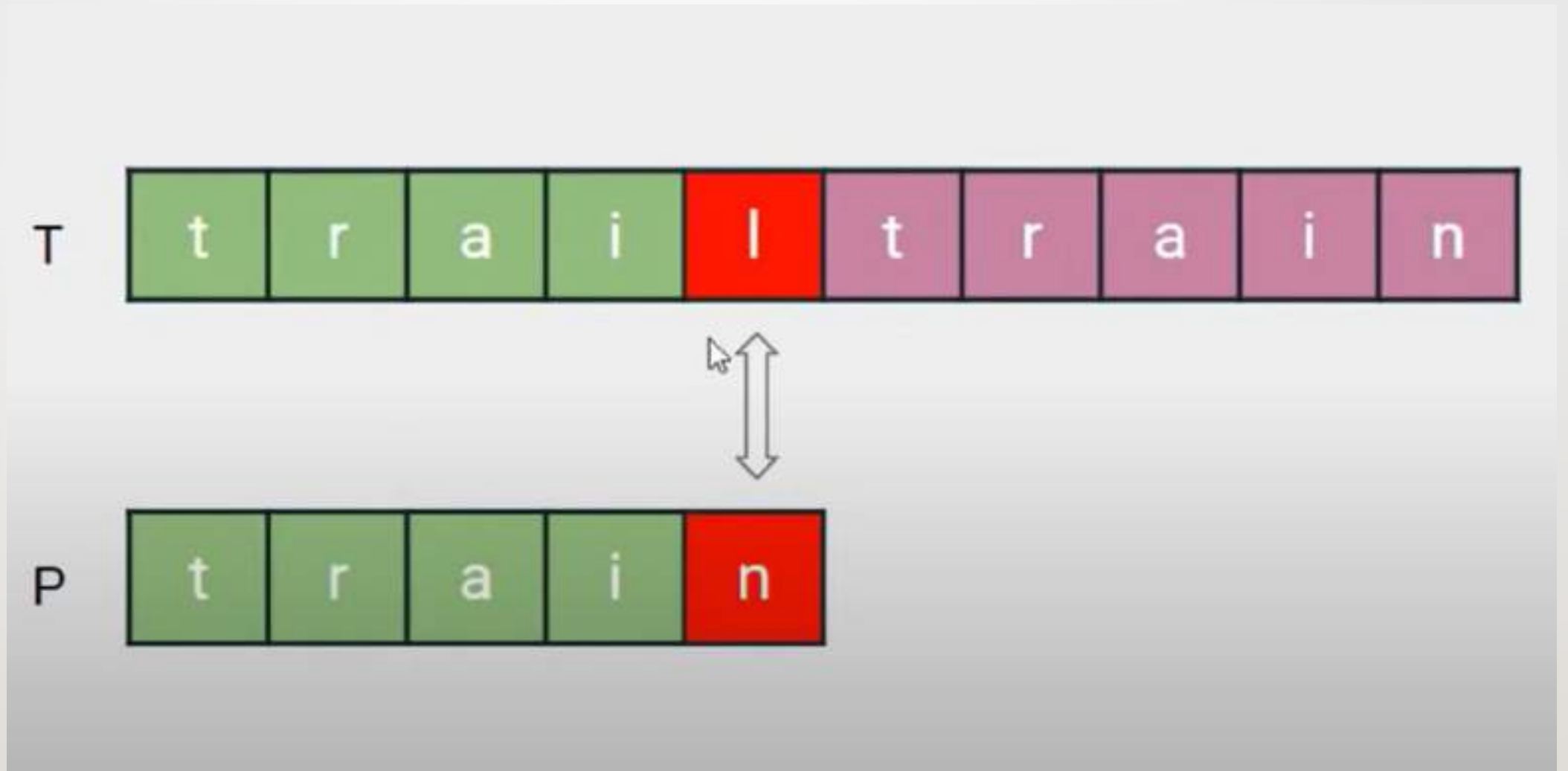


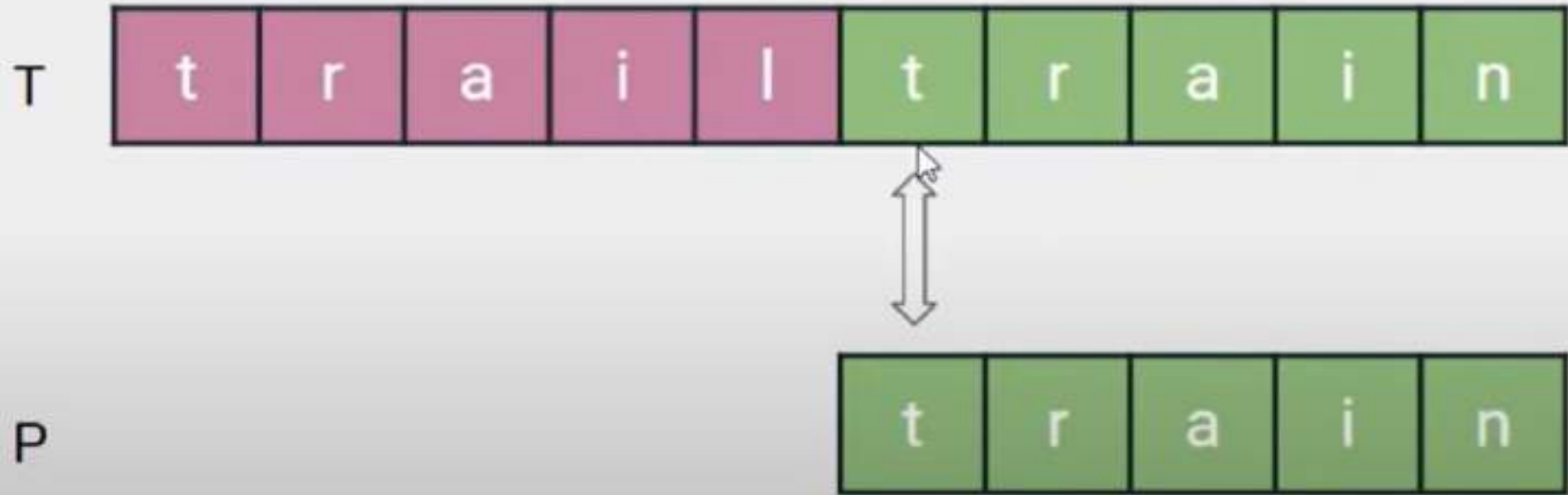
Time Complexity

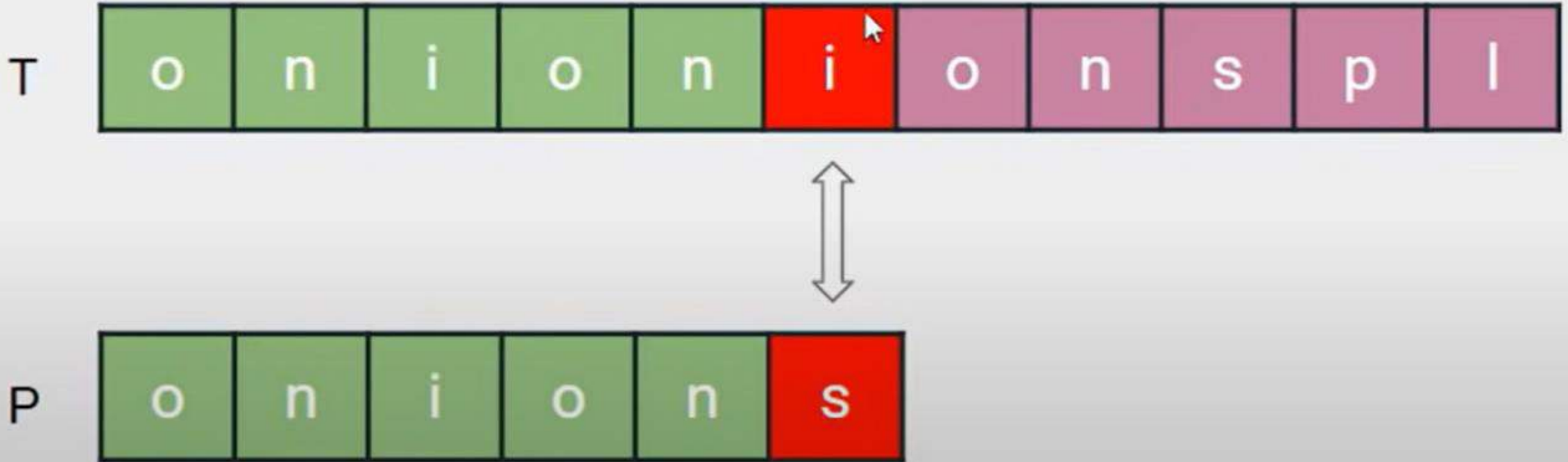
$$O([n-m+1]*m)$$

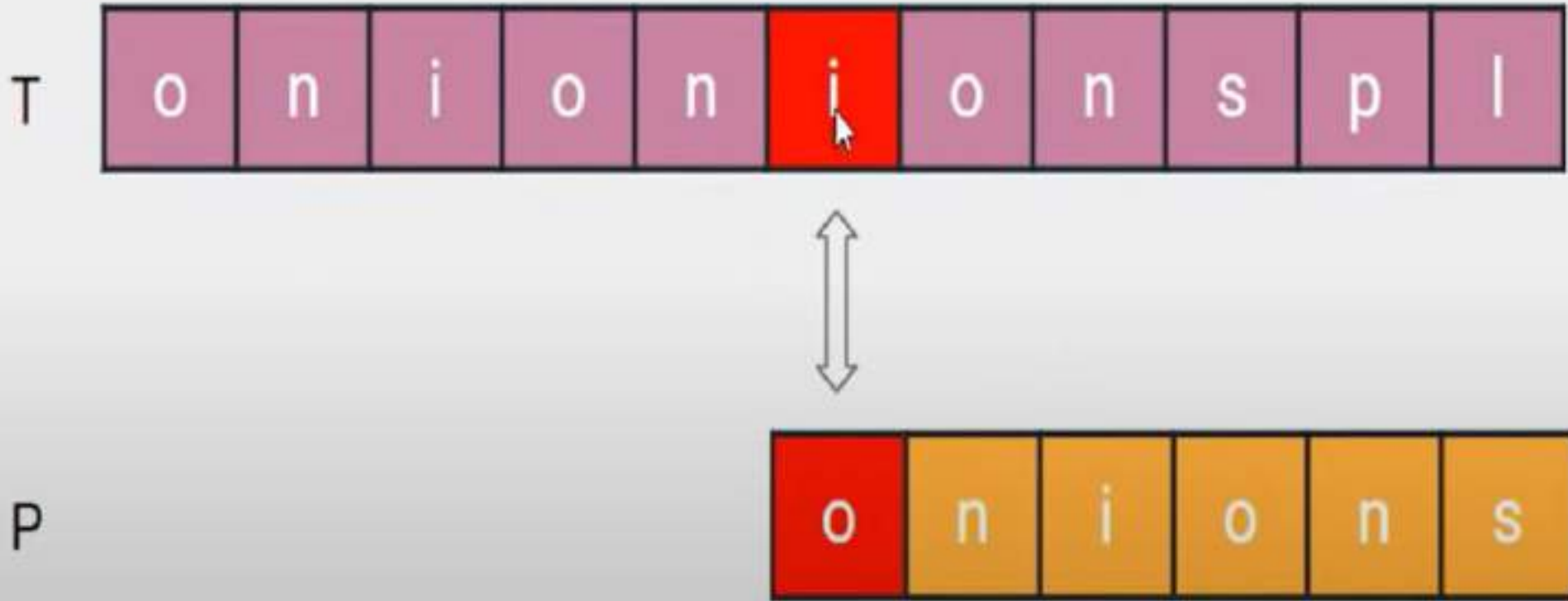
$$\sim O(nm)$$

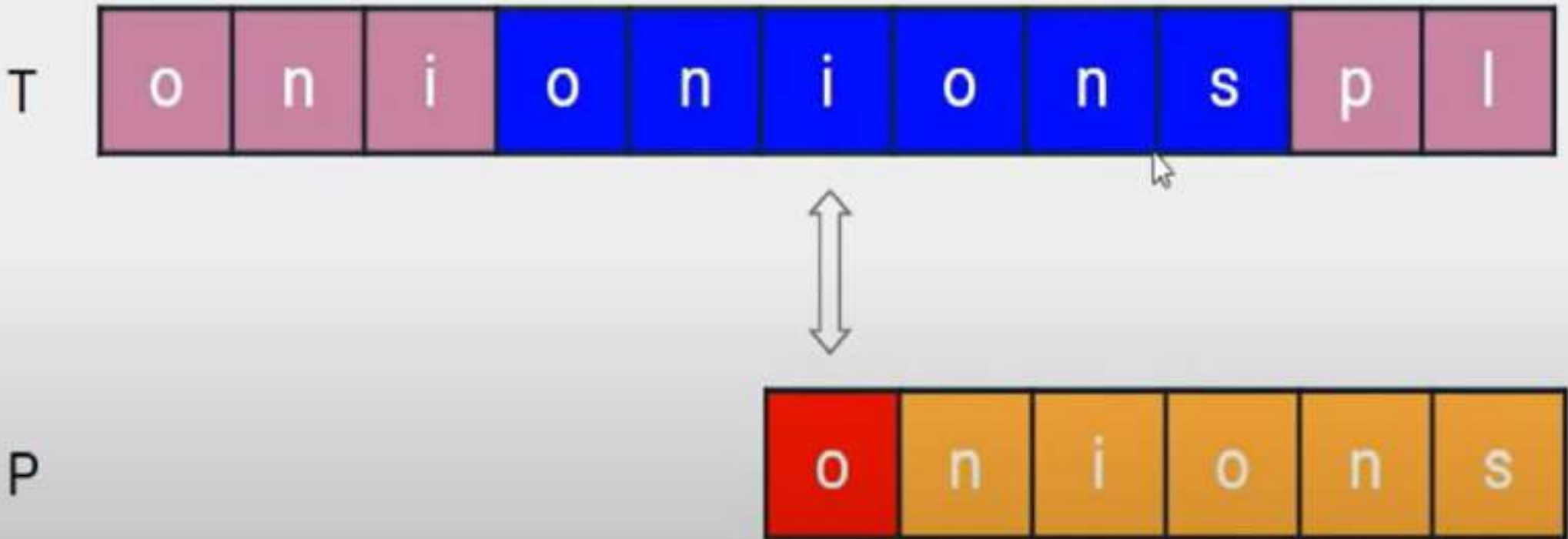
when $n \gg m$



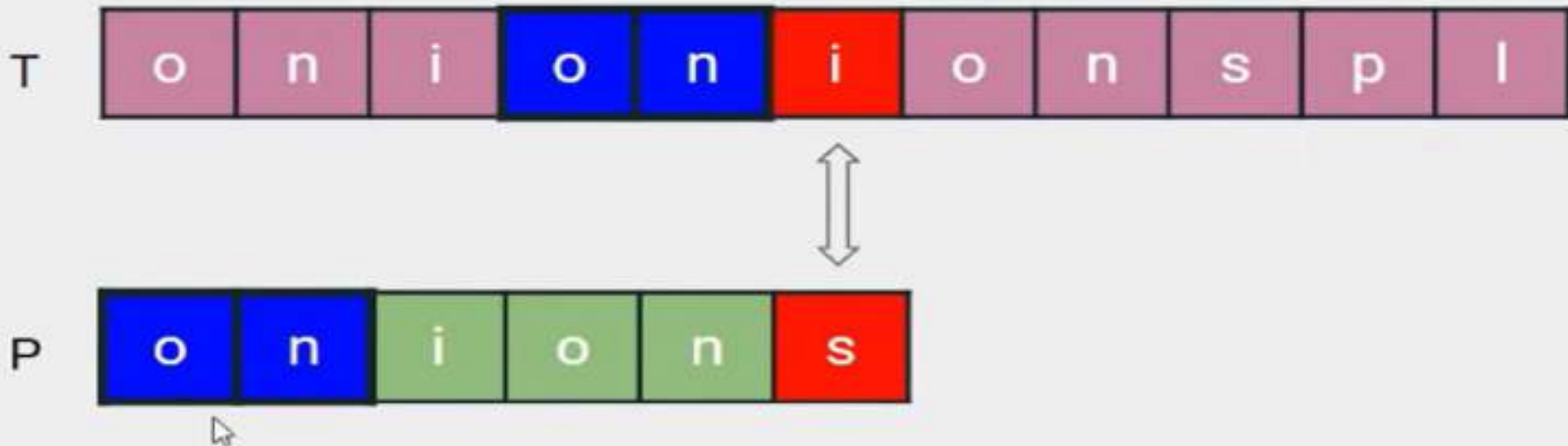








KMP algorithm



Knuth-Morris-Pratt Algorithm

KMP Algorithm is one of the most popular patterns matching algorithms. KMP stands for Knuth Morris Pratt. KMP algorithm was invented by **Donald Knuth** and **Vaughan Pratt** together and independently by **James H Morris** in the year 1970. In the year 1977, all the three jointly published KMP Algorithm.

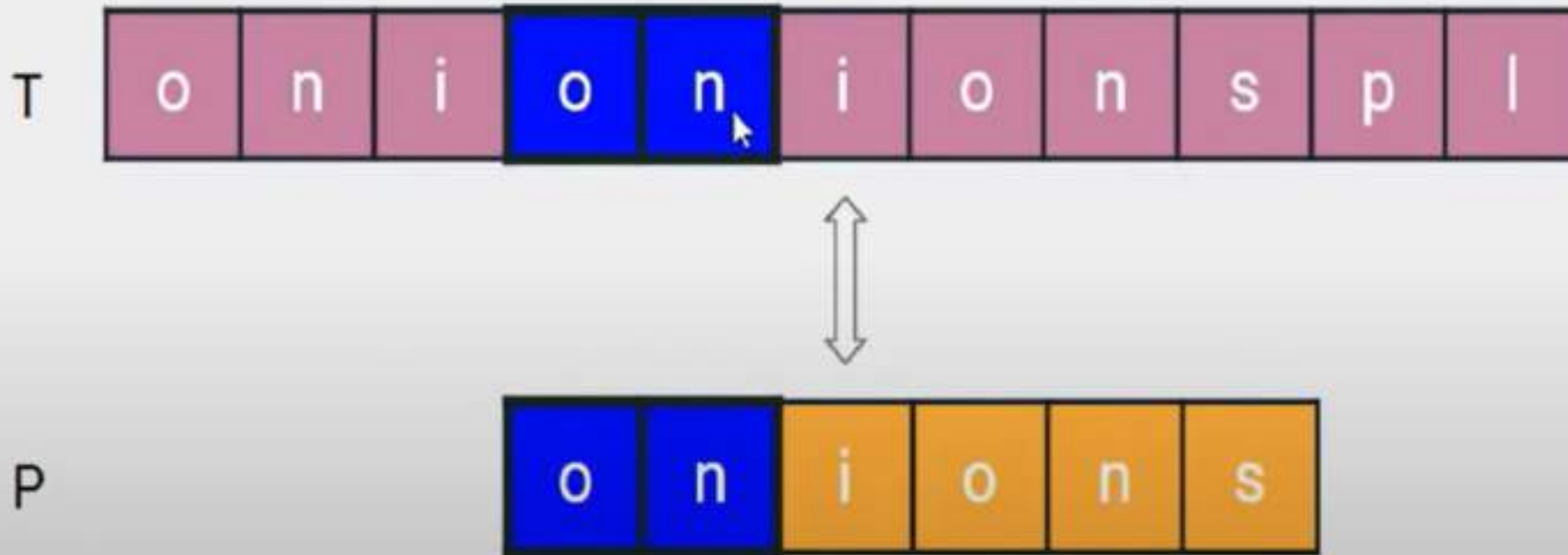
KMP algorithm was the first **linear time complexity** algorithm for string matching.

KMP algorithm is one of the string matching algorithms used to find a Pattern in a Text.

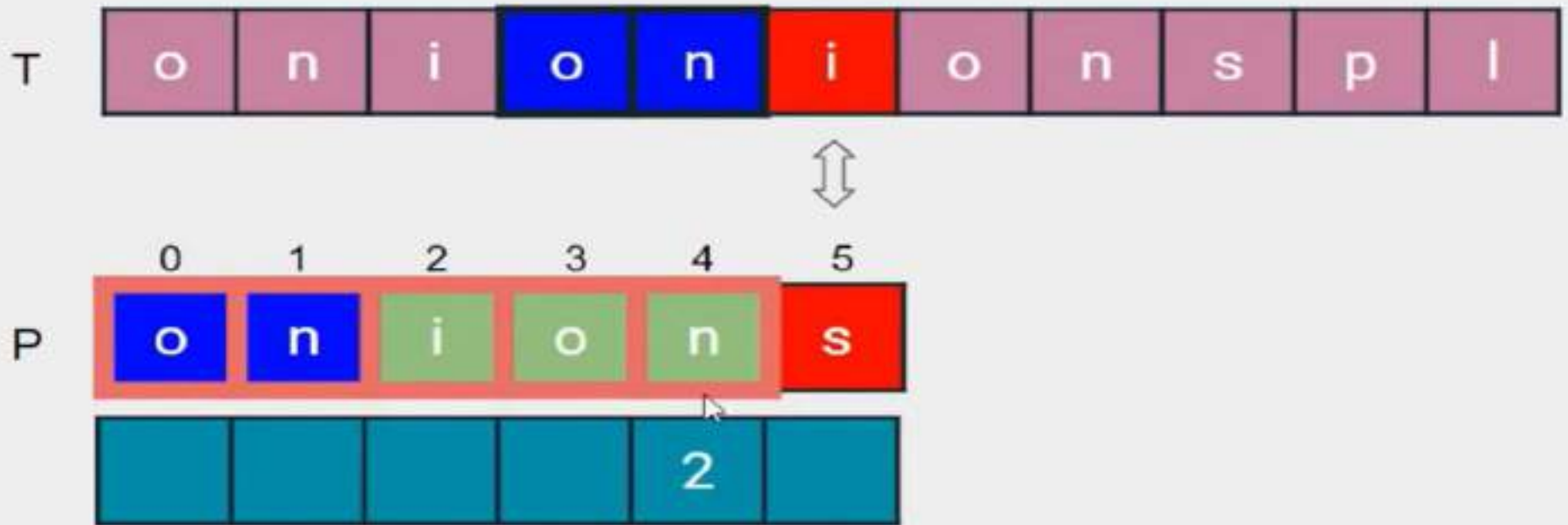
KMP algorithm is used to find a "**Pattern**" in a "**Text**". This algorithm compares character by character from left to right. But whenever a mismatch occurs, it uses a preprocessed table called "**Prefix Table**" to skip characters comparison while matching. Some times prefix table is also known as **LPS Table**. Here LPS stands for "**Longest proper Prefix which is also Suffix**".

KMP algorithm

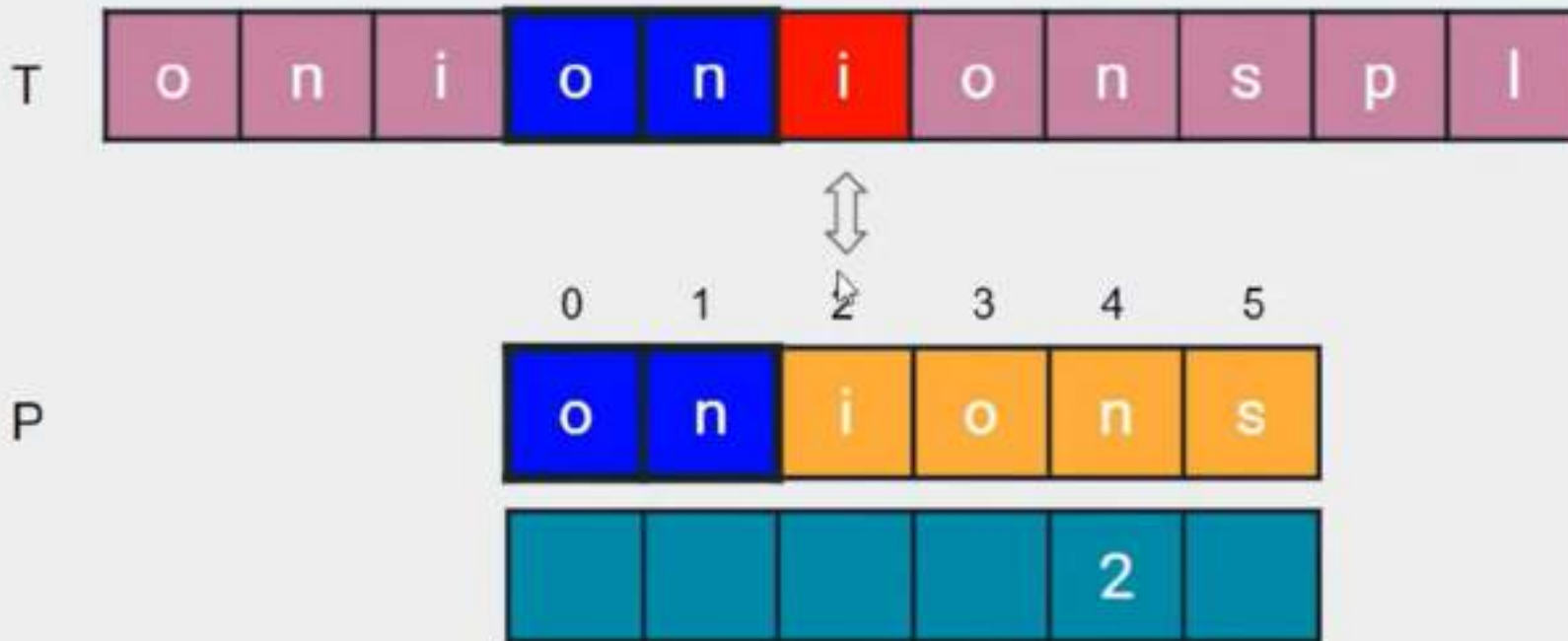
- Knuth-Morris-Pratt is the first linear time algorithm for string matching.
- Prevents the re examination of previously matched characters.



KMP algorithm



KMP algorithm



LPS array

proper

Longest prefix that is also a suffix

P

o	n	i			
---	---	---	--	--	--

LPS

0	0	0			
---	---	---	--	--	--

o
On
oni

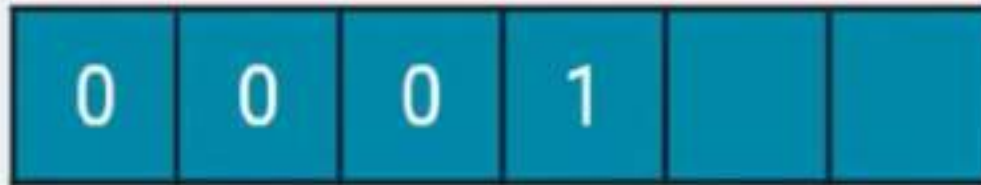
i
ni
oni x

Longest prefix that is also a suffix

P



LPS

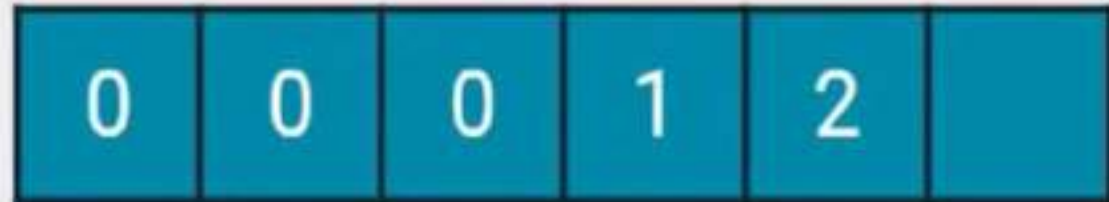


Longest prefix that is also a suffix

P



LPS

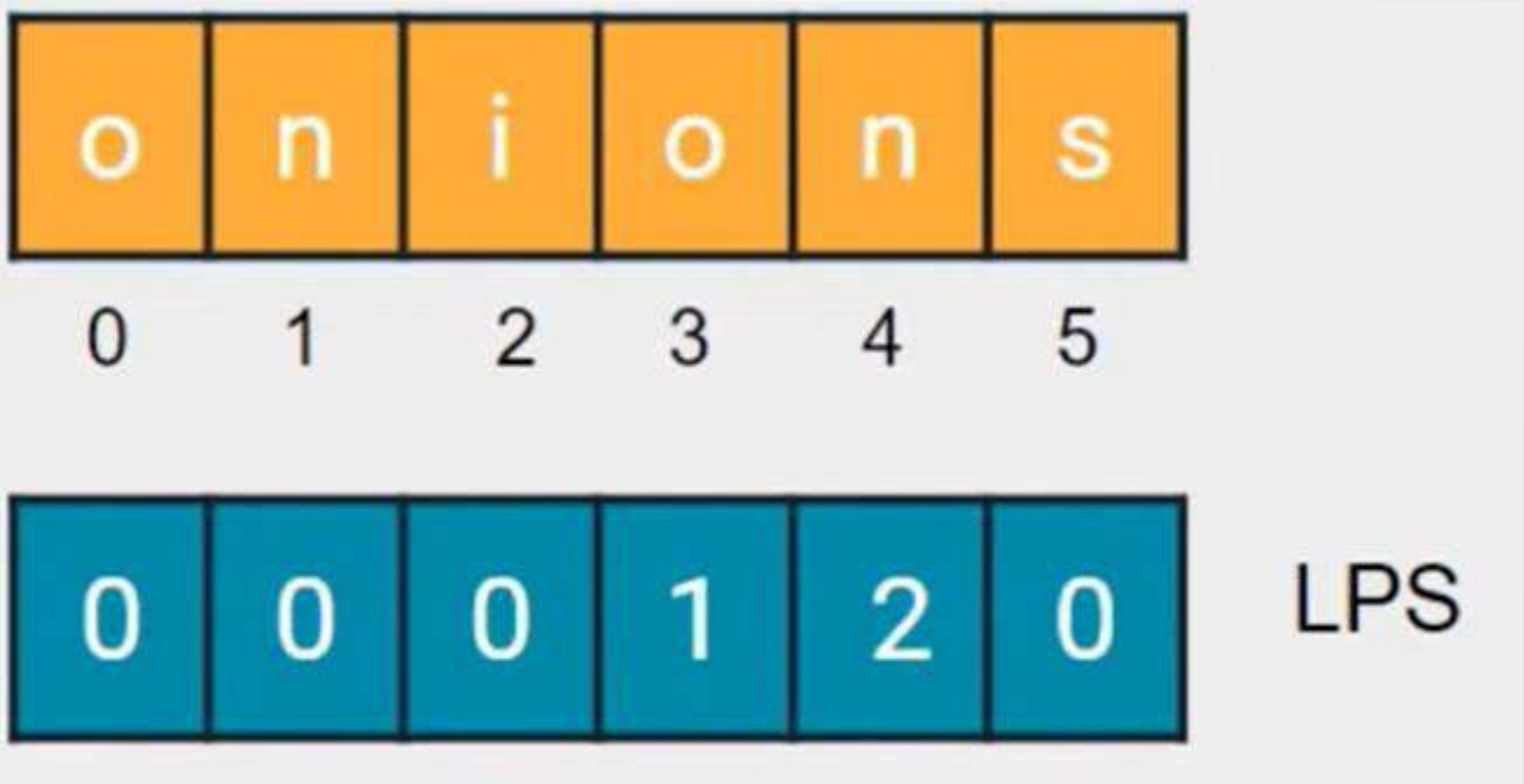


LPS array

Longest prefix that is also a suffix

P	o	n	i	o	n	s
LPS	0	0	0	1	2	

o
 on
 oni
 onio
 onion
 s
 ns
 ons
 ions
 nions



Implementation - python

```
def KMPSearch(pat, txt):
    N = len(txt)
    M = len(pat)
    lps = [0]*M
    computeLPSArray(pat, M, lps)
    i=0
    j=0
    while i < N:
        if txt[i] == pat[j]:
            i += 1
            j += 1
        else:
```

N = 11

M = 6

0	1	2	3	4	5	6	7	8	9	10
o	n	i	o	n	i	o	n	s	p	i

0	1	2	3	4	5
o	n	i	o	n	s

i ↓ j

0	0	0	1	2	0
---	---	---	---	---	---

LPS

Implementation - python

```
def KMPSearch(pat, txt):
    N = len(txt)
    M = len(pat)
    lps = [0]*M
    computeLPSArray(pat, M, lps)
    i=0
    j=0
    while i < N:
        if txt[i] == pat[j]:
            i += 1
            j += 1
        else:
            j = lps[j-1]
```

N = 11

M = 6

0	1	2	3	4	5	6	7	8	9	10
o	n	i	o	n	i	o	n	s	p	i

0	1	2	3	4	5
o	n	i	o	n	s

i ↓
j ↓

0	0	0	1	2	0
---	---	---	---	---	---

LPS

```
def KMPSearch(pat, txt):
    N = len(txt)
    M = len(pat)
    lps = [0]*M
    computeLPSArray(pat, M, lps)
    i=0
    j=0
    while i < N:
        if txt[i] == pat[j]:
            i += 1
            j += 1
        else:
            if j != 0:
                j = lps[j-1]
            else:
```

0	1	2	3	4	5
a	b	c	a	b	c

i
↑
j

x	y	z
---	---	---

0	1	2
---	---	---

Implementation - python

```
def KMPSearch(pat, txt):
    N = len(txt)
    M = len(pat)
    lps = [0]*M
    computeLPSArray(pat, M, lps)
    i=0
    j=0
    while i < N-M+1:
        if txt[i] == pat[j]:
            i += 1
            j += 1
        else:
            if j != 0:
                j = lps[j-1]
            else:
                i += 1
        if j == M:
            print(i-j)
            j = lps[j-1]
```

N = 11

M = 6

0	1	2	3	4	5	6	7	8	9	10
o	n	i	o	n	i	o	n	s	p	l

0	1	2	3	4	5
o	n	i	o	n	s

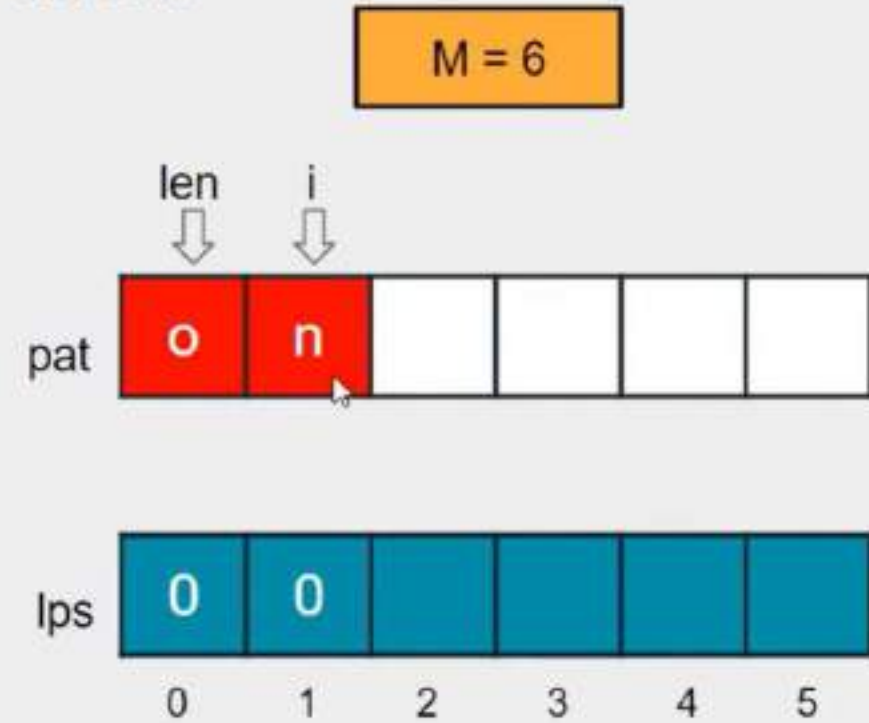
i
↕
j

0	0	0	1	2	0
---	---	---	---	---	---

LPS

Implementation (LPS) - python

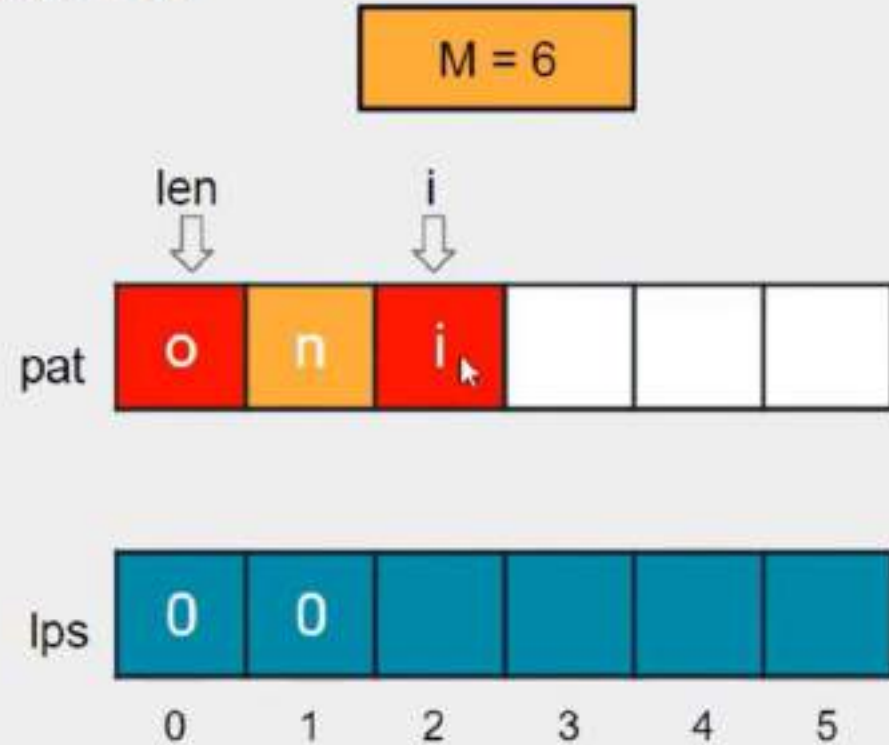
```
def computeLPSArray(pat, m, lps):
    len = 0
    i = 1
    lps[0] = 0
    while i < M:
        if pat[i] == pat[len]:
            # Matching found, increment len and i
            len = len + 1
            i = i + 1
            lps[i] = len
        else:
            # Mismatch, reset len to 0 and increment i
            len = 0
            i = i + 1
            lps[i] = len
```



Implementation (LPS) - python

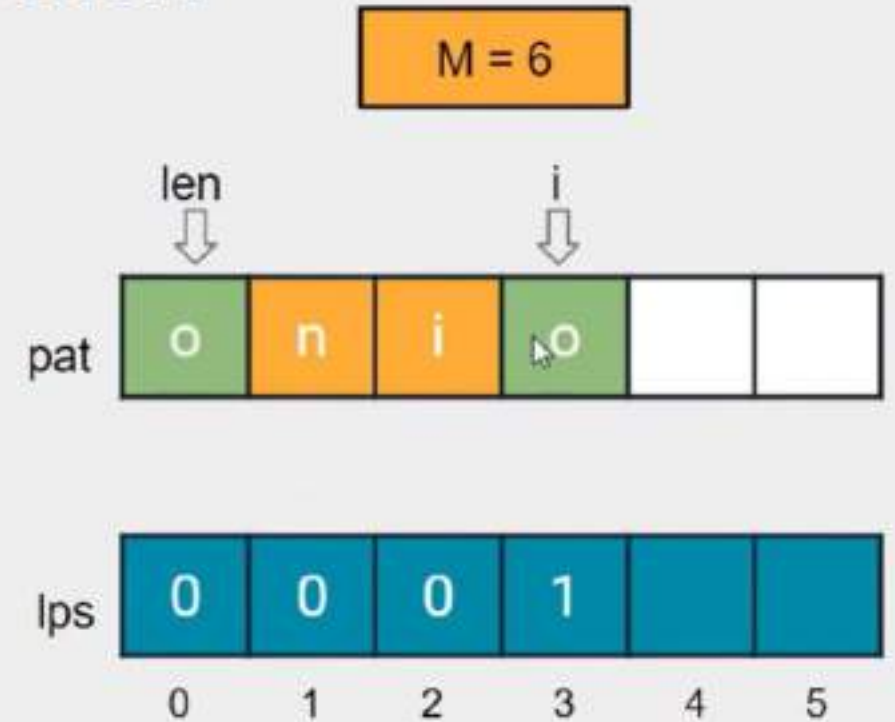
```
def computeLPSArray(pat,m,lps):
    len = 0
    i = 1
    lps[0] = 0
    while i < M:
        if pat[i] == pat[len]:

        else:
            lps[i] = 0
            i += 1
```



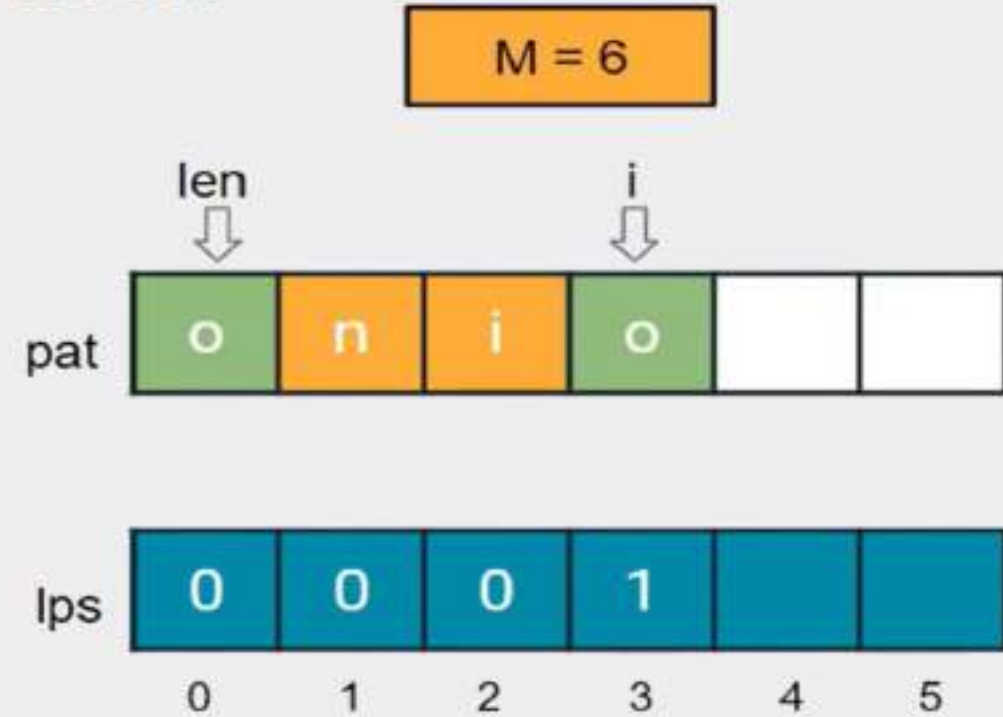
Implementation (LPS) - python

```
def computeLPSArray(pat,m,lps):
    len = 0
    i = 1
    lps[0] = 0
    while i < M:
        if pat[i] == pat[len]:
            lps[i] = len + 1
        else:
            lps[i] = 0
            i += 1
```



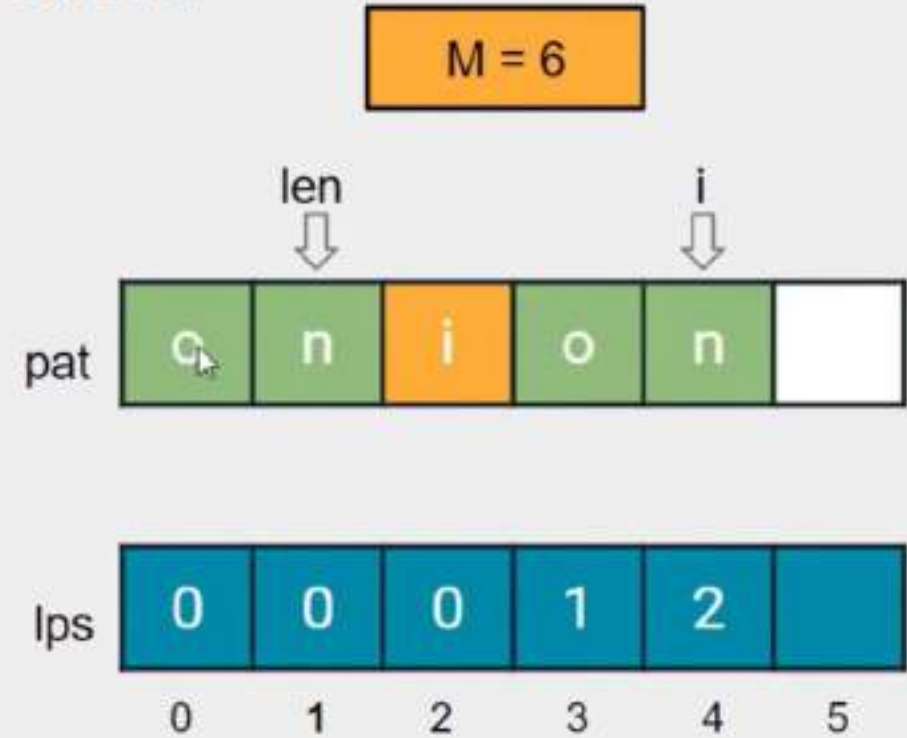
Implementation (LPS) - python

```
def computeLPSArray(pat,m,lps):
    len = 0
    i = 1
    lps[0] = 0
    while i < M:
        if pat[i] == pat[len]:
            lps[i] = len + 1
            i += 1
        else:
            lps[i] = 0
            i += 1
```



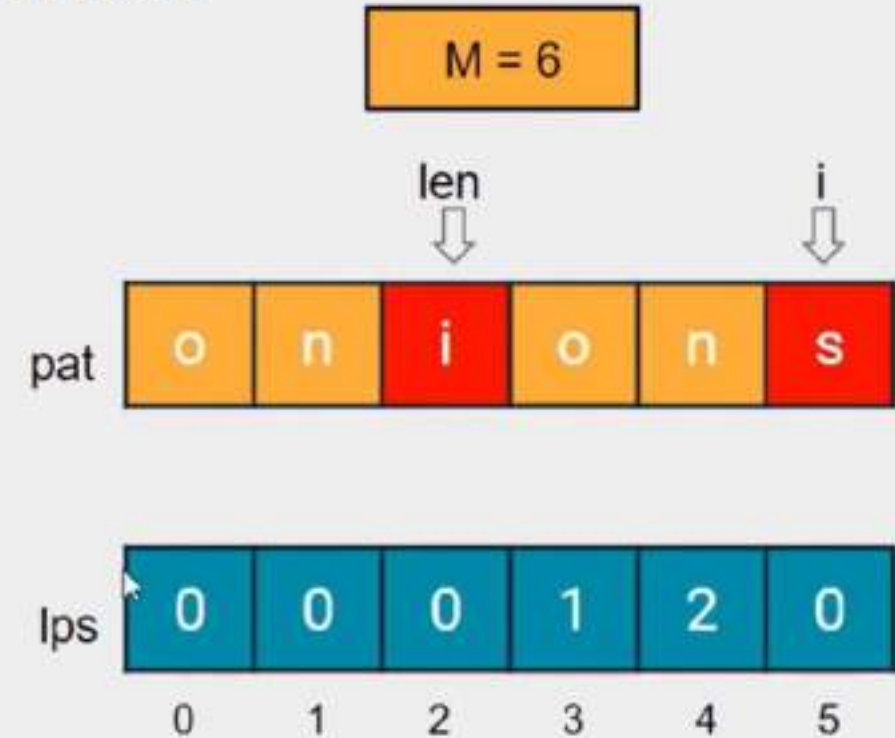
Implementation (LPS) - python

```
def computeLPSArray(pat,m,lps):
    len = 0
    i = 1
    lps[0] = 0
    while i < M:
        if pat[i] == pat[len]:
            lps[i] = len + 1
            len += 1
            i += 1
        else:
            lps[i] = 0
            i += 1
```



Implementation (LPS) - python

```
def computeLPSArray(pat,m,lps):
    len = 0
    i = 1
    lps[0] = 0
    while i < M:
        if pat[i] == pat[len]:
            lps[i] = len + 1
            len += 1
            i += 1
        else:
            lps[i] = 0
            i += 1
```



Implementation (LPS) - python

```
def computeLPSArray(pat,m,lps):  
    len = 0  
    i = 1  
    lps[0] = 0  
    while i < M:  
        if pat[i] == pat[len]:  
            lps[i] = len + 1  
            len += 1  
            i += 1  
        else:  
            if len != 0:  
                len = lps[len-1]  
            else:  
                lps[i] = 0  
                i += 1
```

```
def KMPSearch(pat, txt):  
    N = len(txt)  
    M = len(pat)  
    lps = [0]*M  
    computeLPSArray(pat, M, lps)  
    i=0  
    j=0  
    while i < N-M+1:  
        if txt[i] == pat[j]:  
            i += 1  
            j += 1  
        else:  
            if j != 0:  
                j = lps[j-1]  
            else:  
                i += 1  
    if j == M:  
        print(i-j)  
        j = lps[j-1]
```

Time complexity

For KMPsearch $\rightarrow O(n)$



T

o	n	i	o	n	i	o	n	s	p	l
---	---	---	---	---	---	---	---	---	---	---

P

o	n	i	o	n	s
---	---	---	---	---	---

Time complexity

For computeLPSArray $\rightarrow O(m)$

```
def computeLPSArray(pat,m,lps):  
    len = 0  
    i = 1  
    lps[0] = 0  
    while i < M:  
        if pat[i] == pat[len]:  
            lps[i] = len + 1  
            len += 1  
            i += 1  
        else:  
            if len != 0:  
                len = lps[len-1]  
            else:  
                lps[i] = 0  
                i += 1
```



Time complexity

Overall $\rightarrow O(n+m)$

$\Rightarrow O(n)$ for $n \gg m$

Space complexity

$O(m)$ extra space for LPS array

Applications

Best applied when the text and pattern has a lot of repeated characters.

Eg -> DNA pattern matching

Only the characters A C G T are repeated in DNA.

Example for creating KMP Algorithm's LPS Table (Prefix Table)

Consider the following Pattern

Pattern :

0	1	2	3	4	5	6
A	B	C	D	A	B	D

Let us define LPS[] table with size 7 which is equal to length of the Pattern

LPS

0	1	2	3	4	5	6

Step 1 - Define variables i & j. Set $i = 0$, $j = 1$ and $LPS[0] = 0$.

LPS

0	1	2	3	4	5	6
0						

$i = 0$ and $j = 1$

Step 2 - Compare Pattern[i] with Pattern[j] ==> A with B.

Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0					

i = 0 and j = 2

Step 3 - Compare Pattern[i] with Pattern[j] ==> A with C.

Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0	0				

i = 0 and j = 3

Step 4 - Compare Pattern[i] with Pattern[j] ==> A with D.

Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0			

i = 0 and j = 4

Step 5 - Compare Pattern[i] with Pattern[j] ==> A with A.

Since both are matching set LPS[j] = i+1 and increment both i & j value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1		

i = 1 and j = 5

Step 6 - Compare Pattern[i] with Pattern[j] ==> B with B.

Since both are matching set $LPS[j] = i+1$ and increment both i & j value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	

i = 2 and j = 6

Step 7 - Compare Pattern[i] with Pattern[j] ==> C with D.

Since both are not matching and $i \neq 0$, we need to set $i = LPS[i-1]$

==> $i = LPS[2-1] = LPS[1] = 0$.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	

i = 0 and j = 6

Step 8 - Compare Pattern[i] with Pattern[j] ==> A with D.

Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	0

Here LPS[] is filled with all values so we stop the process. The final LPS[] table is as follows...

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	0

Steps for Creating LPS Table (Prefix Table)

Step 1 - Define a one dimensional array with the size equal to the length of the Pattern. (LPS[size])

Step 2 - Define variables **i** & **j**. Set **i = 0**, **j = 1** and **LPS[0] = 0**.

Step 3 - Compare the characters at **Pattern[i]** and **Pattern[j]**.

Step 4 - If both are matched then set **LPS[j] = i+1** and increment both **i** & **j** values by one. Goto to Step 3.

Step 5 - If both are not matched then check the value of variable '**i**'. If it is '0' then set **LPS[j] = 0** and increment '**j**' value by one, if it is not '0' then set **i = LPS[i-1]**. Goto Step 3.

Step 6- Repeat above steps until all the values of LPS[] are filled.

How to use LPS Table

We use the LPS table to decide how many characters are to be skipped for comparison when a mismatch has occurred.

When a mismatch occurs, check the LPS value of the previous character of the mismatched character in the pattern. If it is '0' then start comparing the first character of the pattern with the next character to the mismatched character in the text. If it is not '0' then start comparing the character which is at an index value equal to the LPS value of the previous character to the mismatched character in pattern with the mismatched character in the Text.

How the KMP Algorithm Works

Let us see a working example of KMP Algorithm to find a Pattern in a Text...

Consider the following Text and Pattern

Text : ABC ABCDAB ABCDABCDABDE
Pattern : ABCDABD

LPS[] table for the above pattern is as follows...

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	0

Step 1 - Start comparing first character of Pattern with first character of Text from left to right

Text:

A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
---	---	---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6

Pattern:

A	B	C	D	A	B	D
---	---	---	---	---	---	---

Here mismatch occurred at Pattern[3], so we need to consider LPS[2] value. Since LPS[2] value is '0' we must compare first character in Pattern with next character in Text.

Step 2 - Start comparing first character of Pattern with next character of Text.

Text:

A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
---	---	---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6

Pattern:

A	B	C	D	A	B	D
---	---	---	---	---	---	---

Here mismatch occurred at Pattern[6], so we need to consider LPS[5] value. Since LPS[5] value is '2' we compare Pattern[2] character with mismatched character in Text.

Step 3 - Since LPS value is '2' no need to compare Pattern[0] & Pattern[1] values

Text

A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
---	---	---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6

Pattern

A	B	C	D	A	B	D
---	---	---	---	---	---	---

Here mismatch occurred at Pattern[2], so we need to consider LPS[1] value. Since LPS[1] value is '0' we must compare first character in Pattern with next character in Text.

Step 4 - Compare Pattern[0] with next character in Text.

Text

A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
---	---	---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6

Pattern

A	B	C	D	A	B	D
---	---	---	---	---	---	---

Here mismatch occurred at Pattern[6], so we need to consider LPS[5] value. Since LPS[5] value is '2' we compare Pattern[2] character with mismatched character in Text.

Step 5 - Compare Pattern[2] with mismatched character in Text.

Text

A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
---	---	---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

															0	1	2	3	4	5	6
															A	B	C	D	A	B	D

Here all the characters of Pattern matched with a substring in Text which is starting from index value 15. So we conclude that given Pattern found at index 15 in Text.

Boyer-Moore Pattern Matching

The **Boyer-Moore Pattern Matching algorithm** is one of the most efficient string-searching algorithm that is the standard benchmark for practical pattern matching. It was developed by **Robert Stephen Boyer** and **J Strother Moore** in the year **1977**.

The Boyer-Moore algorithm works by **pre-processing** the pattern and then scanning the text from right to left, starting with the rightmost characters. It is based on the principle that if a mismatch is found, there is no need to match the remaining characters. This backwards approach significantly reduces the algorithm's time complexity compared to naive string search methods.

Here's a step-by-step explanation of how the Boyer-Moore algorithm works:

1. Preprocessing

Boyer-Moore algorithm uses two heuristics for preprocessing the pattern:

- a. Bad Character Heuristic**
- b. Good Suffix Heuristic**

Bad Character Heuristic (Building Bad Character Table)

Determines how far the pattern can be shifted when a character mismatch occurs.

- Create an array (often called the "**Bad Character Table**") to store the rightmost occurrence of each character in the pattern. If a character is not in the pattern, its value is set to **pattern length**.
- For each character in the pattern, update the table with its last occurrence in the pattern with **pattern length - index - 1**.

$$BCT(ch) = \begin{cases} m & \text{if } ch \text{ is not in } p \\ m-j-1 & \text{otherwise} \end{cases}$$

Example:

$j \rightarrow$	0	1	2	3	4
P	M	I	S	S	I

Length(m)=5

Bad Character Table

M	I	S	*
4	0	1	5

$$M \rightarrow (m-j-1) = 5-0-1 = 4$$

$$I \rightarrow (m-j-1) = 5-1-1 = 3$$

$$S \rightarrow (m-j-1) = 5-2-1 = 2$$

$$S \rightarrow (m-j-1) = 5-3-1 = 1$$

$$I \rightarrow (m-j-1) = 5-4-1 = 0$$

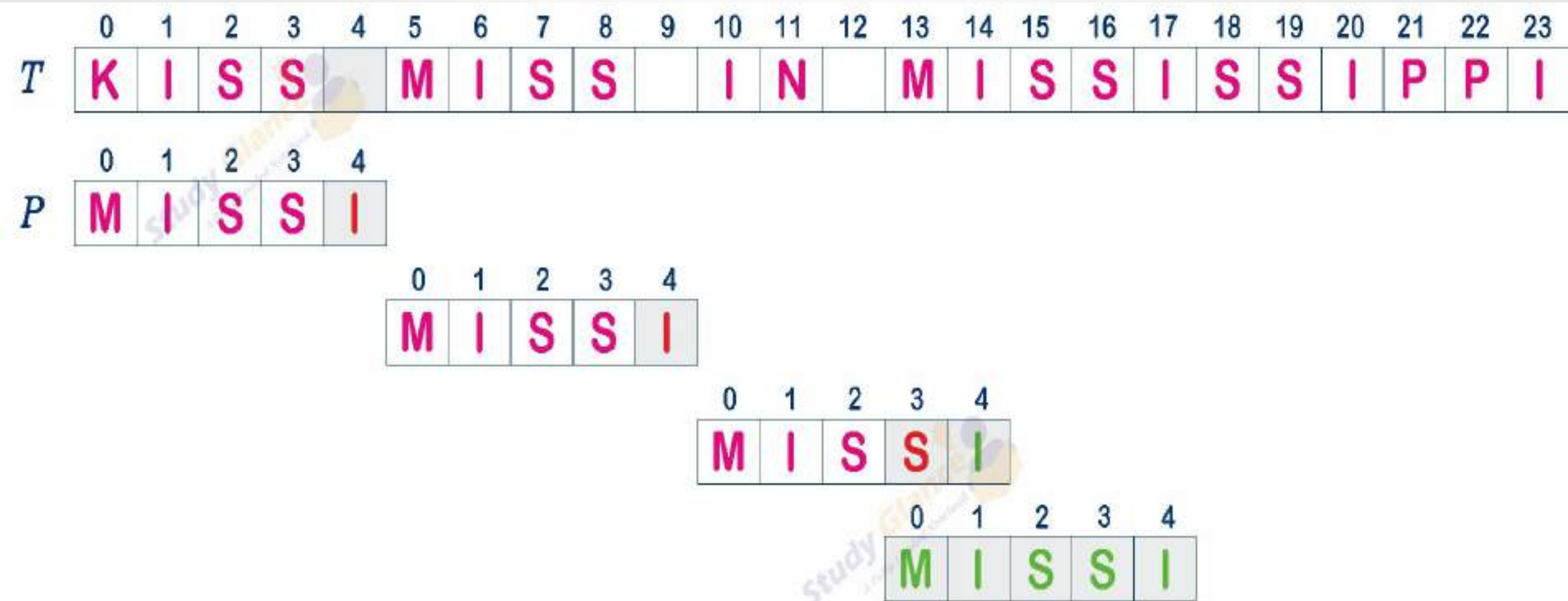
2. Searching

- a. Align the pattern with the beginning of the text.
- b. Compare the pattern with the text from right to left.
- c. If all characters of the pattern match, a valid occurrence is found.

3. Shifting the Pattern:

When a mismatch occurs

- **If the character is in the pattern:** Shift the pattern such that the mismatched character in the text aligns with the rightmost occurrence of it in the pattern.
- **If the character is not in the pattern:** shift the entire pattern past the character.



3. Repeat Comparison

After shifting the pattern according to the above rules, repeat the comparison process:

- a. Continue comparing the pattern with the text from right to left.
- b. Apply the shifting rules whenever a mismatch is encountered.
- c. Continue this process until the end of the text is reached or all occurrences of the pattern are found.

4. Termination

The algorithm terminates when either

- The pattern has been shifted past the **end of the text**, indicating no more matches are possible.
- All occurrences of the **pattern has have been found**.

Pseudo code for Boyer-Moore Pattern Matching algorithm

Pseudo-code

```
Boolean BoyerMoore( T: text, P: pattern)
n: length of T,      i: index of T
m: length of P,      j: index of P
bctable[ch] is an array, for ch = 0 to 127 (ASCII values)

1. Initialize all elements of array bctable to m.
2. Set bctable[Pj] = m-j-1, for j= 0 to m-1.
3. Initialize i = m-1
4. Repeat steps 5 to 8, while(i < n)
5. j := m-1
6. Repeat steps (a) and (b), while (j ≥ 0 and Pj = Ti)
   (a) decrement i
   (b) decrement j
7. if (j = -1) return true      //pattern matching successful
8. i := i+ max(bctable[Ti], m-j)
9. return false                //at the end, if pattern matching fails, return false
```

Bad Character Approach

- ❖ The character of the text which doesn't match with the current character of pattern is called the **Bad Character**.
- ❖ Upon mismatch we shift the pattern until –
 - 1) The mismatch become a match.
 - ❖ If the mismatch occur then we see the Bad-Match table for shifting the pattern.
 - 2) Pattern P move past the mismatch character.
 - ❖ If the mismatch occur and the mismatch character not available in the Bad-Match Table then we shift the whole pattern accordingly.

Boyer-Moore Algorithm

Steps to find the pattern :

Step 1: Construct the bad-symbol shift table.

Step 2: Align the pattern against the beginning of the text.

Step 3: Repeat the following step until either a matching substring is found or the pattern reaches beyond the last character of the text.

1. Construct Bad Match Table:

Formula for constructing Bad Match Table:

- **Formula:**

Values = Length of pattern-index-1

Construct Bad Match Table:

Example:

Text: " WELCOMETOTEAMMAST"

Pattern: 'TEAMMAST'

Pattern T E A M M A S T Length = 8
Index# 0 1 2 3 4 5 6 7

Letter	T	E	A	M	S	*
Values						

Bad Match Table

Construct Bad Match Table:

Pattern T E A M M A S T Length = 8
Index# 0 1 2 3 4 5 6 7

↓

Letter	T	E	A	M	S	*
Values	7	6	5	3		

Values = $\max(1, \text{Length of string} - \text{index} - 1)$

$$M = \max(1, 8 - 4 - 1) = 3$$

Construct Bad Match Table:

Pattern **TEAMMAST** Length = 8
Index# 0 1 2 3 4 5 6 7



Letter	T	E	A	M	S	*
Values	7	6	2	3	1	

Values = Length of string - index - 1

$$S = \max(1, 8 - 6 - 1) = 1$$

Construct Bad Match Table:

Pattern **T E A M M A S T** Length = 8
Index# 0 1 2 3 4 5 6 7

Letter	T	E	A	M	S	*
Values	8	6	2	3	1	8



Any other letter is presented by '*' is taken equal value to length of string i.e 8 here.

2. Align the pattern

Text:

W E L C O M E T O T E A M M A S T

Pattern:

T E A M M A S T



Move 6 spaces toward right

Bad-Match Table

Letter	Values
T	8
E	6
A	2
M	3
S	1
*	8



Matching.....



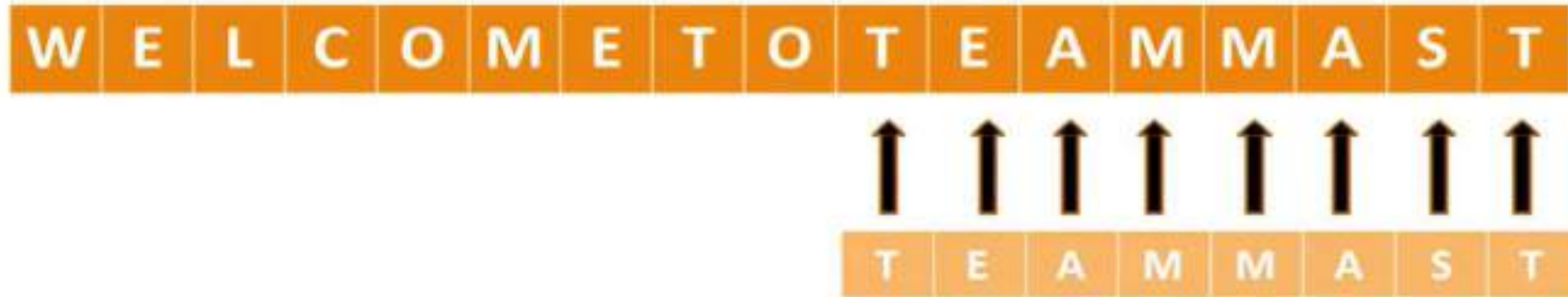
Now move 3 spaces toward right.

Bad-Match Table

Letter	Values
T	8
E	6
A	2
M	3
S	1
*	8



Matching.....



Hence the pattern match.....

Bad-Match Table

Letter	Values
T	8
E	6
A	2
M	3
S	1
*	8

Letter	a	b	c	d	*
Value	3	2	1	4	4

e o v a d a b c d f t o y

a b c d



Move 3 positions

e o v a d a b c d f t o y

a b c d



Move 2 positions

e o v a d a b c d f t o y

a b c d

Good Suffix Rule

Before breaking down the pseudocode, here is an explanation of the idea behind this rule.

Let t be a suffix of P such that it matches with a substring in T . Consider figure 1. We start from $i = 11$ and $j = 11$ then keep comparing $T[i]$ with $P[j]$ by decrementing both i and j until $T[i] \neq P[j]$. In figure 1, $T[10...11]$ is equal to the suffix of P whose starting index is 10 ($P[10...11]$).

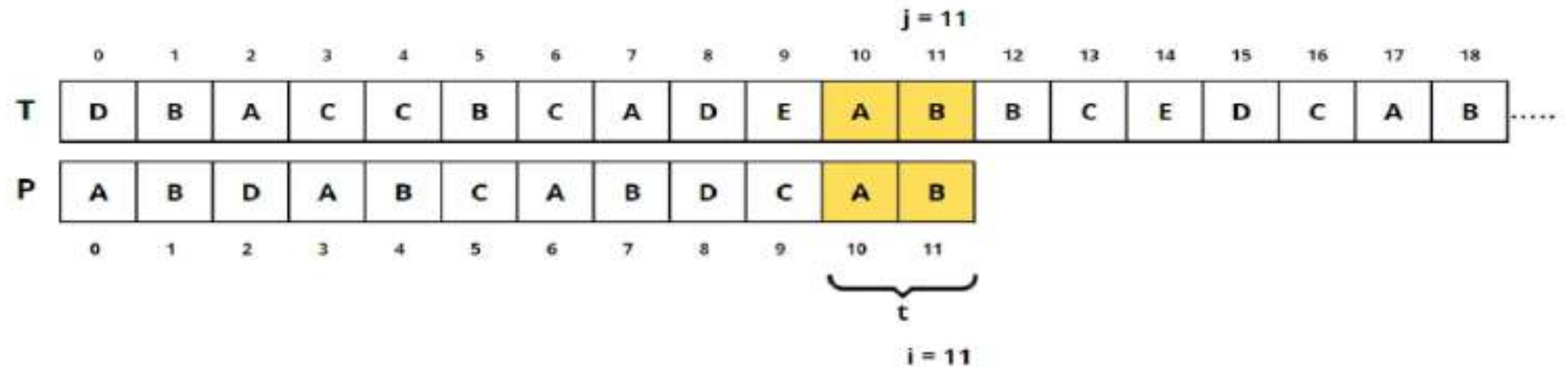


Figure 2 shows the situation in which we have found a mismatch at $i = 9$ and $j = 9$. Now we cannot continue to match T with P . P must be shifted to the right by at least 1 step to continue the search process. When shifting P to the right, we can possibly encounter 3 cases.

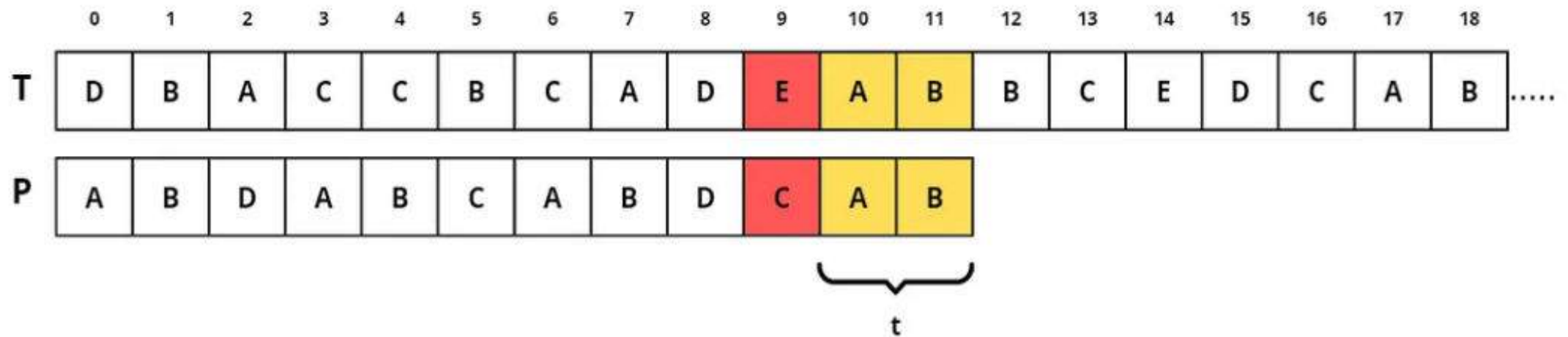
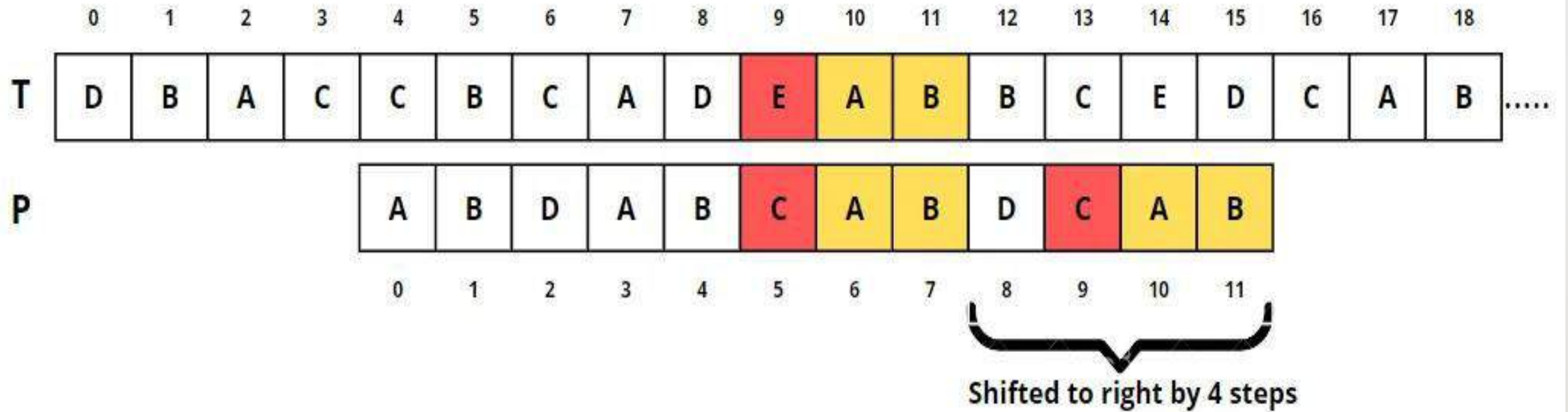


Figure 2

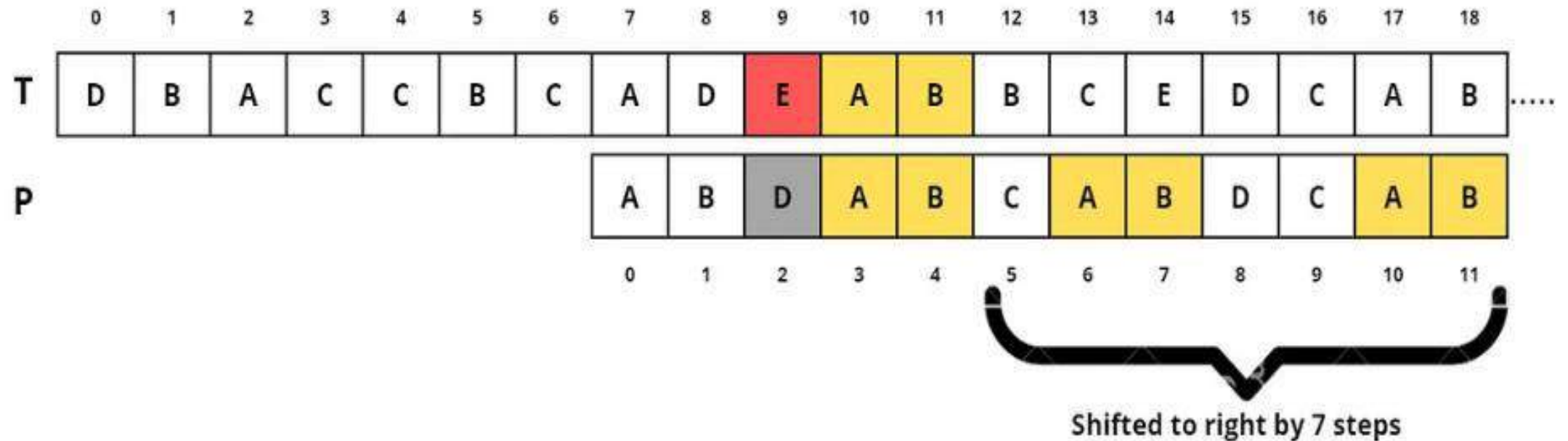
Case 1

There exists a substring u of P such that $u = t$ and u is not preceded by the character which precedes t .

Now we know the fact that there exists a suffix $t: P[10..11]$ such that T has a substring $T[10..11]$ identical to t . And we know we must find a different alignment of P in order to continue the searching process. We could shift P to right by 1 step and continue, but we can use the information we already have to shift P more than 1 step. We can find another occurrence of 'AB' in P such that it is not preceded by a 'C' and align it with $P[10..11]$. It is not useful to align ANY 'AB' with $T[10..11]$ because if that 'AB' in P is preceded by a 'C' then we encounter a mismatch at $T[9]$ again. Figure 3 illustrates both cases.



Shift is not useful since $P[5] = P[9]$ and we know $P[9] \neq T[9]$
 Even if somehow $P[6...11] = T[10...15]$, $P[5]$ will mismatch with $T[9]$



Shift also causes a mismatch. However, we cannot predict it prior to the comparison.
This is more desirable since P has been shifted by 7 steps rather than 4

Figure 3

Case 2

There is no any substring u of P such that $u = t$ and u is preceded by the character other than the one that precedes t .

In this case, we try to find the longest prefix of P which is also a suffix of t .

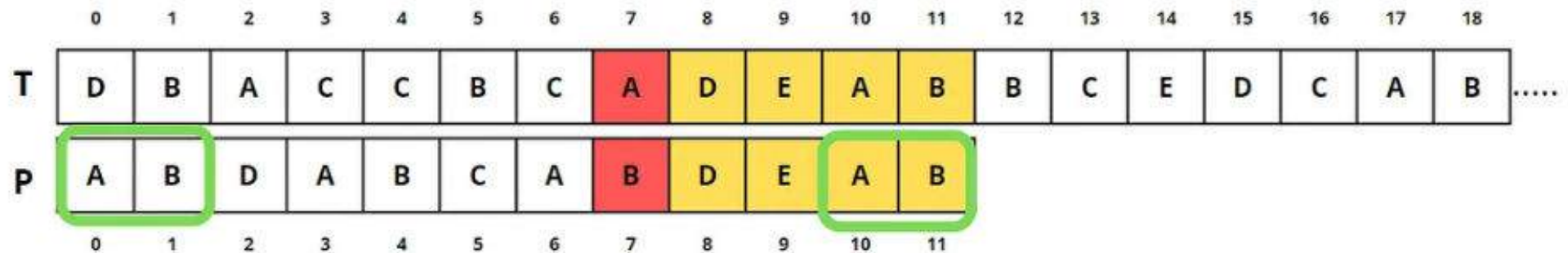


Figure 4

As figure 4 illustrates, we cannot find a substring u of P such that u is equivalent to t : $P[8...11]$. But we can see $P[0...1]$ is the longest prefix of P that is also a suffix of t . Therefore we can simply align that prefix with $T[10...11]$ because any other shift will not align a matching substring of P with $T[10...11]$. Figure 5 illustrates the states T and P after the correct shift.

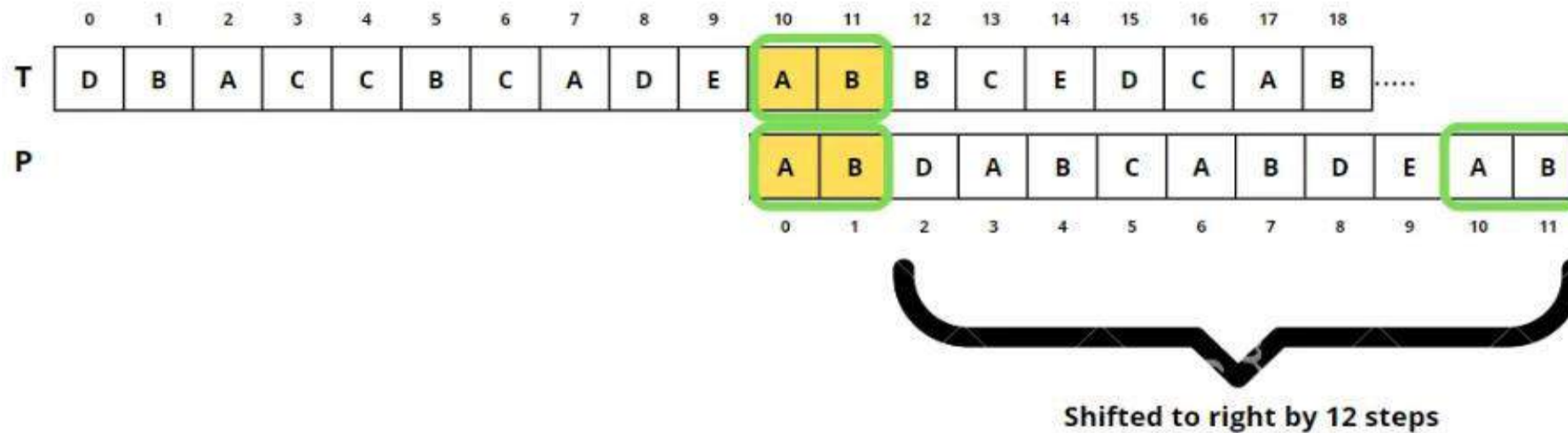
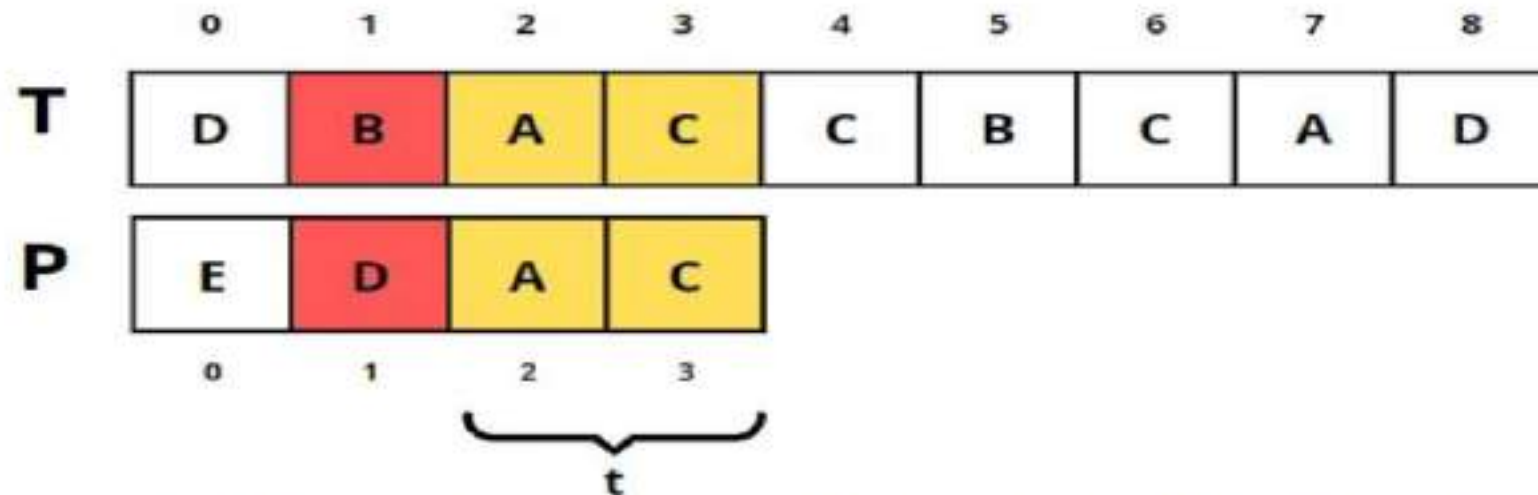


Figure 5

Case 3

There is no substring u of P such that $u = t$ and u is preceded by the character other than the one that precedes t and there is no prefix of P which is also a suffix of t . In this case, we can shift P all the past current window of T (Shift by $|P|$ steps) as figure 6 demonstrates.



Notice that we can conclude there are no chances of occurring P in $T[0...6]$ once we discovered there is no prefix of P which also a suffix of t

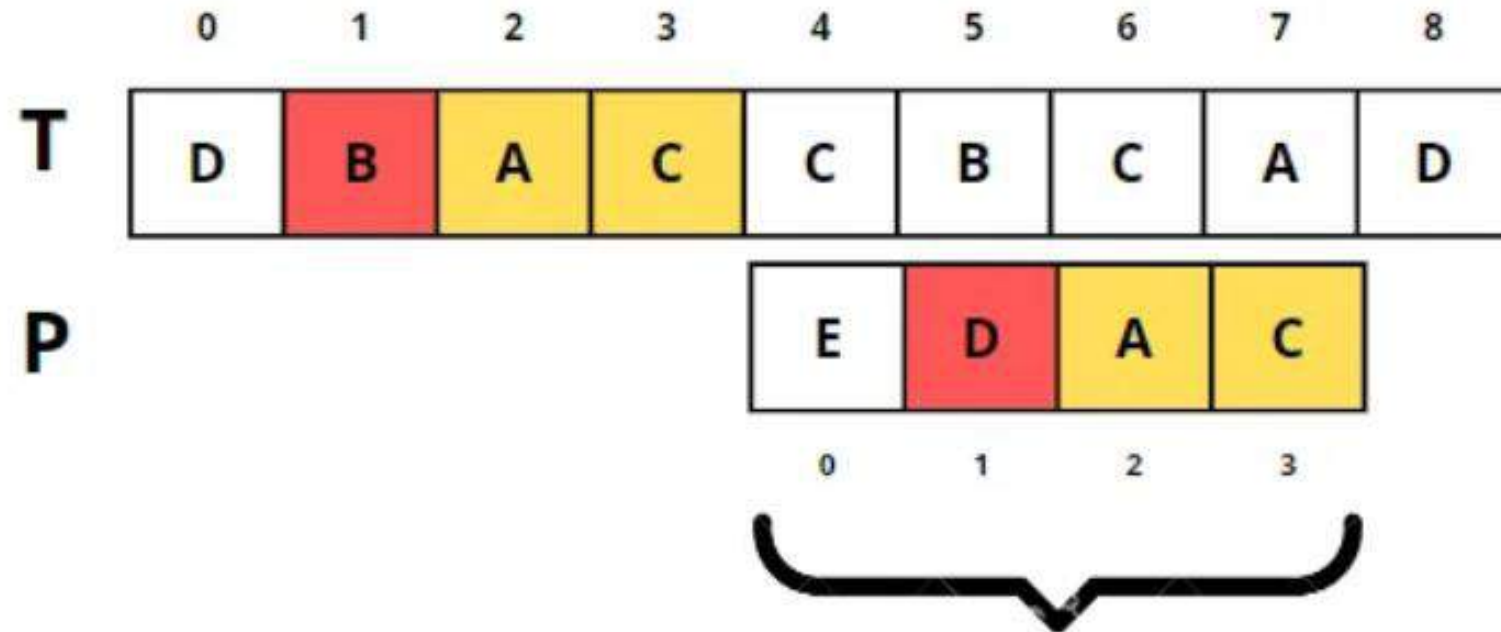


Figure 6

Context of i is used for scan T and j is used for P ends here.

Backtracking Algorithm

- **A backtracking algorithm is a problem-solving algorithm that uses a brute force approach for finding the desired output.**
- **The Brute force approach tries out all the possible solutions and chooses the desired/best solutions.**
- **The term backtracking suggests that if the current solution is not suitable, then backtrack and try other solutions. Thus, recursion is used in this approach.**
- **This approach is used to solve problems that have multiple solutions. If you want an optimal solution, you must go for dynamic programming.**

Example Backtracking Approach :

Problem: You want to find all the possible ways of **arranging 2 boys and 1 girl on 3 benches.**

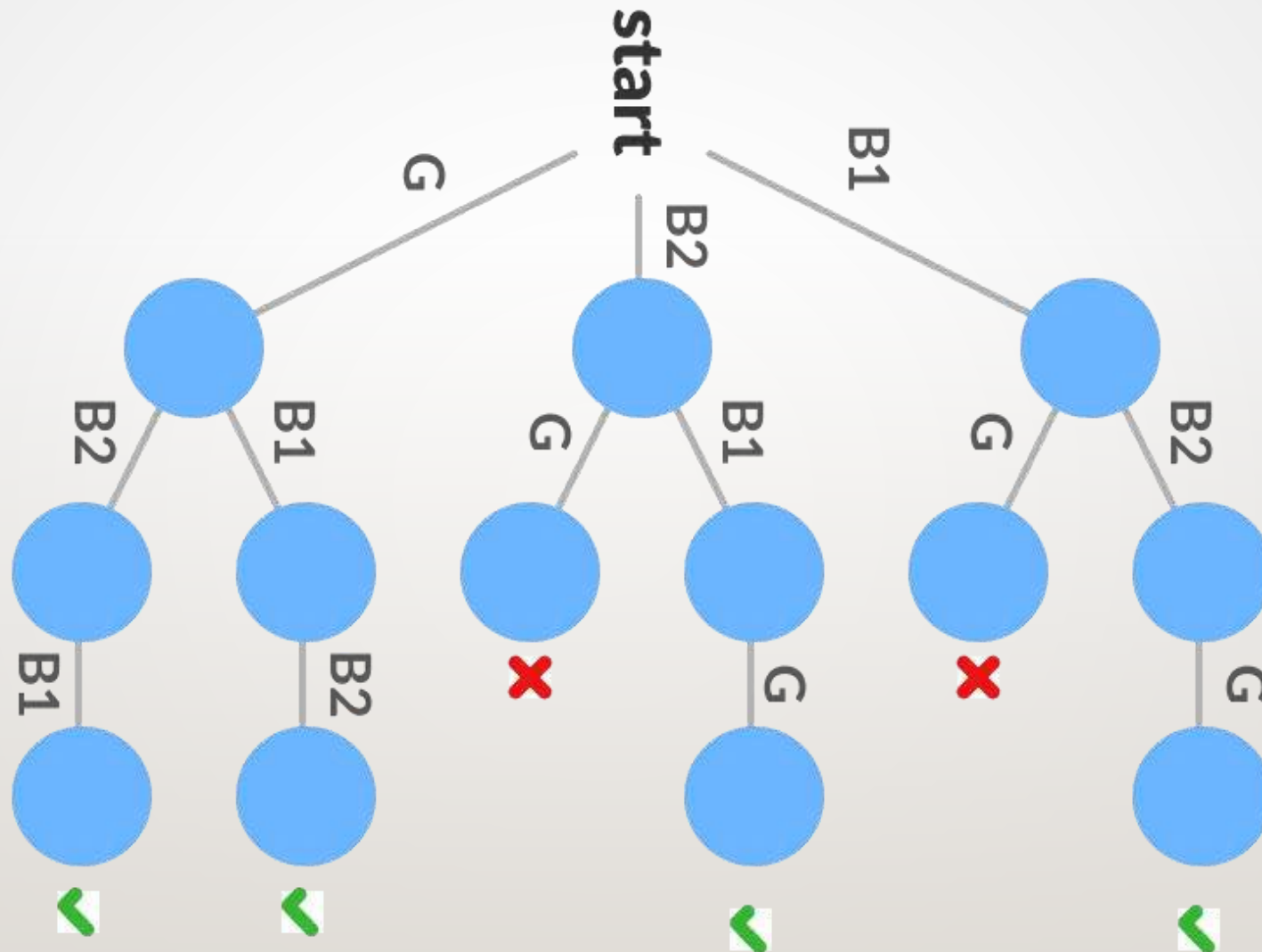
Constraint: Girl should not be on the middle bench.

Solution: There are a total of $3! = 6$ possibilities. We will try all the possibilities and get the possible solutions. We recursively try all the possibilities.

All the possibilities are:

B1	B2	G
B1	G	B2
B2	B1	G
B2	G	B1
G	B1	B2
G	B2	B1

The following **state space tree** shows the possible solutions :



The terms related to the backtracking are:

Many problems can be solved by backtracking strategy, and that problems satisfy complex set of constraints, and these constraints are of two types:

- **Implicit constraint:** It is a rule in which how each element in a tuple is related.
- **Explicit constraint:** The rules that restrict each element to be chosen from the given set.

STATE SPACE TREE TERMINOLOGY

- **Problem state:** Each node in the tree organization defines a problem state.
- **State space tree:** If we represent solution space in the form of a tree then the tree is referred as the state space tree.
- **Solution Space:** All tuples that satisfy the explicit constraints define a possible solution space for a particular instance 'I' of the problem
- **Solution States:** These are those problem states S for which the path from the root to S define a tuple in the solution space. A solution state is a node s for which each path from root node to node s together can represent a tuple in solution set.
- **Answer States:** These solution states S for which the path from the root to S defines a tuple which is a member of the set of solution (i.e. it satisfies the implicit constraints) of the problem.
- **Live node:** A node which has been generated and all of whose children have not yet been generated is live node.
- **E-node:** The live nodes whose children are currently being generated is called E-node (node being expanded)
- **Dead node:** It is a generated node that is either not to be expanded further or one for which all of its children has been generated.
- **Bounding function:** It will be used to kill live nodes without generating all their children

Backtracking methodology Explicit Constraints

- Explicit constraints are rules that restrict each x_i takes values from a given set S_i .
- All solutions to the 4-queens problem can therefore be represented as 4-tuple (x_1, x_2, x_3, x_4) , where x_i is the column on which queen Q_i is placed.

Ex: $S_i = \{1, 2, 3, 4\}$ where $1 \leq i \leq 4$

- The explicit constraints depend on the particular instance I of the problem being solved.

Ex: The placement of Q_2 depend upon the placement of Q_1

All tuples that satisfy the explicit constraints define a possible solution space for I . The solution space, therefore, consists of $4^4 (= 256)$ 4-tuples.

- All tuples that satisfy the explicit constraints define a possible solution space for I .

Ex: $I_1 = \{x_2, x_4, x_1, x_3\}$

Backtracking methodology Implicit Constraints

- The implicit constraints are rules that determine which of the tuples in the solution space I satisfy the criterion function.
- Thus, implicit constraints describe the way in which the x_i must relate to each other.

Ex: (a) No two x_i 's can be the same (i.e. all queens must be on different columns)

(b) No two queens on the same diagonal

(c) No two queens on the same row

The constraints (a) and (b) implies that all solutions are permutations of the 4-tuple (1, 2, 3, 4).

This realization reduces the size of the solution space from 4^4 to $4!$ tuples.

Difference between the Backtracking and Recursion

- **Recursion is a technique that calls the same function again and again until you reach the base case.**
- **Backtracking is an algorithm that finds all the possible solutions and selects the desired solution from the given set of solutions.**

- Backtracking is used when we have multiple solutions, and we require all those solutions.
- Backtracking name itself suggests that we are going back and coming forward; if it satisfies the condition, then return success, else we go back again. It is used to solve a problem in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criteria.

When to use a Backtracking algorithm?

When we have multiple choices, then we make the decisions from the available choices. In the following cases, we need to use the backtracking algorithm:

- A piece of sufficient information is not available to make the best choice, so we use the backtracking strategy to try out all the possible solutions.
- Each decision leads to a new set of choices. Then again, we backtrack to make new decisions. In this case, we need to use the backtracking strategy.

Backtracking Algorithm Applications:

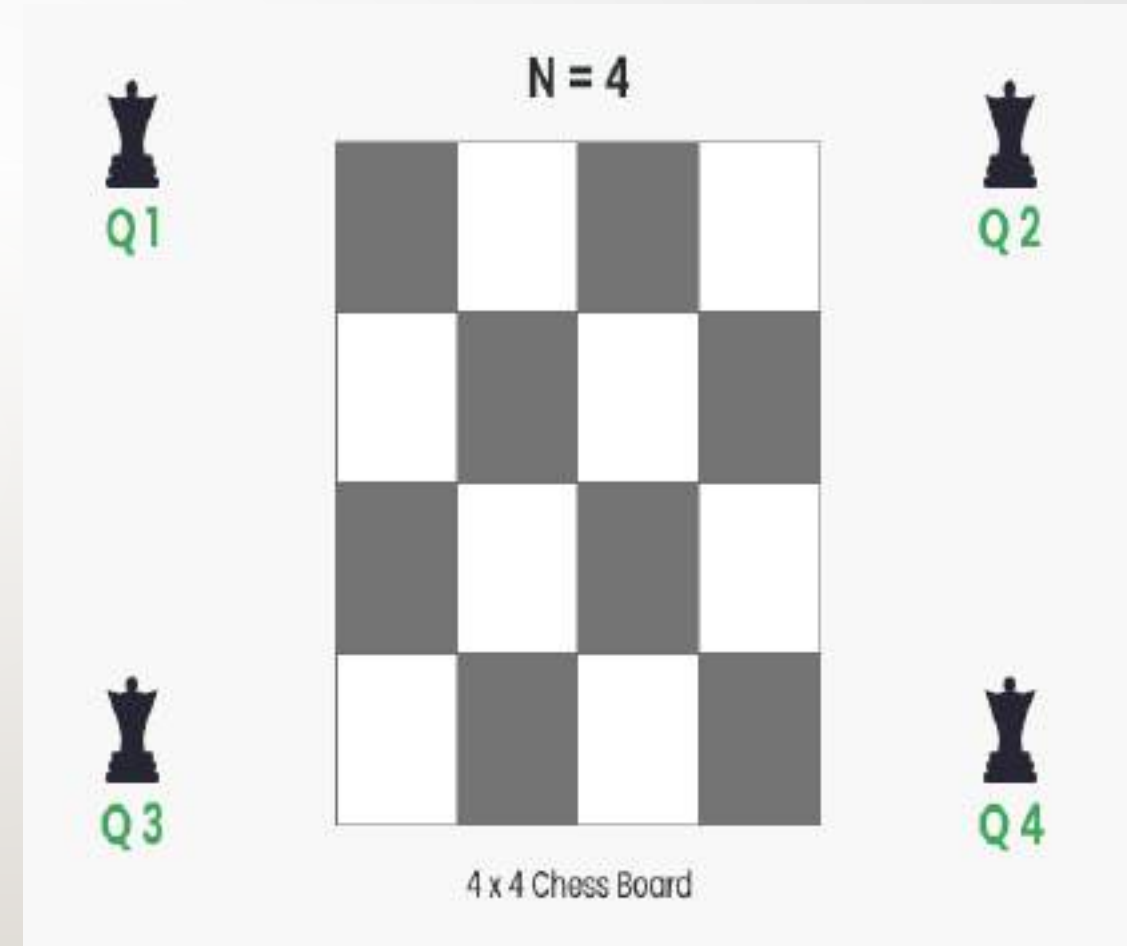
- N-queens problem
- Sum of subset problem
- Graph coloring
- Hamilton cycle

N-Queens Problem

- N - Queens problem is to place n - queens in such a manner on an n x n chessboard that no queens attack each other by being in the same row, column or diagonal.
- It can be seen that for $n = 1$, the problem has a trivial solution, and no solution exists for $n = 2$ and $n = 3$. So first we will consider the 4 queens problem and then generate it to n - queens problem.
- Given a 4 x 4 chessboard and number the rows and column of the chessboard 1 through 4.

4-Queens Problem

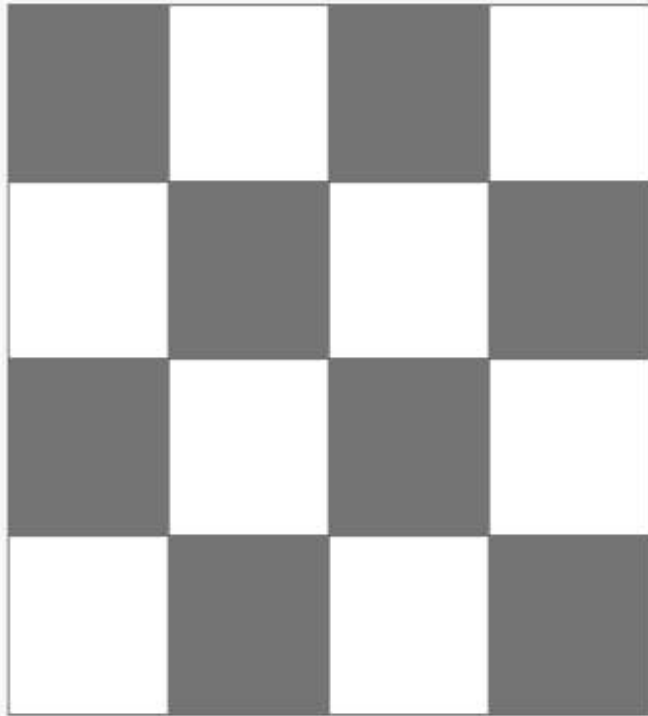
- The 4 Queens Problem consists in placing four queens on a 4 x 4 chessboard so that **no two queens attack each other**. That is, no two queens are allowed to be placed on the same row, the same column or the same diagonal.
- We are going to look for the solution for $n=4$ on a 4 x 4 chessboard :



4 Queens Problem using Backtracking Algorithm:

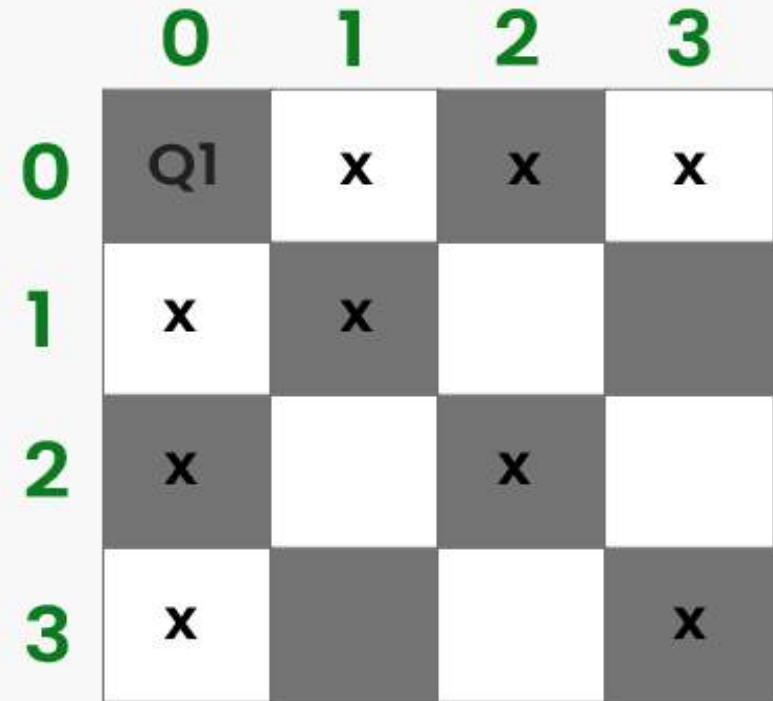
- Place each queen one by one in different rows, starting from the topmost row.
- While placing a queen in a row, check for clashes with already placed queens.
- For any column, if there is no clash then mark this row and column as part of the solution by placing the queen.
- In case, if no safe cell found due to clashes, then backtrack (i.e, undo the placement of recent queen) and return false.

Illustration of 4 Queens Solution



4 x 4 Chess Board

Step 0: Initialize a 4×4 board.



4 x 4 Chess Board

Step 1: Put our first Queen (Q1) in the (0,0) cell . 'x' represents the cells which is not safe i.e. they are under attack by the Queen (Q1). After this move to the next row [0 -> 1].

Step 3: At row 2 there is no cell which are safe to place Queen (Q3) . So, backtrack and remove queen Q2 queen from cell (1, 2).

Step 4: There is still a safe cell in the row 1 i.e. cell (1, 3). Put Queen (Q2) at cell (1, 3).

	0	1	2	3
0	Q1	x	x	x
1	x	x	Q2	x
2	x	x	x	x
3	x		x	x

4 x 4 Chess Board

Step 2: Put our next Queen (Q2) in the (1,2) cell . After this move to the next row [1 -> 2].

	0	1	2	3
0	Q1	x	x	x
1	x	x	x	Q2
2	x		x	x
3	x	x		x

4 x 4 Chess Board

Step 6: There is no any cell to place Queen (Q4) at row 3. Backtrack and remove Queen (Q3) from row 2. Again there is no other safe cell in row 2, So backtrack again and remove queen (Q2) from row 1. Queen (Q1) will be remove from cell (0,0) and move to next safe cell i.e. (0 , 1).

Step 7: Place Queen Q1 at cell (0 , 1), and move to next row.

	0	1	2	3
0	Q1	x	x	x
1	x	x	x	Q2
2	x	Q3	x	x
3	x	x	x	x

4 x 4 Chess Board

Step 5: Put queen (Q3) at cell (2, 1).

	0	1	2	3
0	x	Q1	x	x
1		x	x	
2		x		x
3		x		

4 x 4 Chess Board

Step 8: Place Queen Q2 at cell (1 , 3), and move to next row.

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2		x	x	x
3		x		x

4 x 4 Chess Board

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2	Q3	x	x	x
3	x	x		x

4 x 4 Chess Board

Step 9: Place Queen Q3 at cell (2 , 0), and move to next row.

**Step 10: Place Queen Q4 at cell (3 , 2), and move to next row.
This is one possible configuration of solution.**

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2	Q3	x	x	x
3	x	x	Q4	x

4 x 4 Chess Board

Q1	X	X	X
X	X		
X		X	
X			X

Step 1

Q1	X	X	X
X	X	Q2	X
X	X	X	X
X		X	X

Step 2

Q1	X	X	X
X	X	X	Q2
X		X	X
X	X		X

Step 3

Q1	X	X	X
X	X	X	Q2
X	Q3	X	X
X	X	X	X

Step 4

X	Q1	X	X
X	X	X	
	X		X
	X		

Step 5

X	Q1	X	X
X	X	X	Q2
	X	X	X
	X		X

Step 6

X	Q1	X	X
X	X	X	Q2
Q3	X	X	X
X	X		X

Step 7

X	Q1	X	X
X	X	X	Q2
Q3	X	X	X
X	X	Q4	X

Step 8

	Q1		
			Q2
Q3			
		Q4	

4 Queens Problem Solution 1 - (2, 4, 1, 3)

	Q1		
			Q2
Q3			
		Q4	

Solution 1

		Q1	
Q2			
			Q3
	Q4		

Solution 2

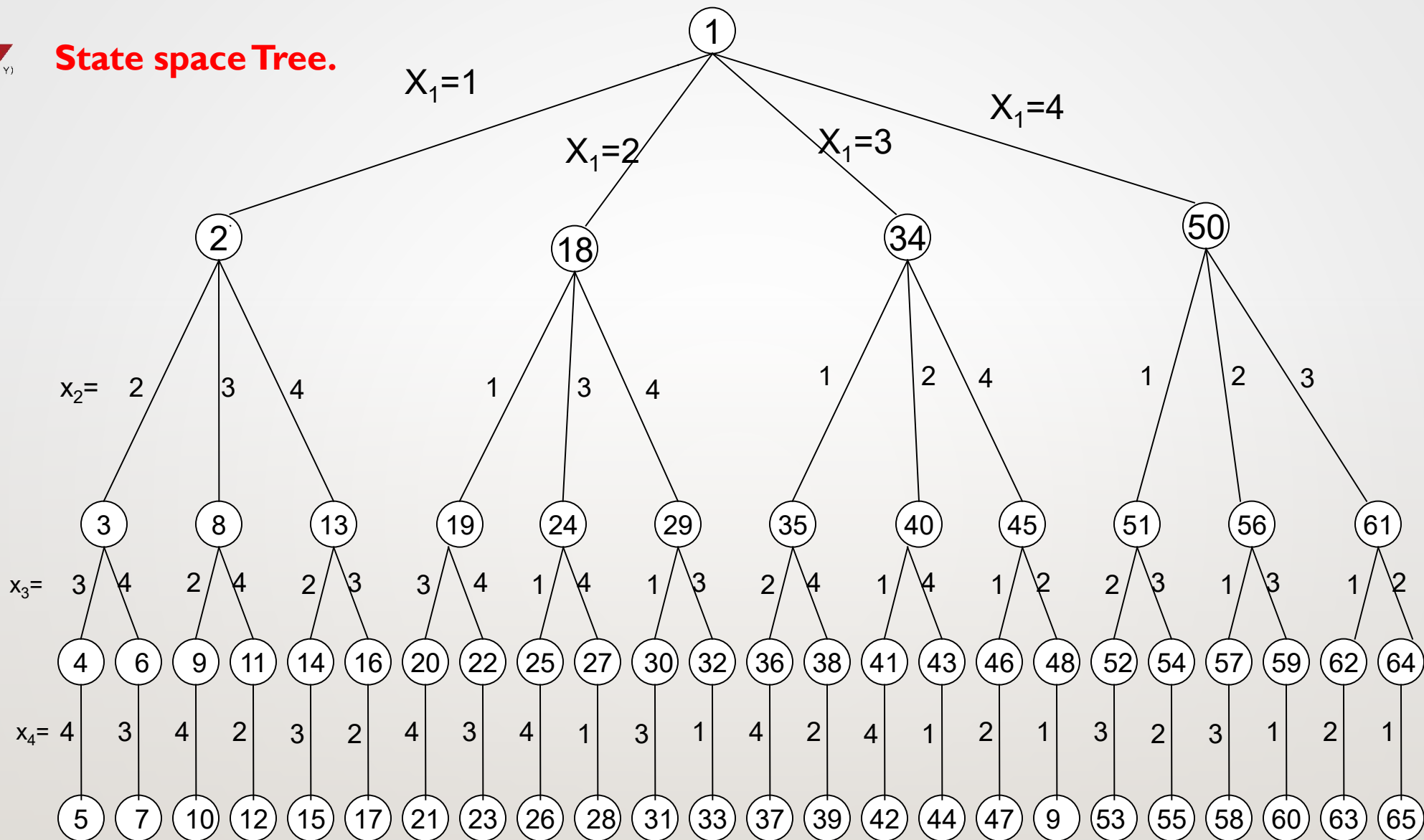
Follow the steps below to implement the idea:

- Make a recursive function that takes the state of the board and the current row number as its parameter.
- Start in the topmost row.
- If all queens are placed, return true
- For every row :

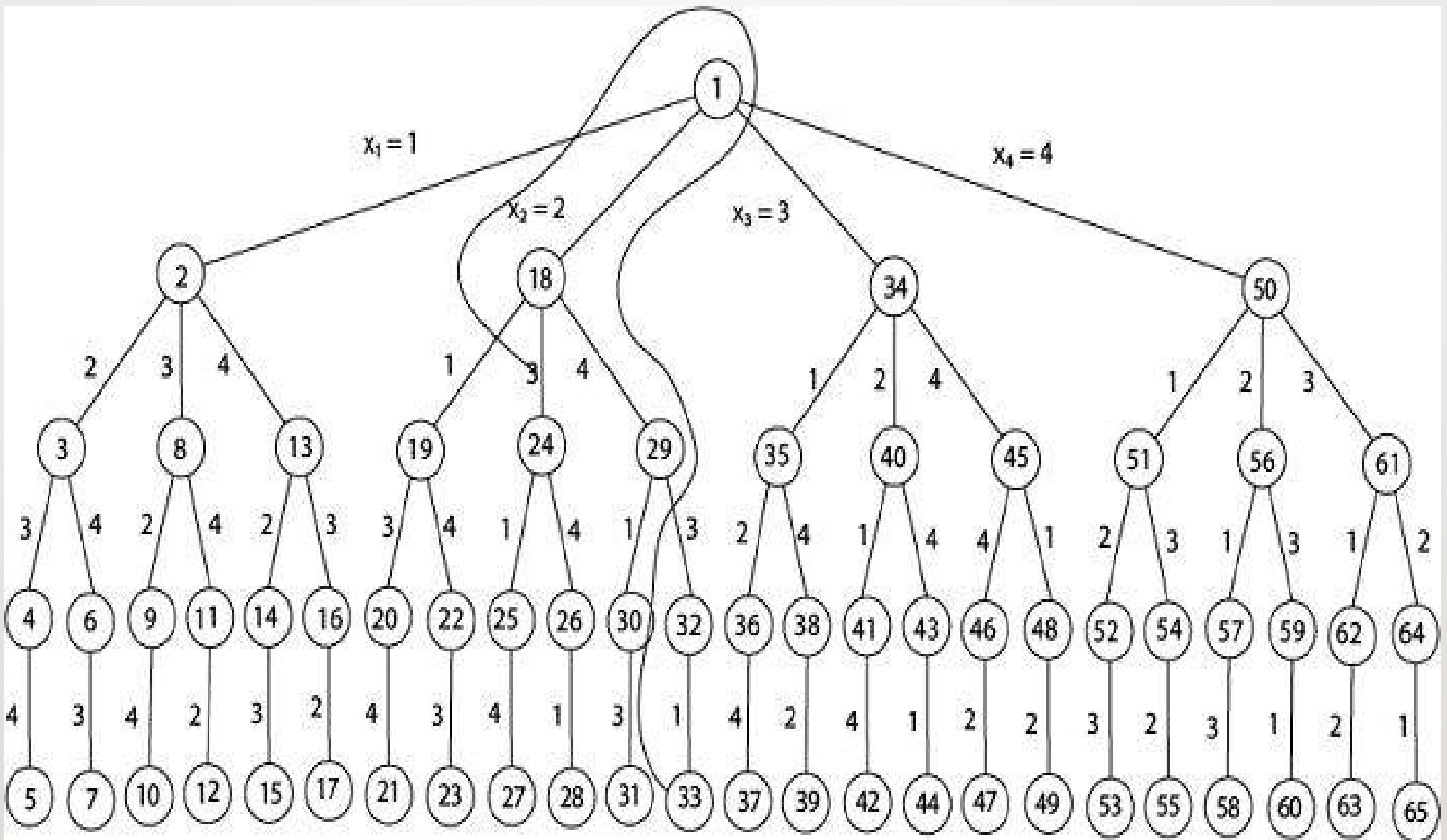
Do the following for each column in current row.

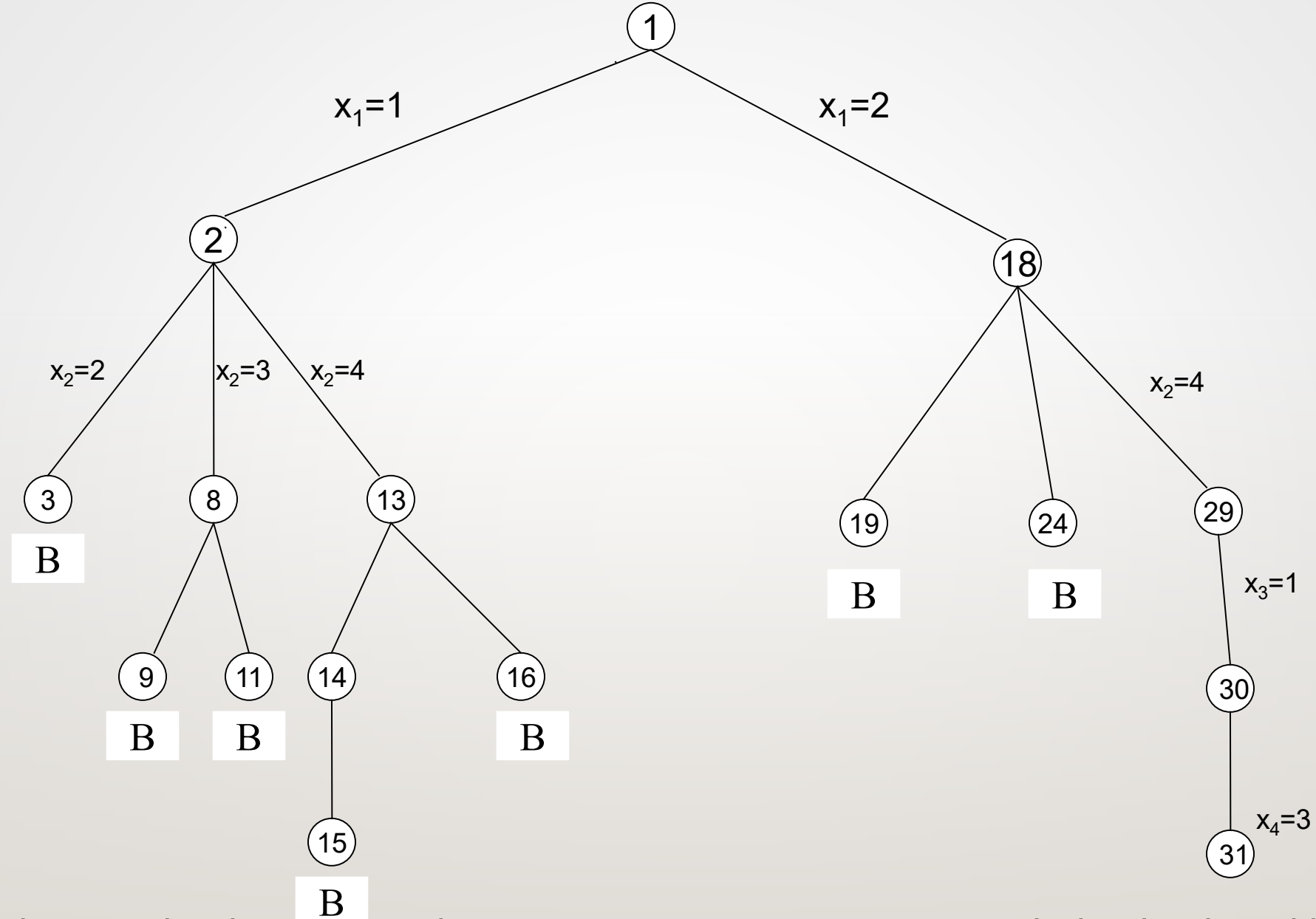
- Then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.
- If placing the queen in [row, column] leads to a solution, return true.
- If placing queen doesn't lead to a solution then unmark this [row, column] and track back and try other columns.

State space Tree.



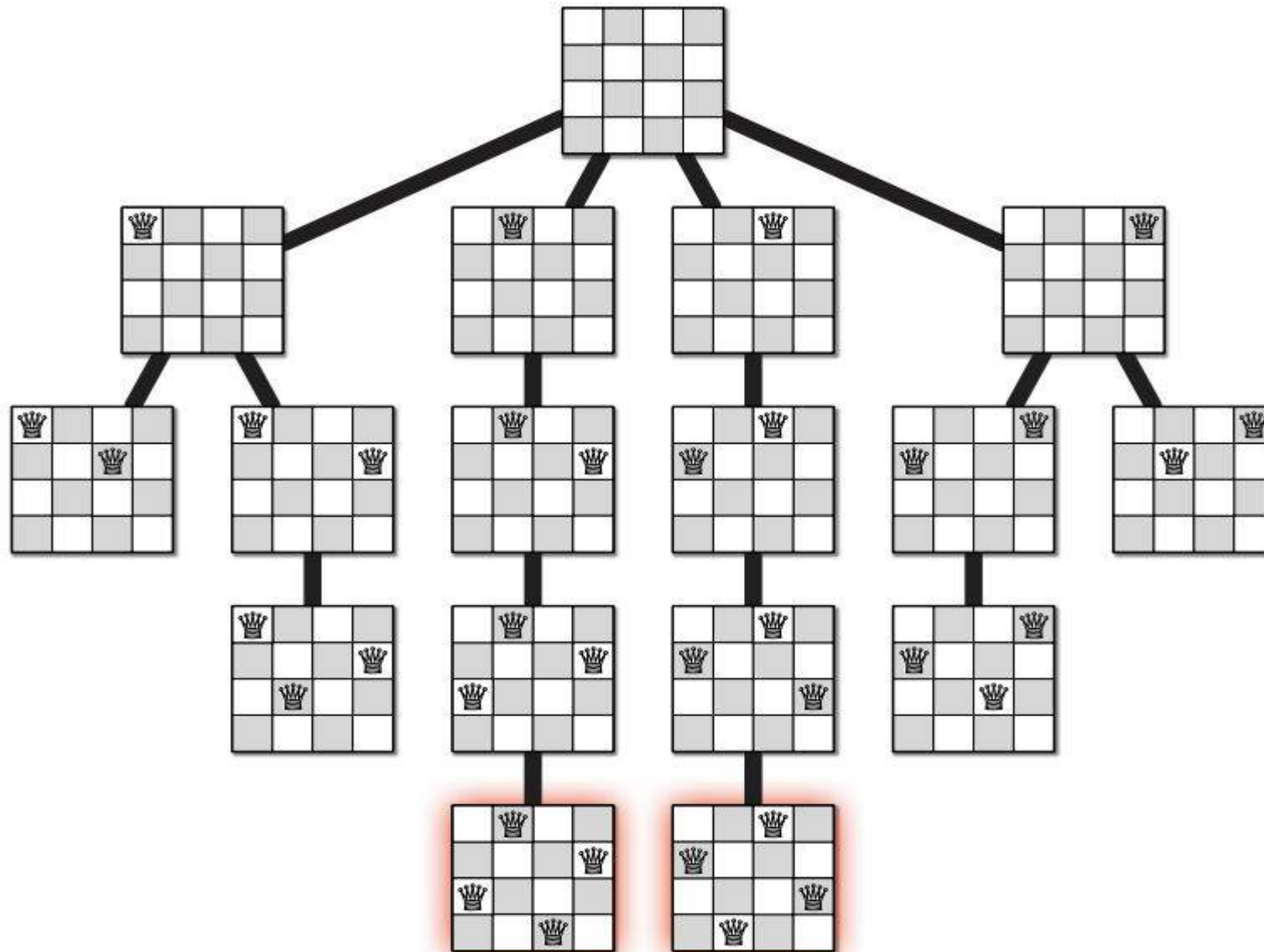
Tree organization of the 4-queens solution space. Nodes are numbered as in **depth first search**.





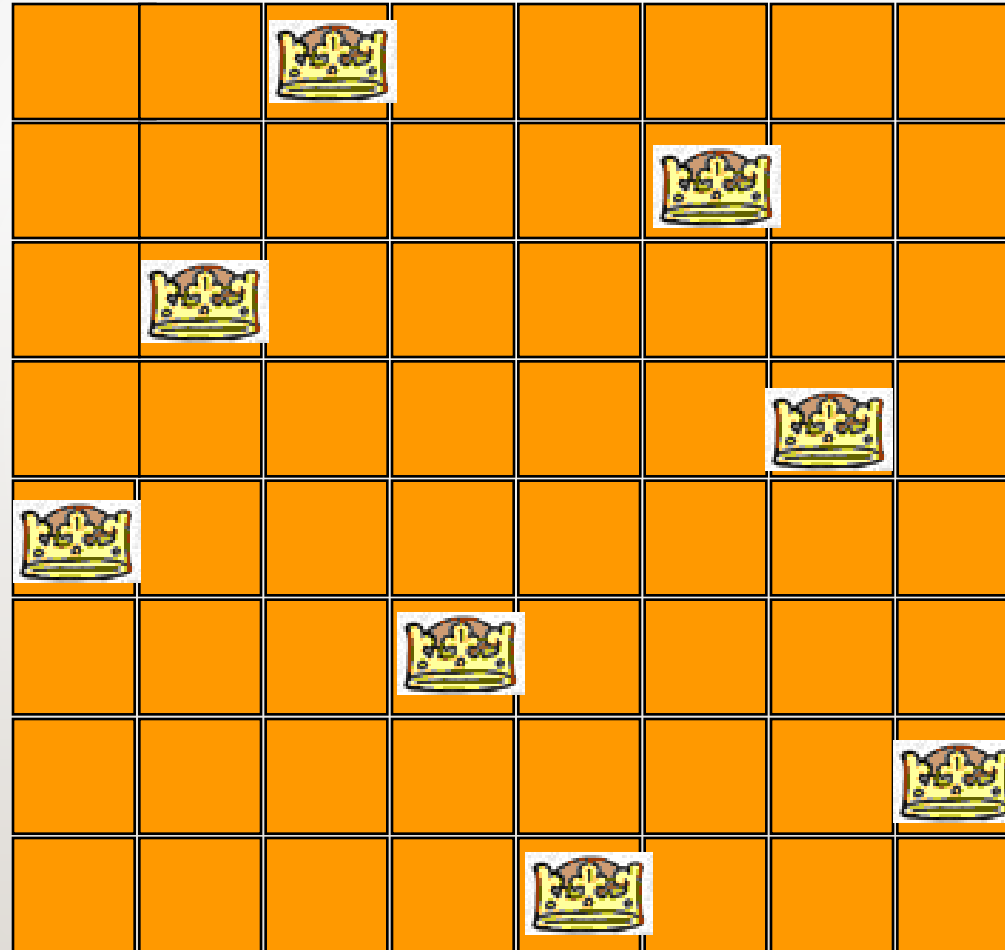
Portion of the tree that is generated

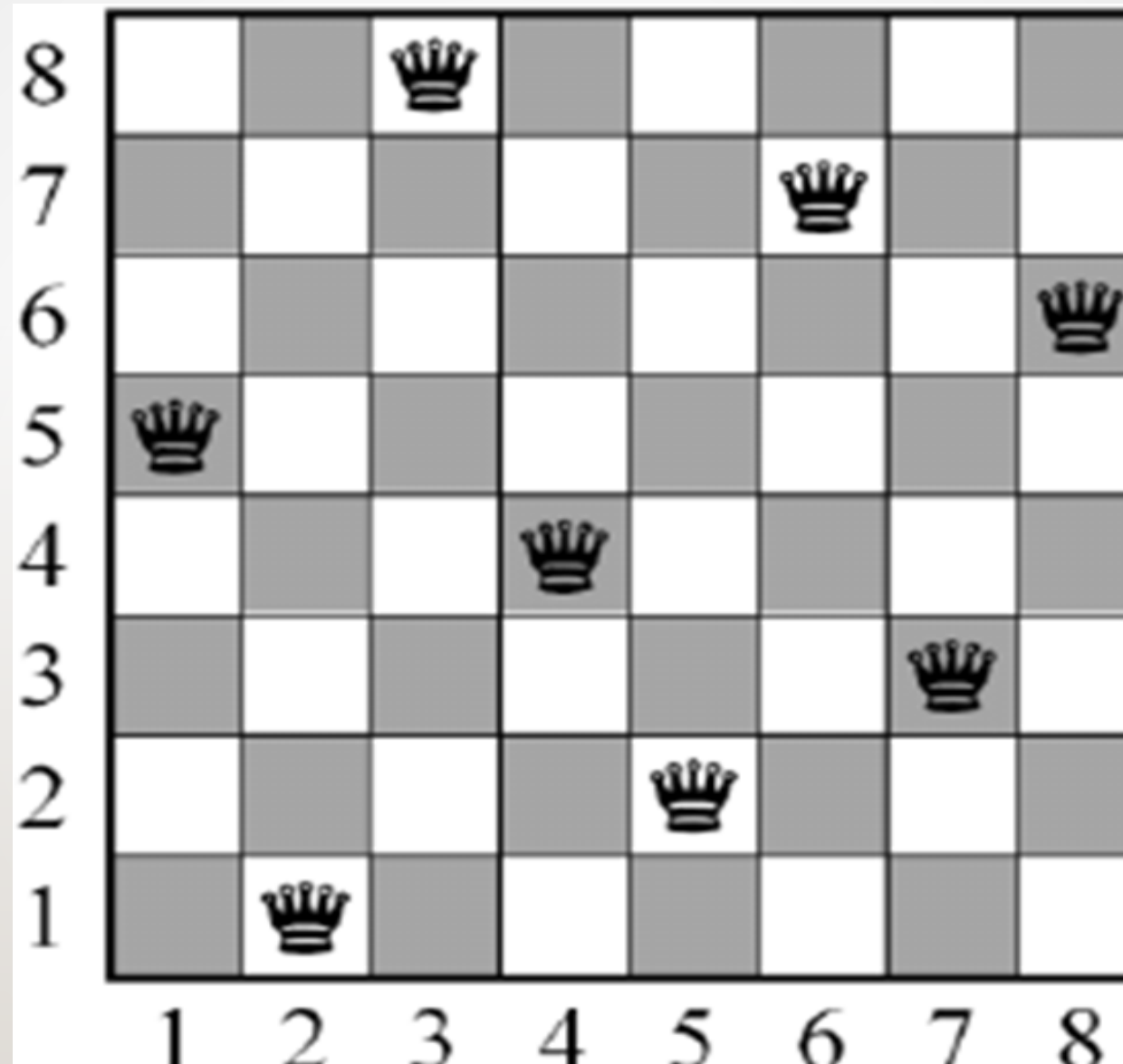
during backtracking($n=4$).

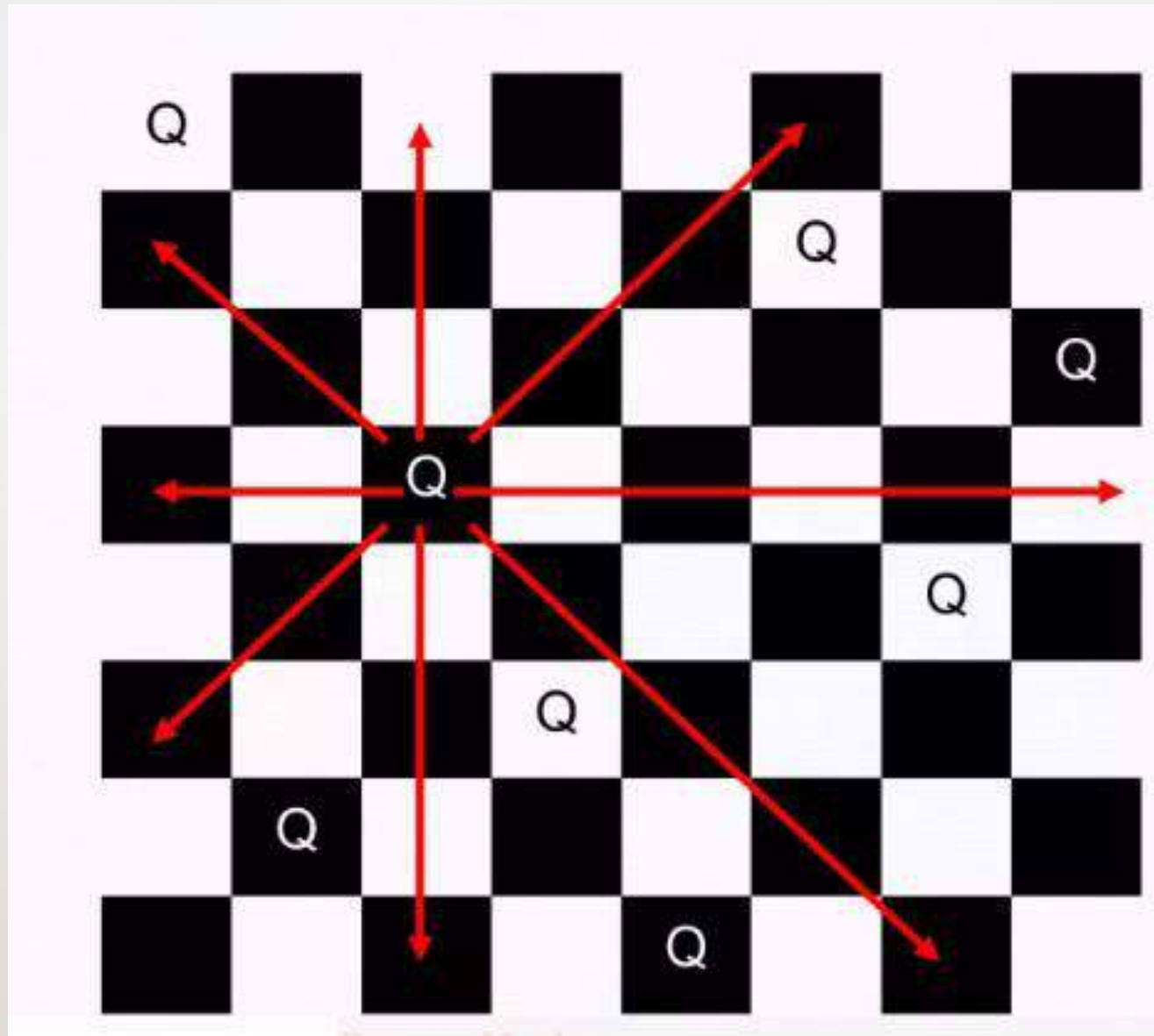




8-QUEENS PROBLEM







1	1	2	3	4	5	6	7	8
2						Q1		
3				Q2				
4	Q4						Q3	
5			Q5					
6					Q6			
7		Q7						
8								Q8

∴ 8 - queen with feasible solution is $\langle 6, 4, 7, 1, 3, 5, 2, 8 \rangle$

The other solution for 8 - queen problem is

$\langle 4, 7, 3, 8, 2, 5, 1, 6 \rangle$, $\langle 7, 2, 6, 3, 1, 4, 8, 5 \rangle$, $\langle 6, 4, 7, 1, 3, 5, 2, 8 \rangle$,
 $\langle 5, 3, 1, 7, 2, 8, 6, 4 \rangle$ $\langle 6, 1, 5, 2, 8, 3, 7, 4 \rangle$ $\langle 5, 8, 4, 1, 3, 6, 2, 7 \rangle$,
 $\langle 8, 2, 5, 3, 1, 7, 4, 6 \rangle$ etc.

- **n– queens : Defining the problem:-**

- The problem is to place n queens on an $n \times n$ chessboard so that no two queens “attack” i.e.no two queens on the same row, column, or diagonal.
- Assume rows and columns of chessboard are numbered 1 through n . Queens also be numbered 1 through n .
- **Condition to ensure - No two queens on same Row:** Each queen must be on a different row ,hence queen i is to be placed on row i . Therefore, all solutions to the n -queens problem can be represented as n -tuples $(x_1, x_2, \dots, x_n)$, where x_i is the column on which queen i is placed.
- **Condition to ensure - No two queens on same Column:**
 - Select a distinct value for each x_i

Condition to ensure - No two queens on same Diagonal

If two queens are placed at positions (i, j) and (k, l) . They are on the same diagonal then following conditions hold good.

1) Every element on the **same diagonal** that runs from the upper left to the lower right has the same row – column value.

$$i - j = k - l \text{ -----(1) // (1,1) (2,2)}$$

2) Similarly, every element on the **same diagonal** that goes from the upper right to the lower left has the same row + column value.

$$i + j = k + l \text{ -----(2) // (1,2) (2,1)}$$

- First equation implies $j - l = i - k$
- Second equation implies $j - l = k - i$

- Therefore, two queens lie on the same diagonal iff $|j - l| = |i - k|$

x	Q1	x	x
x	x	x	Q2
Q3	x	x	x
x	x	Q4	x

4 x 4 Chess Board

Algorithm **place**(k, i)

// It returns true if a queen can be placed in kth row and ith

// column . Otherwise it returns false. x[] is a goal array whose

// first(k-1) values have been set. Abs(r) returns the absolute value of
r.

{

for j = 1 to k-1 do

{ // Two in the same column or in the same diagonal

if ((**x [j] = i**) or (**abs(x[j] - i**) = **abs(j - k**))) then

return false;

}

return true ;

}

Algorithm **Nqueen**(k, n)

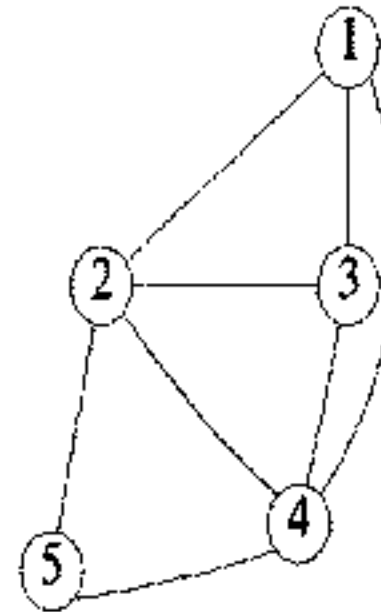
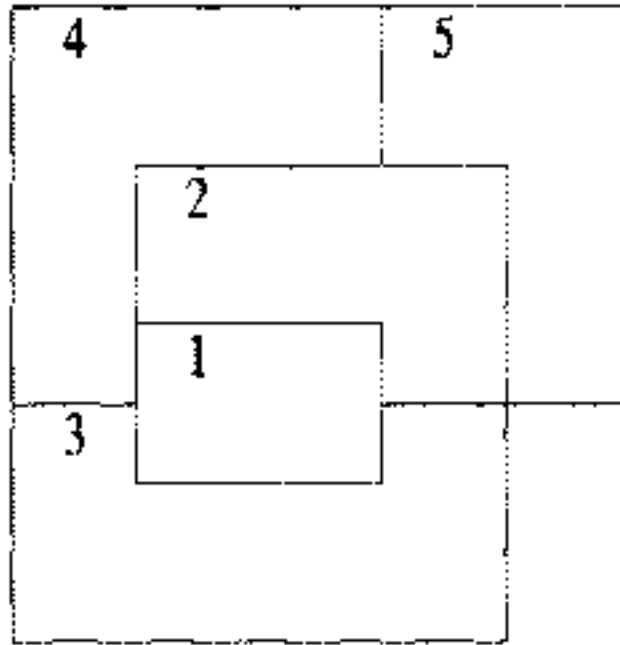
// Using backtracking, this procedure prints all possible placements
// of n queens on an $n \times n$ chessboard so that they are non-attacking.

```
{  
    for i = 1 to n do          // check place of column for queen k  
    {  
        if place( k, i ) then  
        {  
            x[ k ] = i;  
            if( k = n ) then write ( x[1:n] );  
            else Nqueen( k+1, n);  
        }  
    }  
}
```


GRAPH COLORING

- Let 'G' be a graph and 'm' be a given positive integer. If the nodes of 'G' can be colored in such a way that no two adjacent nodes have the same color. Yet only 'M' colors are used. So it's called M-color ability decision problem.
- The graph G can be colored using the smallest integer 'm'. This integer is referred to as chromatic number of the graph.
- A graph is said to be planar iff it can be drawn on plane in such a way that no two edges cross each other.
- Suppose we are given a map then, we have to convert it into planar. Consider each and every region as a node. If two regions are adjacent then the corresponding nodes are joined by an edge.

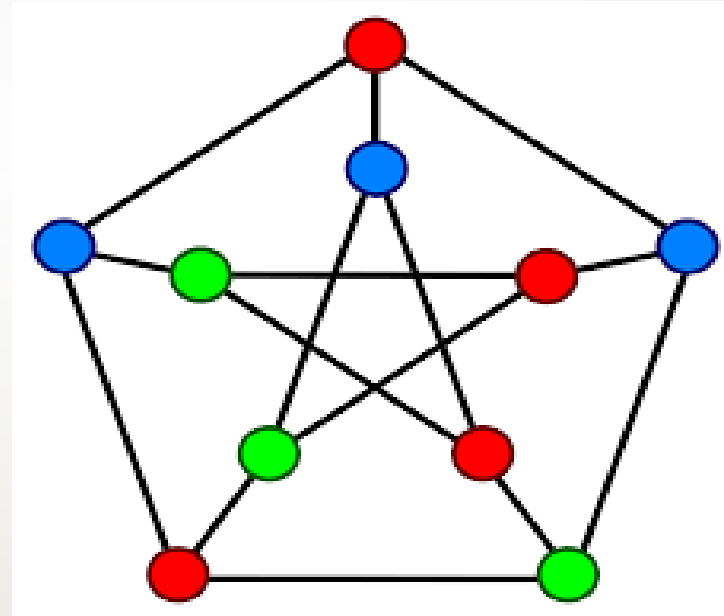
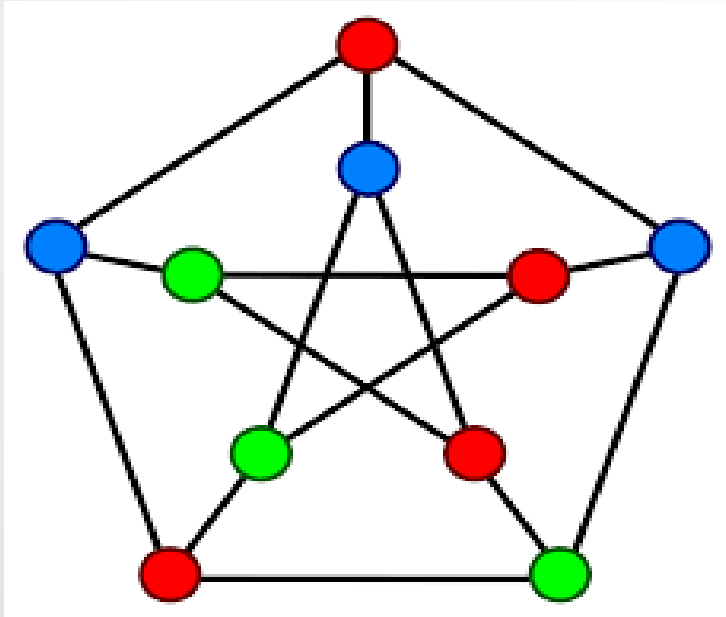
A map and its planar graph representation



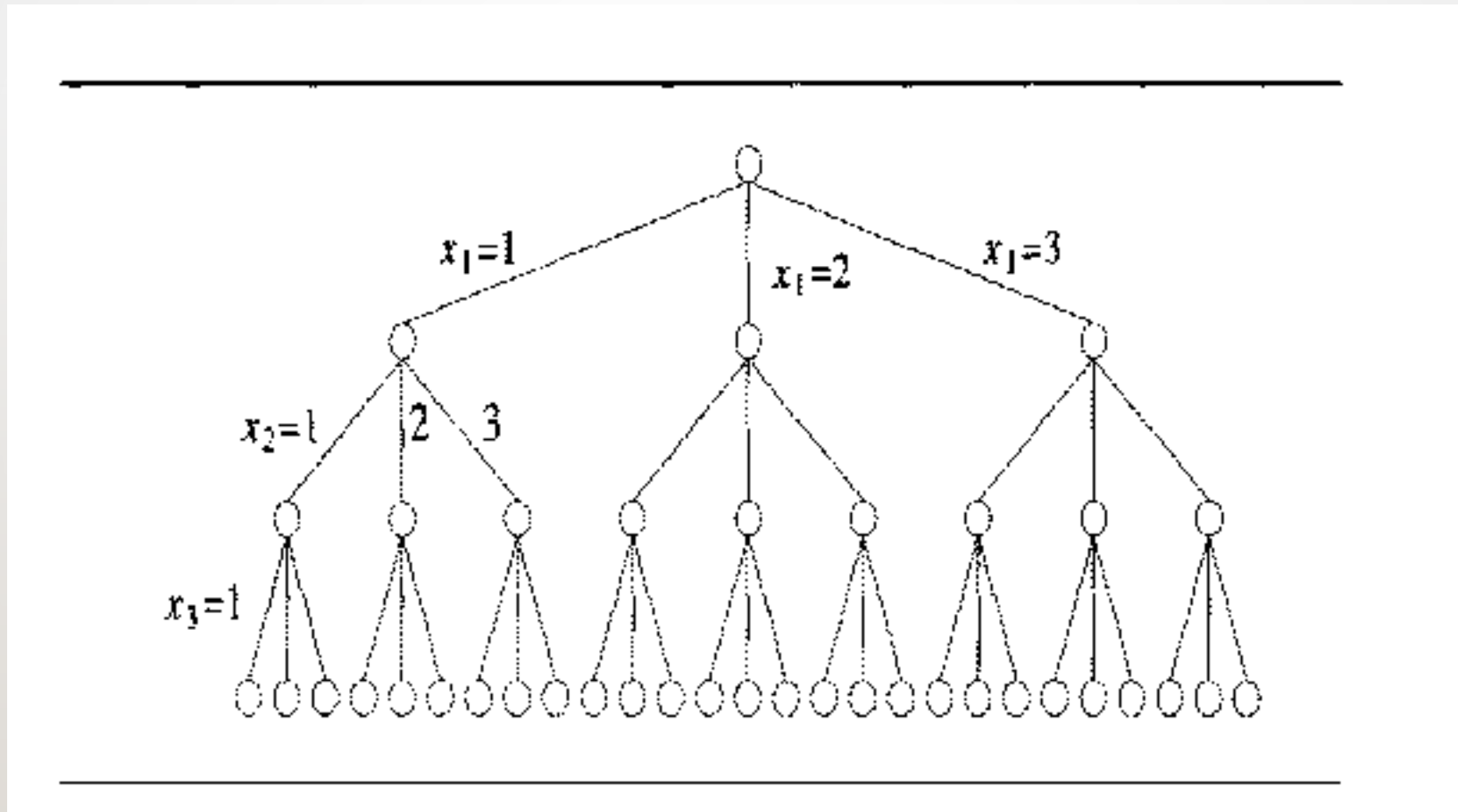
STEPS TO COLOR THE GRAPH:

- First create the adjacency matrix $graph(1:m, 1:n)$ for a graph, if there is an edge between i, j then $C(i, j) = 1$ otherwise $C(i, j) = 0$.
- The Colors will be represented by the integers $1, 2, \dots, m$ and the solutions will be stored in the array $X(1), X(2), \dots, X(n)$, $X(\text{index})$ is the color, index is the node.
- The formula is used to set the color is,
$$X(k) = (X(k) + 1) \% (m + 1)$$
- First one chromatic number is assigned, after assigning a number for 'k' node, we have to check whether the adjacent nodes have got the same values if so then we have to assign the next value.
- Repeat the procedure until all possible combinations of colors are found.
- The function which is used to check the adjacent nodes and same color is,
$$\text{If}((\text{Graph}(k, j) == 1) \text{ and } X(k) = X(j))$$

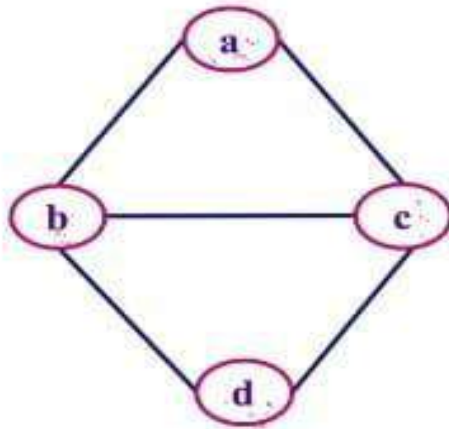
GRAPH COLORING EXAMPLE



STATE SPACE TREE FOR M Coloring WHEN $N=3$ AND $M=3$

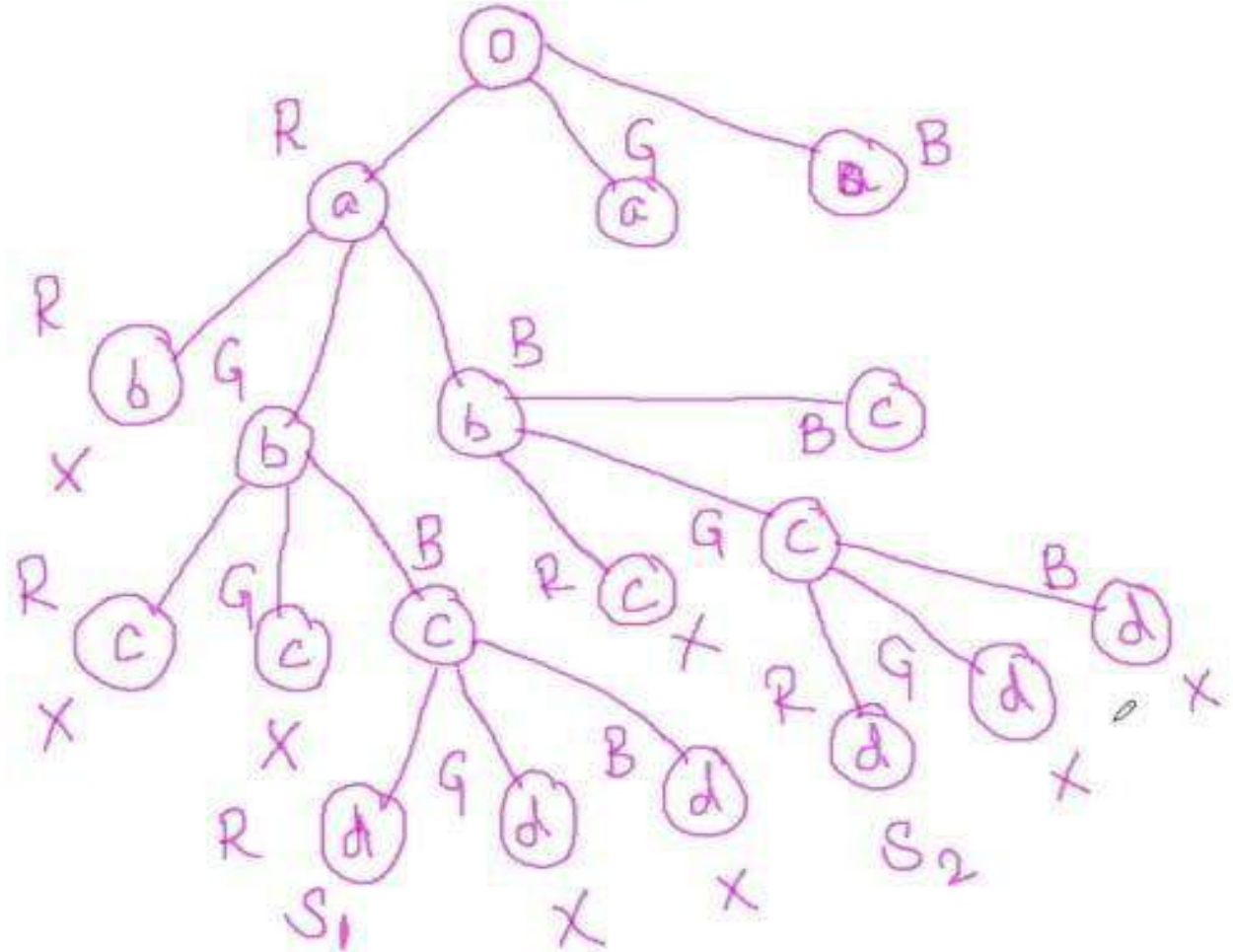


BACKTRACKING - GRAPH COLOURING PROBLEM

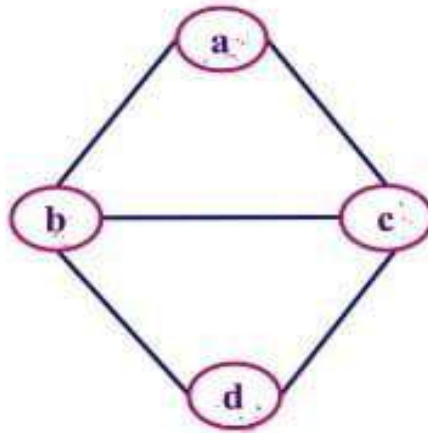


M=3
R,G,B

State Space Tree

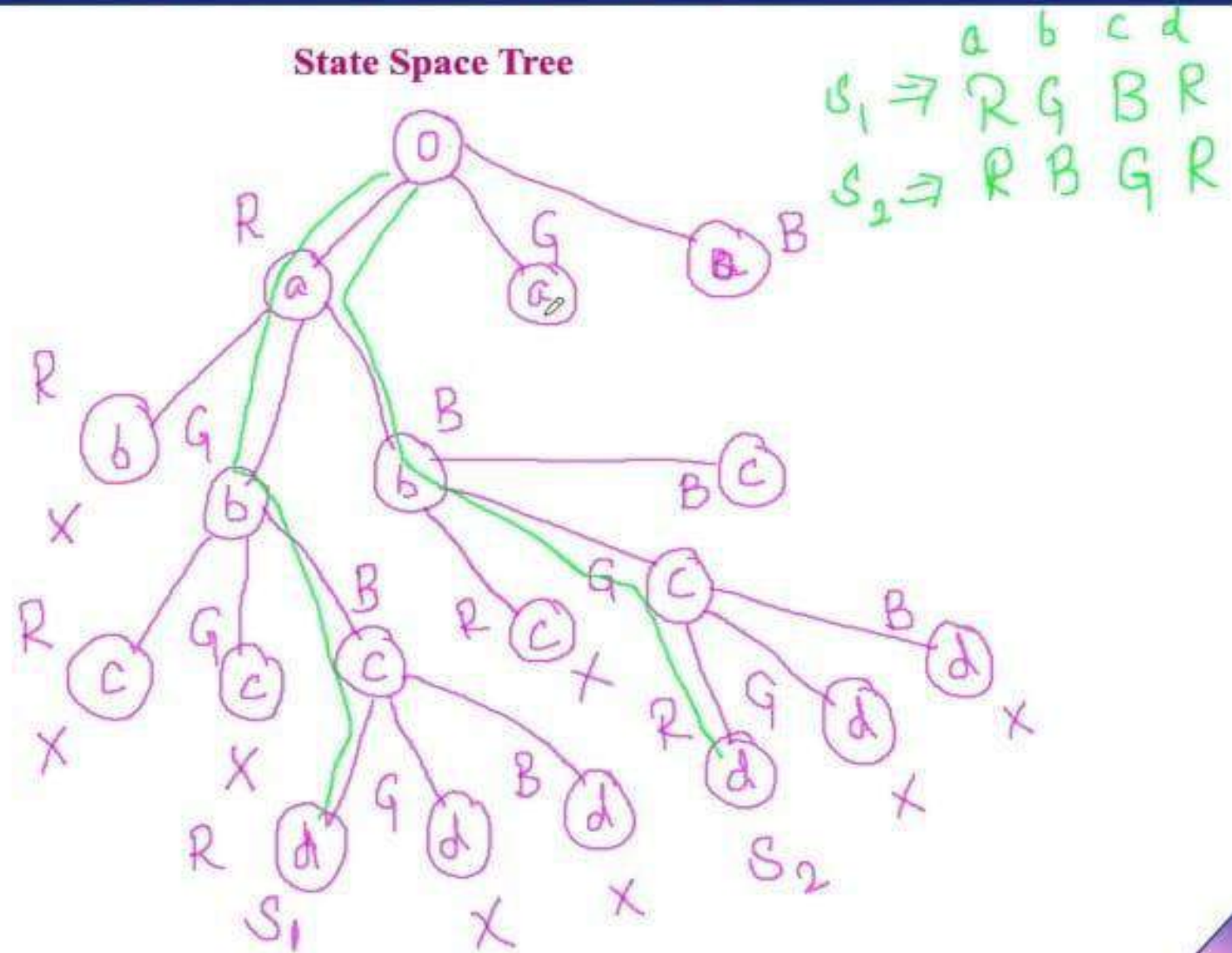


BACKTRACKING - GRAPH COLOURING PROBLEM

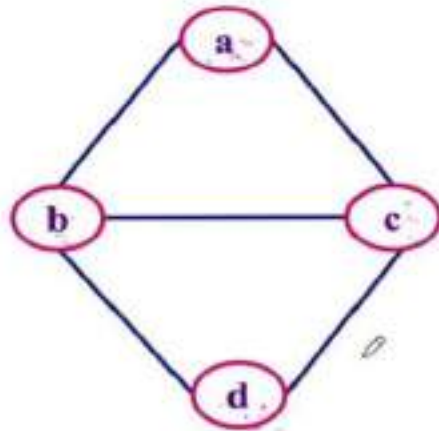


M=3
R,G,B

State Space Tree

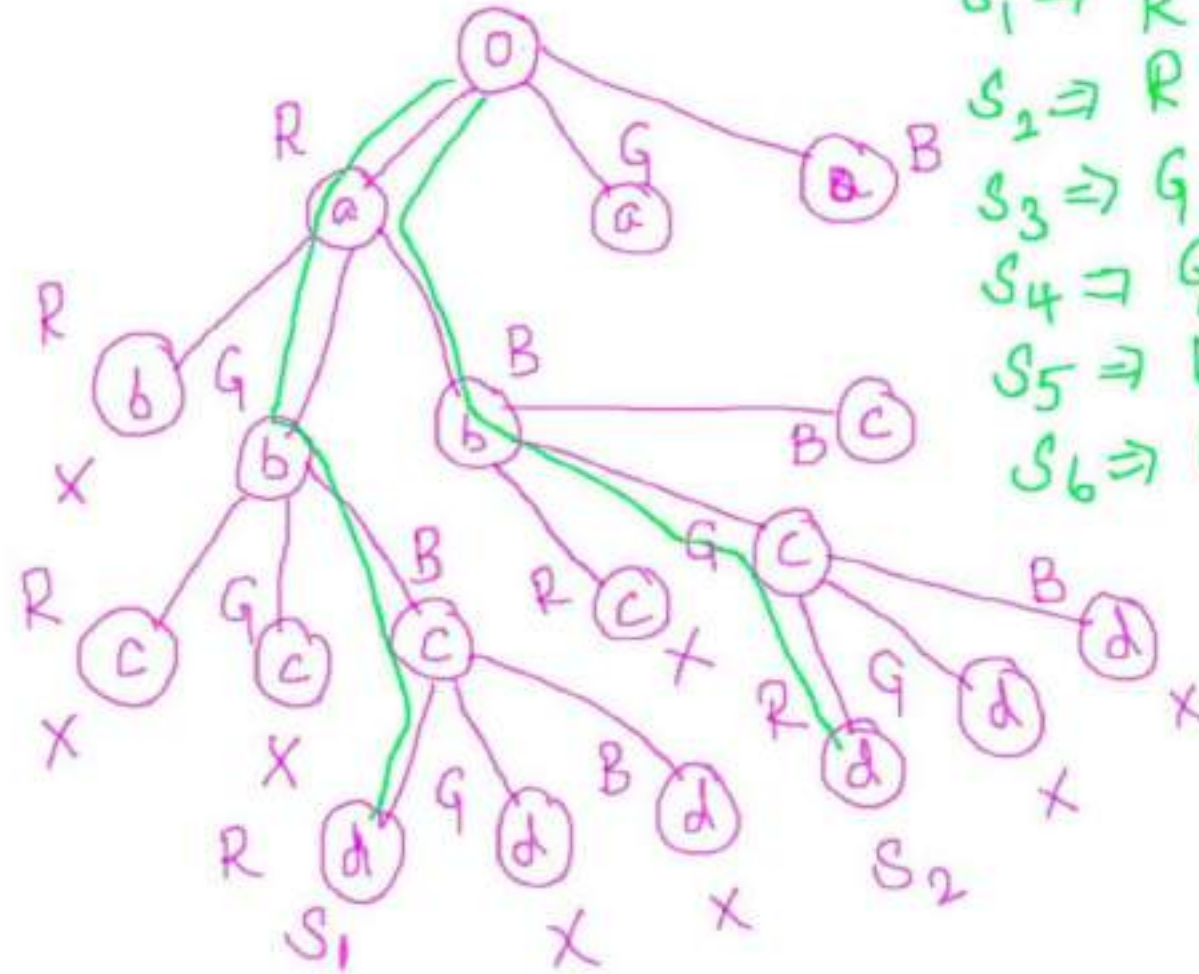


BACKTRACKING - GRAPH COLOURING PROBLEM



$M=3$
R, G, B

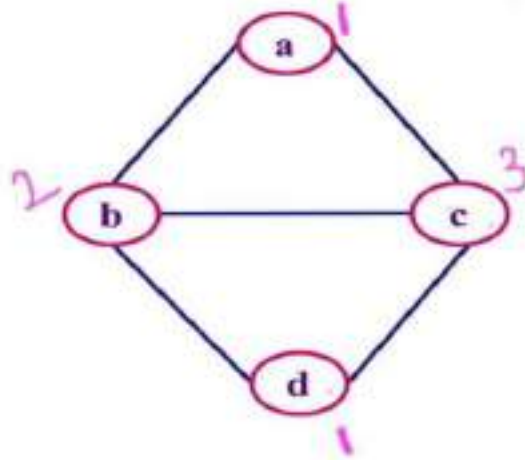
State Space Tree



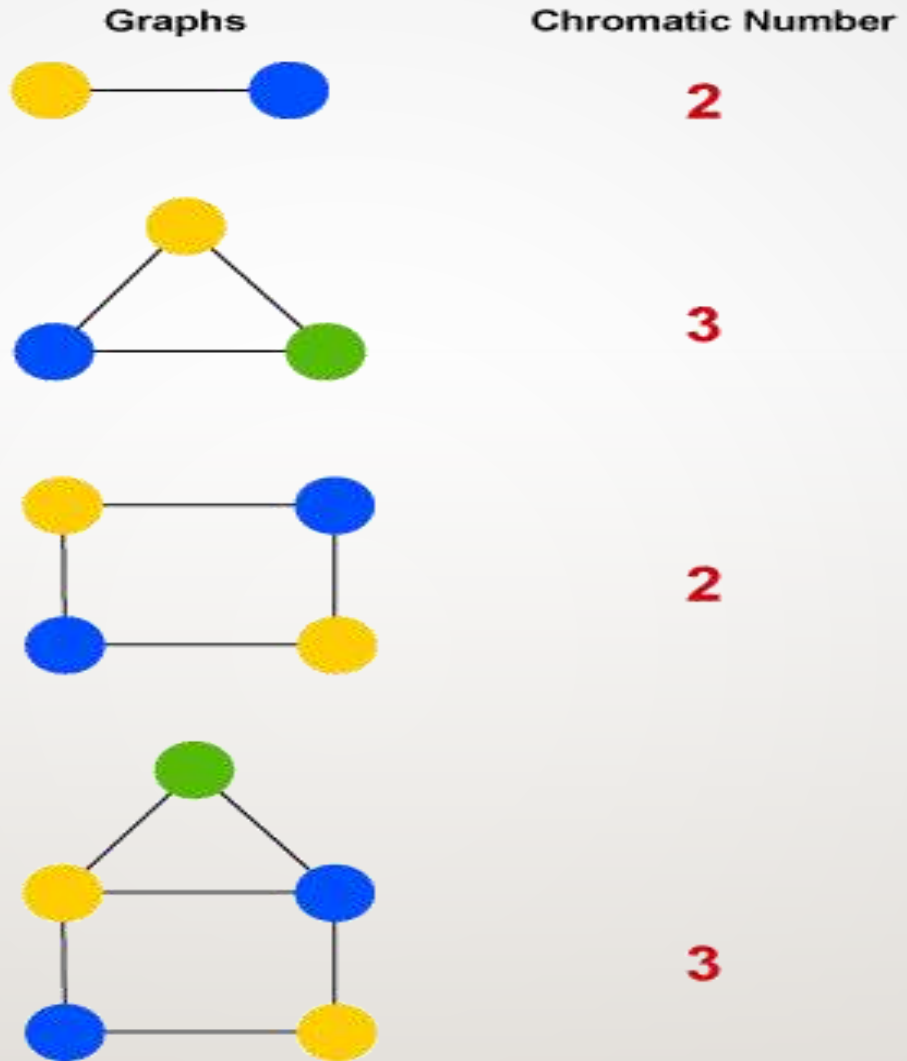
	a	b	c	d
$S_1 \Rightarrow$	R	G	B	R
$S_2 \Rightarrow$	R	B	G	R
$S_3 \Rightarrow$	G	R	B	G
$S_4 \Rightarrow$	G	B	R	G
$S_5 \Rightarrow$	B	G	R	B
$S_6 \Rightarrow$	B	R	G	B

BACKTRACKING - GRAPH COLOURING PROBLEM

What is the chromatic number for this given graph



Minimum No. of Colours Required $\Rightarrow$ 3



Chromatic number of graphs having different number of nodes

Algorithm **mColoring(k)**

// the graph is represented by its Boolean adjacency matrix $G[1:n,1:n]$.All assignments
//of $1,2,\dots,m$ to the vertices of the graph such that adjacent vertices are assigned
//distinct integers are printed. 'k' is the index of the next vertex to color.

{

repeat

{ // generate all legal assignment for $X[k]$.

Nextvalue(k); // Assign to $X[k]$ a legal color.

If ($X[k]=0$) then return; // No new color possible.

If ($k=n$) then // Almost 'm' colors have been used to color the 'n' vertices

Write($x[1:n]$);

Else **mcoloring(k+1);**

}until(false);

}

Algorithm NextValue(k)

```
//  $x[1], \dots, x[k-1]$  have been assigned integer values in  
// the range  $[1, m]$  such that adjacent vertices have distinct  
// integers. A value for  $x[k]$  is determined in the range  
//  $[0, m]$ .  $x[k]$  is assigned the next highest numbered color  
// while maintaining distinctness from the adjacent vertices  
// of vertex  $k$ . If no such color exists, then  $x[k]$  is 0.  
{  
    repeat  
    {  
         $x[k] := (x[k] + 1) \bmod (m + 1)$ ; // Next highest color.  
        if ( $x[k] = 0$ ) then return; // All colors have been used.  
        for  $j := 1$  to  $n$  do  
        {  
            // Check if this color is  
            // distinct from adjacent colors.  
            if (( $G[k, j] \neq 0$ ) and ( $x[k] = x[j]$ ))  
            // If ( $k, j$ ) is an edge and if adj.  
            // vertices have the same color.  
                then break;  
        }  
        if ( $j = n + 1$ ) then return; // New color found  
    } until (false); // Otherwise try to find another color.  
}
```


TIME COMPLEXITY:

- An upper bound on the computing time of **mColoring** can be derived at by noticing that the number of internal nodes in the state space tree is:

$$\sum_{i=0}^{n-1} m^i$$

At each internal node, $O(mn)$ time is spent by **NextValue** to determine the children corresponding to legal colorings.

Hence the total time is bounded by:

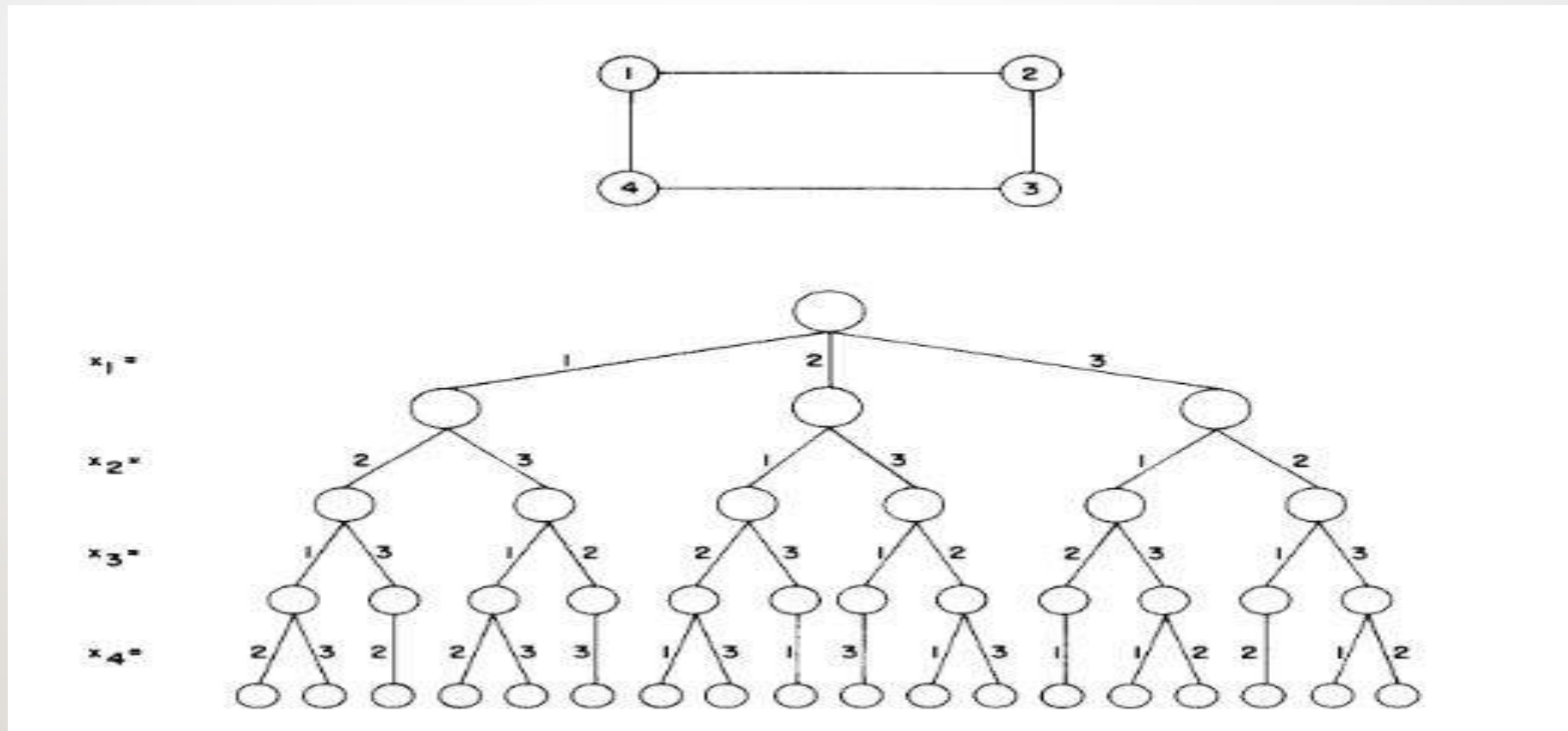
$$\sum_{i=0}^{n-1} m^{i+1} n$$

$$n(m^{n+1} - 2) / (m - 1) = O(nm^n)$$

SAMPLE QUESTIONS

- What is the Eight Queens problem? Explain its objective and constraints.
- Describe the rules and constraints that govern the placement of eight queens on an 8x8 chessboard.
- Explain the concept of backtracking
- What is goal state

A 4 – NODE GRAPH AND ALL POSSIBLE 3 COLORINGS



SUM OF SUBSETS PROBLEM BY BACKTRACKING

SUM OF SUBSETS

Problem Definition: Given n distinct positive numbers w_i , and m , find all subsets whose sums are m .

- **Explicit Constraints :**
- $X_i = \{ j / j \text{ is an integer and } 1 \leq j \leq n \}$
- **Implicit Constraints :**
 1. No two x_i 's are same
 2. Sum of the chosen weights must be equal to m .
 3. To avoid generation of multiple instances of the same subsets)

We can formulate this problem using Fixed tuple or Variable tuples.

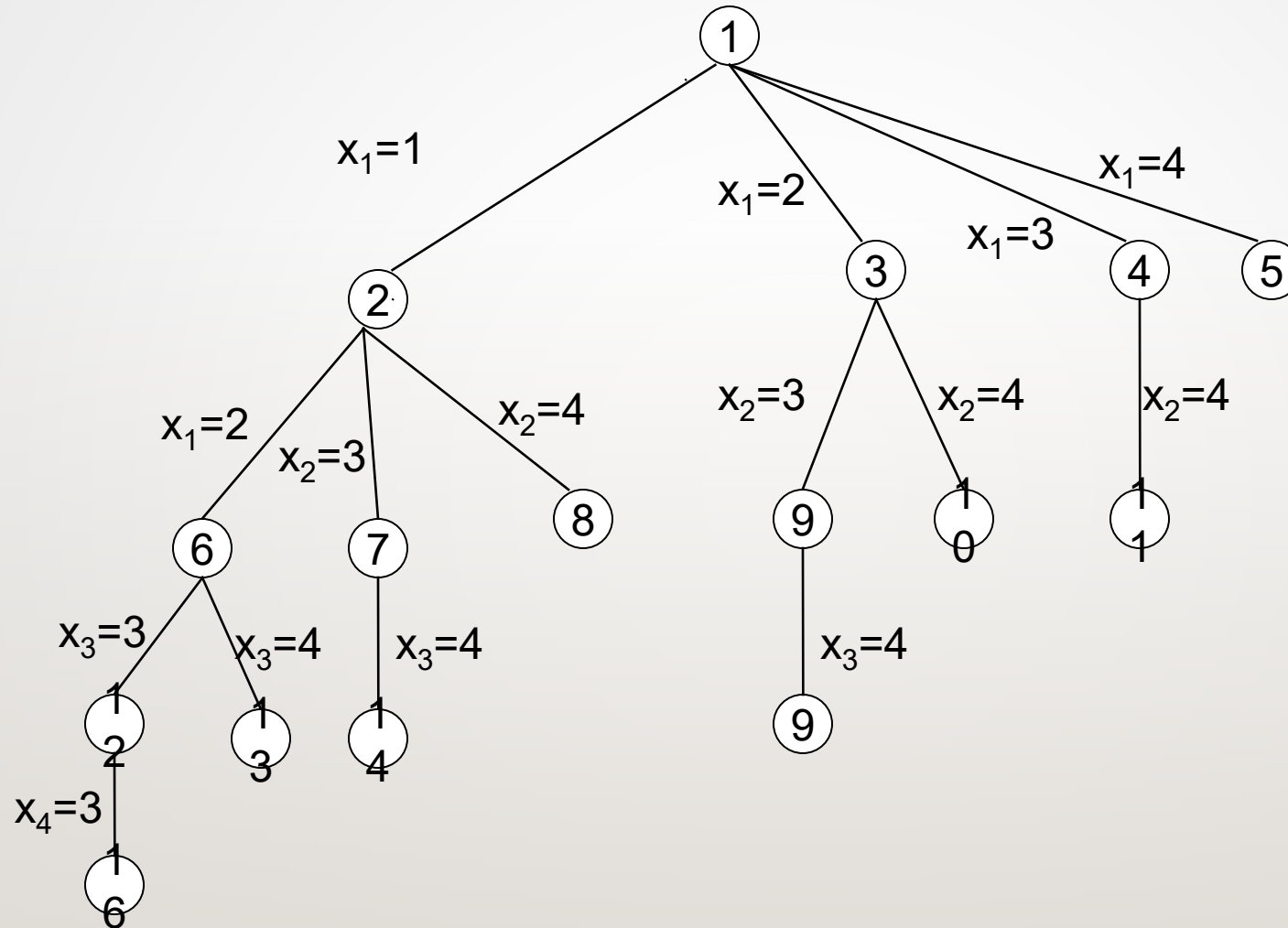
VARIABLE- SIZED TUPLE

Ex:- $n=4$, $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$, $m=31$.

- Solutions are $(11, 13, 7)$ and $(24, 7)$
- Rather than representing the solution by w_i 's, we can represent by giving the indices of these w_i
- Now the solutions are $(1, 2, 4)$ and $(3, 4)$.
- Different solutions may have different-sized tuples.
- We use the following condition to avoid generating multiple instances of the same subset (e.g., $(1, 2, 4)$ and $(1, 4, 2)$)

$$x_i < x_i + 1$$

Subset Tree for $n=4$ (variable- sized tuple)



Nodes are numbered as in Breadth first search.

FIXED- SIZED TUPLE

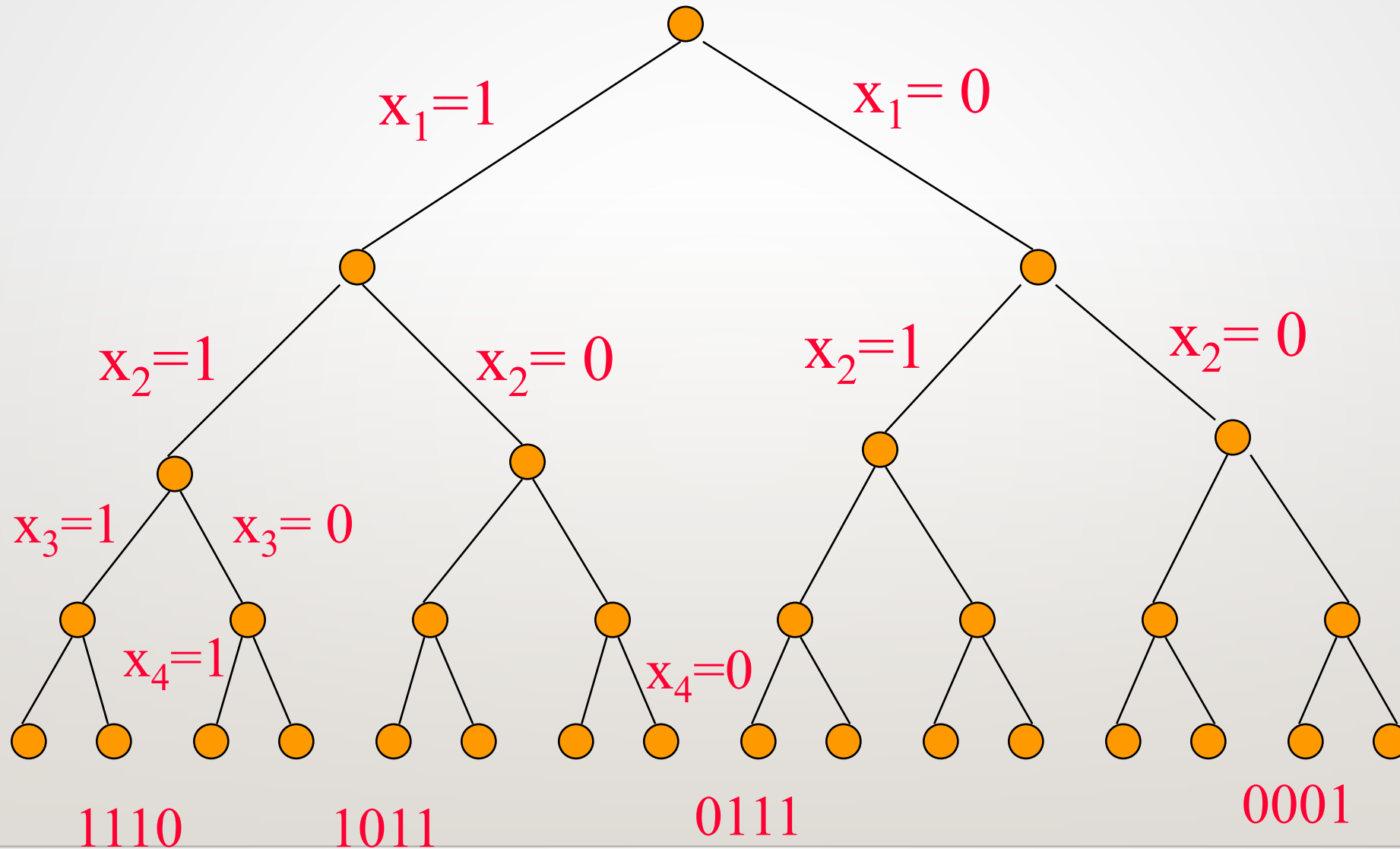
- In this method, each solution subset is represented by an n-tuple $(x_1, x_2, \dots, x_n)$.
- $x_i = 0$ if w_i is not chosen and $x_i = 1$ if w_i is chosen

Ex:- $n=4$, $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$, $m=31$.

- Solutions are $(11, 13, 7)$ and $(24, 7)$

- Solutions are:
- $X=(1,1,0,1)$ and $X=(0,0,1,1)$

SUBSET TREE FOR N = 4 (FIXED – SIZED TUPLE)



BOUNDARY CONDITIONS OF SUM OF SUBSETS

Simple choice for the bounding Function is $B_k(x_1 \dots x_k) = \text{true}$ iff

$$\sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m$$

Clearly $x_1 \dots x_k$ can not lead to an answer node if this condition is not satisfied.

$$\sum_{i=1}^k w_i x_i + w_{k+1} > m$$

Assuming w_i 's in non decreasing order, $(x_1 \dots x_k)$ cannot lead to an answer node if

So, the bounding functions we use are therefore

$$B_k(x_1, \dots, x_k) = \text{true iff } \sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m$$

$$\text{and } \sum_{i=1}^k w_i x_i + w_{k+1} \leq m$$

Algorithm **SumOfSub**(s, k, r)

// Find all subsets of $w[1:n]$ that sum to m

// It is assumed that $w[1] \leq m$ and $\sum w_i \geq m$

{

$x[k]=1$; // left child

 if ($s + w[k] = m$) then write($x[1:k]$); // Subset found

 else if ($s + s[k] + s[k+1] \leq m$)

 then SumOfSub($s + w[k], k+1, r - w[k]$)

 // Generate right child and evaluate Bk

 if (($s + r - w[k] \geq m$) and ($s + w[k+1] \leq m$)) then

 {

$x[k] = 0$;

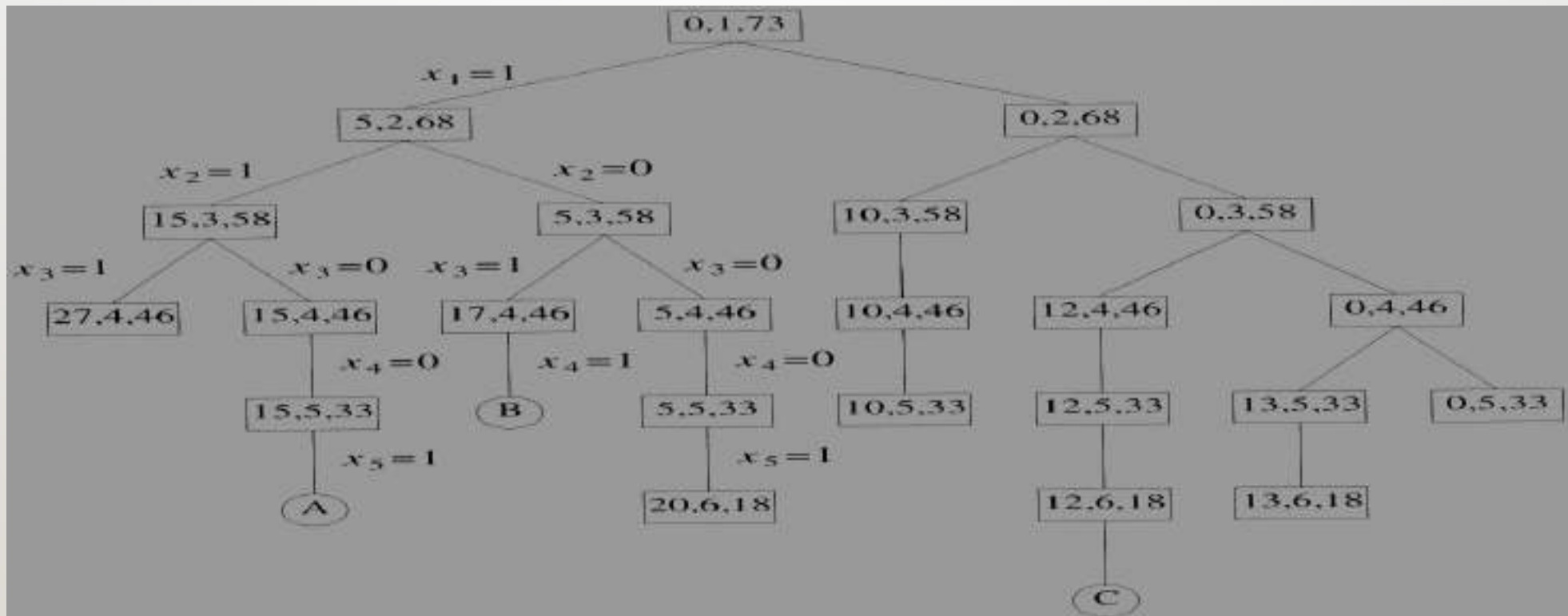
 }

 SumOfSub(s, k+1, r - w[k]);

}

EX:- $N=6$, $M=30$, $W [1:6] = \{ 5,10,12,13,15,18 \}$
 PORTION OF STATE SPACE TREE GENERATED BY SUMOFSUB.
 CIRCULAR NODES INDICATE SUBSETS WITH SUMS EQUAL TO M .

- Example 7.6 (Figure 7.10)
- $n=6$, $w[1:6]=\{5,10,12,13,15,18\}$, $m=30$



SAMPLE QUESTIONS

- What is the Sum of Subsets problem
- Describe the problem statement of the Sum of Subsets. What are the inputs and outputs of the problem
- Explain about state space tree in sum of subsets problem
- Solve sum of subsets by using Input: set[] = {4, 16, 5, 23, 12}, sum = 9

BRANCH & BOUND

Terminology

Live node is a node that has been generated but whose children have not yet been generated.

E -node (Expanded Node) is a live node whose children are currently being explored. In other words, an E-node is a node currently being expanded.

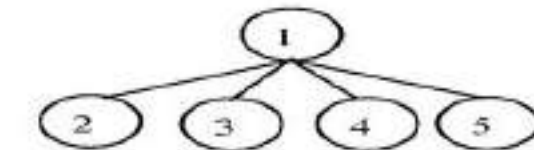
Dead node is a generated node that is not to be expanded or explored any further. All children of a dead node have already been expanded.

Branch-and-bound refers to all state space search methods in which all children of an E-node are generated before any other live node can become the E-node.

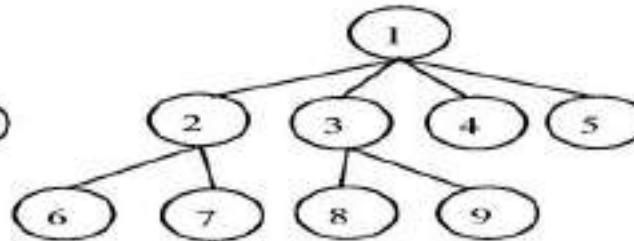
Branch and Bound

Definitions:

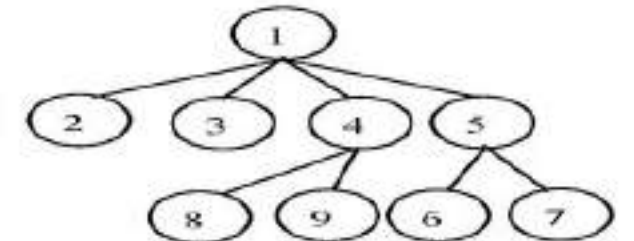
- Branch and Bound is a state space search method in which all the children of a node are generated before expanding any of its children.
- **Live-node:** A node that has not been expanded.
- It is similar to backtracking technique but uses BFS-like search.



Live Node: 2, 3, 4, and 5



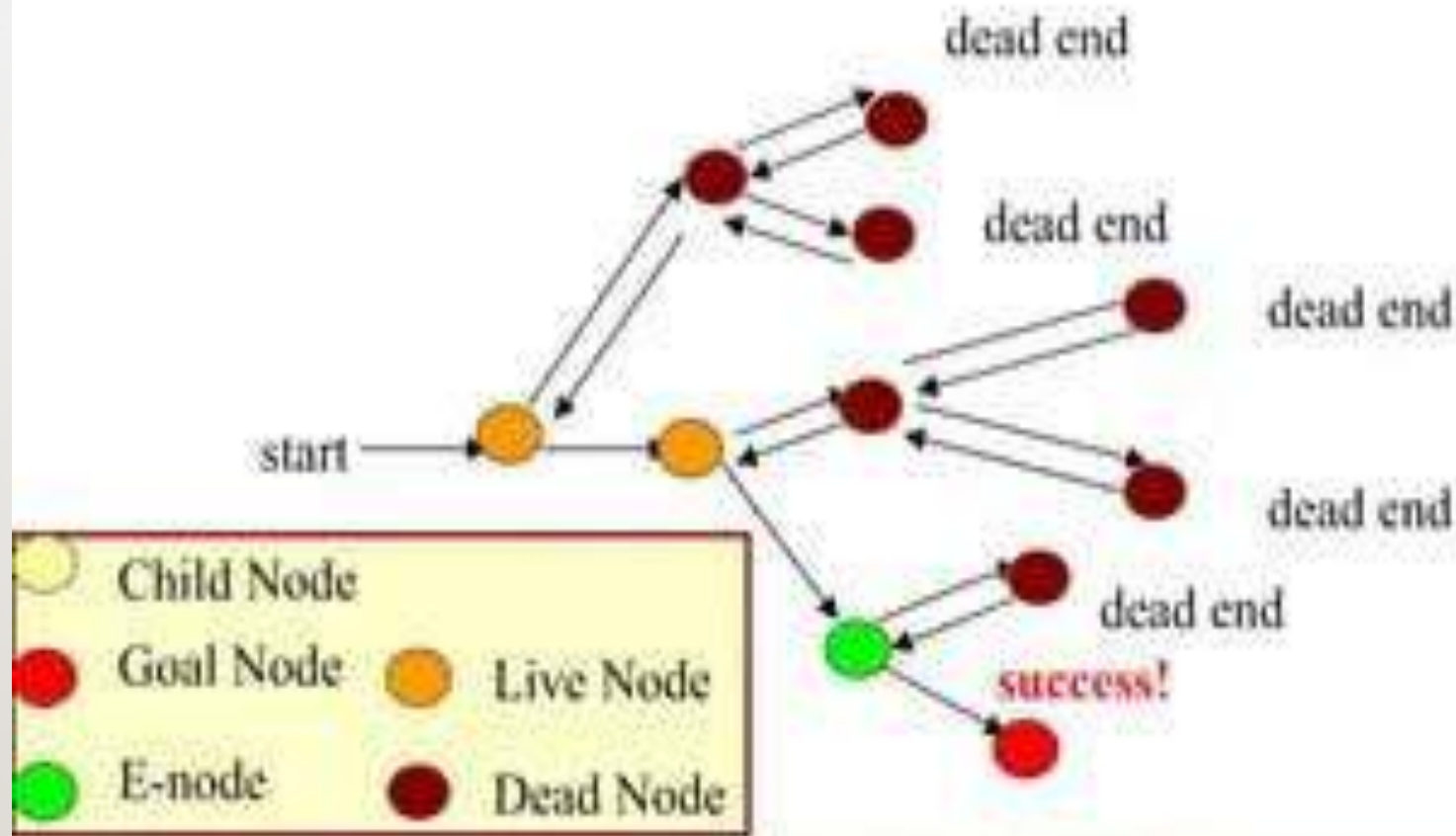
FIFO Branch & Bound (BFS)
Children of E-node are
inserted in a queue.



LIFO Branch & Bound (D-Search)
Children of E-node are inserted in a
stack.

- Dead-node: A node that has been expanded
- Solution-node

Types Of Nodes - Demonstration



BRANCH AND BOUND -- THE METHOD

The term branch-and-bound refers to all state space search methods in which all children of the E-node are generated before any other live node can become the E-node.

In branch-and-bound terminology breadth first search(**BFS**)- like state space search will be called FIFO (First In First Output) search as the list of live nodes is a first-in-first-out list (or queue).

A **D-search** (depth search) state space search will be called LIFO (Last In First Out) search, as the list of live nodes is a last-in-first-out list (or stack).

Bounding functions are used to help avoid the generation of sub trees that do not contain an answer node.

- A search strategy that uses a cost function $c(x) = f(h(x)) + g(x)$ to select the next E-node, would always choose for its next E-node a live node with least cost. Hence, such a search strategy is called an LC-search (Least Cost search).
- **BFS and D-search are the special cases of LC-search.**
- If $g(x) = 0$ and $f(h(x)) = \text{level of node } x$, then a LC-search becomes BFS.
- If $f(h(x)) = 0$ and $g(x) > g(y)$ whenever y is a child of x , then LC search is essentially a D-search.
- **An LC-search coupled with bounding functions is called an LC branch-and-bound search.**
- **Generating state space tree using FIFO search with bounding functions is called FIFOBB**
- **Generating state space tree using LIFO search with bounding functions is called LIFOBB**

One of the problem-solving approaches is the branch and bound. It is similar to backtracking in that it also employs the state space tree. It is employed in the solution of optimization and minimization problems. If we are given a maximization problem, we can use the Branch and bound technique to transform it by simply changing the problem to a minimization problem.

Example: Jobs = $\{j_1, j_2, j_3, j_4\}$

$$P = \{10, 5, 8, 3\}$$

$$d = \{1, 2, 1, 2\}$$

The jobs, problems, and problems given above are listed above. The solutions can be written in two ways, as shown below:

If we want to do jobs j_1 and j_2 , the solution can be represented in two ways:

The subsets of jobs are the first way to represent the solutions.

$$S1 = \{j1, j4\}$$

The second way to represent the solution is that the first job is completed, the second and third jobs are not completed, and the fourth job is completed.

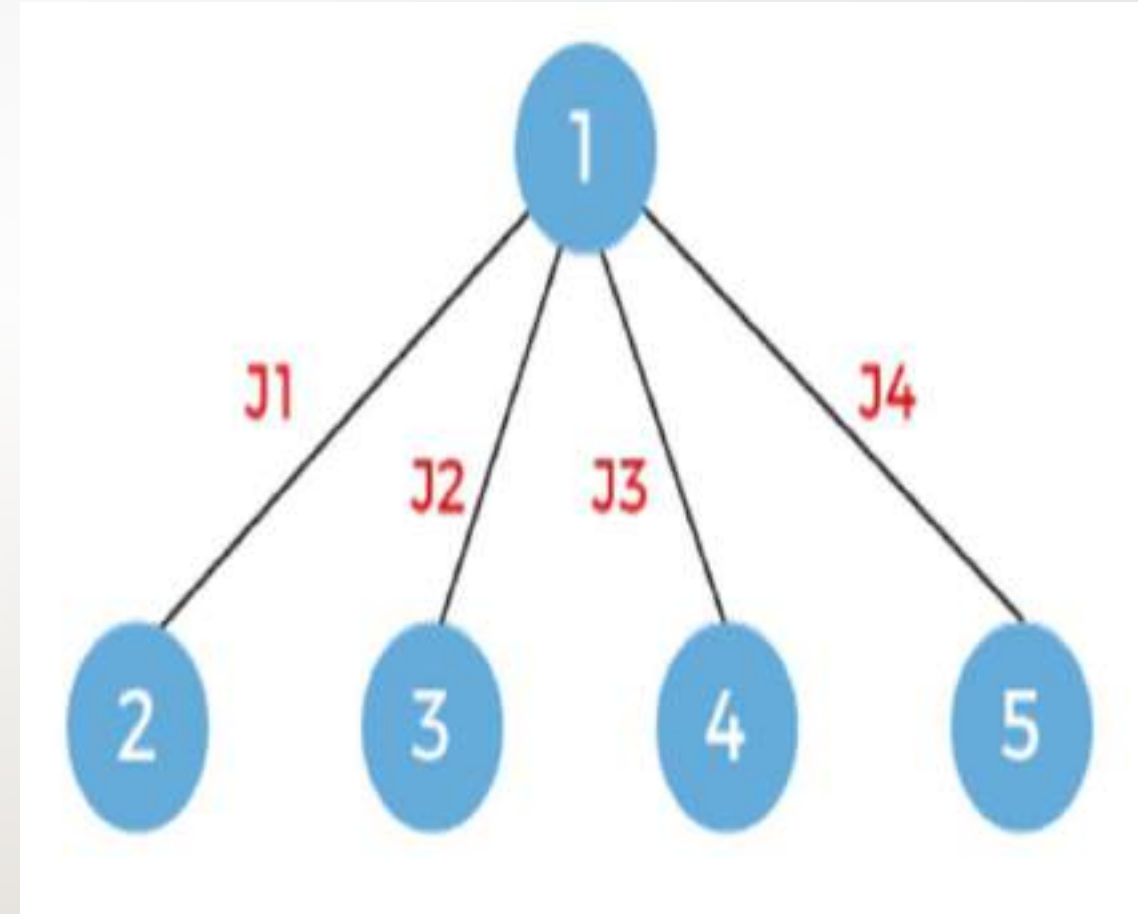
$$S2 = \{1, 0, 0, 1\}$$

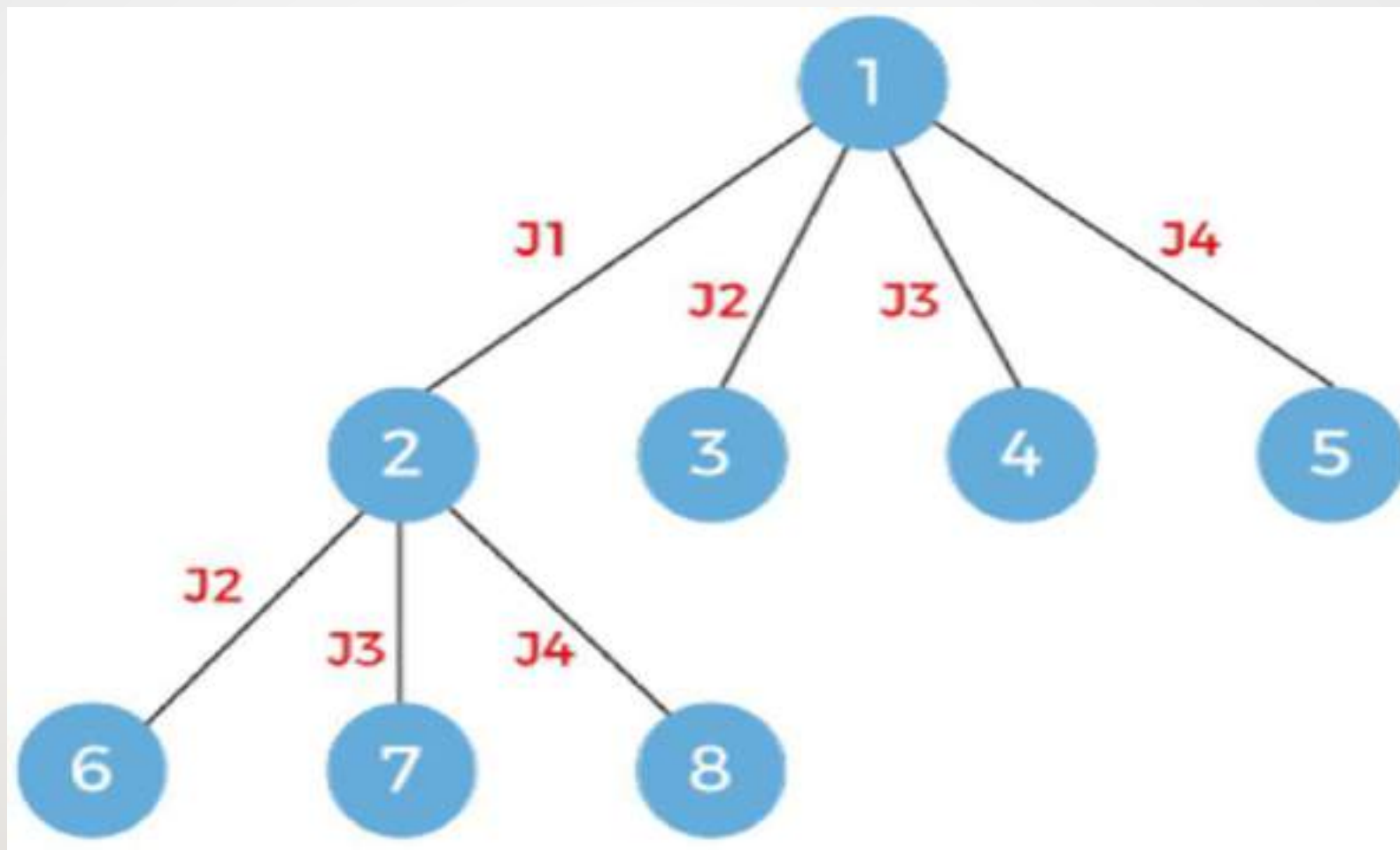
The variable-size solution $s1$ is the variable-size solution, while the fixed-size solution $s2$ is the fixed-size solution.

The variable-size solution is $s1$, and the fixed-size solution is $s2$.

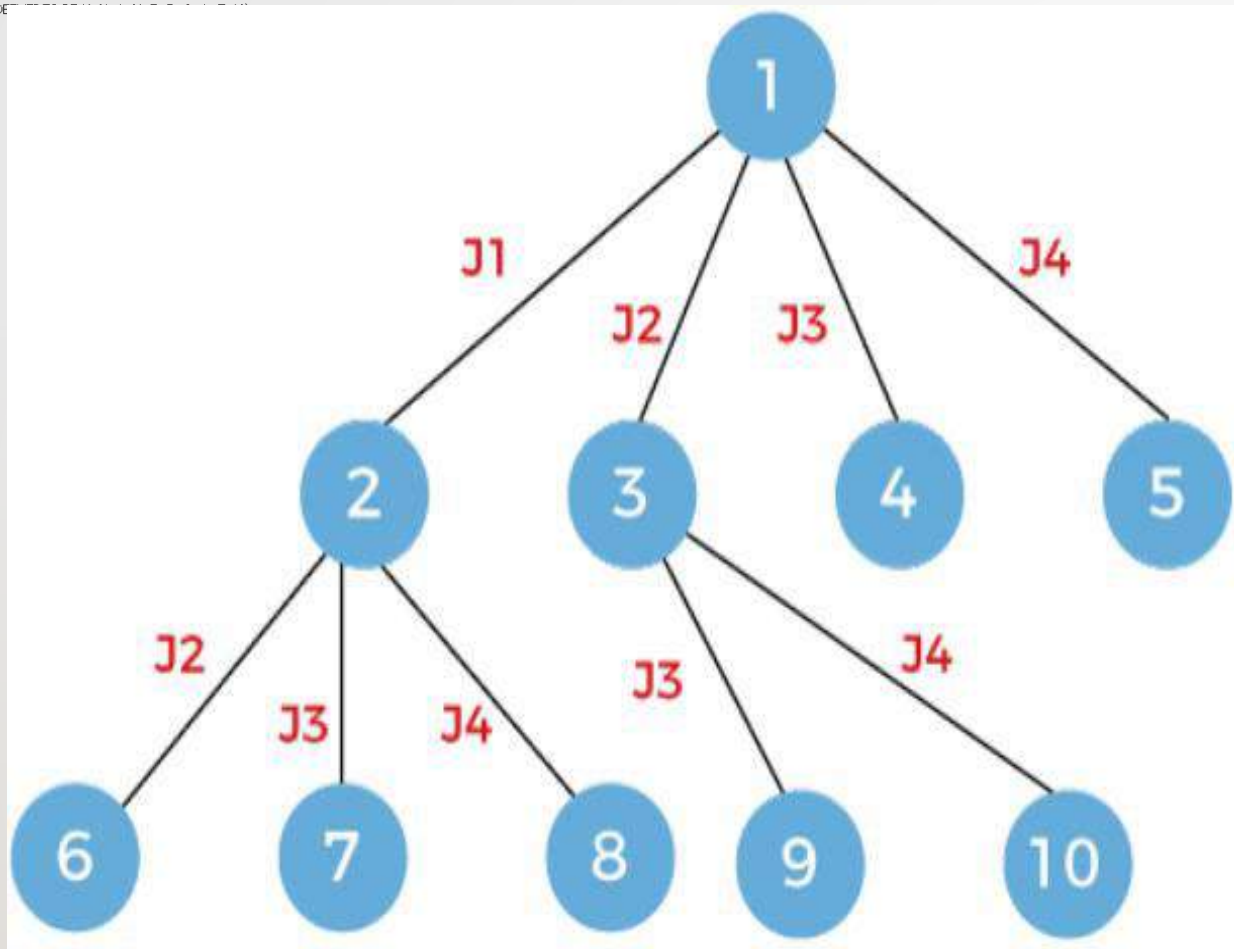
Method I- FIFO BRANCH & BOUND

- In this case, we consider the first job, then the second job, then the third job, and finally the last job.
- As shown in the above figure, the breadth first search is used but not the depth first search. Here, we move in a horizontal direction to investigate the solutions.
- **BACKTRACKING IS DONE IN DEPTH, WHEREAS BRANCH AND BOUND IS DONE IN BREADTH.**

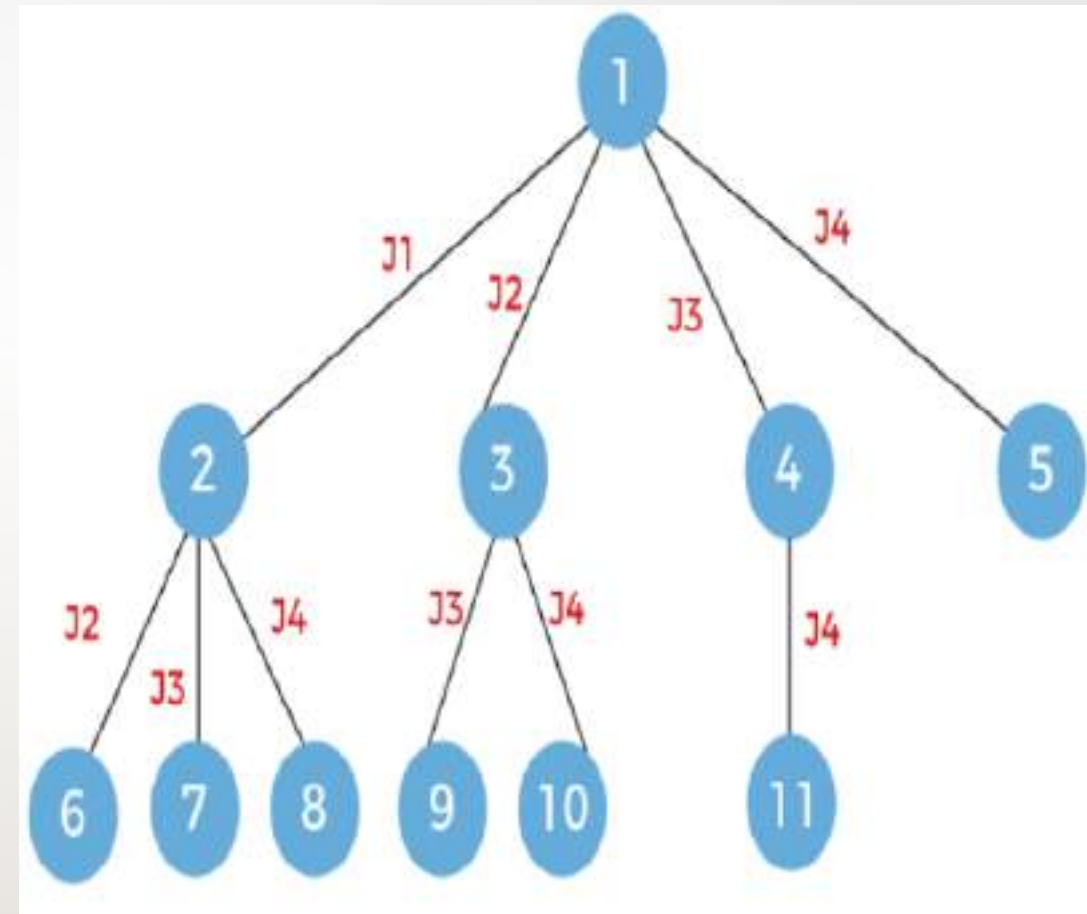




One level has now been completed. After I accept the first job, we can consider j2, j3, or j4. If we follow the path, it says we are doing jobs j1 and j4, so we will disregard jobs j2 and j3.

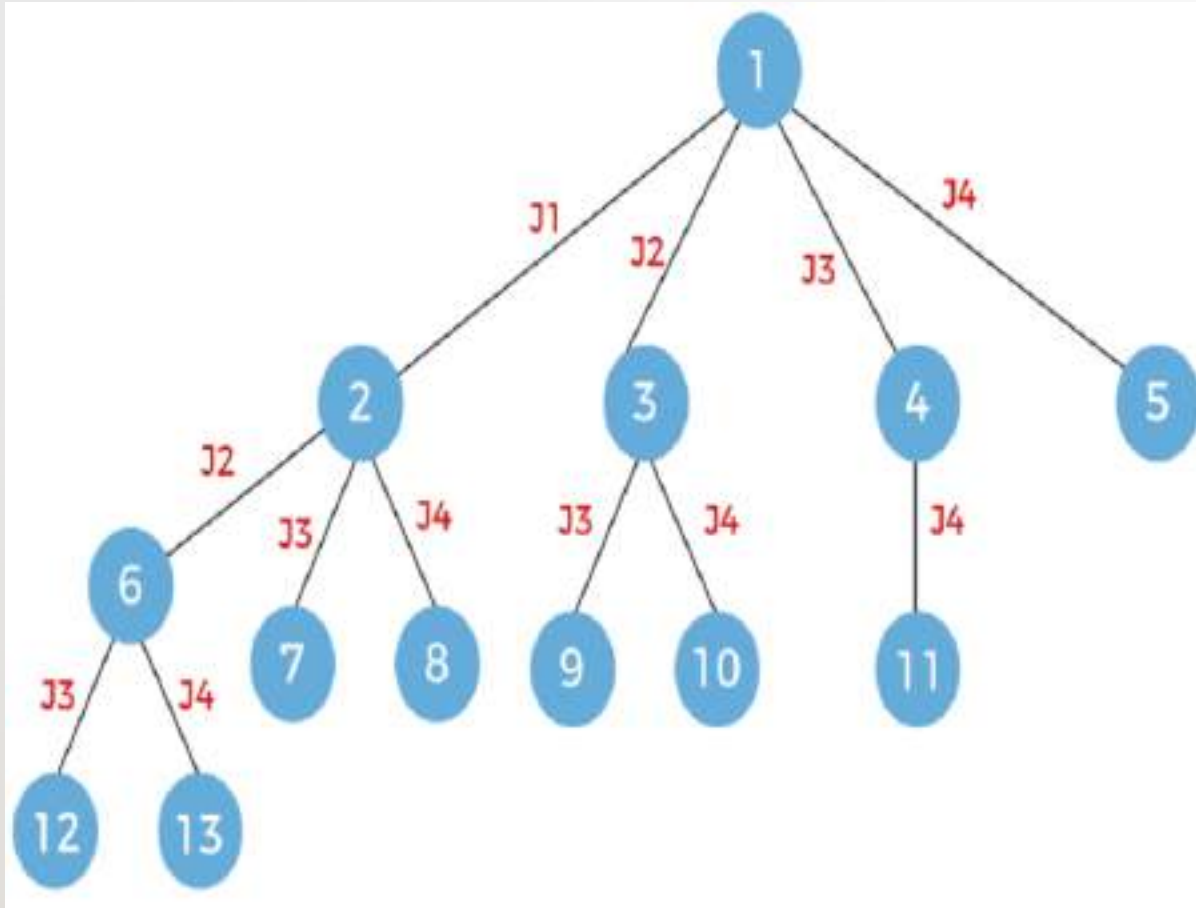


We will now look at node 3. We are doing job j2 in this case, so we can recognize either job j3 or j4. In this case, we have removed the job j1.

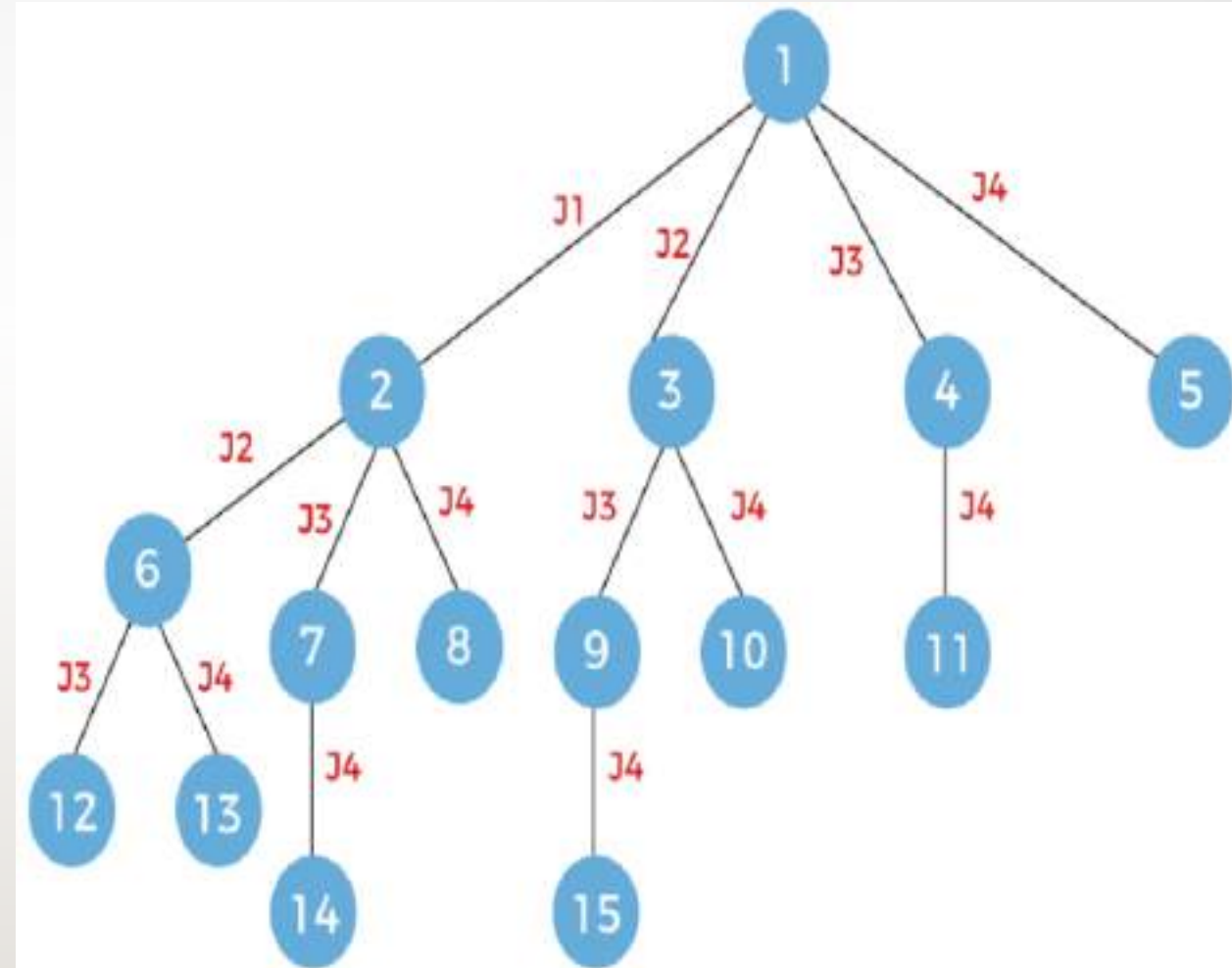


We will now expand node 4. Because we are currently working on job j3, we will only consider job j4.

Now we will expand node 7, and we will look at job j4.



Node 6 will now be expanded, and the jobs j3 and j4 will be considered.

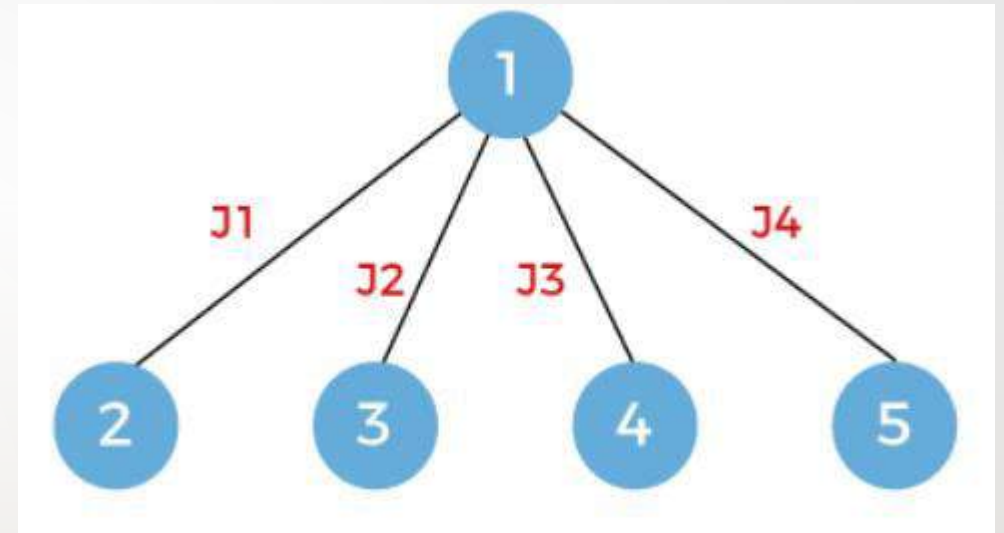


Using the queue and the FIFO concept, we explored all of the nodes in the preceding manner.

Method-2 - LIFO BRANCH & BOUND

We'll look at another technique to tackle the problem to get to solution s1.

First, consider node 1 as indicated below:

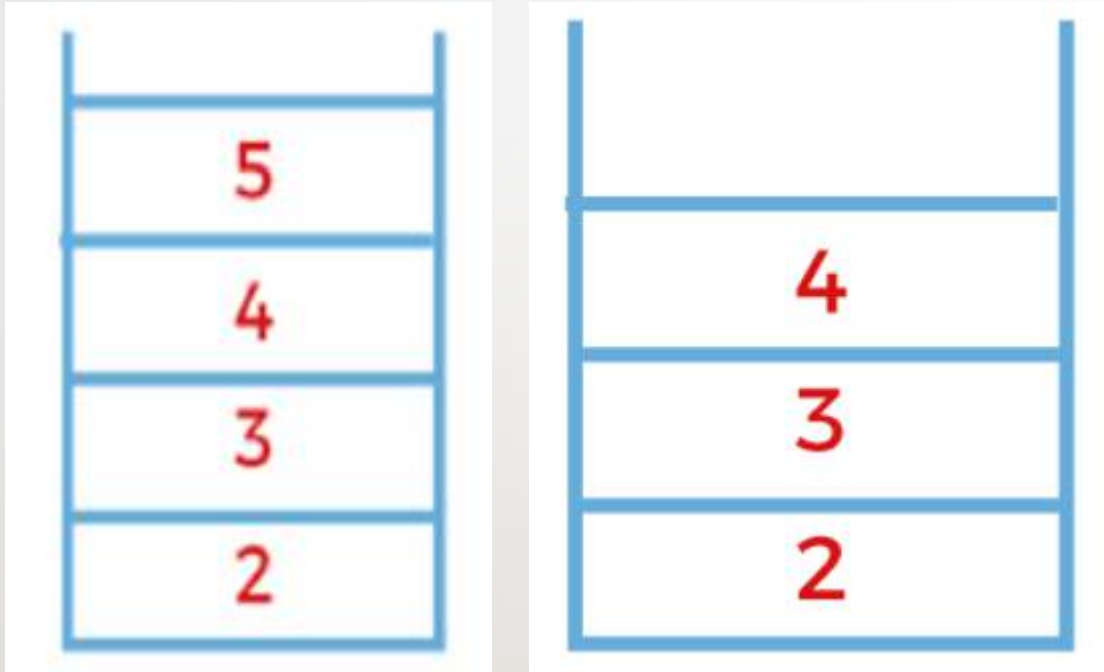


We shall now expand node 1. After expansion, the state space tree might look like this:

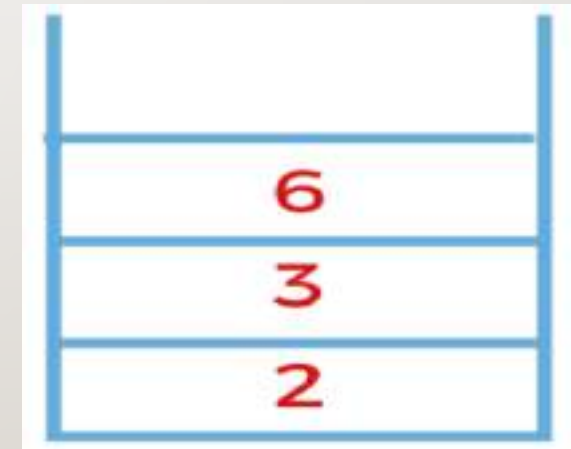
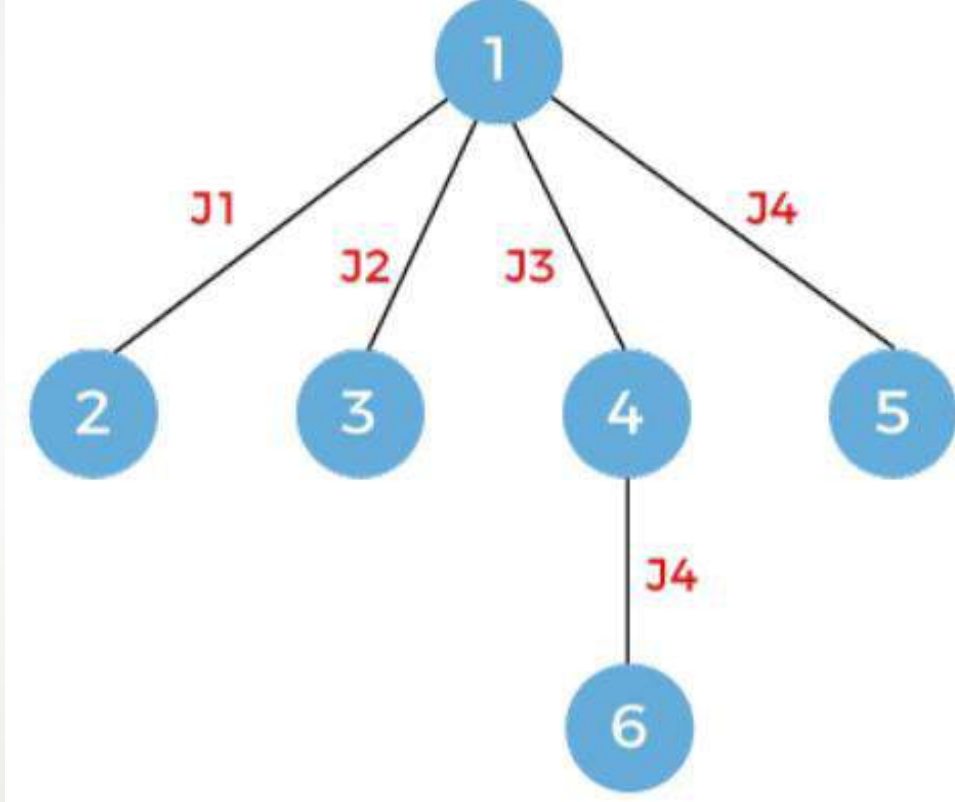


The node will be pushed into the stack indicated below on each expansion:

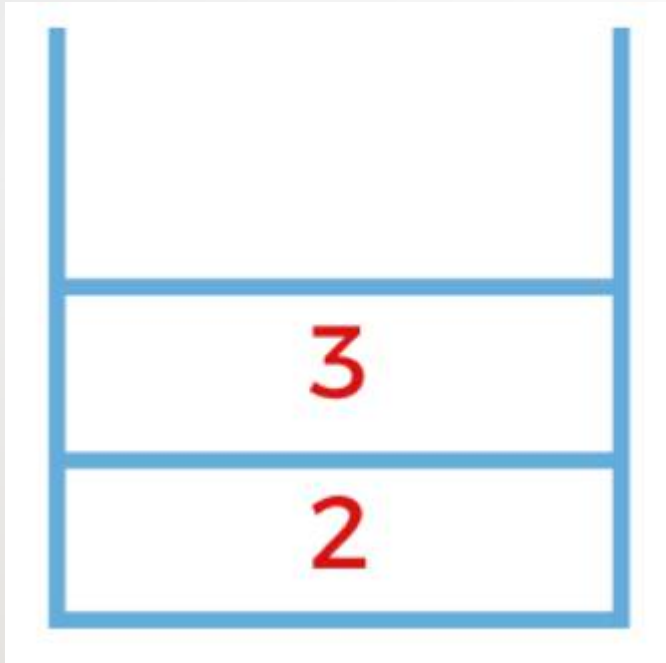
The expansion would now be based on the node at the top of the stack. Because node 5 appears at the top of the stack, we shall enlarge it. We shall remove node 5 from the stack. There is no additional possibility for enlargement because node 5 is in the last job, i.e., j_4 .



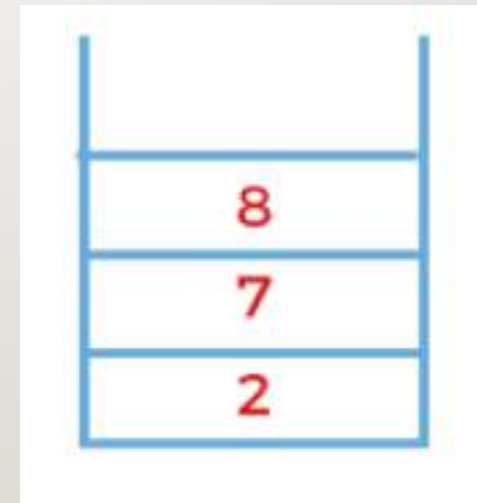
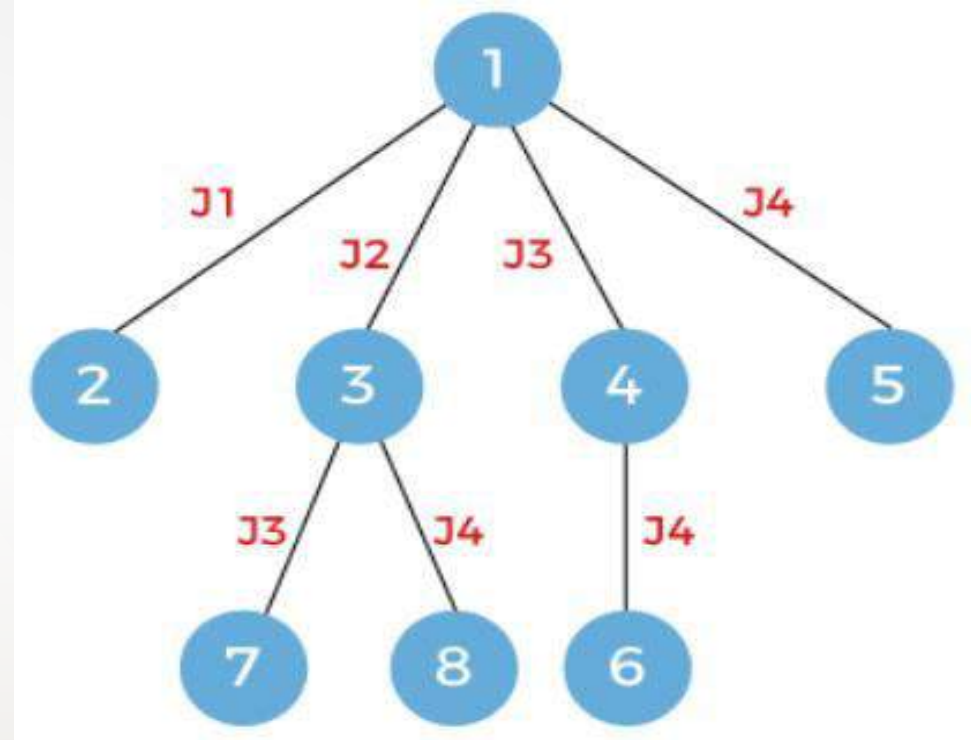
Node 4 is the next node to appear at the top of the stack. Remove node 4 and enlarge it. Job j_4 will be examined for expansion, and node 6 will be added to the stack illustrated below:



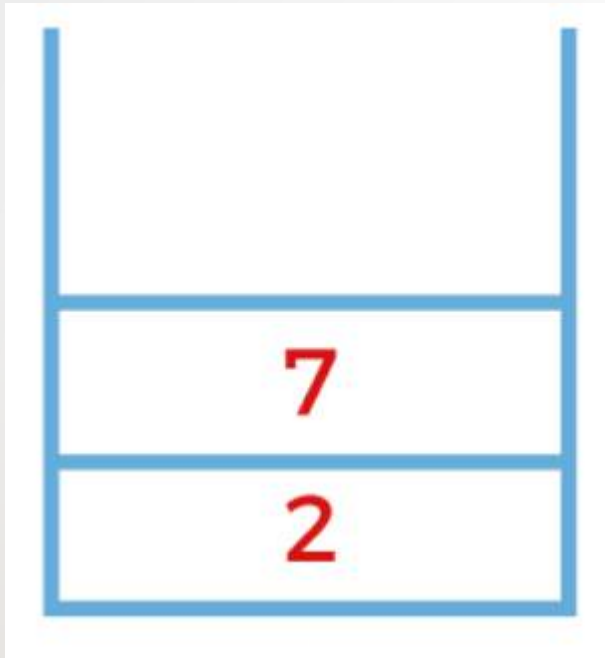
The next node to be expanded is node 6. Remove node 6 and enlarge it. There is no additional possibility for extension because node 6 is in the last job, i.e., j_4 .



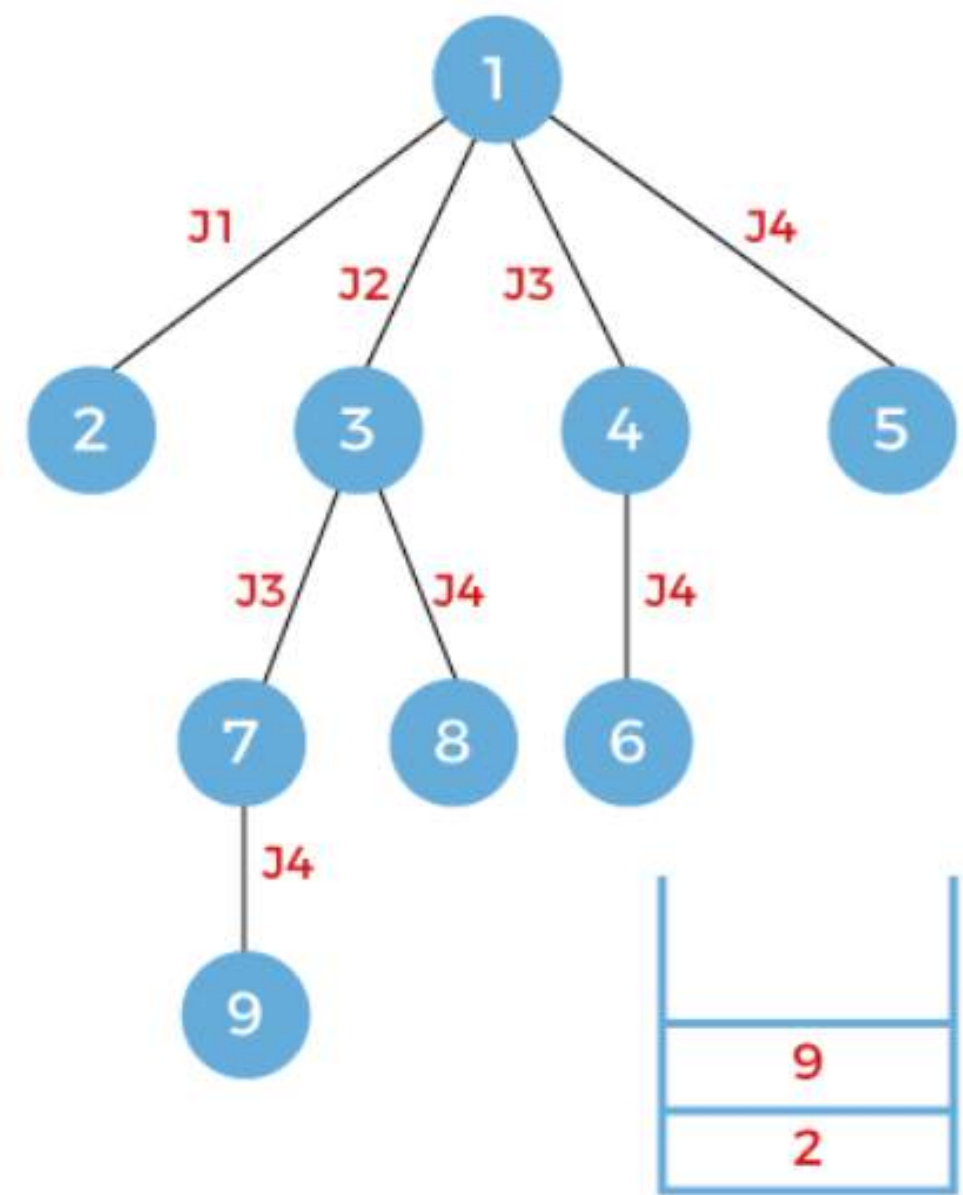
Node 3 is the next to be expanded. Because node 3 works on job j_2 , it will be enlarged to two nodes, namely 7 and 8, which will work on jobs 3 and 4, respectively. Nodes 7 and 8 will be added to the stack illustrated below:



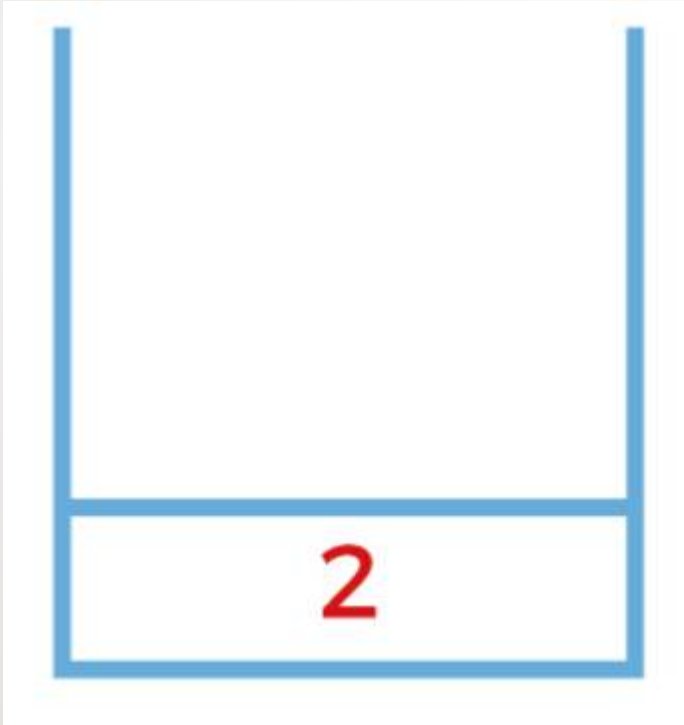
Node 8 is the next node to appear at the top of the stack.
Remove node 8 and enlarge it. There is no additional possibility for enlargement because node 8 works on job j4.



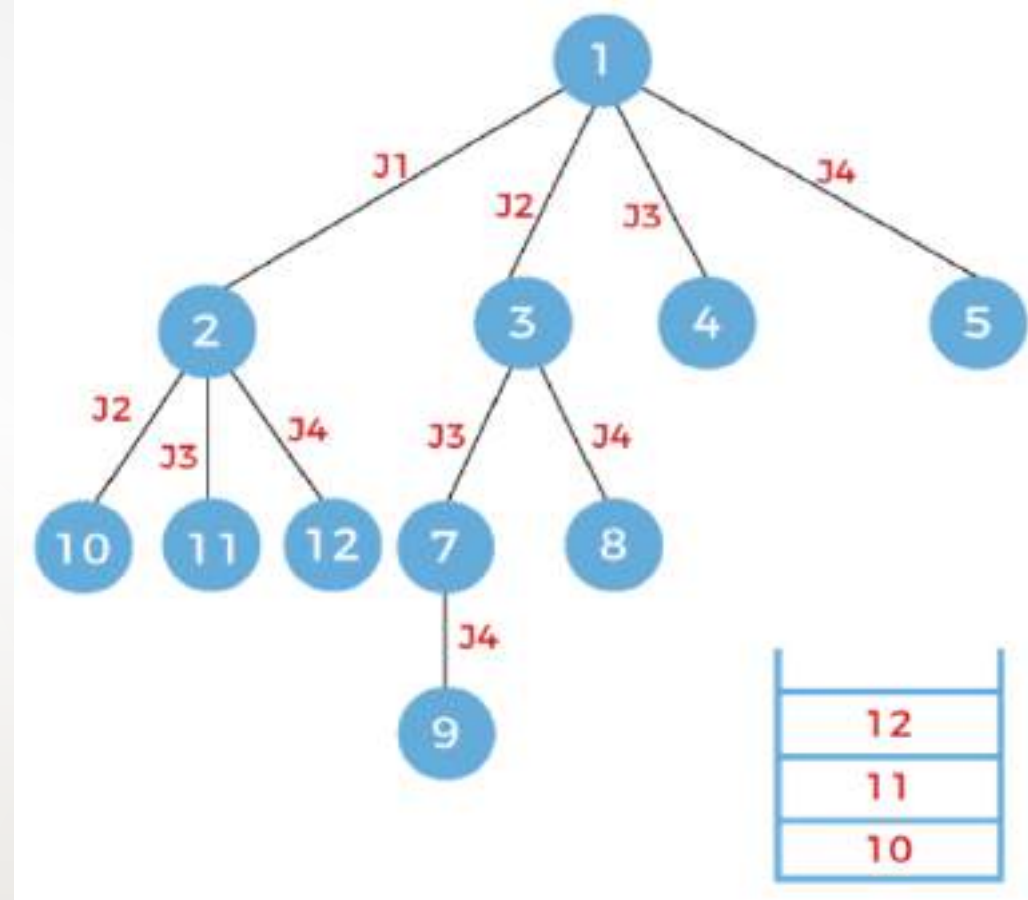
Node 7 is the next node to appear at the top of the stack.
Remove node 7 and enlarge it. Because node 7 works on task j3, node 7 will be enlarged to node 9, which works on job j4, as illustrated below, and node 9 will be pushed into the stacks.



Node 9 is the next node to appear at the top of the stack. There is no more room for expansion because node 9 is working on job 4.



Node 2 is the next node to appear at the top of the stack. Because node 2 is working on job j1, it means that node 2 can be expanded further. It can be expanded to three nodes, 10, 11, and 12, each working on job j2, j3, and j4. New nodes will be added to the stack as illustrated below:



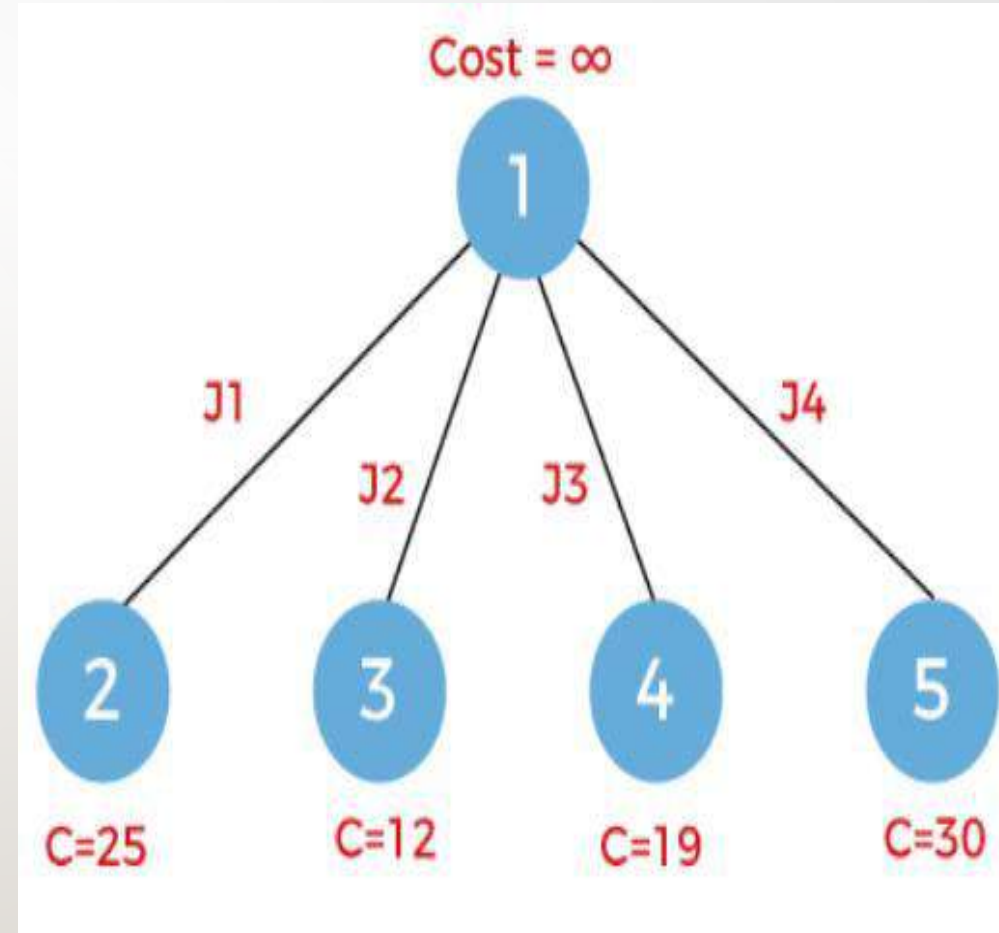
Using the stack and the LIFO concept, we explored all of the nodes in the preceding manner.

Method 3 - LC BRANCH & BOUND

Least cost branch and bound is another strategy that can be utilised to discover the solution. Nodes are investigated using this strategy based on their cost. The cost of the node can be defined using the problem, and the cost function can be defined using the given problem. We can define the node's cost after we define the cost function.

Consider node 1 with cost infinity, as given below:

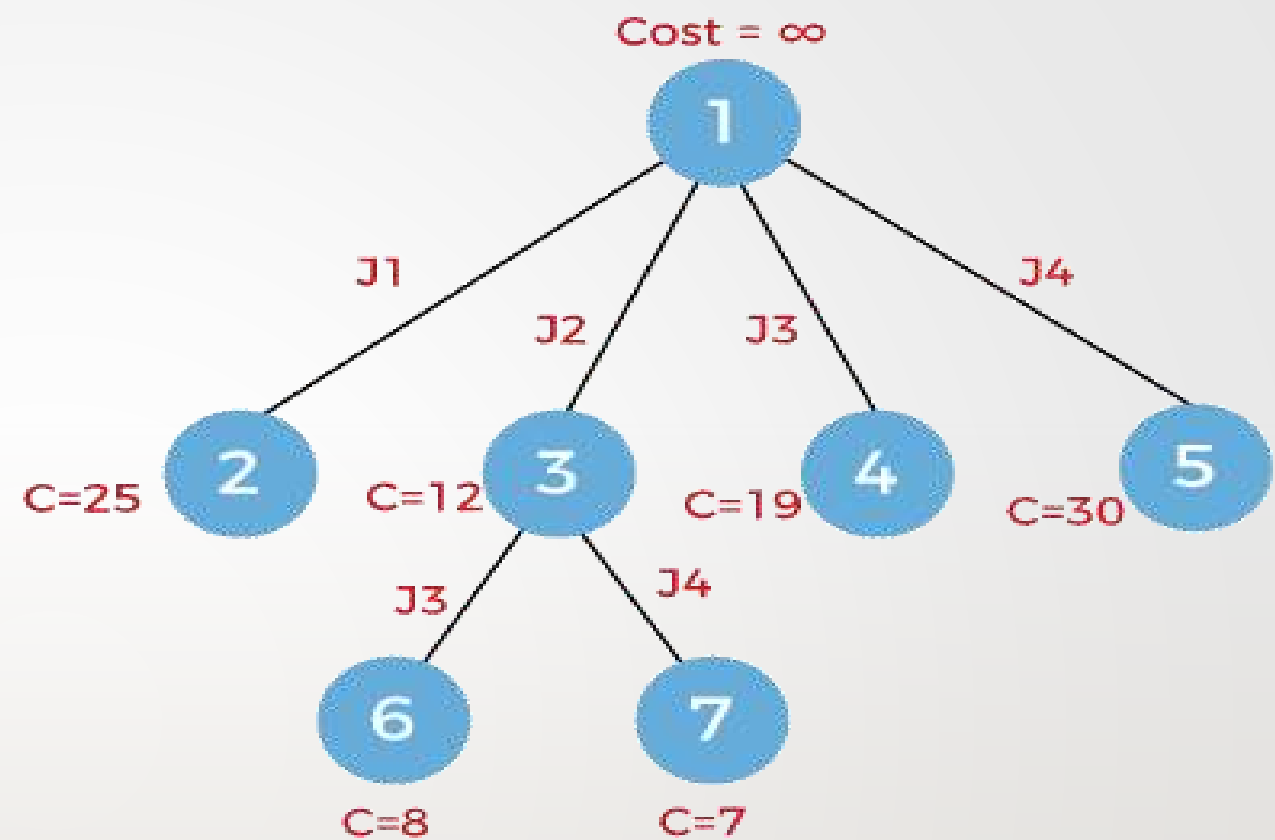
We shall now expand node 1. Node 1 will be divided into four nodes, numbered 2, 3, 4, and 5, as indicated below:



Assume the costs of nodes 2, 3, 4, and 5 are \$25, \$19, and \$30, respectively.

Because it is the least cost branch n bound, we will investigate the node with the lowest cost. In the diagram above, we can see that node 3 has the lowest cost. As a result, we will investigate Node 3 with a cost of \$12.

Because node 3 is working on job j2, it will be split into two nodes, 6 and 7, as indicated below:



Node 6 is working on job j3, while node 7 is working on job j4. The cost of node 6 is \$8, whereas the cost of node 7 is \$7. Now we need to choose the node with the lowest cost. Because node 7 has the lowest cost, we will investigate it. Because node 7 is already working on job j4, there is no room for expansion.

KNAPSACK USING FIFO

7.5 0/1 Knapsack Problem

Problem statement :

The 0/1 Knapsack problem states that - There are 'n' objects given and capacity of Knapsack is 'm'. Then select some objects to fill The Knapsack in such a way that it should not exceed the capacity of Knapsack and maximum profit can be earned. The Knapsack problem is a maximization problem. That means we will always seek for maximum $P_i x_i$ (where P_i represents profit of object x_i). We can also get $\sum P_i x_i$ maximum iff - $\sum P_i x_i$ is minimum.

$$\text{Minimize profit} - \sum_{i=1}^n P_i x_i$$

$$\text{Subject to} \quad \sum_{i=1}^n W_i x_i$$

$$\text{Such that } \sum W_i x_i \leq m \text{ and} \\ x_i = 0 \text{ or } 1 \text{ where } 1 \leq i \leq n$$

Example

Consider the knapsack instance: $n = 4$;

$(p_1, p_2, p_3, p_4) = (10, 10, 12, 18)$;

$(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$ and $M = 15$.

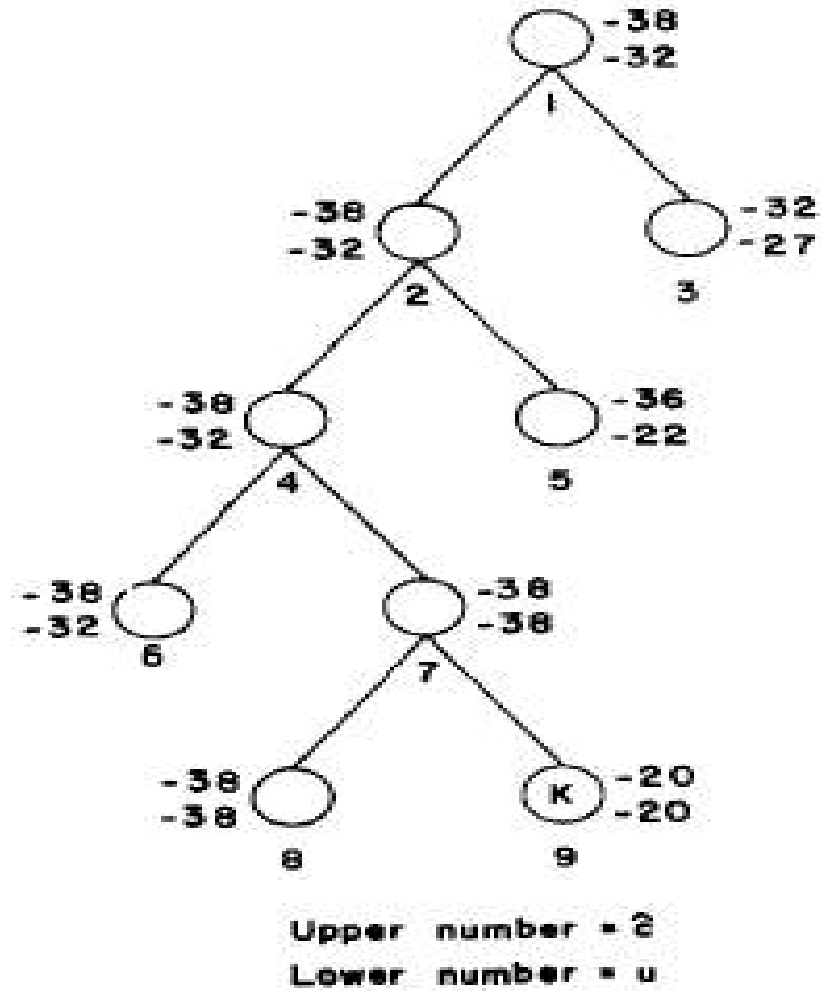


Figure 8.9 LC Branch-and-bound tree for Example 8.2

procedure UBOUND (p, w, k, M)

// p, w, k and M have the same meaning as in Algorithm 7.11*//*

// $W(i)$ and $P(i)$ are respectively the weight and profit of the i th object*//*

global $W(1:n), P(1:n)$; **integer** i, k, n

$b \leftarrow p; c \leftarrow w$

for $i \leftarrow k + 1$ **to** n **do**

if $c + W(i) \leq M$ **then** $c \leftarrow c + W(i); b \leftarrow b + P(i)$ **endif**

repeat

return (b)

end UBOUND

Algorithm 8.5 Function $u(\cdot)$ for knapsack problem

7.5.2 FIFO Branch-and-Bound Solution

The space tree with variable tuple size formulation can be drawn and $c^{\wedge}(\cdot)$ and $u(\cdot)$ is computed (We have considered the same Knapsack problem which is discussed in section 7.5.1).

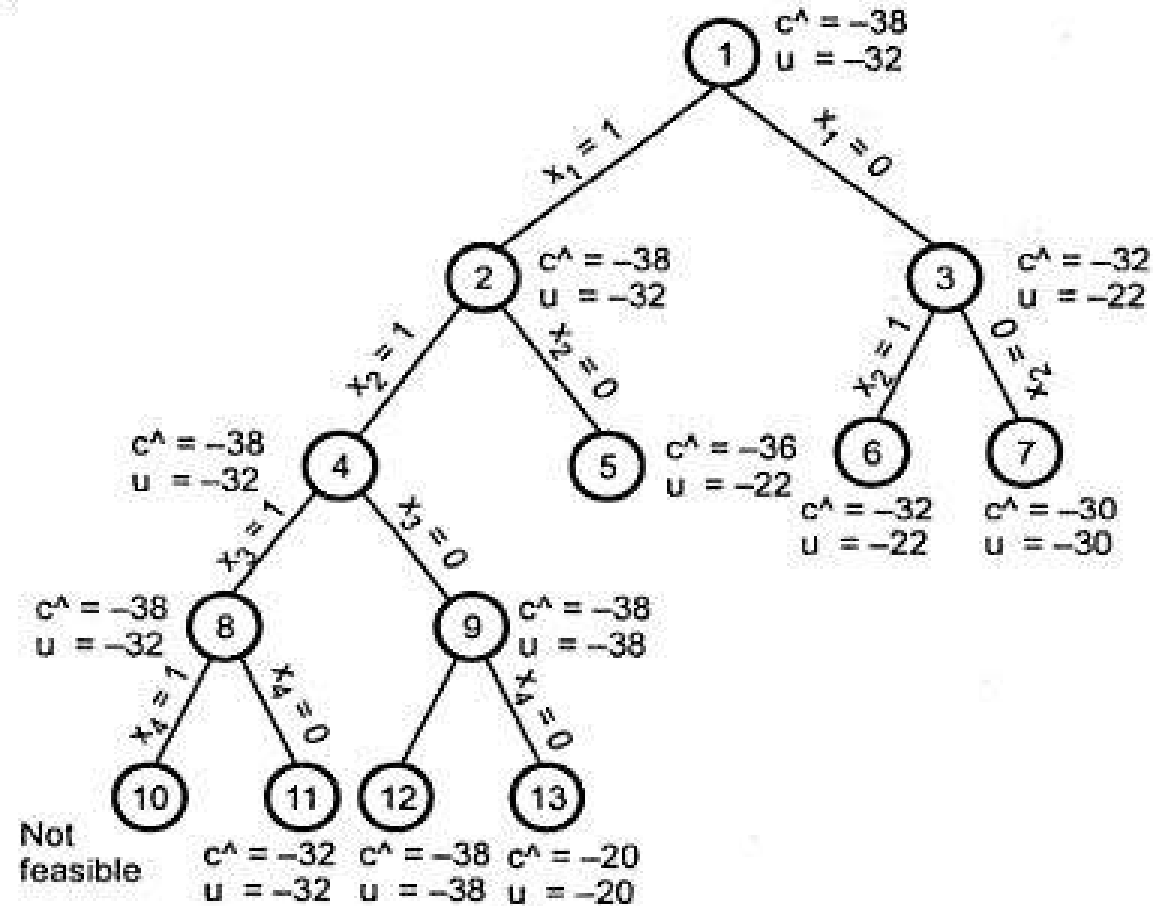


Fig. 7.7 FIFOBB space tree

Initially upper = $u(1) = -32$. Then children of node 1 are generated. Node 2 becomes E-node and hence children 4 and 5 are generated. Node 4 and 5 are added in the list of live nodes. Next, node 3 becomes E-node and children 6 and 7 are generated. As $c^7 > \text{upper}$ we will kill node 7. Hence node 6 will be added in the list of live nodes. Node 4 is E-node and children 8 and 9 are generated. The upper is updated and it is now upper = $u(9) = -38$. Nodes 8 and 9 are added in the list of live nodes. Node 5 and 6 becomes the next E-node but as $c^5 > \text{upper}$ and $c^6 > \text{upper}$, kill nodes 5 and 6. Node 8 becomes next E-node and children 10 and 11 are generated. As node 10 is infeasible do not consider it. $c^{11} > \text{upper}$. Hence kill node 11. Node 9 becomes next E-node and upper = -38 . Children 12 and 13 are generated. But $c^{13} > \text{upper}$. So kill node 13. Finally node 12 becomes an answer node. Therefore solution is $x_1 = 1, x_2 = 1, x_3 = 0$ and $x_4 = 1$.

FIFO branch and bound solution for knapsack problem is derived as follows :

1. Derive state space tree.
2. Compute lower bound : $hatc(.)$ and upper bound : $u(.)$ for each node.
3. If lower bound is greater than upper bound than kill that node.
4. Else, both the children of siblings are inserted in list and most promising node is selected as new E node.
5. Repeat step 3 and 4 until all nodes are examined.
6. The node with minimum lower bound value $hatc(.)$ is the answer node. Trace the path from leaf to root in the backward direction to find the solution tuple.

LC branch and bound solution for knapsack problem is derived as follows :

1. Derive state space tree.
2. Compute lower bound : $\text{hatc}(\cdot)$ and upper bound : $u(\cdot)$ for each node.
3. If lower bound is greater than upper bound than kill that node.
4. Else select node with minimum lower bound as E-node.
5. Repeat step 3 and 4 until all nodes are examined.
6. The node with minimum lower bound value $\text{hatc}(\cdot)$ is the answer node. Trace the path from leaf to root in the backward direction to find the solution tuple.

Sample Questions:

1. In Branch and Bound, 0/1 Knapsack problem is a minimization problem. Explain?
2. With suitable example explain LIFOBB
3. Solve 0/1 Knapsack problem using FIFOBB

- As discussed earlier, the goal of knapsack problem is to maximize

$$\sum_i p_i x_i$$

given the constraints

$$\sum_i w_i x_i \leq M$$

, where M is the size of the knapsack. A maximization problem can be converted to a minimization problem by negating the value of the objective function.

- The modified knapsack problem is stated as,

minimize

$$- \sum_i p_i x_i$$

subjected to

$$\sum_i w_i x_i \leq M$$

Where,

$$x_i \in \{0, 1\} \text{ for } 1 \leq i \leq n$$

- Node satisfying the constraint

$$\sum_i w_i x_i \leq M$$

in state space tree is called the answer state, the remaining nodes are infeasible.

LC branch and bound solution for knapsack problem is derived as follows :

1. Derive state space tree.
2. Compute lower bound : $hatc(.)$ and upper bound : $u(.)$ for each node.
3. If lower bound is greater than upper bound than kill that node.
4. Else select node with minimum lower bound as E-node.
5. Repeat step 3 and 4 until all nodes are examined.
6. The node with minimum lower bound value $hatc(.)$ is the answer node. Trace the path from leaf to root in the backward direction to find the solution tuple.

FIFO branch and bound solution for knapsack problem is derived as follows :

1. Derive state space tree.
2. Compute lower bound : $hatc(.)$ and upper bound : $u(.)$ for each node.
3. If lower bound is greater than upper bound than kill that node.
4. Else, both the children of siblings are inserted in list and most promising node is selected as new E node.
5. Repeat step 3 and 4 until all nodes are examined.
6. The node with minimum lower bound value $hatc(.)$ is the answer node. Trace the path from leaf to root in the backward direction to find the solution tuple.

$u(1) =$

$-\sum p_i$

such that

$\sum w_i \leq M$

$$\hat{u}(1) = u(1) - \frac{M - \text{Weight of selected items}}{\text{Weight of remaining items}} * \text{Profit of remaining item}$$

The bounding function is a heuristic computation. For the same problem, there may be different bounding functions. Apart from the above-discussed bounding function, another very popular **bounding function for knapsack is,**

$$ub = v + (W - w) * (v_{i+1} / w_{i+1})$$

where,

v is value/profit associated with selected items from the first i items.

W is the capacity of the knapsack.

w is the weight of selected items from first i items

THANK YOU

Travelling Salesman Problem

TRAVELLING SALESMAN PROBLEM

- Let $G = (V, E)$ be a directed graph.
- **The problem is to find a tour of minimum cost** and we assume that every tour starts and ends at vertex 1.
- So, the solution space S is given by $S = \{(1, r, 1) \mid r \text{ is a permutation of } (2, 3, \dots, n)\}$.
- Therefore $S = (n - 1)!$

$$c(A) = \begin{cases} \text{length of tour defined by the path from the root to } A, & \text{if } A \text{ is a leaf} \\ \text{cost of a minimum-cost leaf in the subtree } A, & \text{if } A \text{ is not a leaf} \end{cases}$$

$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

(a) Cost matrix

$$\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

(b) Reduced cost
matrix

$$L = 25$$

Figure 8.11 An example

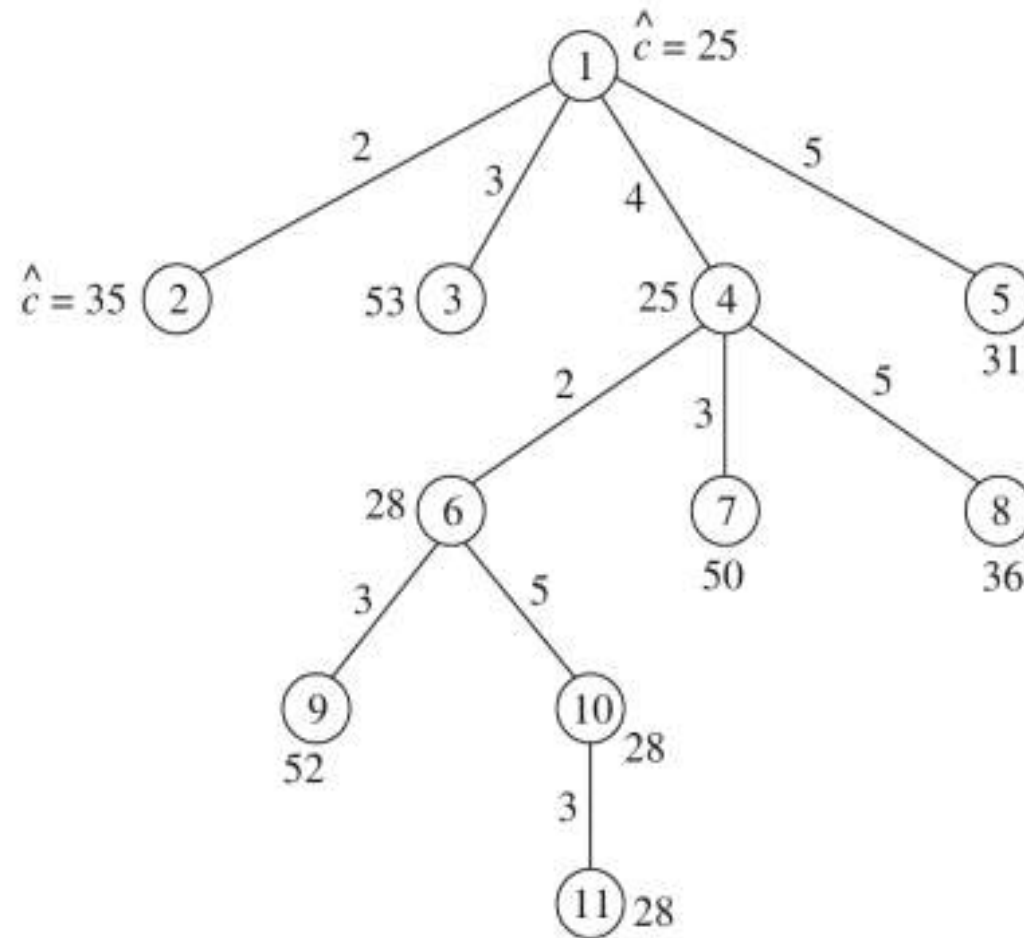


Figure 9.14 State space tree generated by LCBB procedure.

To compute $\hat{c}$ at node 2, we change the entries of first row and second column to ∞ , and $A(2, 1)$ is also set to ∞ [Figure 9.15(a)]. Since the matrix after changing the entries are found to be already reduced, no reduction is required. Therefore, $\hat{c}(2)$ can be computed as

$$\hat{c}(1) + A(1, 2)$$

i.e. $25 + 10 = 35$

Similarly, $\hat{c}$ at node 3 can be computed by changing the entries of 1st row, 3rd column and $A(3, 1)$ to ∞ as shown in Figure 9.15(b). The matrix is reduced by subtracting 11 from the 1st column. Hence, $\hat{c}(3)$ is computed as

$$\hat{c}(1) + A(1, 3) + r$$

i.e., $25 + 17 + 11 = 53$

Similarly, $\hat{c}(4)$ and $\hat{c}(5)$ are computed to 25 and 31 respectively. The matrix and its corresponding reduced matrix are shown in Figure 9.15(c) and 9.15(d) respectively. Since $\hat{c}(4)$ is minimum, node 4 is considered as the next E -node as u remains unchanged.

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & 2 & 0 \\ 0 & \infty & \infty & 0 & 2 \\ 15 & \infty & 12 & \infty & 0 \\ 11 & \infty & 0 & 12 & \infty \end{bmatrix}$$

(a) Matrix at node 2, no reduction required

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 15 & 3 & \infty & \infty & 0 \\ 11 & 0 & \infty & 12 & \infty \end{bmatrix} \Rightarrow$$

(b) Matrix at node 3 after changing the entries

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 4 & 3 & \infty & \infty & 0 \\ 0 & 0 & \infty & 12 & \infty \end{bmatrix}$$

Reduced matrix at node 3, $r = 11$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix}$$

(c) Matrix at node 4

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & 2 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 15 & 3 & 12 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix} \Rightarrow$$

(d) Matrix at node 5

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 10 & \infty & 9 & 0 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 12 & 0 & 9 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix}$$

Reduced matrix at node 5, $r = 5$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & \infty & 0 \\ 0 & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & 0 & \infty & \infty \end{bmatrix}$$

(e) Matrix at node 6

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & \infty & 0 \\ \infty & 3 & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & 0 & \infty & \infty & \infty \end{bmatrix}$$

(f) Matrix at node 7

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & \infty & 0 \\ \infty & 1 & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 0 & 0 & \infty & \infty & \infty \end{bmatrix}$$

Reduced matrix at
node 7, $r = 13$

Node 4 is expanded to nodes 6, 7 and 8 that correspond to the inclusion of edges (4, 2), (4, 3) and (4, 5) respectively. The matrix at node 6 can be obtained by changing the entries of row 1, column 4, row 4, column 2 and $A(2, 1)$ to ∞ . Since, the matrix is already reduced, no further reduction is required. Hence, $\hat{c}(6)$ can be computed as

$$\hat{c}(4) + A(4, 2)$$

i.e., $25 + 3 = 28$

The matrix at node 6 is shown in Figure 9.15(e).

Similarly, to compute $\hat{c}(7)$ the entries at row 1, column 4, row 4, column 3 and $A(3, 1)$ are changed to ∞ . Then, the matrix is reduced by subtracting 2 from row 3 and 11 from column 1. Hence, $\hat{c}(7)$ is computed to be

$$\hat{c}(4) + A(4, 3) + r = 25 + 12 + 13 = 50$$

Similarly, $\hat{c}(8)$ is computed to 36. The matrix at node 7 and 8 are shown in Figure 9.15(f) and 9.15(g) respectively. Currently, the live nodes are 2, 3, 5, 6, 7 and 8. Since node 6 has the minimum $\hat{c}$ value, it is considered as the next E -node. Node 6 is expanded to nodes 9 and 10 that correspond to edges (2, 3) and (2, 5) respectively.

$\hat{c}(9)$ is computed by changing the entries of row 1, column 4, row 4, column 2, row 2, column 3 and $A(3, 1)$ to ∞ . The matrix is reduced by subtracting 2 from row 3 and 11 from column 1. Hence, $\hat{c}(9)$ is computed as

$$\hat{c}(6) + A(2, 3) + 2 = 28 + 11 + 13 = 52$$

Similarly, $\hat{c}(10)$ is computed to 28. The matrices at nodes 9 and 10 are shown in Figure 9.15(h) and 9.15(i) respectively.

Now, the live nodes are 2, 3, 5, 7, 8, 9 and 10. Since $\hat{c}(10)$ is the minimum node, 10 is considered as the next E -node. The only node generated from node 10 is node 11 that corresponds to edge (5, 3). The cost matrix at node 11 is shown in Figure 9.15(j), where each entry is ∞ . $\hat{c}(11)$ is computed as

$$\hat{c}(10) + A(5, 3) = 28 + 0 = 28$$

Node 11 is the solution node. Hence, U is updated to 28. The node whose $\hat{c}$ is greater than U must be killed. Therefore, all the live nodes are killed, and node 11 is considered as the answer node. Hence, LCBB terminates with 1, 4, 2, 5, 3, 1 as the shortest length tour.

Sample Questions:

1. Explain steps involved in reduced cost matrix.
2. Adjacency matrix is given, Solve the following Travelling Salesman problem and find minimum cost of the tour.

	C0	C1	C2	C3	C4
C0	INF	20	30	10	11
C1	15	INF	16	4	2
C2	3	5	INF	2	4
C3	19	6	18	INF	3
C4	16	4	7	16	INF

TRAVELLING SALESMAN PROBLEM

- Let $G = (V, E)$ be a directed graph.
- **The problem is to find a tour of minimum cost** and we assume that every tour starts and ends at vertex 1.
- So, the solution space S is given by $S = \{(1, r, 1) \mid \text{where } r \text{ is a permutation of } (2, 3, \dots, n)\}$.
- Therefore $S = (n - 1)!$

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

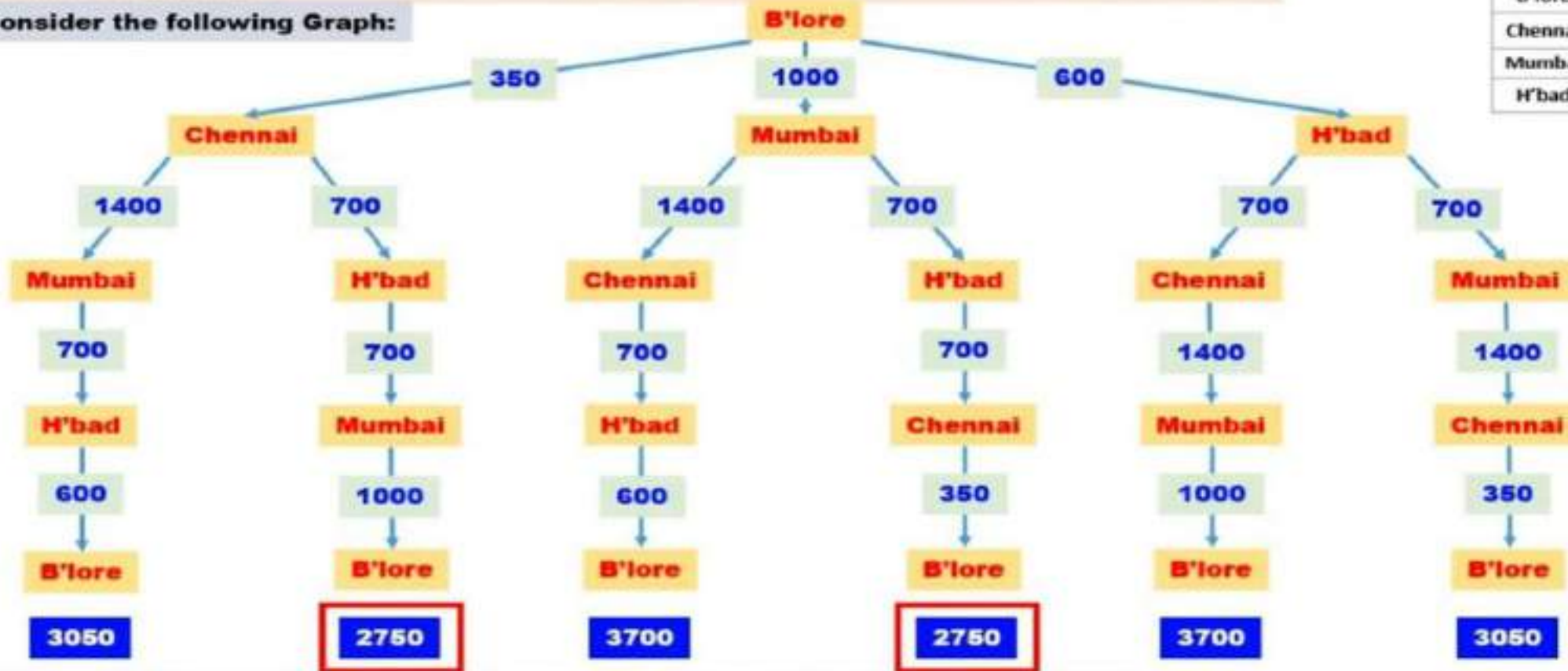
In Traveling Salesman Problem, there are n cities and a Traveling Salesman

The Traveling Salesman should start from a source city, travel through all the other cities only once and at last travel to the source city

Now, the Traveling Salesman should travel in such a manner that the total traveling cost or distance is minimum

Consider the following Graph:

	B'lore	Chennai	Mumbai	H'bad
B'lore	—	350	1000	600
Chennai	350	—	1400	700
Mumbai	1000	1400	—	700
H'bad	600	700	700	—



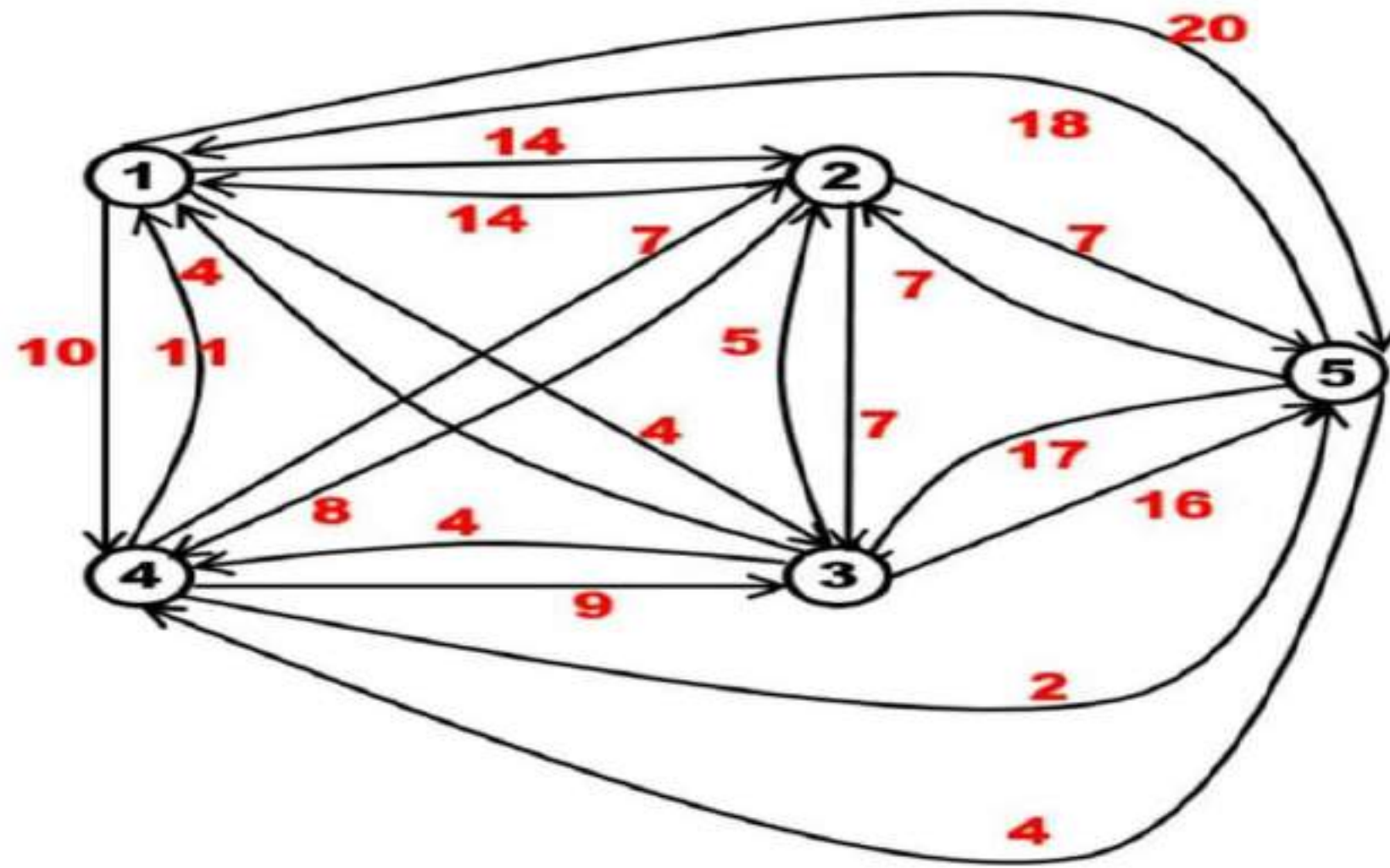
The Best Solution for the Traveling Salesman with Minimum Cost is as follows:

B'lore → Chennai → H'bad → Mumbai → B'lore = 350 + 700 + 700 + 1000 = 2750

B'lore → Mumbai → H'bad → Chennai → B'lore = 1000 + 700 + 700 + 350 = 2750

Using Least Cost Branch and Bound]

Consider the following Graph:



TRAVELING SALESMAN PROBLEM

The Cost Matrix for the Graph is as follows:

	1	2	3	4	5
1	∞	14	4	10	20
2	14	∞	7	8	7
3	4	5	∞	4	16
4	11	7	9	∞	2
5	18	7	17	4	∞

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

The First Step is to get a Reduced Matrix by Reducing the Cost Matrix

To Reduce the Cost Matrix, We have to do Row Reduction and then Column Reduction

To do Row Reduction, find the minimum number in each Row and this number is subtracted from all the numbers in that Row

	1	2	3	4	5	
1	∞	10	0	6	16	4
2	7	∞	0	1	0	7
3	0	1	∞	0	12	4
4	9	5	7	∞	0	2
5	14	3	13	0	∞	4

To do Column Reduction, find the minimum number in each Column and this number is subtracted from all the numbers in that Column

	1	2	3	4	5	
1	∞	9	0	6	16	
2	7	∞	0	1	0	
3	0	0	∞	0	12	
4	9	4	7	∞	0	
5	14	2	13	0	∞	
	0	1	0	0	0	

$$\text{Row Reduction Cost} = 4 + 7 + 4 + 2 + 4 = 21$$

$$\text{Column Reduction Cost} = 0 + 1 + 0 + 0 + 0 = 1$$

Cost Matrix

	1	2	3	4	5
1	∞	14	4	10	20
2	14	∞	7	8	7
3	4	5	∞	4	16
4	11	7	9	∞	2
5	18	7	17	4	∞

$$\begin{aligned}\text{Reduced Cost} &= \text{Row Reduction Cost} \\ &+ \text{Column Reduction Cost} \\ &= 21 + 1 = 22\end{aligned}$$

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]

The First Step is to get a Reduced Matrix by Reducing the Cost Matrix

To Reduce the Cost Matrix, We have to do Row Reduction and then Column Reduction

To do Row Reduction, find the minimum number in each row and this number is subtracted from all the numbers in that row

	1	2	3	4	5	
1	∞	10	0	6	16	4
2	7	∞	0	1	0	7
3	0	1	∞	0	12	4
4	9	5	7	∞	0	2
5	14	3	13	0	∞	4

To do Column Reduction, find the minimum number in each column and this number is subtracted from all the numbers in that column

	1	2	3	4	5	
1	∞	9	0	6	16	
2	7	∞	0	1	0	
3	0	0	∞	0	12	
4	9	4	7	∞	0	
5	14	2	13	0	∞	
	0	1	0	0	0	

This is the Reduced Matrix

Cost Matrix

	1	2	3	4	5
1	∞	14	4	10	20
2	14	∞	7	8	7
3	4	5	∞	4	16
4	11	7	9	∞	2
5	18	7	17	4	∞

$$\begin{aligned} \text{Reduced Cost} &= \text{Row Reduction Cost} \\ &+ \text{Column Reduction Cost} \\ &= 21 + 1 = 22 \end{aligned}$$

$$\begin{aligned} \text{Row Reduction Cost} &= \\ 4 + 7 + 4 + 2 + 4 &= 21 \end{aligned}$$

$$\text{Column Reduction Cost} = 0 + 1 + 0 + 0 + 0 = 1$$

	1	2	3	4	5
1	∞	9	0	6	16
2	7	∞	0	1	0
3	0	0	∞	0	12
4	9	4	7	∞	0
5	14	2	13	0	∞

Node 1: Reduced Cost is 22

	1	2	3	4	5
1	∞	9	0	6	16
2	7	∞	0	1	0
3	0	0	∞	0	12
4	9	4	7	∞	0

Let us Consider the Reduced Matrix

We have the Cost Matrix value 22 for this Reduced Matrix

We Compute the Reduced Cost for each Node [corresponding to an edge (i, j)] in the State Space Tree as follows:

1: Change all entries in Row i and Column j to ∞ to avoid any edges leaving i or entering j

2. Set (j,1) to ∞ to prevent the use of edge (j, 1)

3. Then Reduce all rows and columns in the resulting Matrix except for the Rows and

This is the Reduced Matrix for Node 12 [For edge (1, 2)]

This is the Reduced Matrix for Node 13 [For edge (1, 3)]

This is the Reduced Matrix for Node 14 [For edge (1, 4)]

This is the Reduced Matrix for Node 15 [For edge (1, 5)]

Node 12: For edge (1, 2)

Node 13: For edge (1, 3)

Node 14: For edge (1, 4)

Node 15: For edge (1, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	1	0
3	0	∞	∞	0	12
4	9	∞	7	∞	0
5	14	∞	13	0	∞

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	12	0	11	∞	∞

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	1	∞
3	0	0	∞	0	∞
4	5	0	3	∞	∞
5	∞	2	13	0	∞

Reduced Cost at Node 12 (1, 2)
= 0 + 9 + 22 = 31

Reduced Cost at Node 13 (1, 3)
= 7 + 0 + 22 = 29

Reduced Cost at Node 14 (1, 4)
= 2 + 6 + 22 = 30

Reduced Cost at Node 15 (1, 5)
= 4 + 16 + 22 = 42

Columns containing only ∞

5	14	2	13	0	∞
---	----	---	----	---	----------

4. The Reduced Cost for the Node corresponding to edge (i, j) = Reduced Cost at Node j + Cost of [i, j] + Reduced Cost at Node i

Node 1: Reduced Cost is 22

Node 12: For edge (1, 2)

Node 13: For edge (1, 3)

Node 14: For edge (1, 4)

Node 15: For edge (1, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	1	0
3	0	∞	∞	0	12
4	9	∞	7	∞	0
5	14	∞	13	0	∞

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	12	0	11	∞	∞

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	1	∞
3	0	0	∞	0	∞
4	5	0	3	∞	∞
5	∞	2	13	0	∞

Reduced Cost at Node 12 (1, 2)
= 0 + 9 + 22 = 31

Reduced Cost at Node 13 (1, 3)
= 7 + 0 + 22 = 29

Reduced Cost at Node 14 (1, 4)
= 2 + 6 + 22 = 30

Reduced Cost at Node 15 (1, 5)
= 4 + 16 + 22 = 42

**Reduced Matrix
for Node 13 (1,3)**



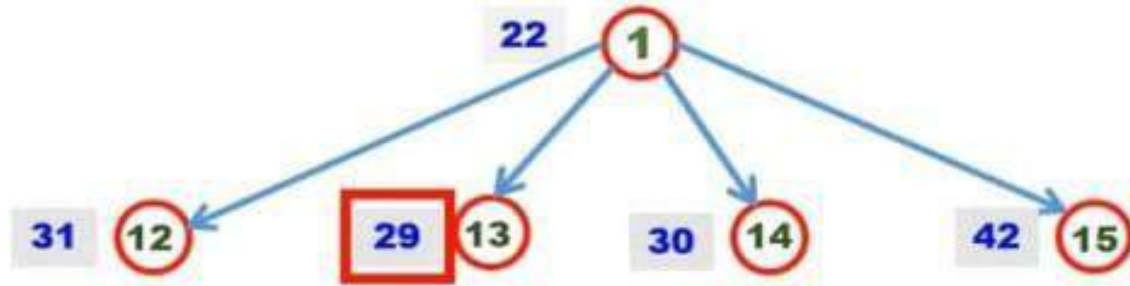
	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost = 29

Node with

have to explore

Now, We have the Partial Space Tree as follows:



Reduced Matrix
for Node 13 (1,3)



	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	7	2	∞	0	∞

Reduced Cost = 29

In Least Cost Branch and Bound, we have to explore and branch the Node with Least Cost among all the Leaf Nodes

Now, the Least Cost among 31, 29, 30 and 42 is 29. Therefore, we have to explore the Node 13 [edge (1, 3)]

Node 132: For edge (3, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	1	0
3	∞	∞	∞	∞	∞
4	0	∞	∞	∞	0
5	5	∞	∞	0	∞

Reduced Cost at Node 132 (3, 2)
= 2 + 0 + 29 = 31

Node 134: For edge (3, 4)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	0
3	∞	∞	∞	∞	∞
4	∞	4	∞	∞	0
5	5	0	∞	∞	∞

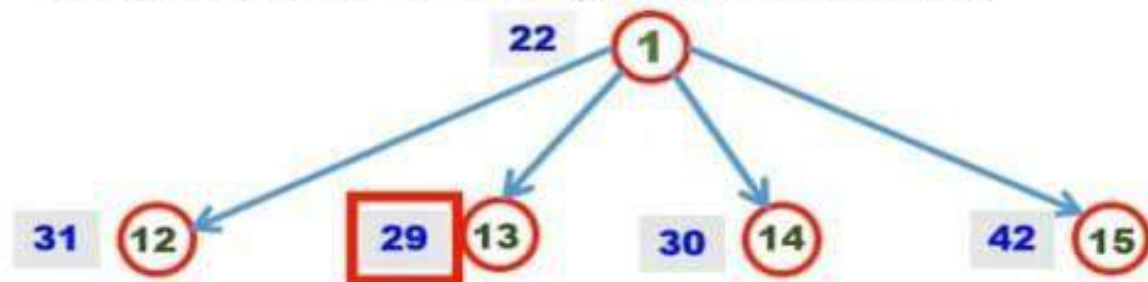
Reduced Cost at Node 134 (3, 4)
= 2 + 0 + 29 = 31

Node 135: For edge (3, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	∞
3	∞	∞	∞	∞	∞
4	0	0	∞	∞	∞
5	∞	0	∞	0	∞

Reduced Cost at Node 135 (3, 5)
= 4 + 12 + 29 = 45

Now, We have the Partial Space Tree as follows:



Reduced Matrix
for Node 13 (1,3) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	0
3	∞	0	∞	0	12
4	2	4	∞	∞	0
5	∞	0	∞	∞	∞

In L...
Lea...
Now...
the...
we have...
des...
9, 30 and...
to expl...

This is the Reduced Matrix
for Node 132
For edge (3, 2)

This is the Reduced Matrix
for Node 134
For edge (3, 4)

This is the Reduced Matrix
for Node 135
For edge (3, 5)

Cost = 29

Node 132: For edge (3, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	1	0
3	∞	∞	∞	∞	∞
4	0	∞	∞	∞	0
5	5	∞	∞	0	∞

Reduced Cost at Node 132 (3, 2)
= 2 + 0 + 29 = 31

Node 134: For edge (3, 4)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	0
3	∞	∞	∞	∞	∞
4	∞	4	∞	∞	0
5	5	0	∞	∞	∞

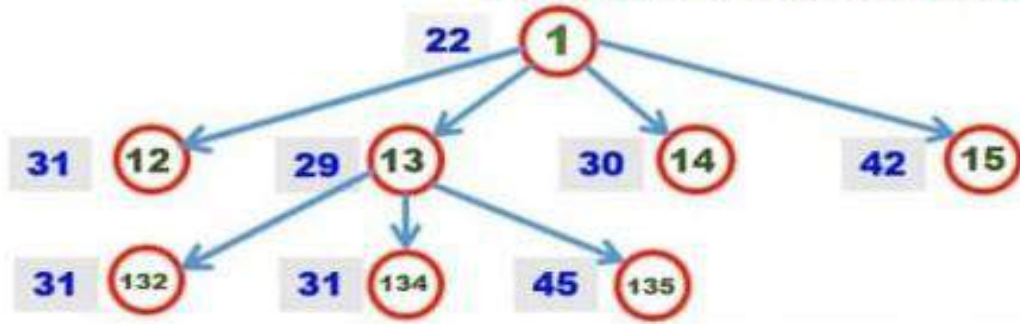
Reduced Cost at Node 134 (3, 4)
= 2 + 0 + 29 = 31

Node 135: For edge (3, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	1	∞
3	∞	∞	∞	∞	∞
4	0	0	∞	∞	∞
5	∞	0	∞	0	∞

Reduced Cost at Node 135 (3, 5)
= 4 + 12 + 29 = 45

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



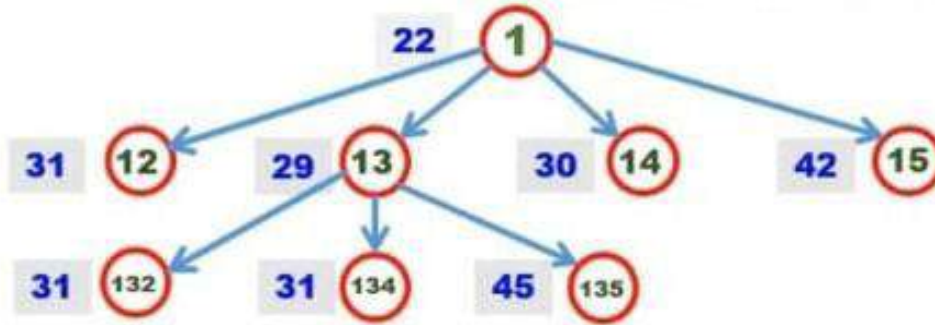
Reduced Matrix
for Node 14 (1, 4) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	12	0	11	∞	∞

Reduced Cost = 30

Among all the Leaf Node Values, find the Least Cost Node to explore and branch
Among the Leaf node Values 31, 31, 31, 45, 30 and 42, 30 is the Least Cost.
Therefore, we explore and branch from Node 14

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Reduced Matrix
for Node 14 (1, 4) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	12	0	11	∞	∞

Among all the Leaf Node Values, find the Least Cost Node to explore and branch
Among the Leaf node Values 31, 31, 31, 45, 30 and 42, 30 is the Least Cost.
Therefore, we explore and branch from Node 14

Reduced Cost = 30

Node 142: For edge (4, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	0
3	0	∞	∞	∞	12
4	∞	∞	∞	∞	∞
5	1	∞	0	∞	∞

Reduced Cost at Node 142 (4, 2)
 $= 11 + 4 + 30 = 45$

Node 143: For edge (4, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	0
3	∞	0	∞	∞	12
4	∞	∞	∞	∞	∞
5	5	0	∞	∞	∞

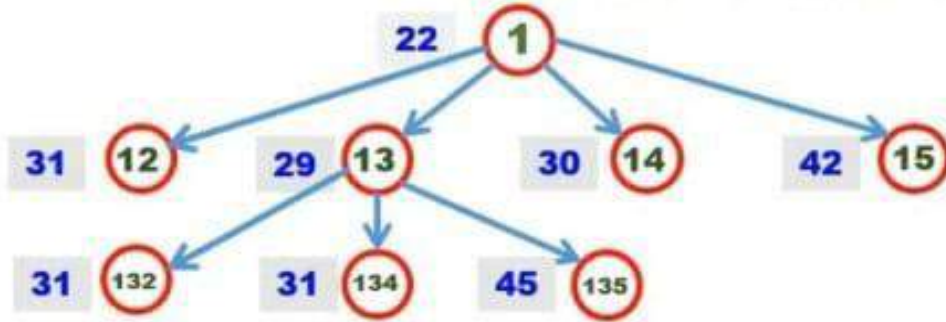
Reduced Cost at Node 143 (4, 3)
 $= 7 + 7 + 30 = 44$

Node 145: For edge (4, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	∞
3	0	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	11	∞	∞

Reduced Cost at Node 145 (4, 5)
 $= 0 + 0 + 30 = 30$

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Reduced Matrix
for Node 14 (1, 4) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	0
3	0	0	∞	∞	12
4	∞	4	7	∞	0
5	∞	∞	11	∞	∞

This is the Reduced Matrix
for Node 142
For edge (4, 2)

Node 142: For edge (4, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	0
3	0	∞	∞	∞	12
4	∞	∞	∞	∞	∞
5	1	∞	0	∞	∞

Reduced Cost at Node 142 (4, 2)
= 11 + 4 + 30 = 45

This is the Reduced Matrix
for Node 143
For edge (4, 3)

Node 143: For edge (4, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	0
3	∞	0	∞	∞	12
4	∞	∞	∞	∞	∞
5	5	0	∞	∞	∞

Reduced Cost at Node 143 (4, 3)
= 7 + 7 + 30 = 44

This is the Reduced Matrix
for Node 145
For edge (4, 5)

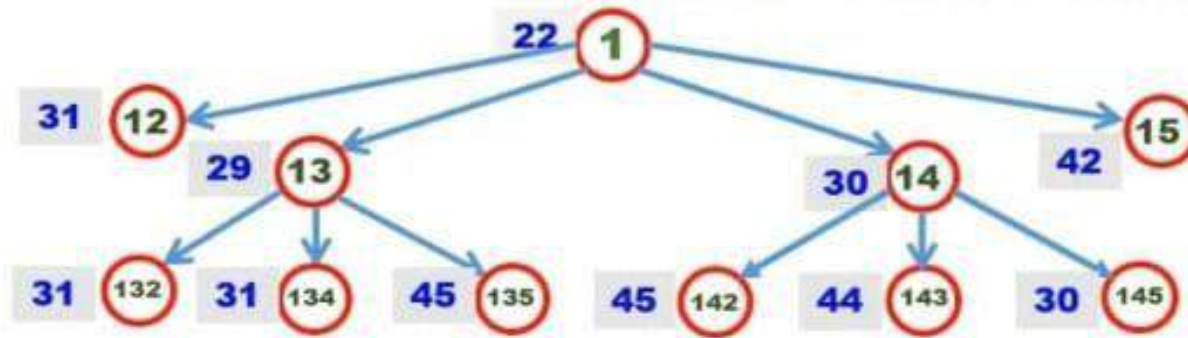
Node 145: For edge (4, 5)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	∞
3	0	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	11	∞	∞

Reduced Cost at Node 145 (4, 5)
= 0 + 0 + 30 = 30

Reduced Cost = 30

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Reduced Matrix
for Node 145
(4, 5)

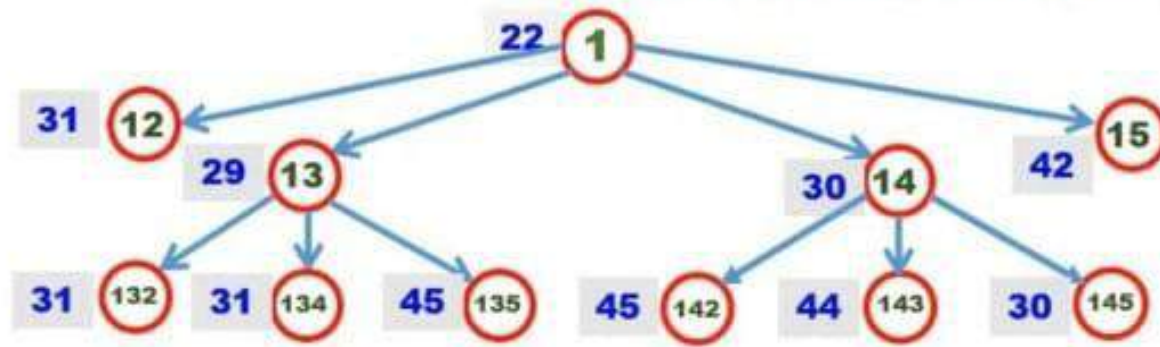
	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	∞
3	0	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	11	∞	∞

Reduced Cost = 30

Among all the Leaf Node Values, find the Least Cost Node to explore and branch

Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30 and 42, 30 is the Least Cost.

Therefore, we explore and branch from Node 145



Reduced Matrix
for Node 145
(4, 5) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	∞
3	0	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	11	∞	∞

Reduced Cost = 30

Among all the Leaf Node Values, find the Least Cost.
Among the Leaf node Values 31, 31, 31, 45, 45, 44
Therefore, we explore and branch from Node 145

Now, We shall
Reduce this Matrix

Node 1452: For edge (5, 2)

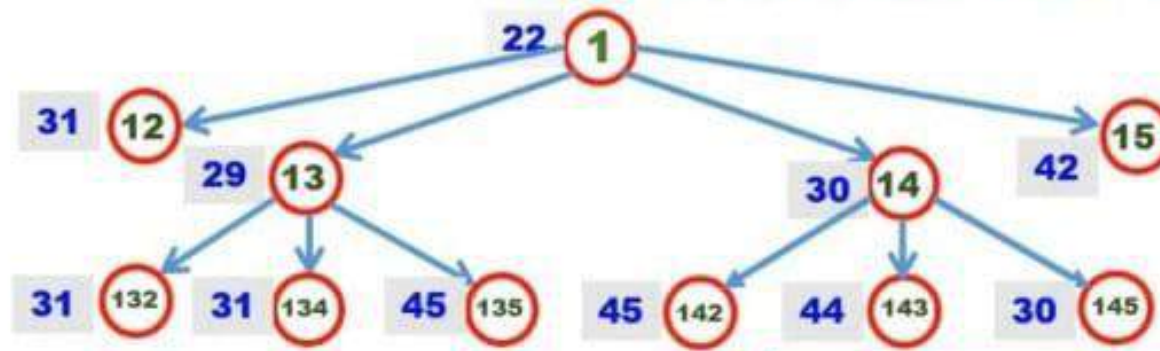
	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost at Node 1452 (5, 2)
= 0 + 0 + 30 = 30

Node 1453: For edge (5, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	∞
3	∞	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost at Node 1453 (5, 3)
= 7 + 11 + 30 = 48



Reduced Matrix
for Node 145
(4, 5) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	7	∞	0	∞	∞
3	0	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	11	∞	∞

Reduced Cost = 30

This is the Reduced Matrix
for Node 1452
For edge (5, 2)

Node 1452: For edge (5, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost at Node 1452 (5, 2)

$$= 0 + 0 + 30 = 30$$

This is the Reduced Matrix
for Node 1453
For edge (5, 3)

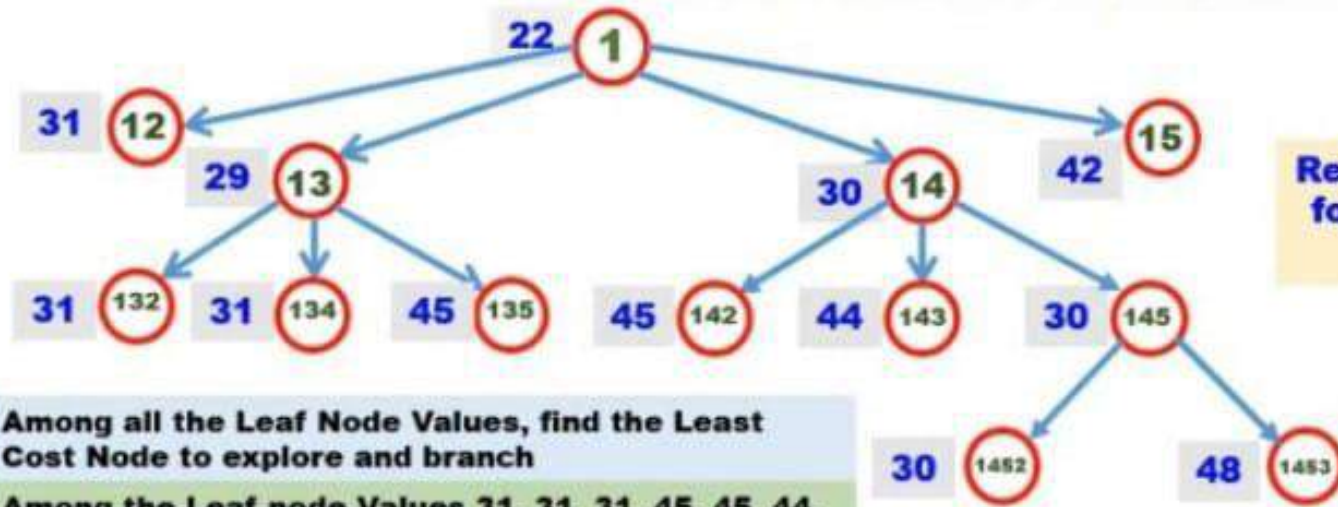
Node 1453: For edge (5, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	0	∞	∞	∞	∞
3	∞	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost at Node 1453 (5, 3)

$$= 7 + 11 + 30 = 48$$

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



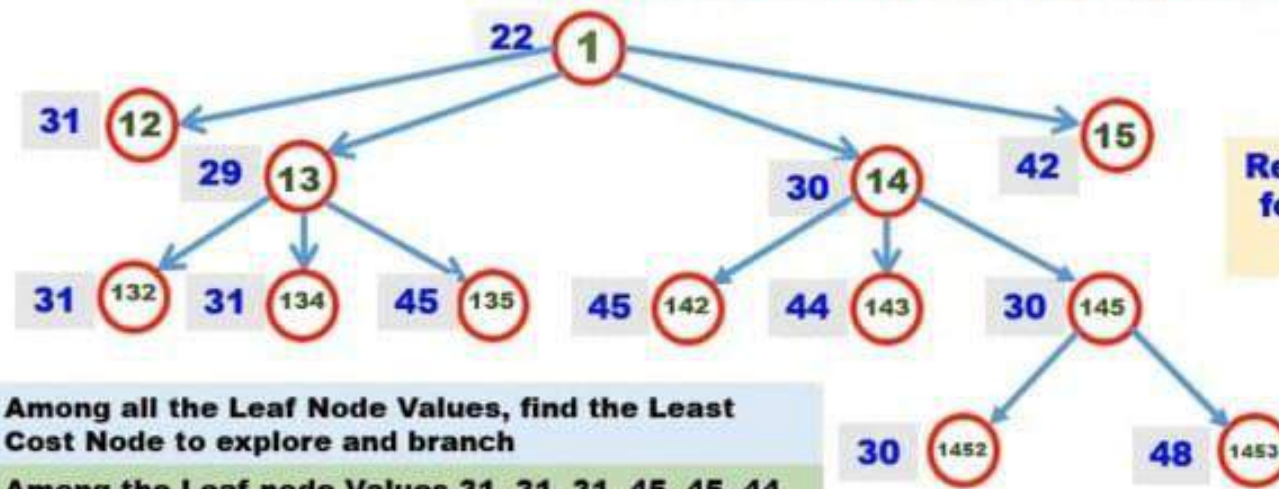
Reduced Matrix
for Node 1452
(5, 2) →

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Among all the Leaf Node Values, find the Least Cost Node to explore and branch

Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30, 48 and 42, 30 is the Least Cost.

Therefore, we explore and branch from Node 1452



Reduced Matrix
for Node 1452
(5, 2)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	0	∞	∞
3	0	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost = 30

Among all the Leaf Node Values, find the Least Cost Node to explore and branch

Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30, 48 and 42, 30 is the Least Cost.

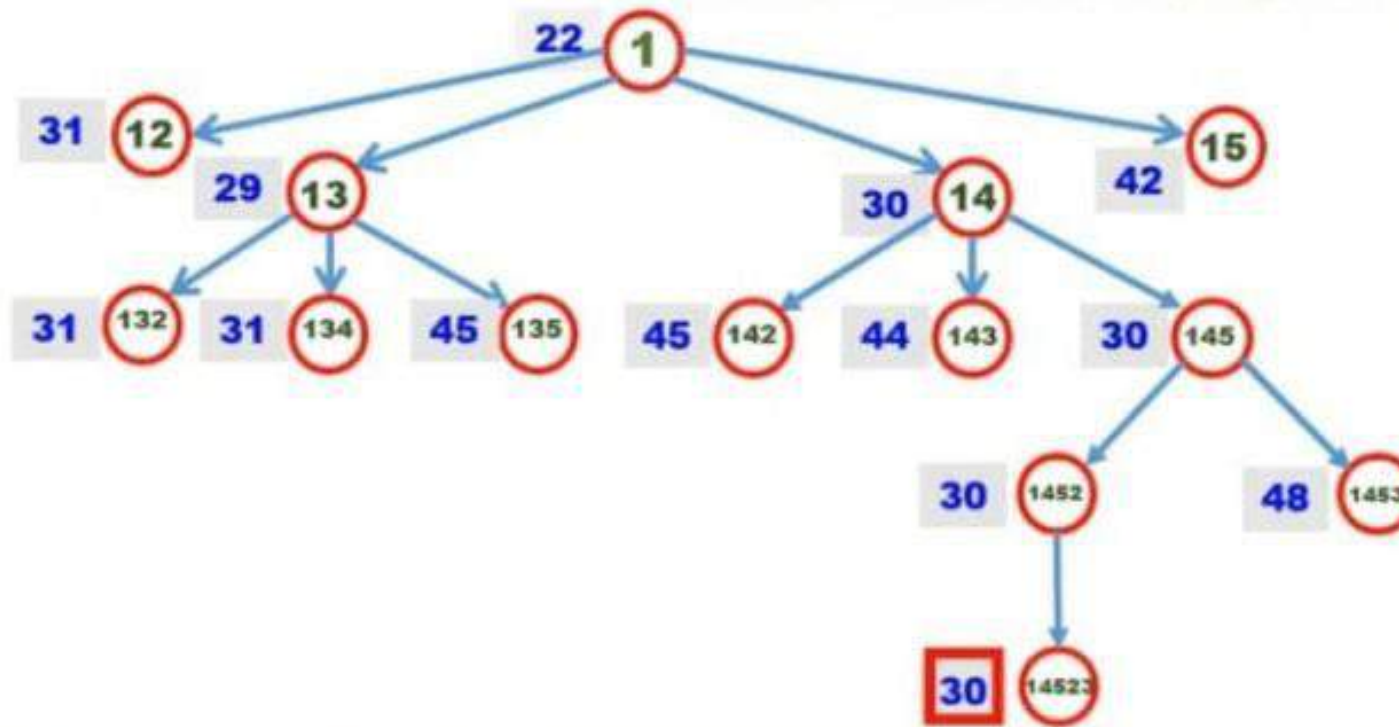
Therefore, we explore and branch from Node 1452

Node 14523: For edge (2, 3)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	∞	∞
3	∞	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

Reduced Cost at Node 14523 (2, 3) = 0 + 0 + 30 = 30

TRAVELING SALESMAN PROBLEM [using Least Cost Branch and Bound]



Among the Leaf node Values 31, 31, 31, 45, 45, 44, 30, 48 and 42, 30 is the Least Cost.

We have got the solution as 14523 and again back to 1. The Solution is as follows:

1 → 4 → 5 → 2 → 3 → 1

FINAL COST = 10 + 2 + 7 + 7 + 4 = 30

Reduction

Definition: Let L1 and L2 be two problems.

Problem L1 is reducible to L2 if and only if there is a way to solve L1 by a deterministic polynomial time algorithm using a deterministic algorithm that solves L2 in polynomial time.

Reducability between L1 and L2 is represented as:

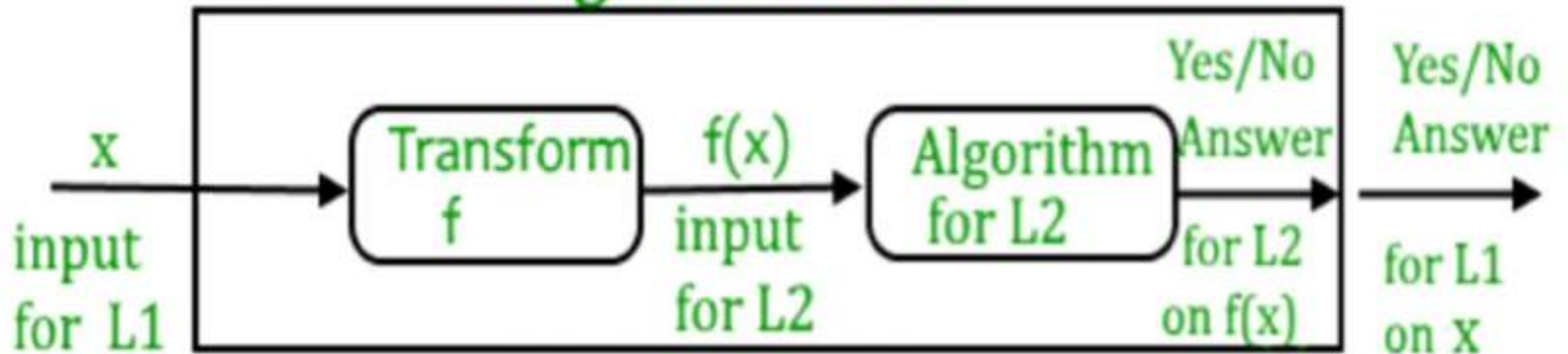
$$L1 \preceq L2$$

Example of Reduction

Let L_1 and L_2 be two decision problems. Suppose algorithm A_2 solves L_2 . That is, if y is an input for L_2 then algorithm A_2 will answer Yes or No depending upon whether y belongs to L_2 or not.

The idea is to find a transformation from L_1 to L_2 so that the algorithm A_2 can be part of an algorithm A_1 to solve L_1 .

Algorithm for L_1



NP-HARD

A problem X is NP-hard if every problem $Y \in NP$ reduces to X .

Example: Halting Problem

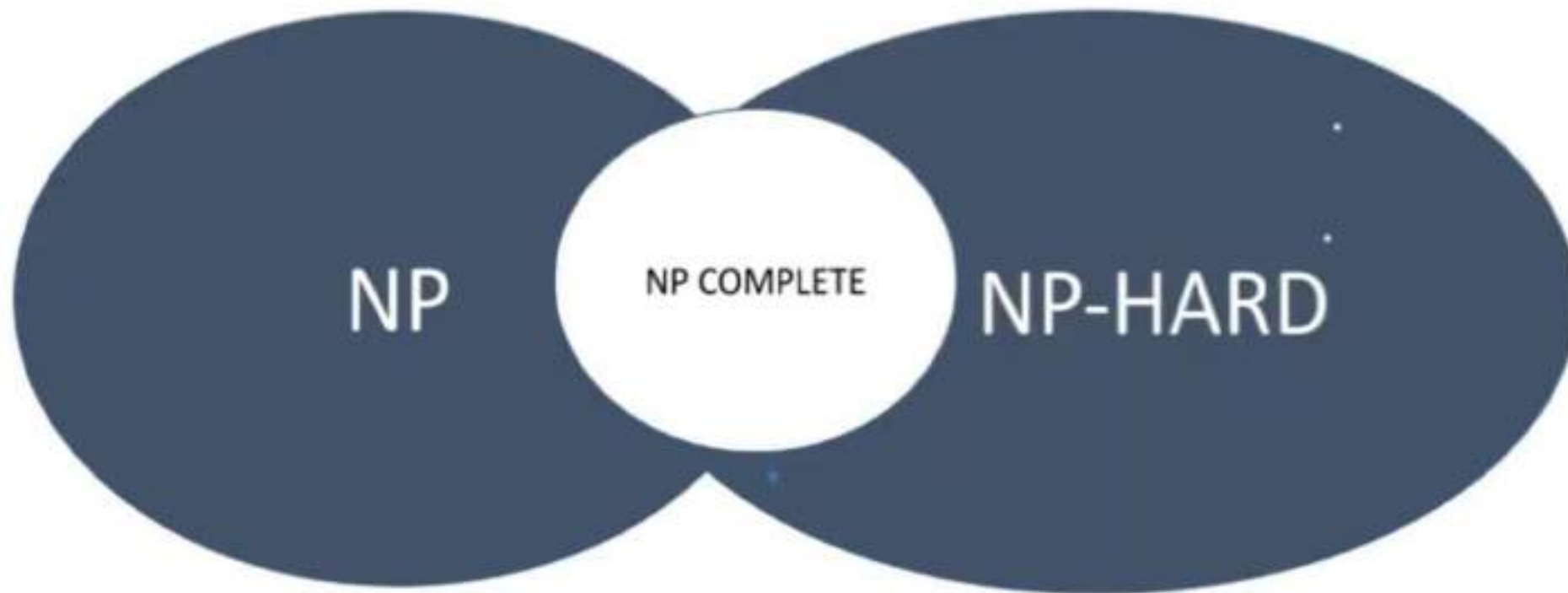
The problem is to determine for an arbitrary deterministic algorithm A and an input I whether algorithm A with input I ever terminates or enters an infinite loop.

Steps to show that a Problem is NP-hard

- 1. Pick a problem L1 already known to be NP-hard.**
- 2. Show that L1 is reducible to L2.**
- 3. Thus L2 is NP-hard.**

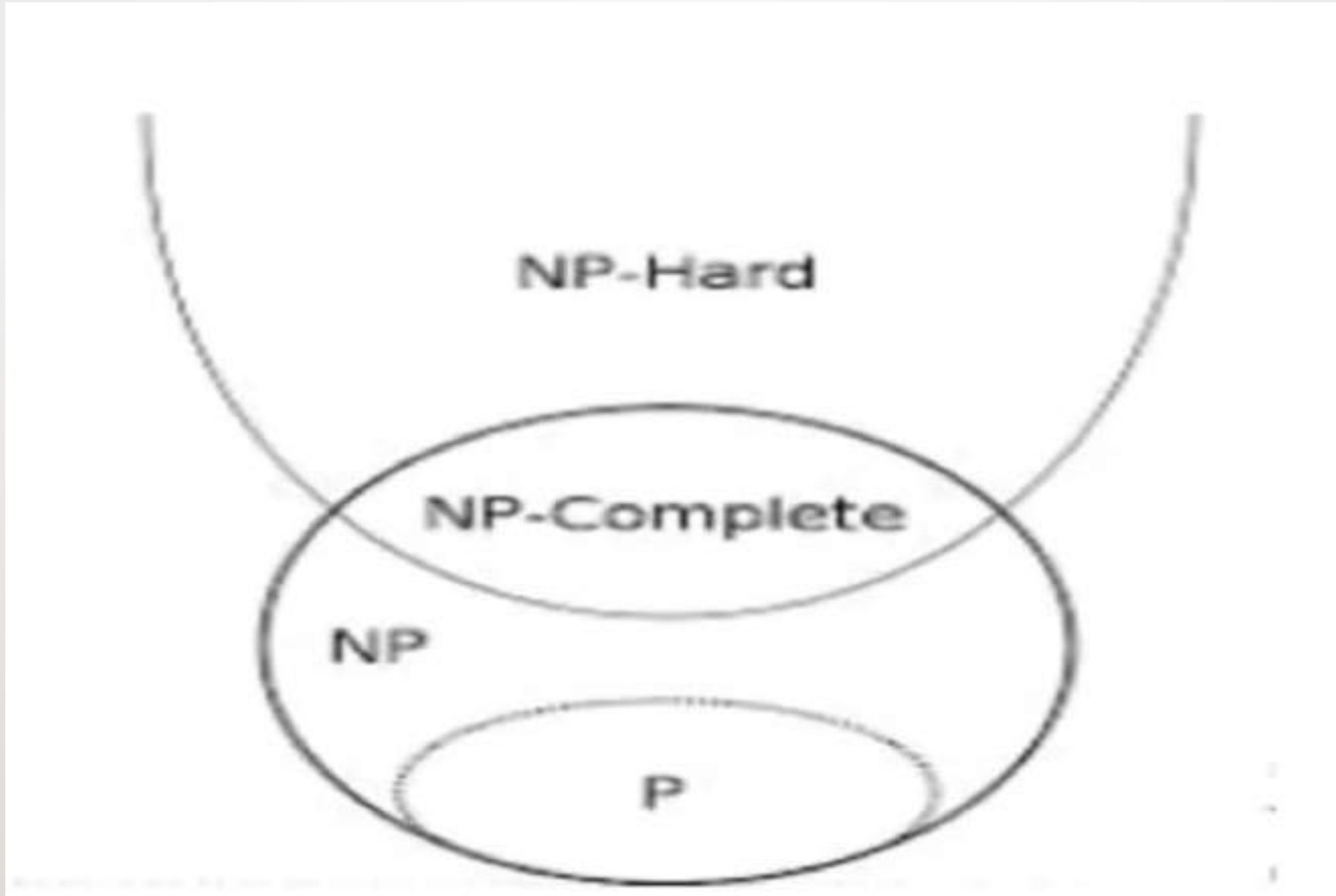
NP-Complete

Def: A problem that is both NP and NP-hard is known as NP-Complete problem.



NP-Complete

Example: SAT (Boolean satisfiability problem)
Knapsack Problem



BASIC CONCEPTS

The computing times of algorithms fall into two groups.

- **Group1**– Consists of problems whose solutions are bounded by the **polynomial of small degree**. Example – Binary search $O(\log n)$, sorting $O(n \log n)$, matrix multiplication $O(n^{2.81})$
- **Group2** – Contains problems whose best known algorithms are non polynomial.

Example –Traveling salesperson problem $O(n^2 2^n)$, knapsack problem $O(2^{n/2})$ etc.

Polynomial & Exponential Complexity

Polynomial Complexity: An algorithm A is of polynomial complexity if there exists a polynomial $p()$ such that the computing time of A is $O(p(n))$ for every input of size n.

$p(n) = (n^k)$ for some non-negative integer k.

Exponential Complexity: An algorithm A is of exponential complexity if complexity is of the form: $O(2^n)$

Eg. If n is equal to 1000, then notice 1000^3 is 1 billion where k is 3 and n is 1000, and 2^{1000} is simply huge.

Deterministic Algorithm

- The result of every operation is uniquely defined.
- The working of algorithm is known.
- Such algorithms agree with the way programs are executed on a computer.

Deterministic and Non-Deterministic Algorithms

Algorithms with the property that the result of every operation is **uniquely** defined are termed deterministic.

- Such algorithms agree with the way programs are executed on a computer.
- When the outcome is not uniquely defined but is limited to a specific set of possibilities, we call it non deterministic algorithm

Nondeterministic Algorithm

- The algorithm contains operations whose outcomes are not uniquely defined but are limited to specific set of possibilities.
- The machine executing such operations is allowed to choose any one of these outcomes subject to a termination condition to be defined for the problem.
- To specify these algorithms, three functions are introduced:
 1. Choice(s): arbitrarily chooses one of elements of s.
 2. Failure(): signals an unsuccessful completion.
 3. Success(): signals a successful completion.

A nondeterministic algorithm terminates unsuccessfully if and only if there exists no set of choices leading to success signal.

Nondeterministic Algorithm

Example: Consider the problem of searching for an element x in a given set of elements $A[1:n]$. A nondeterministic algorithm for this is as follows:

Sol:

$j := \text{Choice}(1, n)$

If $A[j] = x$, then {write (j); Success();}

Write(0); Failure();

EXAMPLE OF A DETERMINISTIC ALGORITHM

```
Algorithm Lsearch (A, n, x) {  
    for i:= 1 to n do {  
        if(a[i]=x) then  
            return i;  
    }  
    return 0;  
}
```


EXAMPLE OF A NON DETERMINISTIC ALGORITHM

```
Algorithm Search(A, n, x) {  
    j:= choice(1,n);  
    if(a[j]=x) then {  
        return j;  
        success();  
    }  
    return 0;  
    failure();  
}
```


DEFINITIONS

- **Decision problem**

Any problem whose answer is yes or no

- **Decision algorithm**

An algorithm for a decision problem is termed as a decision algorithm

- **Optimization problem**

Any problem that involves the identification of an optimal (either minimum or maximum) values of a given cost function is known as an optimization problem.

- **Optimization algorithm**

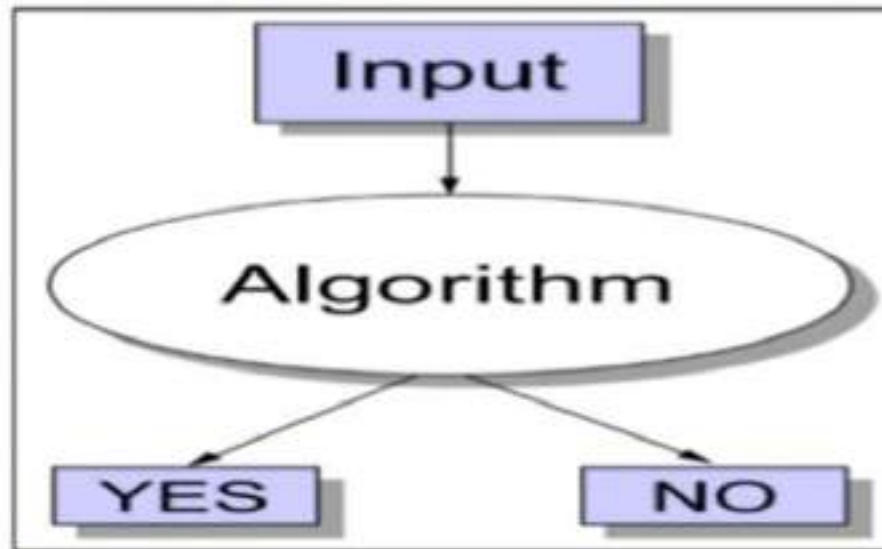
An optimization algorithm is used to solve an optimization problem.

Decision vs. Optimization Problems

- **Decision Problems**
 - Any problem for which the answer is either zero or one is called a decision problem.
 - An algorithm for a decision problem is termed a decision algorithm.
- **Optimization Problems**
 - Any problem that involves the identification of an optimal (either minimum or maximum) value of a given cost function is known as an optimization problem.
 - An optimization algorithm is used to solve an optimization problem.

Decision Problem: Any problem for which the answer is either yes or no.

Example: Deciding whether a given natural number is prime.



Classes P and NP

- **P** is the set of all decision problems solvable by deterministic algorithms in polynomial time.
- **NP** is the set of all decision problems solvable by nondeterministic algorithms in polynomial time.
- Since deterministic algorithms are just a special case of nondeterministic ones $P \subseteq NP$.
- Perhaps the most famous unsolved problem in computer science, is whether $P = NP$ or $P \neq NP$.
- Most computer scientists believe that $P \neq NP$, but we do not have a proof.

➤ **P** is the set of all decision problems solvable by a deterministic algorithm in polynomial time.

Sample Problems in P :

Fractional Knapsack, MST , Sorting.

➤ **NP** is the set of all decision problems solvable by a nondeterministic algorithm in polynomial time.

Sample Problems in NP :

Fractional Knapsack, MST , Sorting and Hamiltonian Cycle (Traveling Salesman), Graph Coloring

Class NP

Definition: Class NP is the set of all decision problems solvable by nondeterministic algorithms in polynomial time.

Example: To find out whether a salesman can visit multiple cities and come back to the home city in given amount of fuel..

Note: Class P is a subset of class NP. Any problem that can be solved by deterministic algorithm in polynomial time can also be solved by non deterministic algorithm in polynomial time.

Reduction

Definition: Let L1 and L2 be two problems.

Problem L1 is reducible to L2 if and only if there is a way to solve L1 by a deterministic polynomial time algorithm using a deterministic algorithm that solves L2 in polynomial time.

Reducability between L1 and L2 is represented as:

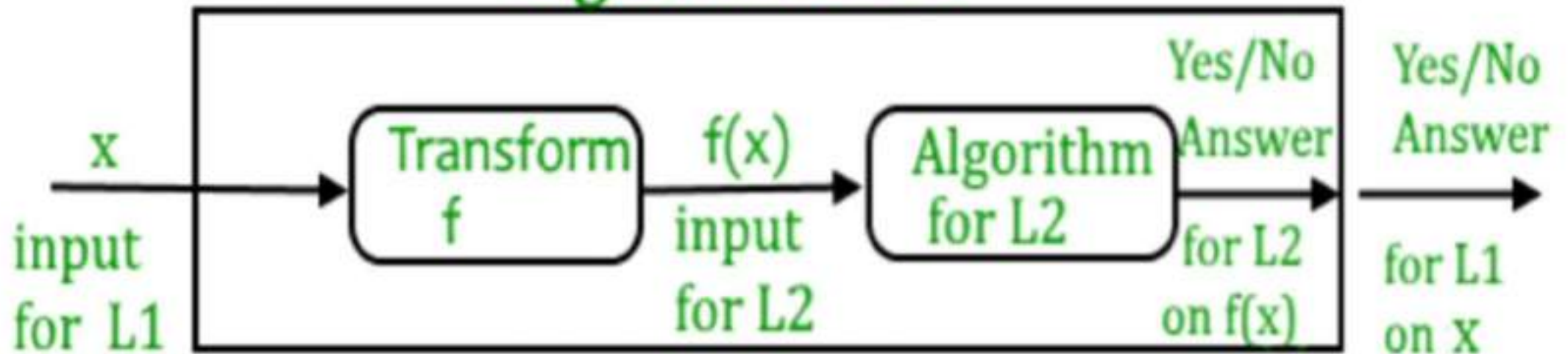
$$L1 \preceq L2$$

Example of Reduction

Let L_1 and L_2 be two decision problems. Suppose algorithm A_2 solves L_2 . That is, if y is an input for L_2 then algorithm A_2 will answer Yes or No depending upon whether y belongs to L_2 or not.

The idea is to find a transformation from L_1 to L_2 so that the algorithm A_2 can be part of an algorithm A_1 to solve L_1 .

Algorithm for L_1



NP-HARD

A problem X is NP-hard if every problem $Y \in NP$ reduces to X .

Example: Halting Problem

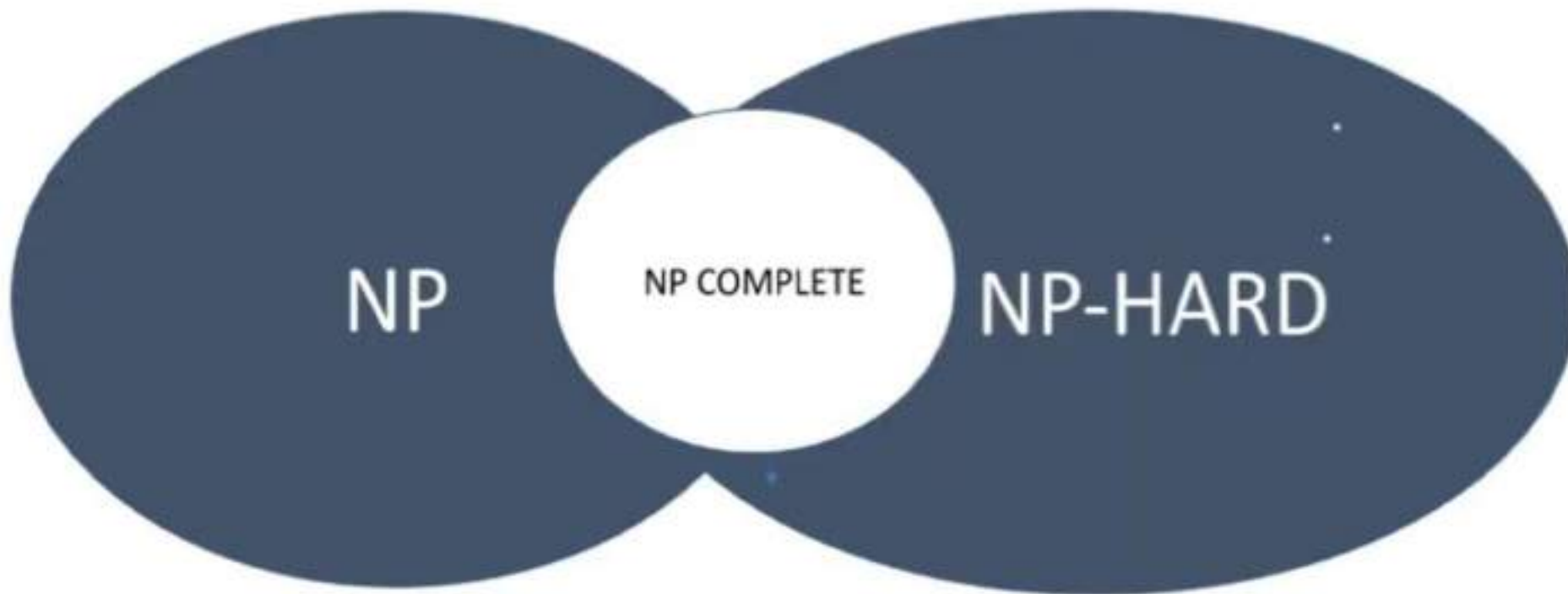
The problem is to determine for an arbitrary deterministic algorithm A and an input I whether algorithm A with input I ever terminates or enters an infinite loop.

Steps to show that a Problem is NP-hard

- 1. Pick a problem L1 already known to be NP-hard.**
- 2. Show that L1 is reducible to L2.**
- 3. Thus L2 is NP-hard.**

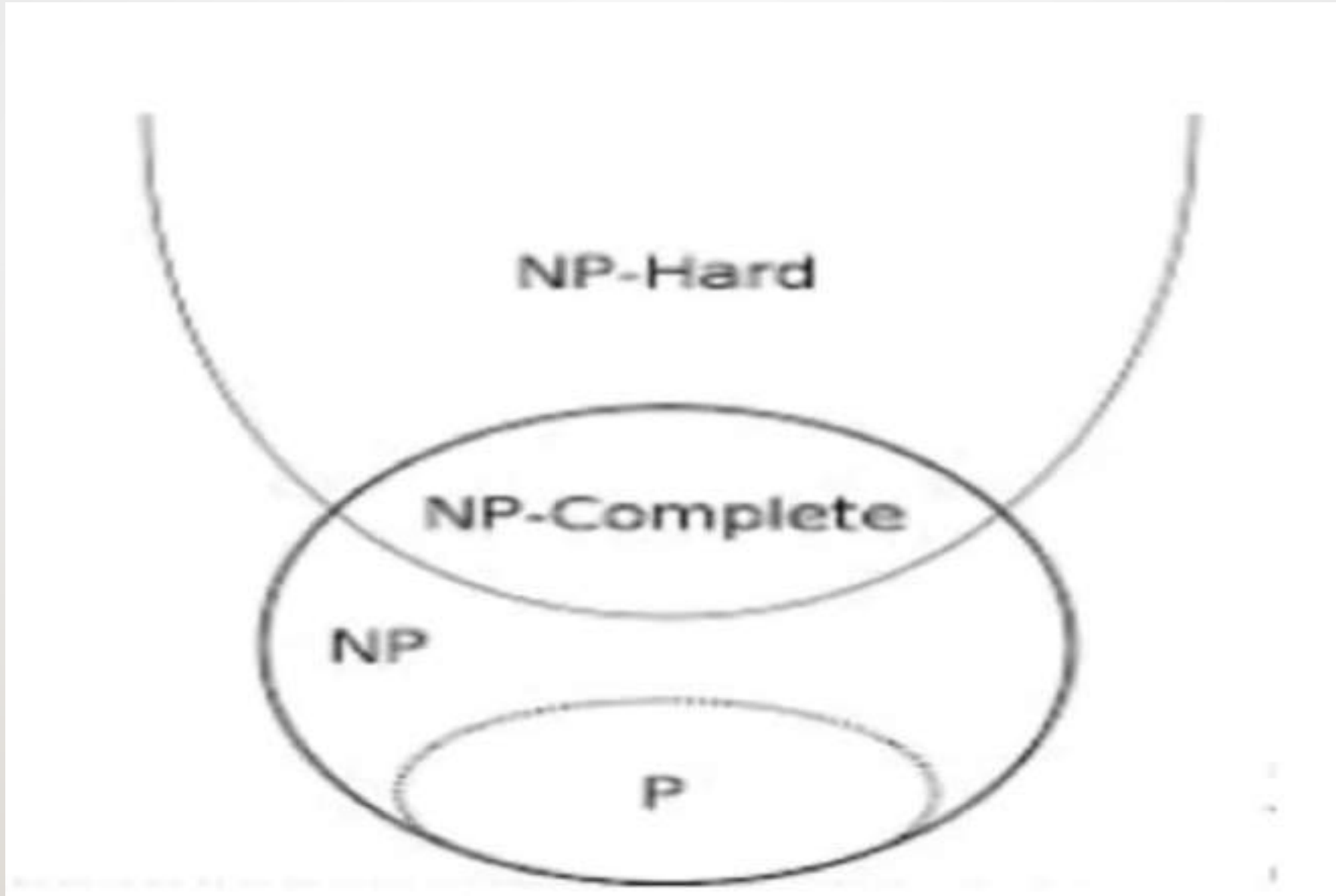
NP-Complete

Def: A problem that is both NP and NP-hard is known as NP-Complete problem.



NP-Complete

Example: SAT (Boolean satisfiability problem)
Knapsack Problem



SATISFIABILITY

- Let $x_1, x_2, x_3, \dots, x_n$ denotes Boolean variables.
- Let $\neg x_i$ denotes the negation of x_i .
- A literal is either a variable or its negation.
- A formula in the propositional calculus is an expression that can be constructed using literals and the operators $\wedge$ (AND) and $\vee$ (OR).

- A formula is in Conjunctive Normal Form (CNF) iff it is represented as $\bigwedge C_i$, where the C_i are clauses each represented as $\bigvee l_{ij}$.
- It is in Disjunctive Normal Form (DNF) iff it is represented as $\bigvee C_i$ and each clause is represented as $\bigwedge l_{ij}$
- The satisfiability problem is to determine if a formula is true for some assignment of truth values to the variables
- CNF-Satisfiability is the satisfiability problem for CNF formulas

Classes NP-hard and NP-complete

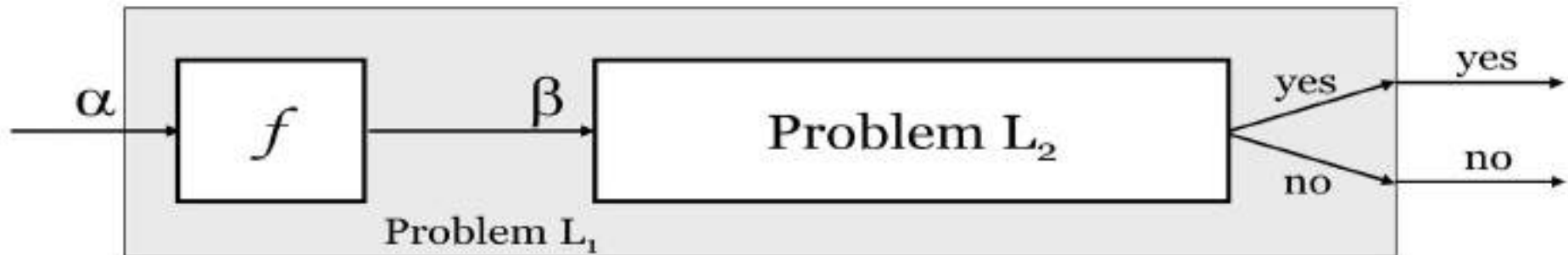
- Satisfiability Problem: Let $x_1, x_2 \dots$ denote boolean variables. A literal is either a variable or its negation. A formula in the propositional calculus is an expression that can be constructed using literals and the operations **and** and **or**.
 - Example: $(x_1 \wedge x_2) \vee (x_3 \wedge x_4)$
- The satisfiability problem is to determine whether a formula is true for some assignment of truth values to the variables.
- **Cook Theorem:** Satisfiability is in P if and only if $P = NP$. (See Horowitz for a proof)

Classes NP-hard and NP-complete

- A problem L is NP-hard if and only if satisfiability reduces to L ($\text{satisfiability} \leq L$).
- A problem L is NP-complete if and only if L is NP-hard and $L \in \text{NP}$.
- Another Definition:
 - A problem B is **NP-complete** if:
 - $B \in \text{NP}$
 - $A \leq_p B$ for all $A \in \text{NP}$

Reduction

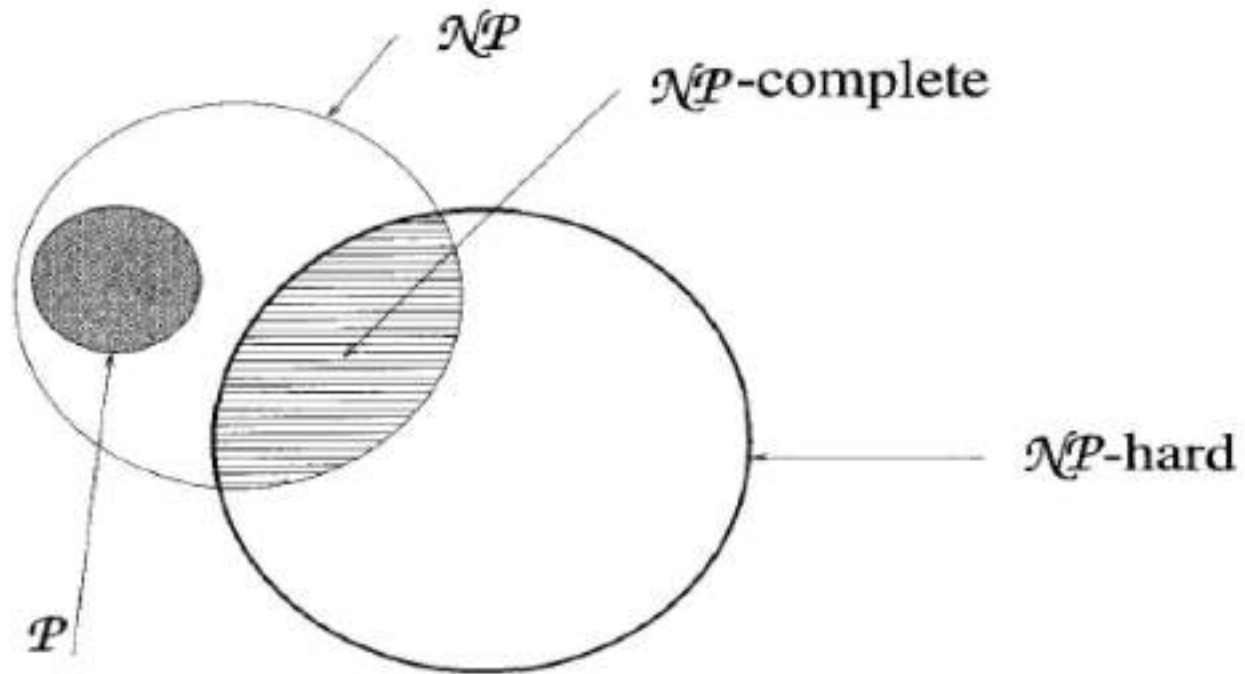
- Reduction is a way of saying that one problem is **easier** than another.
- Definition: Let L_1 and L_2 be problems. Problem L_1 reduces to L_2 (also written $L_1 \leq L_2$) if and only if there is a way to solve L_1 by a deterministic polynomial time algorithm using a deterministic algorithm that solves L_2 in polynomial time.
- **Idea:** transform the inputs of L_1 to inputs of L_2



Polynomial Reductions

- Given two problems L_1 and L_2 , we say that L_1 is polynomially **reducible** to L_2 ($L_1 \leq_p L_2$) if:
 - There exists a function f that converts the input of L_1 to inputs of L_2 in polynomial time
 - $L_1(i) = \text{YES} \Leftrightarrow L_2(f(i)) = \text{YES}$

Classes NP-hard and NP-complete



Classes NP-hard and NP-complete

- To show that a problem L_2 is NP-hard, it is adequate to show $L_1 \leq L_2$, where L_1 is some problem already known to be NP-hard.
 - Since $\leq$ is a transitive relation, it follows that if satisfiability $\leq L_1$ and $L_1 \leq L_2$, then satisfiability $\leq L_2$.
- To show that an NP-hard decision problem is NP-complete, we have just to exhibit a polynomial time nondeterministic algorithm for it.
- **Theorem:** If any NP-Complete problem can be solved in polynomial time $\Rightarrow$ then $P = NP$.

ALGORITHM FOR SATISFIABILITY

Algorithm EVAL(E, n)

{

for i := 1 to n do

 xi := choice(true, false);

if E(x1,...,xn)

 then success();

else failure();

}

Reducibility

Let L_1 and L_2 be problems. L_1 reduces to L_2 ($L_1 \alpha L_2$) if and only if there is a deterministic polynomial time algorithm to solve L_1 that solves L_2 in polynomial time.

➤ If $L_1 \alpha L_2$ and $L_2 \alpha L_3$ then $L_1 \alpha L_3$.

COOK's Theorem

Satisfiability is in P if and only if $P = NP$.

NP-Hard

A problem L is NP-hard if any only if satisfiability reduces to L .

NP-complete

A problem L is NP-complete if and only if L is NP-hard and $L \in NP$.

There are two classes of non polynomial time problems

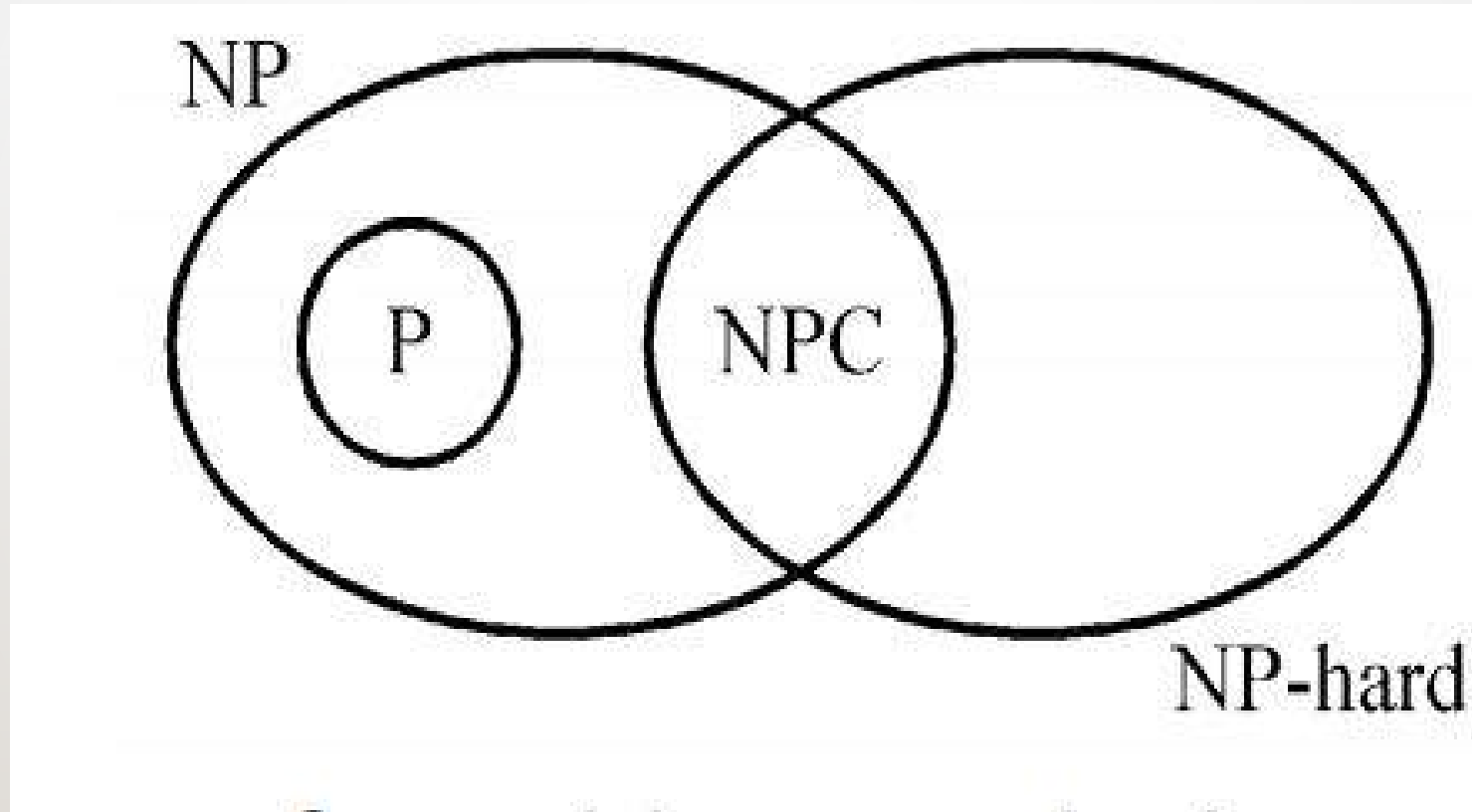
1. NP-Complete: Have the property that it can be solved in polynomial time if all other NP-Complete problems can be solved in polynomial time.

2. NP-Hard- If it can be solved in polynomial time then all NP-Complete can be solved in polynomial time.

“All NP-Complete problems are NP-Hard but not all NP Hard problems are not NP-Complete”

RELATIONSHIP BETWEEN P, NP, HARD AND NP-COMPLETE

NP-



COOK'S THEOREM

- Theorem states that satisfiability is in P if and only if $P = NP$.
- To prove this, we show how to obtain from any polynomial time nondeterministic decision algorithm A and input I a formula $Q(A, I)$ such that Q is satisfiable if A has a successful termination with input I .
- If the length of I is n and the time complexity of A is $p(n)$ for some polynomial $p()$, then the length of Q is $O(p^3(n) \log n) = O(p^4(n))$.

- The time needed to construct Q is also $O(p^3(n)\log n)$.
- A deterministic algorithm Z to determine the outcome of A on any input I can be easily obtained.
- Algorithm Z simply computes Q and then uses a deterministic algorithm for the satisfiability problem to determine whether Q is satisfiable.
- If $O(q(m))$ is the time needed to determine whether a formula of length m is satisfiable, then the complexity of Z is $O(p^3(n) \log n + q(p^3(n) \log n))$.

- If satisfiability is in P, then $q(m)$ is a polynomial function of m and the complexity of Z becomes $O(r(n))$ for some polynomial $r()$. Hence, if satisfiability is in P, then for every nondeterministic algorithm A in NP we can obtain a deterministic Z in P.
- So, the above construction shows that if satisfiability is in P, then $P = NP$.

Difference Between NP-Hard and NP-Complete

NP-Hard	NP-Complete
NP-Hard problems(say X) can be solved if and only if there is a NP-Complete problem(say Y) that can be reducible into X in polynomial time.	NP-Complete problems can be solved by a non-deterministic Algorithm/Turing Machine in polynomial time.
To solve this problem, it do not have to be in NP .	To solve this problem, it must be both NP and NP-hard problems.
Do not have to be a Decision problem.	It is exclusively a Decision problem.
Example: Halting problem, Vertex cover problem, etc.	Example: Determine whether a graph has a Hamiltonian cycle, Determine whether a Boolean formula is satisfiable or not, Circuit-satisfiability problem, etc.

Example : Halting problem is NP-hard decision problem, but it is not NP-complete.

Halting problem is NP-hard

To show that Halting problem is NP-hard, we show that

satisfiability α halting problem.

For this let us construct an algorithm A

whose input is a propositional formula X.

- Suppose X has n variables.
- Algorithm A tries out all 2^n possible truth assignments and verifies if X is satisfiable.

Halting problem is un-decidable.

- Hence there exists no algorithm to solve this problem.
- So, it is not in NP. Therefore, it is not NP-complete.

Questions:

1. With suitable example, explain nondeterministic algorithm.
2. Explain terminology used in Satisfiability Problem.
3. Explain Cook's theorem.
4. Differentiate NP-Hard with NP-Completeness.

THANK YOU

Clique Decision Problem(CDP)

NP-HARD GRAPH PROBLEMS

The strategy to show that a problem L_2 is NP-hard is

- (i) Pick a problem L_1 already known to be NP-hard.
- (ii) Show how to obtain an instance I^1 of L_2 from any instance I of L_1 such that from the solution of I^1
 - We can determine (in polynomial deterministic time) the solution to instance I of L_1 .
- (iii) Conclude from (ii) that $L_1 \alpha L_2$.
- (iv) Conclude from (i),(ii), and the transitivity of α that satisfiability αL_1
 $L_1 \alpha L_2$

$\therefore$ Satisfiability αL_2

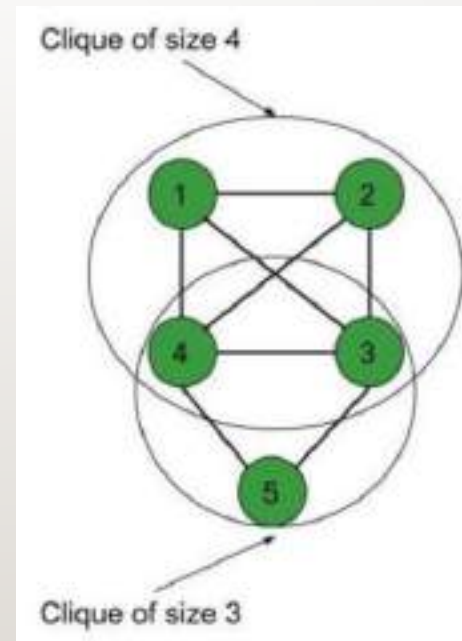
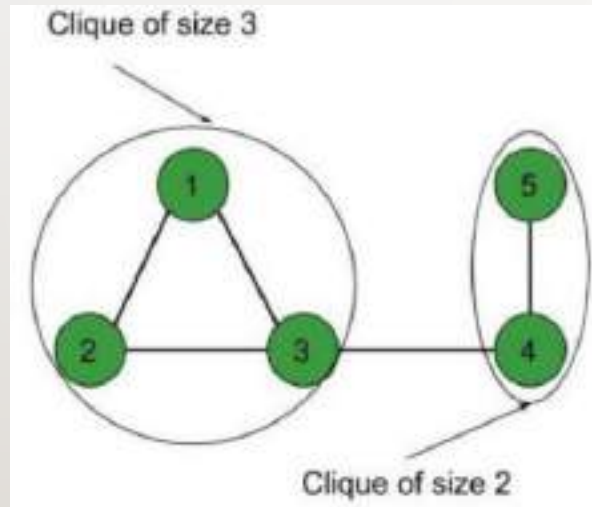
$\therefore L_2$ is NP-hard

Clique Decision Problem(CDP) :

Clique:

Clique is a maximal complete sub graph of a graph $G = (V,E)$

– Size of a clique is the number of vertices in it



Theorem: CNF- satisfiability α Clique Decision Problem

Proof: Let $F = \bigwedge_{1 \leq i \leq k} C_i$ be a propositional formula in CNF. Let x_i , $1 \leq i \leq n$ be the variables in F . We shall show how to construct from F a graph $G = (V, E)$ such that G will have a clique of size at least k iff F is satisfiable. If the length of F is m , then G will be obtainable from F in $O(m)$ time. Hence, if we have a polynomial time algorithm for CDP, then we can obtain a polynomial time algorithm for CNF-satisfiability using this construction.

For any F , $G = (V, E)$ is defined as follows: $V = \{ \langle \sigma, i \rangle \mid \sigma \text{ is a literal in clause } C_i \}$; $E = \{ (\langle \sigma, i \rangle, \langle \delta, j \rangle) \mid i \neq j \text{ and } \sigma \neq \bar{\delta} \}$. A sample construction is given in Example 11.11.

Example 11.11 Consider $F = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$. The construction of Theorem 11.2 yields the graph:

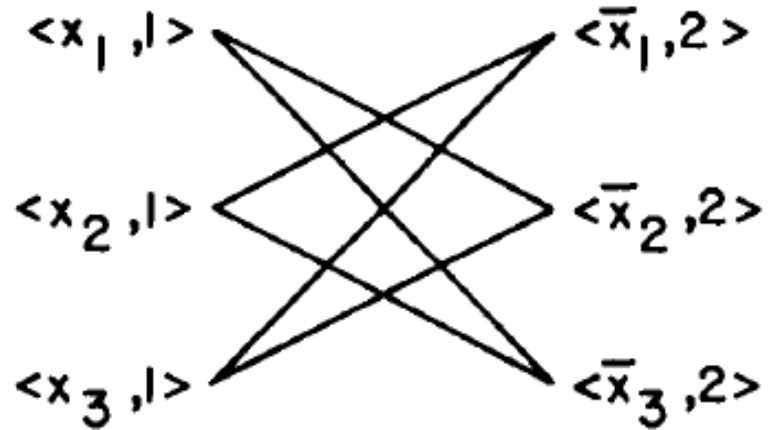


Figure 11.1 A sample graph and satisfiability

This graph contains six cliques of size two. Consider the clique with vertices $\{\langle x_1, 1 \rangle, \langle \bar{x}_2, 2 \rangle\}$. By setting $x_1 = \text{true}$ and $\bar{x}_2 = \text{true}$ (i.e. $x_2 = \text{false}$) F is satisfied. x_3 may be set either to true or false. $\square$

Questions:

- 1.1. Discuss in detail about Clique Decision Problem
2. Reduce CNF-Satisfiability problem into CDP and Solve

THANK YOU

NODE COVER DECISION PROBLEM(NCDP)

Node Cover Decision Problem(NCDP):

A set $S \subseteq V$ is a *node cover* for a graph $G = (V,E)$ if and only if all edges in E are incident to at least one vertex in S . The size $|S|$ of the cover is the number of vertices in S .

Example 11.12 Consider the graph:

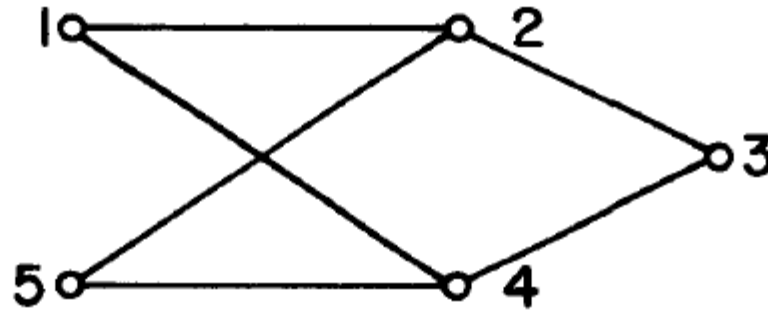


Figure 11.2 A sample graph and node cover

$S = \{2,4\}$ is a node cover of size 2.

$S = \{1,3,5\}$ is a node cover of size 3.

In a node cover decision problem we are given a graph G and an integer k .

We are required to determine whether G has a node cover of size at most k .

Theorem: The clique decision problem α the node cover decision problem.

Proof: let $G = (V, E)$ and k define an instance of CDP. Assume that $|V| = n$.

We Construct a Graph G' such that G' has a node cover of size at most $n-k$ if and only if G has a clique of size at least k .

Graph $G' = (V, E') =$, where $E' = \{(u, v) \mid u \in V, v \in V \text{ and } (u, v) \notin E\}$.

The set G' is known as the complement of G

Now, we shall show that G has a clique of size at least k iff G' has a node cover of size at most $n - k$. Let K be any clique in G . Since there are no edges in $\bar{E}$ connecting vertices in K , the remaining $n - |K|$ vertices in G' must cover all edges in $\bar{E}$. Similarly, if S is a node cover of G' then $V - S$ must form a complete subgraph in G .

Since G' can be obtained from G in polynomial time, CDP can be solved in polynomial deterministic time if we have a polynomial time deterministic algorithm for NCDP. $\square$

Note that since $\text{CNF-satisfiability} \propto \text{CDP}$, $\text{CDP} \propto \text{NCDP}$ and $\propto$ is transitive, it follows that NCDP is NP-hard.

Example 11.11 Consider $F = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$. The construction of Theorem 11.2 yields the graph:

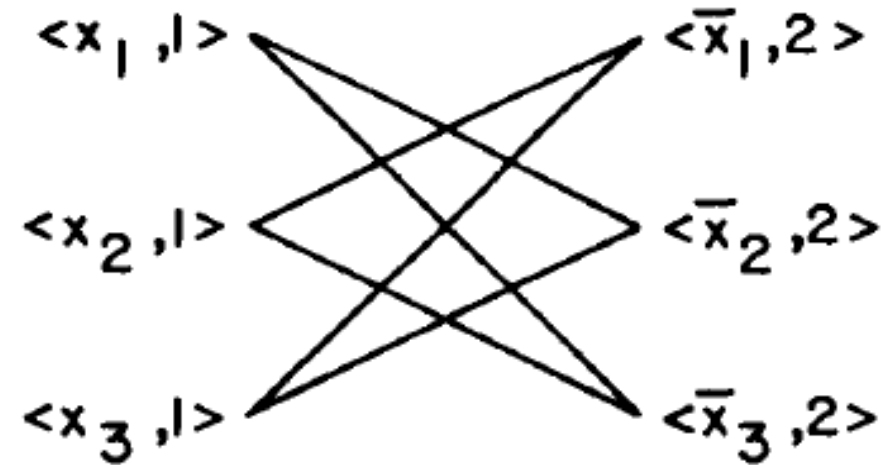
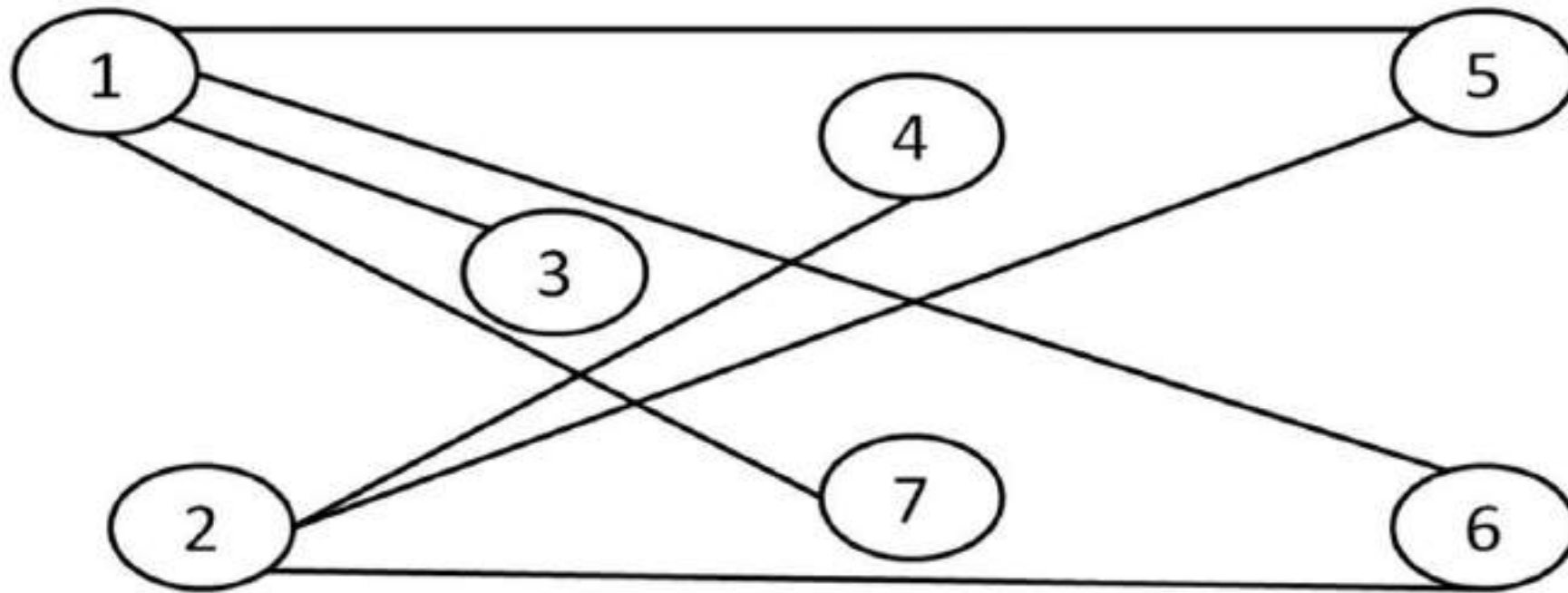


Figure 11.1 A sample graph and satisfiability

This graph contains six cliques of size two. Consider the clique with vertices $\{\langle x_1, 1 \rangle, \langle \bar{x}_2, 2 \rangle\}$. By setting $x_1 = \text{true}$ and $\bar{x}_2 = \text{true}$ (i.e. $x_2 = \text{false}$) F is satisfied. x_3 may be set either to true or false. $\square$

Prove that VERTEX COVER Problem is NP-Complete

- $G=(V, E)$



Prove that VERTEX COVER Problem is NP-Complete

- **Step 1:** Write a polynomial time verification algorithm to prove that the given problem is NP
- **Inputs:** $\langle G, k, V' \rangle$
- **Verifier Algorithm:**
 1. count = 0
 2. for each vertex v in V' remove all edges adjacent to v from set E
 1. increment count by 1
 3. if count = k and E is empty then the given solution is correct
 4. else the given solution is wrong
- This algorithm will execute in polynomial time. Therefore **VERTEX COVER problem is a NP problem.**

Prove that VERTEX COVER Problem is NP-Complete

- **Step 2:** Write a polynomial time reduction algorithm from CLIQUE problem to VERTEX COVER problem

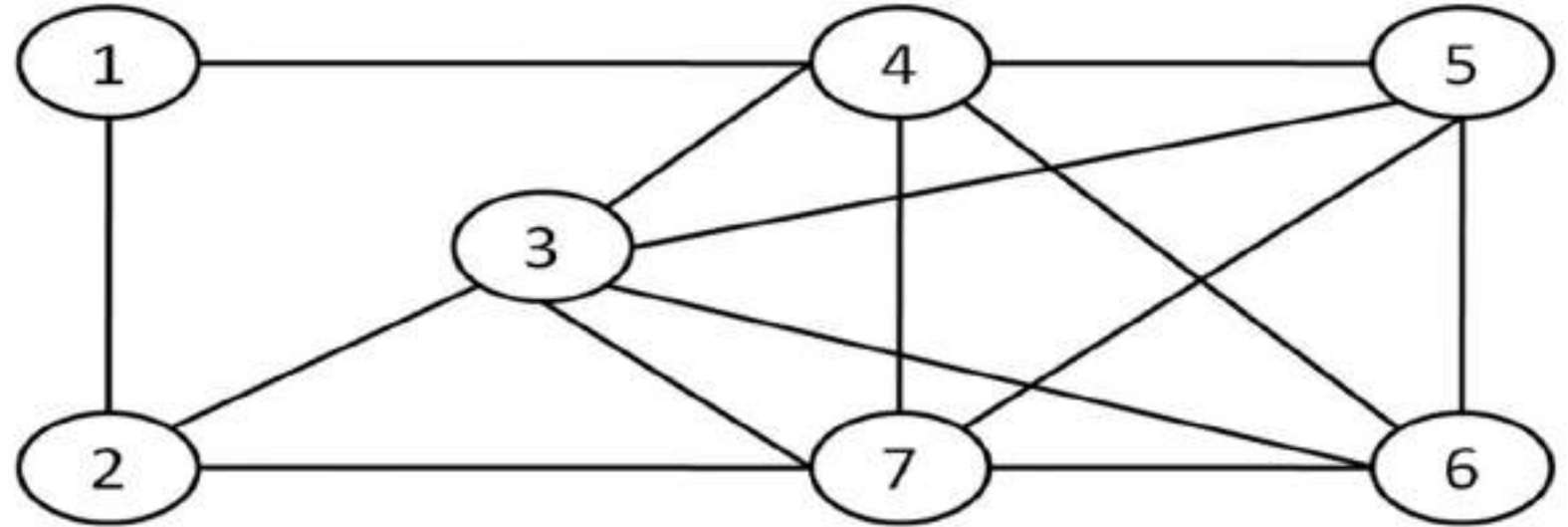
- **Algorithm**

Inputs: $\langle G=(V,E), k \rangle$

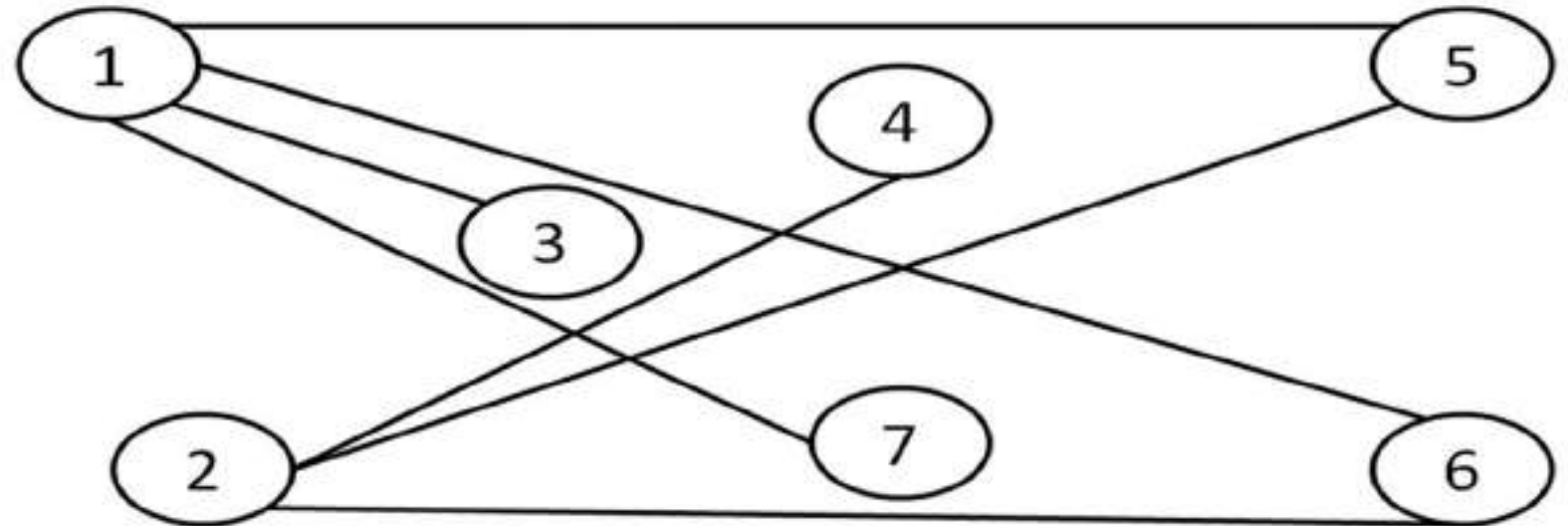
1. Construct a graph G' , which is the complement of Graph G
 2. If G' has a vertex cover of size $|V| - k$, then G has a clique of size k .
- This reduction algorithm (CLIQUE to VERTEX COVER) is a polynomial time algorithm
 - So **VERTEX COVER problem is NP Hard.**

Prove that VERTEX COVER Problem is NP-Complete

• $G=(V, E)$

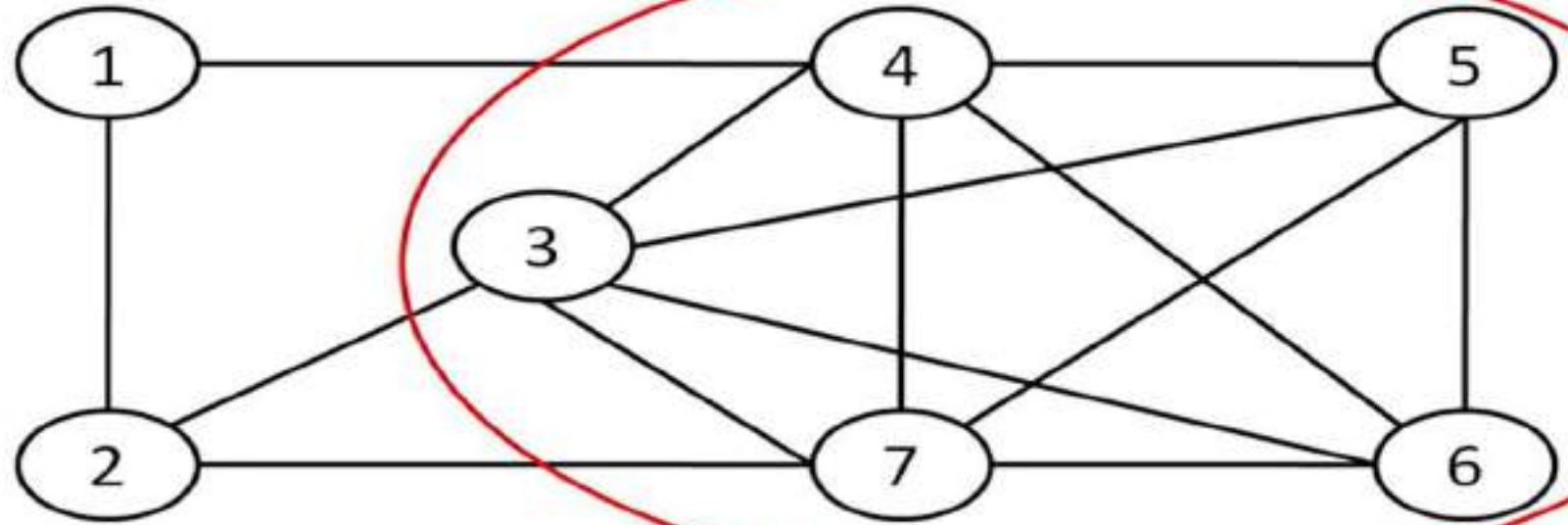


• $G'=(V, E')$

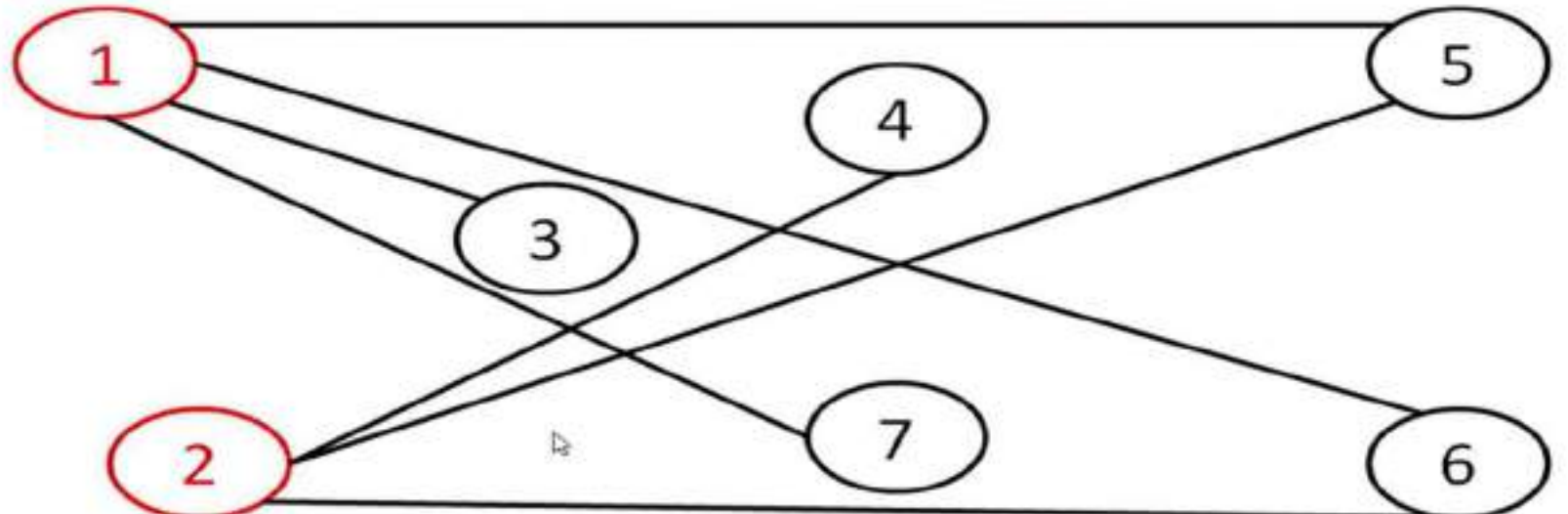


Prove that VERTEX COVER Problem is NP-Complete

• $G=(V, E)$



• $G'=(V, E')$



Prove that VERTEX COVER Problem is NP-Complete

- **Step 2:** Write a polynomial time reduction algorithm from CLIQUE problem to VERTEX COVER problem
- **Algorithm**
Inputs: $\langle G=(V,E), k \rangle$
 1. Construct a graph G' , which is the complement of Graph G
 2. If G' has a vertex cover of size $|V| - k$, then G has a clique of size k .
- This reduction algorithm (CLIQUE to VERTEX COVER) is a polynomial time algorithm
- So **VERTEX COVER** problem is NP Hard.

Prove that VERTEX COVER Problem is NP-Complete

- **Conclusion**
 - VERTEX COVER problem is NP and NP-Hard.
 - So it is NP-Complete

Approximate Algorithms

Introduction:

An Approximate Algorithm is a way of approach **NP-COMPLETENESS** for the optimization problem. This technique does not guarantee the best solution. The goal of an approximation algorithm is to come as close as possible to the optimum value in a reasonable amount of time which is at the most polynomial time. Such algorithms are called approximation algorithm or heuristic algorithm.

- For the traveling salesperson problem, the optimization problem is to find the shortest cycle, and the approximation problem is to find a short cycle.
- For the vertex cover problem, the optimization problem is to find the vertex cover with fewest vertices, and the approximation problem is to find the vertex cover with few vertices.

Approximation Algorithm

- **Approximate Solution:** A feasible solution with value close to the value of optimal solution is called an approximate solution
- **Approximation Algorithms:** An algorithm that returns near optimal solution is called Approximation Algorithm.
- Approximation algorithms have two main properties:
 - They run in polynomial time
 - They produce solutions close to the optimal solutions
- Approximation algorithms are useful to give approximate solutions to NP complete optimization problems.
- It is also useful to give fast approximations to problems that run in polynomial time.

Approximation algorithms are algorithms designed to solve problems that are not solvable in polynomial time for approximate solutions. These problems are known as NP complete problems. These problems are significantly effective to solve real world problems, therefore, it becomes important to solve them using a different approach.

NP complete problems can still be solved in three cases: the input could be so small that the execution time is reduced, some problems can still be classified into problems that can be solved in polynomial time, or use approximation algorithms to find near-optima solutions for the problems.

This leads to the concept of performance ratios of an approximation problem.

Approximation Algorithm

- **Approximation Ratio / Approximation Factor**
 - For given problem, C is the result obtained by the algorithm and C^* is the optimal result.
 - The approximation ratio of an algorithm is the ratio between the result obtained by the algorithm and the optimal result.
 - For **maximization problem**, $0 < C \leq C^*$,
Approximation Ratio = C^*/C
 - For **minimization problem**, $0 < C^* \leq C$,
Approximation Ratio = C/C^*

Performance Ratios

The main idea behind calculating the **performance ratio** of an approximation algorithm, which is also called as an **approximation ratio**, is to find how close the approximate solution is to the optimal solution.

The approximate ratio is represented using **(n)** where n is the input size of the algorithm, C is the near-optimal solution obtained by the algorithm, C^* is the optimal solution for the problem. The algorithm has an approximate ratio of $\rho(n)$ if and only if –

$$\max \left\{ \frac{C}{C^*}, \frac{C^*}{C} \right\} \leq \rho(n)$$

The algorithm is then called a $\rho(n)$ -approximation algorithm. Approximation Algorithms can be applied on two types of optimization problems: minimization problems and maximization problems. If the optimal solution of the problem is to find the maximum cost, the problem is known as the maximization problem; and if the optimal solution of the problem is to find the minimum cost, then the problem is known as a minimization problem.

For maximization problems, the approximation ratio is calculated by C^*/C since $0 < C \leq C^*$. For minimization problems, the approximation ratio is calculated by C/C^* since $0 < C^* \leq C$.

Assuming that the costs of approximation algorithms are all positive, the performance ratio is well defined and will not be less than 1. If the value is 1, that means the approximate algorithm generates the exact optimal solution.

Approximation Algorithm

- **Approximation Ratio / Approximation Factor**
 - The approximation ratio of an approximation algorithm is never less than 1.
 - Approximation ratio and computational time are inversely proportional.
 - Approximation ratio and quality of the result are also inversely proportional

Approximation Algorithm

- **k-Approximation Algorithm:** An algorithm with approximation ratio k is called a k -approximation algorithm.
- 1-approximation algorithm produces an optimal solution
- An approximation algorithm with a large approximation ratio may return a solution that is much worse than optimal.

Have you ever wondered about the placement of traffic cameras? That how they are efficiently placed without wasting too much budget from the government? The answer to that comes in the form of **vertex-cover algorithm**. The positions of the cameras are chosen in such a way that one camera covers as many roads as possible, i.e., we choose junctions and make sure the camera covers as much area as possible.

A **vertex-cover** of an undirected graph $G = (V, E)$ is the subset of vertices of the graph such that, for all the edges (u, v) in the graph, u and $v \in V$. The junction is treated as the node of a graph and the roads as the edges. The algorithm finds the minimum set of junctions that cover maximum number of roads.

It is a minimization problem since we find the minimum size of the vertex cover the size of the vertex cover is the number of vertices in it. The optimization problem is an NP-Complete problem and hence, cannot be solved in polynomial time; but what can be found in polynomial time is the near optimal solution.

Vertex Cover Algorithm

The vertex cover approximation algorithm takes an undirected graph as an input and is executed to obtain a set of vertices that is definitely twice as the size of optimal vertex cover.

The vertex cover is a 2-approximation algorithm.

Algorithm

Step 1 – Select any random edge from the input graph and mark all the edges that are incident on the vertices corresponding to the selected edge.

Step 2 – Add the vertices of the arbitrary edge to an output set.

Step 3 – Repeat Step 1 on the remaining unmarked edges of the graph and add the vertices chosen to the output until there's no edge left unmarked.

Step 4 – The final output set achieved would be the near-optimal vertex cover.

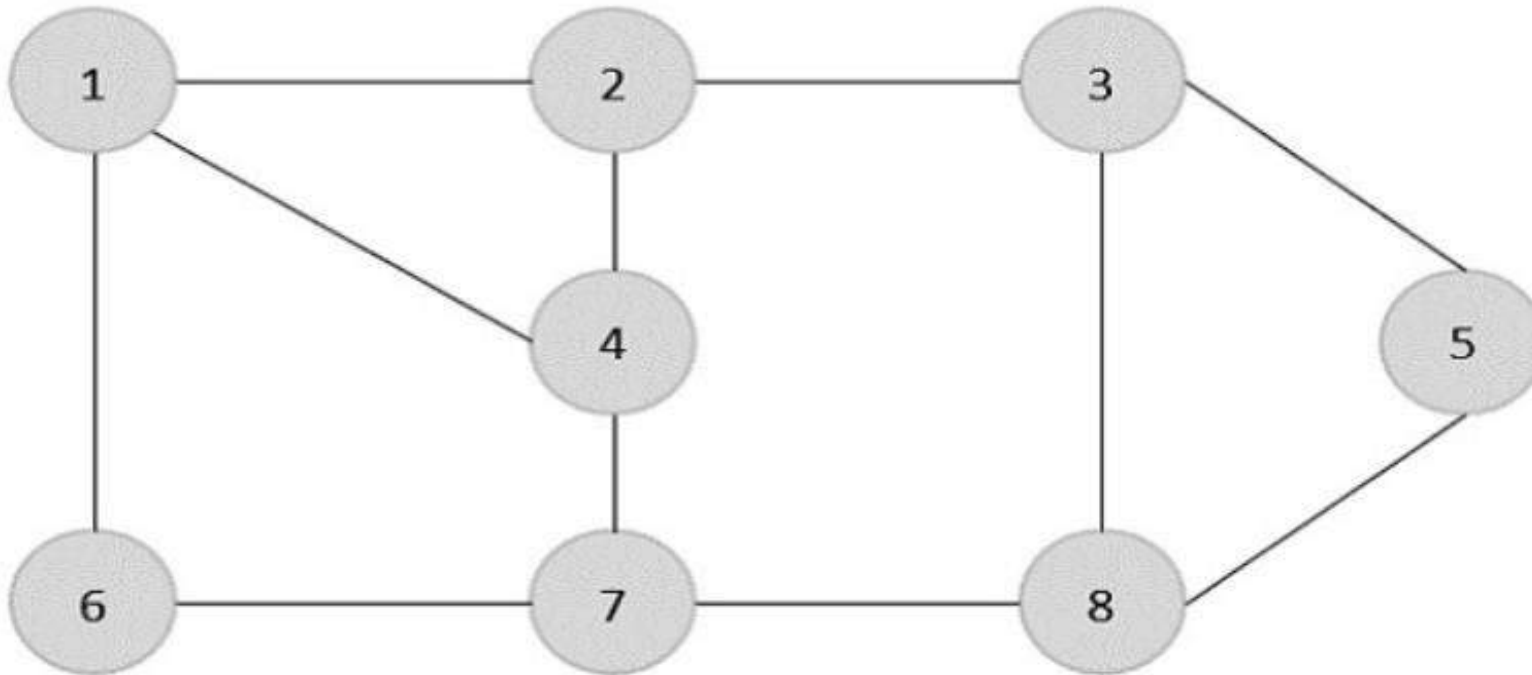
Pseudocode

```
APPROX-VERTEX_COVER (G: Graph)
c ← { }
E ← E[G]
while E is not empty do
    Let (u, v) be an arbitrary edge of E
    c ← c U {u, v}
    Remove from E every edge incident on either u or v
return c
```

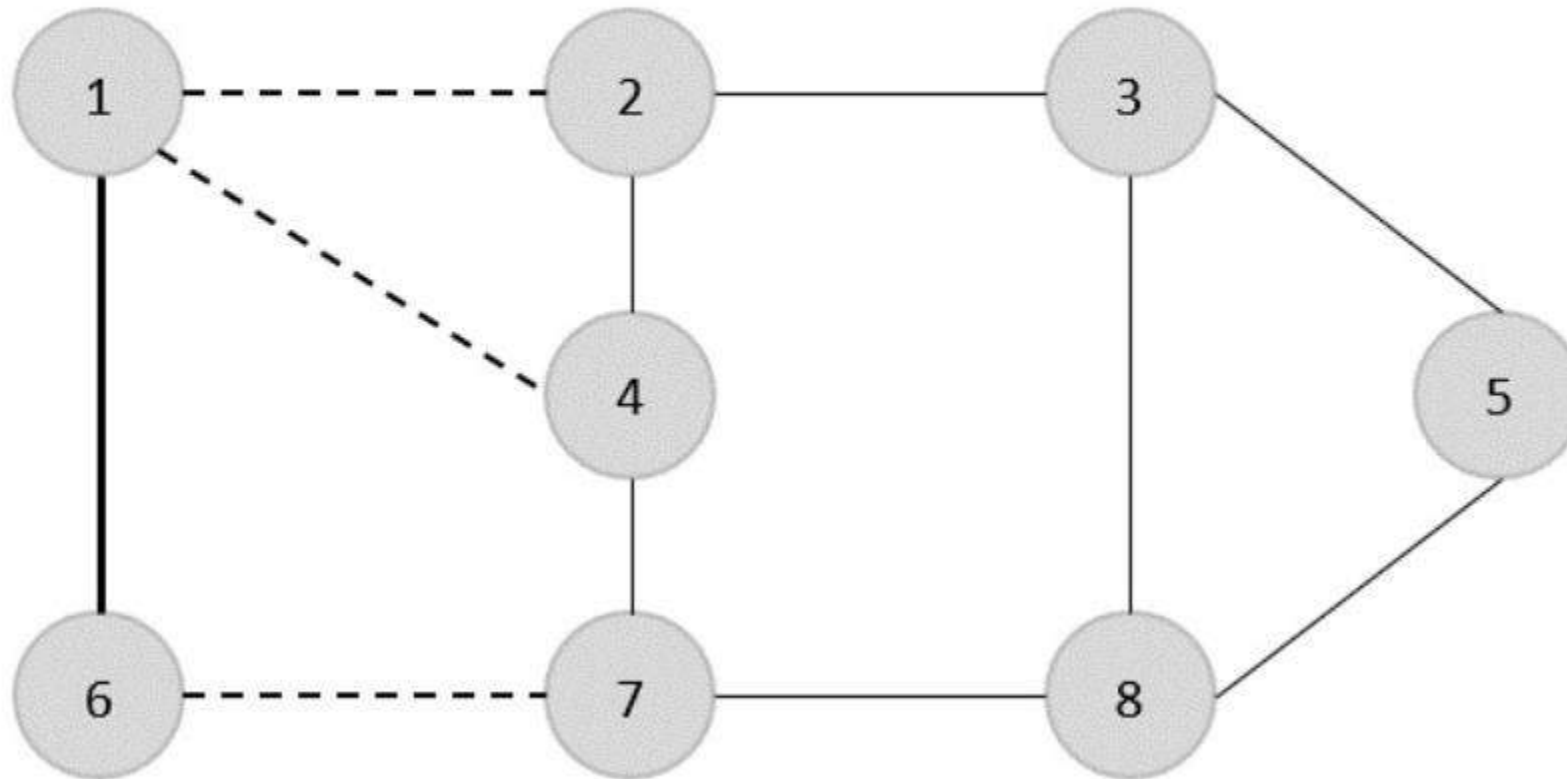
Example

The set of edges of the given graph is –

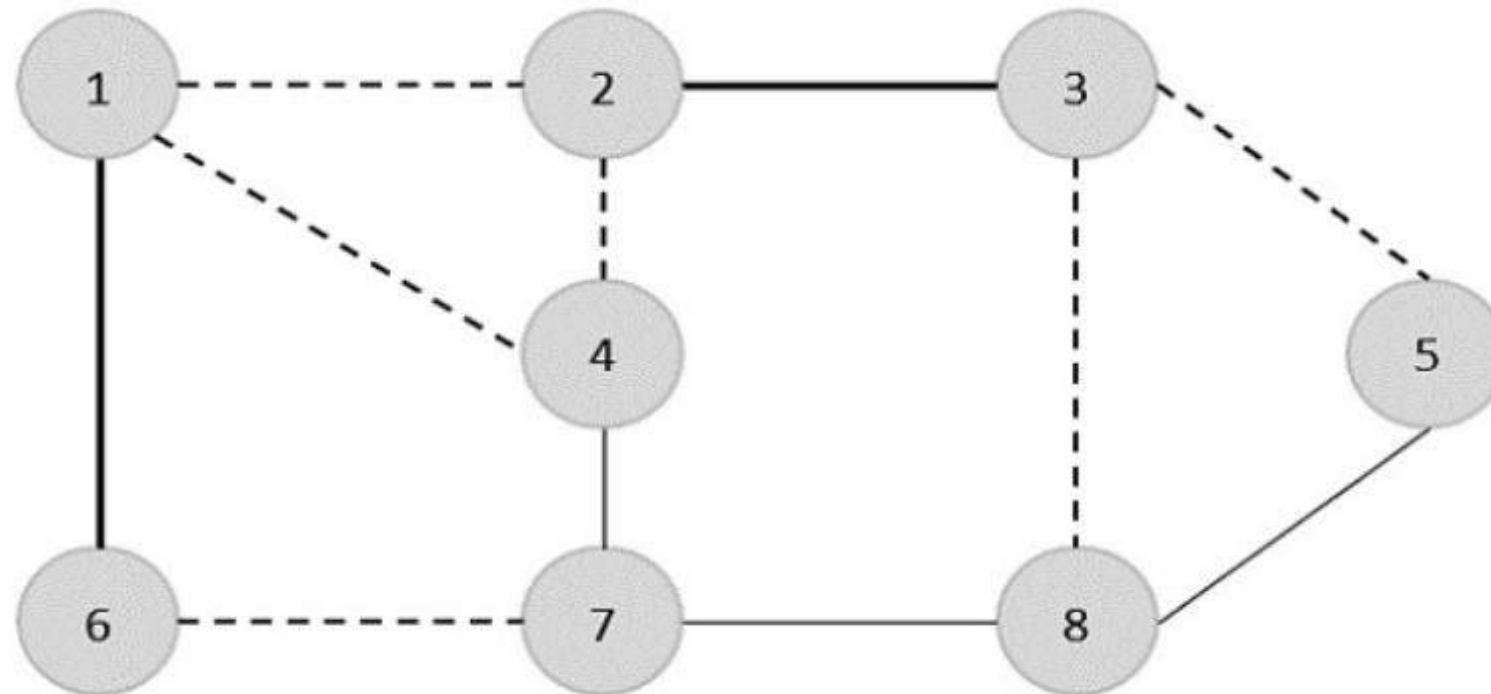
$\{(1,6), (1,2), (1,4), (2,3), (2,4), (6,7), (4,7), (7,8), (3,5), (8,5)\}$



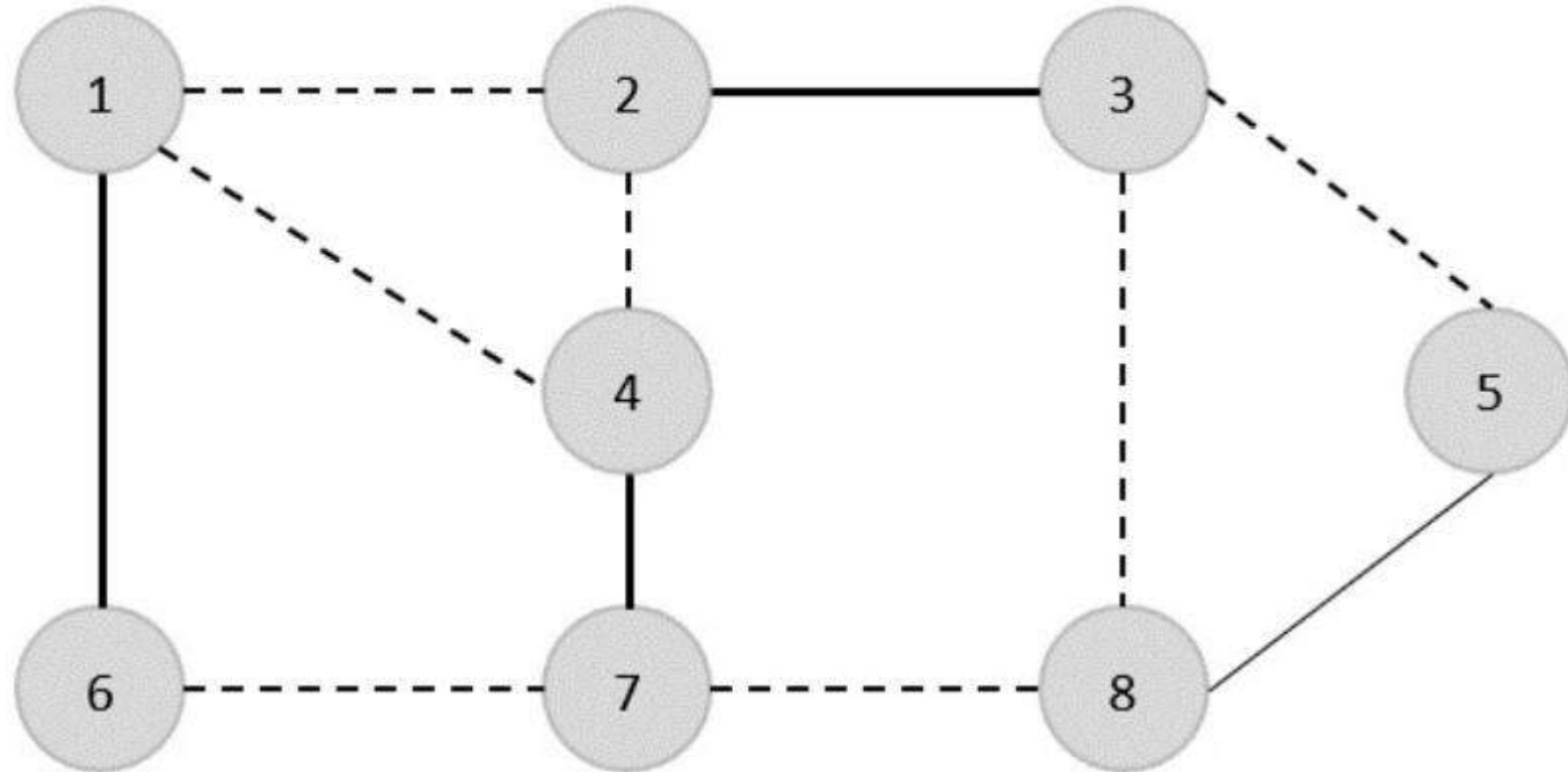
Now, we start by selecting an arbitrary edge (1,6). We eliminate all the edges, which are either incident to vertex 1 or 6 and we add edge (1,6) to cover.



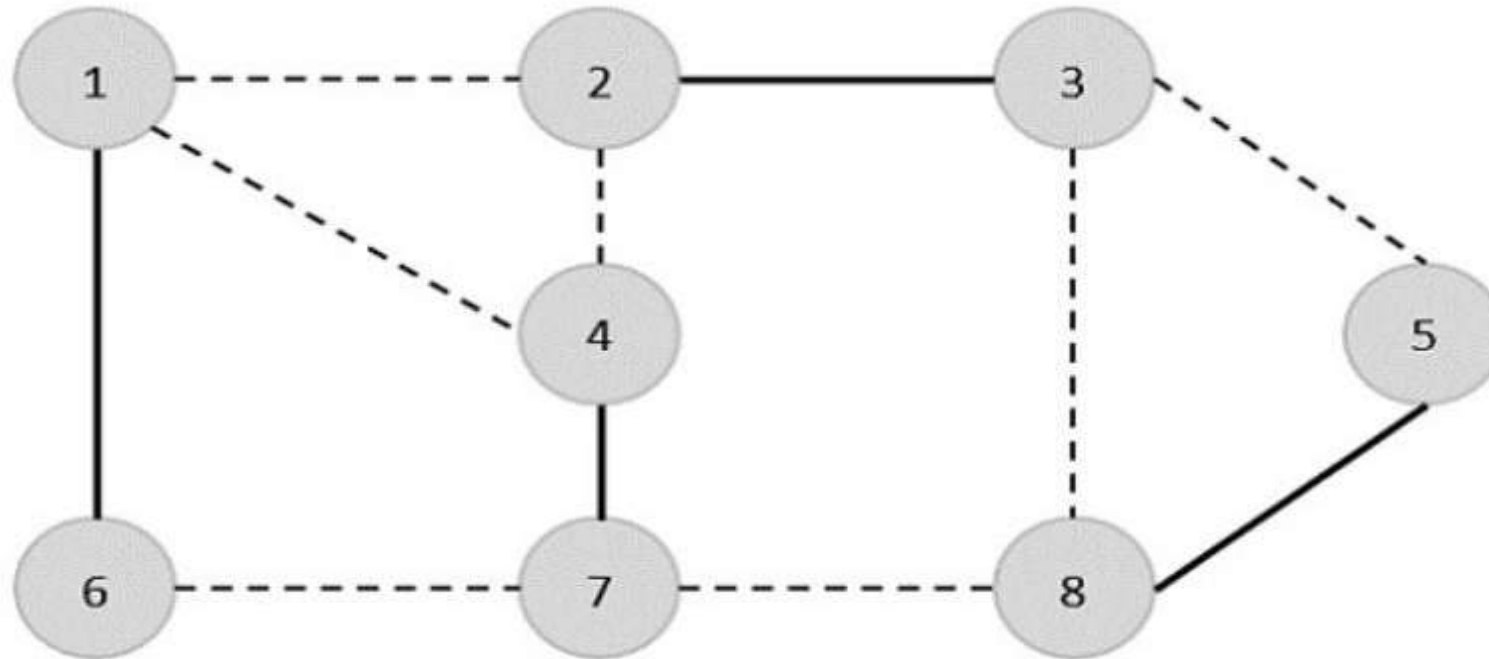
In the next step, we have chosen another edge (2,3) at random.



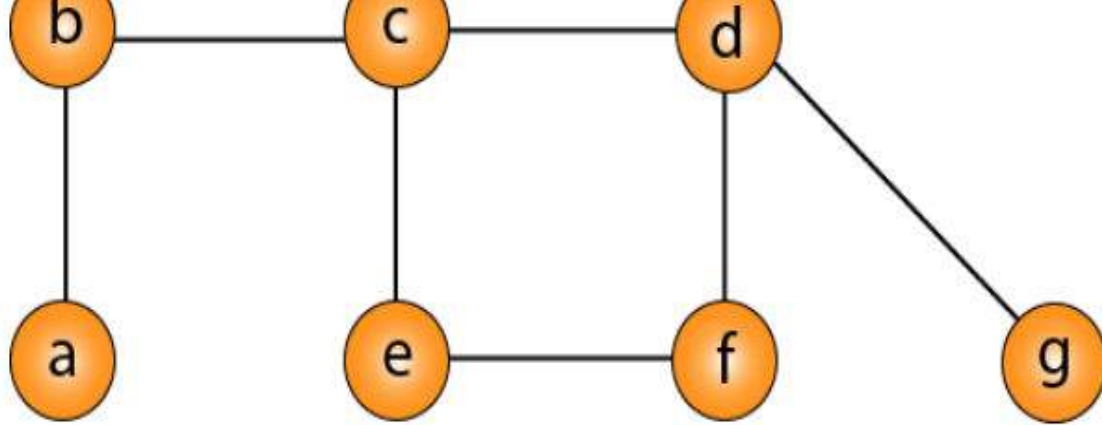
Now we select another edge (4,7).



We select another edge (8,5).



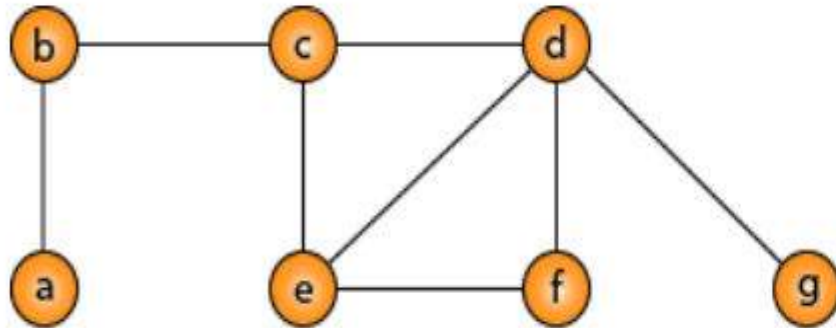
Hence, the vertex cover of this graph is $\{1,6,2,3,4,7,5,8\}$.



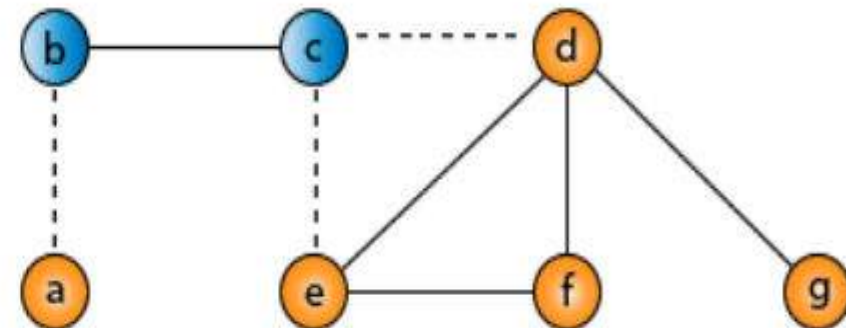
An approximate algorithm for vertex cover:

```
Approx-Vertex-Cover ( $G = (V, E)$ )  
{  
     $C = \text{empty-set};$   
     $E' = E;$   
    While  $E'$  is not empty do  
    {  
        Let  $(u, v)$  be any edge in  $E'$ : (*)  
        Add  $u$  and  $v$  to  $C$ ;  
        Remove from  $E'$  all edges incident to  
         $u$  or  $v$ ;  
    }  
    Return  $C$ ;  
}
```

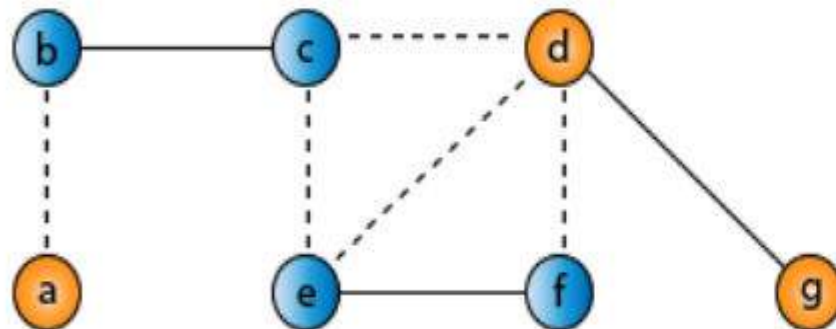
The idea is to take an edge (u, v) one by one, put both vertices to C , and remove all the edge incident to u or v . We carry on until all edges have been removed. C is a VC. But how good is C ?



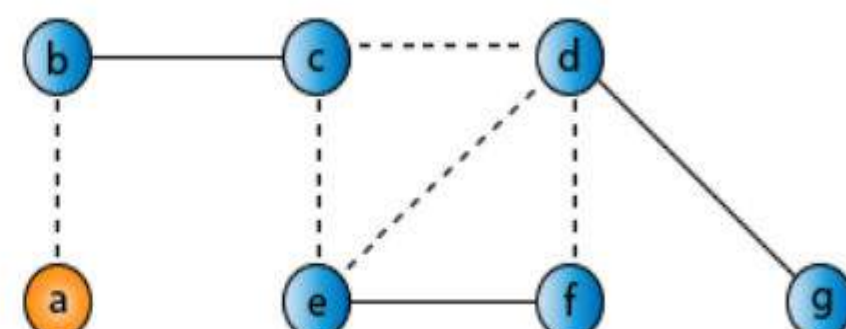
(1)



(2)



(3)



(4)

VC = {b, c, d, e, f, g}

Analysis

It is easy to see that the running time of this algorithm is **$O(V + E)$** , using adjacency list to represent **E'** .

Questions:

1. Explain in detail about NCDP?
2. Prove that The clique decision problem α the node cover decision problem.

THANK YOU